

Base de Datos I (CS2041)

2026-2

Proyecto Final

Sistema de Monitoreo Sismológico y Alerta Temprana

Profesor a cargo: Wilder Nina

Integrantes

Chala Lastra, Jared Eloy Daniel	202520040
Landeo Huatay, Leonard Alexander	202520151
Tumialan Maihua, Fabricio Nick	202520045

2026

Lima, Perú

Índice

1. Requisitos **4**

1.1. Introducción 4

1.2. Descripción general del problema/organización/empresa 4

1.3. Necesidad/usos de la base de datos 4

1.4. ¿Cómo se resuelve el problema hoy? 5

1.4.1. ¿Cómo se almacenan/procesan los datos hoy? 5

1.4.2. Flujo de datos 6

1.5. Descripción detallada del sistema 8

1.5.1. Objetos de información actuales 8

1.5.2. Características y funcionalidades esperadas 9

1.5.3. Tipos de usuarios existentes/necesarios 11

1.5.4. Tipos de consulta, actualizaciones 11

1.5.5. Tamaño estimado de la base de datos 12

1.6. Objetivos del proyecto 13

1.7. Referencias del proyecto 14

1.8. Eventualidades 15

1.8.1. Problemas que pudieran encontrarse en el proyecto 15

1.8.2. Límites y alcances del proyecto 15

2. Modelo Entidad–Relación **16**

2.1. Reglas semánticas 16

2.2. Modelo Entidad–Relación 18

2.3. Especificaciones y consideraciones sobre el modelo 20

3. Modelo Relacional **23**

3.1. Modelo Relacional 23

3.2. Especificaciones de transformación 25

3.2.1. Entidades 25

3.2.2. Entidades débiles 26

3.2.3. Entidades superclase/subclases 26

3.2.4. Relaciones binarias 26
3.2.5. Relaciones ternarias 27
3.3. Diccionario de datos 27

4. Implementación de la base de datos 32
4.1. Creación de Tablas en PostgreSQL 32
4.2. Carga de datos 34
4.3. Simulación de datos faltantes 36
4.4. Vistas (Views) 38
4.5. Procedimientos almacenados 40
4.6. Triggers 42

5. Optimización y experimentación 44
5.1. Consultas SQL para el experimento 44
5.1.1. Descripción del tipo de consultas seleccionadas 44
5.1.2. Implementación de consultas en SQL 45
5.2. Metodología del experimento 48
5.2.1. Diseño experimental y escalas 48
5.2.2. Procedencia y alcance de los datos 49
5.2.3. Condiciones comparadas y protocolo de medición 49
5.2.4. Controles realizados y reproducibilidad 50
5.2.5. Limitaciones de la comparación 50
5.3. Optimización de consultas 50
5.3.1. Plan de índices para la Consulta 1 51
5.3.2. Plan de índices para la Consulta 2 52
5.3.3. Plan de índices para la Consulta 3 53
5.3.4. Plan de índices para la Consulta 4 55
5.3.5. Plan de índices para la Consulta 5 57
5.4. Plataforma de pruebas 59
5.5. Medición de tiempos 61
5.5.1. Sin índices secundarios 61
5.5.2. Con índices candidatos 62

5.6. Resultados 63
5.6.1. Consulta 1: Actividad sísmica anual del catálogo del IGP 63
5.6.2. Consulta 2: Eventos sísmicos en ventanas geográficas 64
5.6.3. Consulta 3: Calidad de lecturas por estación 65
5.6.4. Consulta 4: Tiempos de primera detección y primera alerta 66
5.6.5. Consulta 5: Sensores con lecturas y sin detecciones 67
5.7. Análisis y Discusión 68
5.7.1. Balance de las veinte comparaciones y decisiones de indexación . . . 68
5.7.2. Trazabilidad y alcance de las mediciones 69
5.7.3. Análisis adicional: crecimiento más allá del millón 69
5.7.4. Limitaciones y reproducibilidad 70

6. Conclusiones **71**

7. Anexos **72**
7.1. Código fuente y reproducibilidad de la implementación 72
7.2. Respaldo de las cuatro bases experimentales 72
7.3. Demostración SQL sobre un millón de registros 74
7.4. Prototipo web de consultas 74
7.5. Verificación tabular de la consulta geográfica 77
7.6. Código SQL de la implementación y los experimentos 78
7.6.1. Estructura y restricciones 78
7.6.2. Carga, catálogo IGP y simulación 82
7.6.3. Vistas, procedimientos, disparadores y consultas 94
7.6.4. Bases experimentales 116
7.6.5. Planes de ejecución e índices 127
7.6.6. Medición de rendimiento 136
7.6.7. Demostración de la escala máxima 143

1. Requisitos

1.1. Introducción

El presente proyecto desarrolla el diseño y la implementación de una base de datos relacional para el monitoreo sismológico y la gestión de alertas tempranas. Su estructura integra información de estaciones sísmicas, sensores, lecturas instrumentales, eventos sísmicos, detecciones y alertas emitidas.

La propuesta busca representar de manera estructurada el flujo de información generado durante el monitoreo de la actividad sísmica, permitiendo relacionar cada estación con sus respectivos sensores, conservar las mediciones obtenidas, registrar las características principales de los eventos sísmicos y mantener un historial de las alertas asociadas a dichos eventos.

A partir del análisis de la información se definieron el modelo conceptual y el modelo relacional, con las entidades, relaciones y restricciones necesarias para representar el dominio. Ambos modelos sustentan la implementación del esquema en PostgreSQL y las consultas utilizadas para su evaluación.

1.2. Descripción general del problema/organización/empresa

El monitoreo sismológico requiere recopilar y administrar información proveniente de múltiples estaciones distribuidas geográficamente. Estas estaciones contienen sensores encargados de registrar el movimiento del suelo y producir mediciones que posteriormente pueden ser utilizadas para identificar y analizar eventos sísmicos.

Para el desarrollo del proyecto se plantea una organización ficticia encargada del monitoreo sísmico, cuya función es administrar una red de estaciones y sensores, conservar las mediciones generadas por estos dispositivos, registrar los eventos sísmicos identificados y mantener información sobre las alertas asociadas a dichos eventos.

El principal problema consiste en gestionar de manera consistente información de distinta naturaleza pero estrechamente relacionada. La organización necesita poder determinar, por ejemplo, qué sensores participaron en la detección de un evento, a qué estaciones pertenecen dichos sensores, qué características presenta el evento registrado y qué alertas fueron generadas a partir de este. Por ello, se requiere una estructura de datos que permita integrar y relacionar esta información de forma organizada.

1.3. Necesidad/usos de la base de datos

La organización requiere una base de datos que permita centralizar y relacionar la información generada durante el monitoreo sismológico. Debido a que los datos corresponden a distintos objetos, como estaciones, sensores, lecturas, eventos sísmicos y alertas, resulta necesario mantener una estructura que permita conservar sus relaciones y garantizar la consistencia de la información almacenada.

La base de datos permitirá mantener un registro histórico de las mediciones generadas por

los sensores, identificar qué dispositivos y estaciones participaron en la detección de un determinado evento sísmico y conservar las características asociadas a cada evento. Asimismo, permitirá registrar las alertas vinculadas a estos eventos y mantener información sobre su emisión y estado.

Además de facilitar el registro y recuperación de información, la base de datos servirá como soporte para consultas operativas e históricas, así como para la generación de estadísticas sobre la actividad sísmica registrada. La centralización de los datos también permitirá reducir la duplicidad de información, facilitar su actualización y establecer restricciones que mantengan la integridad de los registros.

1.4. ¿Cómo se resuelve el problema hoy?

Para efectos del proyecto, se asume que la organización gestiona actualmente la información mediante archivos independientes y herramientas de almacenamiento no integradas. Los datos correspondientes a estaciones, sensores, lecturas, eventos sísmicos y alertas se mantienen en fuentes separadas y deben ser relacionados posteriormente cuando se requiere realizar una consulta que involucre más de un tipo de información.

Este esquema puede resultar suficiente mientras el volumen de datos es reducido; sin embargo, a medida que aumenta la cantidad de estaciones, sensores y mediciones registradas, se vuelve más difícil mantener la información consistente y realizar consultas de manera eficiente. La ausencia de una estructura centralizada también incrementa el riesgo de duplicidad, inconsistencias y errores durante la búsqueda o actualización de los datos.

1.4.1. ¿Cómo se almacenan/procesan los datos hoy?

Se asume que la información de las estaciones y sensores se mantiene principalmente en hojas de cálculo, mientras que las mediciones generadas por los sensores se almacenan en archivos CSV debido a su naturaleza tabular y a su mayor volumen. Por otra parte, los eventos sísmicos identificados y las alertas emitidas se conservan en reportes independientes.

Debido a esta separación, las operaciones que requieren combinar información de distintos objetos deben realizarse mediante búsquedas y cruces manuales entre varios archivos. Este procedimiento puede implicar comparar identificadores, fechas, horas y otros atributos para determinar qué registros se encuentran relacionados.

Por ejemplo, para responder a la consulta "¿qué sensores detectaron un determinado evento sísmico y con qué aceleración máxima?", un operador tendría que localizar primero el evento en el registro correspondiente, revisar los datos disponibles de los sensores dentro del intervalo temporal asociado al evento y, posteriormente, consultar la información de cada sensor para determinar la estación a la que pertenece.

A medida que aumenta el número de sensores y lecturas registradas, este procedimiento requiere mayor tiempo y aumenta la posibilidad de errores, como omitir registros relevantes, relacionar datos incorrectos o mantener versiones inconsistentes de la misma información. Estas limitaciones justifican la necesidad de sustituir el manejo fragmentado de los datos por una estructura relacional centralizada.

1.4.2. Flujo de datos

El flujo de información del sistema comienza en las estaciones sísmicas, donde se encuentran instalados los sensores encargados de registrar el movimiento del suelo. Cada sensor genera lecturas de manera periódica o continua de acuerdo con su frecuencia de muestreo, y cada una de estas mediciones queda asociada al dispositivo que la produjo.

Las lecturas generadas constituyen la información instrumental utilizada durante el monitoreo. Cuando se identifica un evento sísmico, se registra un único evento con sus principales características, como fecha y hora de origen, ubicación aproximada del epicentro, profundidad y magnitud.

Debido a que un mismo evento puede ser observado por sensores pertenecientes a distintas estaciones, el sistema registra individualmente la participación de cada sensor mediante una detección. Cada detección relaciona un sensor con el evento sísmico correspondiente y puede conservar información específica del registro, como la hora de detección, la aceleración máxima observada y un indicador de calidad.

De esta manera, varios sensores pueden encontrarse asociados a un mismo evento sin duplicar la información general de este. Asimismo, un mismo sensor puede participar en la detección de múltiples eventos a lo largo del tiempo.

Finalmente, cuando un evento cumple con los criterios establecidos para la emisión de una alerta, se registra una alerta asociada a dicho evento. Esta puede almacenar información como la fecha y hora de emisión, el nivel asignado, el mensaje, la magnitud estimada al momento de su emisión y su estado.

De manera general, el flujo de información se representa en la Figura 1.

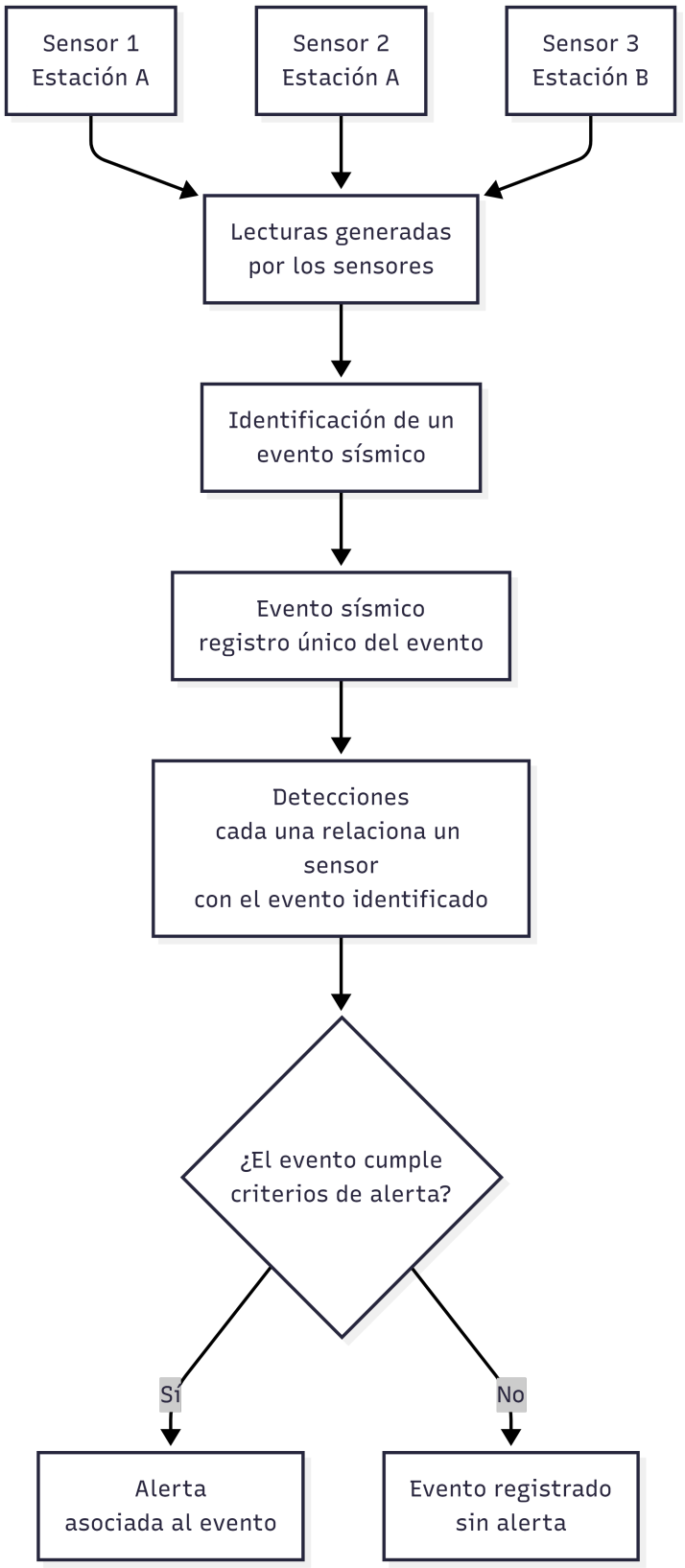


Figura 1: Flujo general de datos del sistema de monitoreo sismológico y alerta temprana.

El flujo descrito corresponde al proceso esperado del sistema, no a un mecanismo de detección automatizada ya implementado. Se distingue del manejo actual por archivos independientes. La siguiente tabla vincula las dificultades de dicho manejo con las funcio-

nalidades que debe resolver la base de datos.

Situación asumida actualmente	**Problema que debe resolver la base de datos**	**Requerimientos**
Estaciones y sensores en hojas de cálculo separadas.	Evitar referencias inconsistentes y duplicación de identificadores, conservando la instalación actual de cada sensor.	RF–01, RF–02
Lecturas almacenadas en archivos CSV independientes.	Recuperar registros por sensor y período, evitar duplicaciones y conservar la marca de validez sin eliminar mediciones.	RF–04, RF–05, RF–09
Eventos, detecciones y alertas conservados en reportes distintos.	Vincular los sensores participantes con eventos y alertas mediante identificadores consistentes, facilitando consultas históricas.	RF–06 a RF–10

Cuadro 1: Relación entre el estado actual asumido del manejo de datos y las funcionalidades previstas.

1.5. Descripción detallada del sistema

El sistema propuesto tiene como finalidad gestionar de forma centralizada la información generada durante el monitoreo sismológico. Para ello, integra los datos correspondientes a estaciones sísmicas, sensores, lecturas, eventos sísmicos, detecciones y alertas, manteniendo las relaciones necesarias entre estos elementos.

La información almacenada presenta diferentes características y frecuencias de actualización. Mientras que los datos de estaciones y sensores corresponden principalmente a información estructural y operativa, las lecturas son generadas de manera continua o periódica y constituyen el mayor volumen de registros. Por su parte, los eventos sísmicos, las detecciones y las alertas se incorporan cuando se identifican situaciones asociadas a la actividad sísmica monitoreada.

A partir de esta estructura, el sistema deberá permitir tanto operaciones de registro y actualización como consultas operativas, históricas y analíticas, manteniendo la consistencia e integridad de la información almacenada.

1.5.1. Objetos de información actuales

Para efectos del proyecto, se identifican como objetos de información aquellos elementos del dominio que deben ser registrados, relacionados o consultados durante el monitoreo sismológico y la gestión de alertas. Cada uno representa información con un significado específico dentro del sistema.

Objeto	Descripción
Estación sísmica	Punto físico de monitoreo ubicado en una determinada zona geográfica. Almacena información de identificación, ubicación, fecha de instalación y estado operativo.
Sensor	Dispositivo instalado en una estación sísmica y encargado de registrar el movimiento del suelo. Contiene información sobre su identificación, características técnicas, frecuencia de muestreo, fecha de instalación, fecha del último mantenimiento y estado operativo.
Lectura de onda	Medición individual generada por un sensor en un instante determinado. Permite conservar el valor registrado, la información temporal asociada a la medición y un indicador que señale si la lectura fue considerada válida después de su revisión.
Evento sísmico	Registro de un sismo identificado dentro del sistema. Contiene sus principales características, como fecha y hora de origen, ubicación del epicentro, profundidad, magnitud y referencia geográfica.
Detección	Asociación que representa la participación de un sensor específico en el registro de un evento sísmico determinado. Puede conservar información propia de dicha detección, como la hora de registro, la aceleración máxima observada y un indicador de calidad.
Alerta	Registro informativo generado a partir de un evento sísmico cuando corresponde. Incluye información como fecha y hora de emisión, nivel, mensaje, magnitud estimada y estado.

Estos objetos de información constituyen la base del diseño conceptual del sistema. En el modelo Entidad–Relación se determinará cuáles serán representados como entidades y cuáles mediante relaciones, junto con sus atributos, multiplicidades y restricciones de participación.

1.5.2. Características y funcionalidades esperadas

El sistema deberá permitir gestionar de manera centralizada la información relacionada con el monitoreo sismológico, manteniendo relaciones consistentes entre estaciones sísmicas, sensores, lecturas, eventos, detecciones y alertas.

Con base en las reglas semánticas de la Sección 2.1, los requerimientos funcionales se especifican de forma verificable a continuación. Los códigos RF permiten identificar qué operación debe satisfacer la implementación y qué restricciones deben comprobarse. La identificación de los actores se corresponde con los perfiles descritos en la siguiente sección.

Código	Requerimiento funcional y condición de cumplimiento	Reglas
RF–01	El administrador registra y actualiza estaciones con identificador y código no repetidos. Toda estación registrada debe quedar asociada por lo menos a un sensor, conforme al alcance del modelo.	1–3
RF–02	El administrador registra y actualiza sensores asociados a una sola estación, sin repetir identificador ni número de serie. Se conserva la instalación actual, sin historial de traslados.	3–5, 26
RF–03	El administrador registra los cambios de estado y la última fecha de mantenimiento de cada sensor. Esta fecha puede ser nula; cuando exista, debe ser igual o posterior a la fecha de instalación.	6–8, 25
RF–04	El proceso de captura incorpora lecturas con sensor existente, fecha y hora, canal, valor y validez. No se duplica la combinación de sensor, instante y canal.	9–11, 26
RF–05	El operador puede marcar lecturas como no válidas sin eliminarlas. La admisión de nuevas lecturas válidas para el análisis considera el estado operativo del sensor al momento de registrarlas.	12–13, 24
RF–06	El operador registra y actualiza eventos sísmicos con fecha y hora de origen, epicentro, profundidad y magnitud, respetando los dominios definidos.	14, 22–23
RF–07	El operador registra la detección de un evento por un sensor, con hora, aceleración máxima y calidad, sin repetir la pareja sensor–evento. Se admite que existan sensores sin detecciones y eventos sin detecciones.	15–19
RF–08	El operador registra las alertas emitidas, sus niveles, mensajes y estados. Cada alerta se asocia a un evento existente y un evento puede no generar alertas o generar varias.	20–21, 26
RF–09	Los usuarios autorizados pueden consultar estados, mantenimientos, lecturas por intervalo, eventos por identificador y alertas. El alcance de lectura o actualización depende del perfil de usuario.	25–26
RF–10	Los analistas pueden ejecutar consultas de detecciones por evento, conteo de eventos por estación, promedios de aceleración, sensores sin lecturas y actividad histórica por sensor.	9, 15–19

Condiciones de integridad y alcance. Las llaves primarias y foráneas, los dominios numéricos, la unicidad y las participaciones obligatorias se derivan de las reglas semánticas y del modelo relacional. La existencia de al menos un sensor por estación requiere control adicional durante el registro conjunto; una llave foránea en `SENSOR` no garantiza por sí sola esta condición. Los mecanismos concretos se definirán en la implementación de procedimientos y disparadores. El sistema conserva registros de alertas, pero no implementa

su difusión física o pública.

Frecuencia de operación. Estaciones y sensores se incorporan o modifican de manera ocasional; lecturas se cargan de manera continua o periódica; eventos, detecciones y alertas se registran cuando ocurre una identificación o emisión. Esta diferencia justifica la prioridad de carga y consulta sobre `LECTURA_ONDA` y `DETECCION` al realizar las pruebas de rendimiento.

1.5.3. Tipos de usuarios existentes/necesarios

El sistema contempla distintos perfiles de usuario de acuerdo con las operaciones que realizan sobre la información almacenada y el nivel de acceso que requieren. La diferenciación de estos perfiles permitirá posteriormente establecer permisos adecuados sobre las distintas operaciones de consulta, registro y actualización.

Administrador del sistema. Será responsable de gestionar la información estructural y operativa del sistema, incluyendo el registro y actualización de estaciones sísmicas, sensores y sus respectivos estados. Asimismo, podrá supervisar la consistencia general de los datos y realizar operaciones de mantenimiento administrativo.

Operador de monitoreo. Utilizará el sistema para consultar información relacionada con lecturas, detecciones, eventos sísmicos y alertas. También podrá registrar o actualizar información operativa cuando corresponda al proceso de monitoreo.

Analista o investigador. Accederá principalmente a información histórica con el objetivo de realizar análisis sobre los eventos registrados, comparar el comportamiento de estaciones o sensores y obtener resultados estadísticos a partir de los datos almacenados.

Usuario de consulta. Tendrá acceso principalmente de lectura a información de carácter informativo, como eventos sísmicos registrados, alertas emitidas y resultados generales disponibles en el sistema, sin capacidad para modificar la información estructural de la base de datos.

Además de los usuarios anteriores, el sistema considera la interacción de un **proceso automático de captura de datos**. Este no corresponde a un usuario humano, sino a un componente encargado de incorporar de manera continua o periódica las lecturas generadas por los sensores y, cuando corresponda, registrar información derivada del proceso de monitoreo.

La definición de estos perfiles permitirá establecer posteriormente diferentes niveles de acceso, evitando que todos los usuarios o procesos puedan realizar las mismas operaciones sobre la información almacenada.

1.5.4. Tipos de consulta, actualizaciones

La base de datos deberá permitir consultas operativas para recuperar información específica y consultas históricas o analíticas que relacionen varios objetos del sistema.

Entre las consultas operativas se consideran la búsqueda de un evento sísmico por su identificador, la consulta del estado actual de estaciones y sensores, la revisión de la fecha

del último mantenimiento de un sensor, la identificación de lecturas marcadas como no válidas y la recuperación de las lecturas generadas por un sensor durante un intervalo de tiempo.

Como consultas de mayor nivel se plantean las siguientes:

- Identificar los sensores y estaciones que participaron en la detección de un evento sísmico determinado.
- Determinar la cantidad de eventos sísmicos distintos detectados por cada estación durante un periodo determinado.
- Obtener, para cada estación, la aceleración máxima promedio registrada en las detecciones de eventos sísmicos.
- Identificar los sensores que no hayan generado lecturas durante un determinado número de días, con el fin de detectar posibles periodos de inactividad.
- Obtener, para cada sensor, la cantidad de eventos sísmicos en los que participó y la fecha de su detección más reciente.

Respecto a las actualizaciones, la información de estaciones y sensores tendrá cambios poco frecuentes, principalmente por instalación, mantenimiento o modificación de su estado operativo. Cuando se realice un mantenimiento preventivo o correctivo se actualizará la fecha del último mantenimiento del sensor. Las lecturas serán incorporadas de manera continua o periódica y constituirán el flujo de datos de mayor frecuencia; adicionalmente, podrán marcarse como válidas o no válidas cuando se detecten mediciones anómalas o errores instrumentales.

Los eventos sísmicos y sus detecciones se registrarán cuando se identifique un nuevo evento, y sus datos podrán actualizarse posteriormente si se dispone de información revisada. Las alertas se incorporarán cuando corresponda su emisión y podrán actualizarse de acuerdo con su estado.

En cuanto a las velocidades de respuesta esperadas, las consultas operativas simples, como la búsqueda de un evento sísmico por su identificador o la revisión del estado de una estación o sensor, deberán responder aproximadamente en un segundo, incluso cuando la base de datos alcance los mayores volúmenes estimados de lecturas y detecciones. Las consultas de mayor nivel, que combinan varias relaciones o recorren intervalos temporales amplios, podrán tolerar tiempos de respuesta de algunos segundos, debido a su carácter histórico o analítico y no estrictamente operativo. Estas expectativas de rendimiento serán evaluadas posteriormente mediante la experimentación con índices sobre las cinco consultas intermedias seleccionadas para el proyecto.

1.5.5. Tamaño estimado de la base de datos

El tamaño de la base de datos dependerá principalmente de la cantidad de estaciones y sensores administrados y, sobre todo, de la frecuencia con la que se registren las lecturas. Debido a que estas mediciones pueden generarse de manera continua o periódica, la relación `LECTURA_ONDA` será la de mayor crecimiento dentro del sistema.

Para efectos del proyecto se combinan datos reales y datos sintéticos. El catálogo público del IGP aporta los eventos sísmicos, mientras que las estaciones, sensores, lecturas, detecciones y alertas operativas se representan mediante registros simulados. Las bases experimentales también utilizan datos generados de forma controlada para estudiar el rendimiento.

Como referencia, se estima el siguiente volumen de información para el dominio planteado:

Objeto de información	**Cantidad estimada**
Estaciones sísmicas	100 – 500
Sensores	300 – 1 500
Eventos sísmicos	10 000 – 100 000
Detecciones	100 000 – 500 000
Alertas	1 000 – 20 000
Lecturas de onda	1 000 000 o más

Estas cantidades representan una proyección de crecimiento del sistema y no describen la distribución exacta de la base utilizada en los ensayos. De manera independiente, se prepararon cuatro conjuntos experimentales con 1 000, 10 000, 100 000 y 1 000 000 de registros, como se detalla en el Capítulo 5.

El mayor volumen de datos se concentrará en las lecturas y detecciones, por lo que las consultas relacionadas con estas relaciones serán especialmente relevantes para evaluar posteriormente el rendimiento de la base de datos.

1.6. Objetivos del proyecto

Objetivo general

Diseñar e implementar en PostgreSQL una base de datos relacional para gestionar la información de estaciones sísmicas, sensores, lecturas, eventos, detecciones y alertas, garantizando la coherencia de sus relaciones y restricciones de integridad.

Objetivos específicos

- Identificar los principales objetos de información, atributos y reglas semánticas presentes en el dominio del monitoreo sismológico.
- Diseñar un modelo Entidad–Relación que represente adecuadamente las entidades, relaciones, atributos llave, multiplicidades y restricciones de participación del sistema.
- Transformar el modelo conceptual en un modelo relacional normalizado, definiendo correctamente llaves primarias, llaves foráneas y restricciones de integridad.
- Definir los dominios y restricciones principales de los atributos mediante un diccionario de datos coherente con el modelo relacional.

- Implementar consultas y mecanismos de integridad en PostgreSQL y evaluar su rendimiento mediante conjuntos de datos de distinta escala.

1.7. Referencias del proyecto

El proyecto toma como referencia fuentes oficiales relacionadas con la actividad sísmica en el Perú, así como documentación técnica y bibliografía académica sobre diseño e implementación de bases de datos.

- **Instituto Geofísico del Perú (IGP) – Centro Sismológico Nacional (CENSIS).**

  Fuente de los datos de eventos sísmicos utilizados en la carga oficial: fecha, hora, ubicación geográfica, magnitud y profundidad.

  https://censis.igp.gob.pe/repositorio/datos-sismicos

- **Instituto Geofísico del Perú (IGP) – Sistema de Alerta Sísmica Peruano (SASPe).**

  Referencia conceptual para la organización de redes de monitoreo orientadas a la detección sísmica y a la alerta temprana.

  https://www.igp.gob.pe/servicios/saspe/estaciones.php

- **Instituto Geofísico del Perú (IGP) – ACELDAT-PERÚ.**

  Referencia técnica sobre estaciones, registros acelerométricos y parámetros de medición instrumental.

  https://www.igp.gob.pe/servicios/aceldat-peru/reportes-registros-acelerometricos

- **PostgreSQL Global Development Group – Documentación oficial de PostgreSQL.**

  Referencia técnica para la implementación de restricciones, consultas, índices y demás componentes utilizados en PostgreSQL.

  https://www.postgresql.org/docs/

- **Ramakrishnan, R. y Gehrke, J. (2003).** *Database Management Systems* (3.ª ed.). McGraw-Hill.

  Referencia teórica para el modelo Entidad–Relación, su transformación al modelo relacional y los principios de normalización e integridad de datos.

El catálogo oficial del IGP permitió incorporar 24 289 eventos sísmicos. La información operativa no disponible públicamente se representó mediante datos sintéticos, manteniendo explícita la diferencia entre ambas procedencias.

1.8. Eventualidades

Durante el desarrollo del proyecto pueden presentarse dificultades relacionadas con la obtención, calidad, integración y volumen de los datos. Asimismo, es necesario establecer límites claros para mantener el alcance del sistema dentro de los objetivos del curso.

1.8.1. Problemas que pudieran encontrarse en el proyecto

Durante el desarrollo del proyecto podrían presentarse las siguientes dificultades:

- **Disponibilidad limitada de datos reales:** las fuentes públicas pueden proporcionar información sobre eventos sísmicos, estaciones o registros instrumentales, pero no necesariamente todos los atributos y volúmenes requeridos por el modelo.
- **Heterogeneidad de formatos:** los datos obtenidos de distintas fuentes pueden encontrarse en formatos como XLSX, CSV o reportes web, por lo que será necesario realizar procesos de limpieza y transformación antes de incorporarlos a la base de datos.
- **Datos incompletos, inconsistentes, duplicados o erróneos:** algunos registros pueden contener valores faltantes, diferencias de formato, información repetida o mediciones anómalas producidas por ruido o fallas instrumentales. Por ello, será necesario establecer criterios de validación y conservar un indicador que permita distinguir las lecturas consideradas válidas de aquellas que requieren revisión.
- **Asociación entre mediciones y eventos:** puede resultar difícil determinar qué registros instrumentales corresponden a un evento sísmico determinado, especialmente cuando varios sensores generan lecturas de manera continua.
- **Gran volumen de información:** la generación frecuente de lecturas puede producir una cantidad elevada de registros y afectar el rendimiento de determinadas consultas, aspecto que deberá considerarse durante la implementación y experimentación.
- **Generación de datos sintéticos coherentes:** cuando no exista información real suficiente, será necesario generar datos simulados que respeten los dominios, restricciones y relaciones establecidas en el modelo.

Estas dificultades deberán considerarse durante la preparación, carga y validación de los datos con el fin de mantener su consistencia y utilidad dentro del sistema.

1.8.2. Límites y alcances del proyecto

El proyecto comprende el diseño y la implementación de una base de datos para administrar información relacionada con el monitoreo sismológico y la gestión de alertas tempranas. El sistema comprenderá información sobre estaciones sísmicas, sensores, lecturas

instrumentales, eventos sísmicos, detecciones y alertas, utilizando datos reales cuando estén disponibles y datos sintéticos cuando sea necesario completar o ampliar los conjuntos de prueba.

La base de datos permitirá almacenar, relacionar, consultar y actualizar dicha información, así como realizar consultas históricas y analíticas sobre los registros. Además, se evaluó con distintos volúmenes de datos durante las pruebas de rendimiento realizadas en PostgreSQL.

El proyecto no contempla el desarrollo de algoritmos científicos para predecir sismos, determinar automáticamente epicentros o magnitudes, modelar la propagación de ondas sísmicas ni realizar análisis geológicos especializados.

Asimismo, el sistema no controlará dispositivos físicos, sirenas, redes de telecomunicaciones ni mecanismos reales de difusión de alertas a la población. Las alertas serán representadas únicamente como información almacenada dentro de la base de datos.

Finalmente, el dominio se restringe al monitoreo sismológico y no contempla otros fenómenos naturales, como tsunamis, erupciones volcánicas, huaicos o inundaciones.

2. Modelo Entidad–Relación

2.1. Reglas semánticas

Las reglas semánticas establecen las condiciones que deben cumplirse entre los objetos de información del sistema y sirven como base para definir las multiplicidades, restricciones de participación y reglas de integridad del modelo Entidad–Relación.

Para el sistema de monitoreo sismológico se consideran las siguientes reglas:

1. Cada estación sísmica posee un identificador único y un código que no puede repetirse dentro del sistema.
2. Cada estación sísmica debe tener uno o varios sensores instalados.
3. Cada sensor debe pertenecer a una única estación sísmica y no puede existir en el sistema sin estar asociado a una de ellas.
4. La asociación entre sensor y estación representa la instalación actual del sensor; el sistema no conserva el historial de traslados de sensores entre estaciones.
5. Cada sensor posee un identificador y un número de serie únicos.
6. Cada sensor registra su fecha de instalación y puede registrar la fecha de su último mantenimiento. Si el sensor todavía no ha recibido mantenimiento, dicha fecha puede permanecer sin valor.
7. La fecha del último mantenimiento de un sensor, cuando exista, no puede ser anterior a su fecha de instalación.

8. Los mantenimientos preventivos se realizan de acuerdo con el plan de mantenimiento establecido por la organización o con las recomendaciones del fabricante. También pueden realizarse mantenimientos correctivos cuando se detecten fallas, comportamiento anómalo o problemas en las mediciones. Para el alcance del proyecto se conserva únicamente la fecha del último mantenimiento y no un historial completo de intervenciones.

9. Un sensor puede generar cero o múltiples lecturas a lo largo del tiempo.

10. Cada lectura de onda debe ser generada por un único sensor y no puede existir sin estar asociada a este.

11. Cada lectura debe registrar, como mínimo, la fecha y hora de medición, el canal, el valor obtenido y un indicador que establezca si la lectura es válida. La combinación de sensor, fecha y hora y canal de medición no puede repetirse.

12. Una lectura puede marcarse como no válida cuando presente valores anómalos, errores instrumentales o condiciones que impidan utilizarla de manera confiable en el análisis. Una lectura no válida se conserva en la base de datos para mantener la trazabilidad del registro.

13. Solo las lecturas consideradas válidas podrán utilizarse como sustento de las detecciones y análisis de eventos sísmicos.

14. Un evento sísmico posee un identificador único y almacena información sobre su fecha y hora de origen, ubicación del epicentro, profundidad y magnitud.

15. Un sensor puede participar en la detección de cero o múltiples eventos sísmicos.

16. Un evento sísmico puede estar asociado con cero o múltiples sensores mediante detecciones registradas.

17. Cada detección relaciona exactamente un sensor con un evento sísmico determinado.

18. La combinación de un sensor y un evento sísmico no puede repetirse dentro de las detecciones registradas.

19. Cada detección registra su hora de detección y puede almacenar información propia de dicha asociación, como la aceleración máxima y un indicador de calidad. Cuando se informa la aceleración máxima como magnitud, esta no puede ser negativa.

20. Un evento sísmico puede no generar ninguna alerta o puede estar asociado con múltiples alertas.

21. Cada alerta debe corresponder obligatoriamente a un único evento sísmico.

22. La latitud de una estación o de un epicentro debe encontrarse entre -90 y 90 grados, mientras que la longitud debe encontrarse entre -180 y 180 grados.

23. La profundidad de un evento sísmico no puede ser negativa y la frecuencia de muestreo de un sensor debe ser mayor que cero.

24. Los sensores que se encuentren en mantenimiento, en revisión o fuera de servicio no deberán aportar nuevas lecturas válidas para la detección de eventos hasta volver a un estado operativo. Un cambio posterior de estado no invalida retroactivamente las lecturas históricas.

25. La información de estaciones, sensores, eventos y alertas podrá actualizarse cuando cambie su estado o se disponga de información revisada.

26. No deberán existir sensores, lecturas, detecciones o alertas con referencias a registros inexistentes de las entidades de las que dependen.

Estas reglas serán utilizadas para definir las cardinalidades, participaciones y restricciones de integridad del modelo conceptual y posteriormente serán trasladadas, cuando corresponda, al modelo relacional.

2.2. Modelo Entidad–Relación

El modelo Entidad–Relación propuesto para el sistema de monitoreo sismológico y alerta temprana se presenta en la Figura 2. El diagrama representa las entidades principales del dominio, sus atributos, las relaciones existentes entre ellas y las restricciones de multiplicidad y participación definidas a partir de las reglas semánticas establecidas previamente.

Los atributos llave se muestran subrayados. Las flechas representan restricciones de clave y, en las relaciones uno a muchos, se dibujan desde la entidad del lado muchos hacia la relación. Las relaciones muchos a muchos no presentan flechas. La participación total de una entidad en una relación se representa mediante línea doble y la participación parcial mediante línea simple. Cuando una participación total coincide con una restricción de clave, las dos líneas paralelas terminan en una única punta de flecha, siguiendo la notación utilizada en clase.

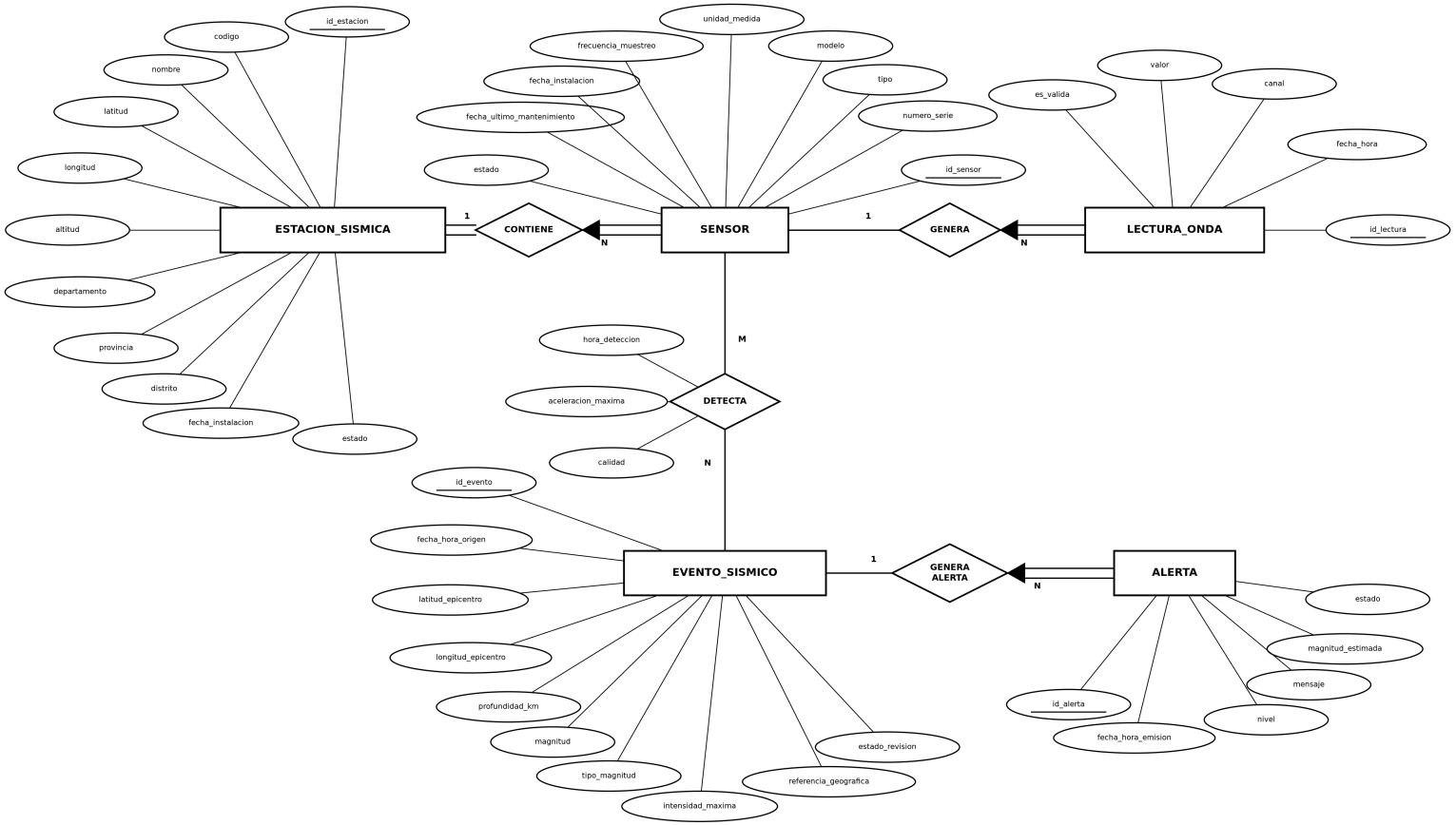


Figura 2: Modelo Entidad–Relación del sistema de monitoreo sismológico y alerta temprana. Las llaves aparecen subrayadas y se respetan las convenciones de participación y multiplicidad utilizadas en clase.

2.3. Especificaciones y consideraciones sobre el modelo

El modelo Entidad–Relación propuesto representa los principales elementos involucrados en el monitoreo sismológico y en la gestión de alertas tempranas. Se consideran como entidades principales `ESTACION_SISMICA`, `SENSOR`, `LECTURA_ONDA`, `EVENTO_SISMICO` y `ALERTA`. Además, se incluye la relación `DETECTA`, la cual posee atributos propios debido a que representa información específica de la participación de un sensor en la detección de un evento sísmico.

La entidad `ESTACION_SISMICA` representa cada punto físico de monitoreo y se identifica mediante el atributo llave `id_estacion`. La relación `CONTIENE` vincula una estación con los sensores instalados en ella y presenta una multiplicidad uno a muchos. Tanto `ESTACION_SISMICA` como `SENSOR` poseen participación total en esta relación, ya que toda estación registrada debe disponer de al menos un sensor y todo sensor debe estar asociado a una única estación. Para el alcance del proyecto, esta asociación representa la instalación actual del sensor y no se conserva el historial de traslados entre estaciones.

La entidad `SENSOR` se identifica mediante `id_sensor` y representa los dispositivos encargados de producir las mediciones. Entre sus atributos se incluye `fecha_ultimo_mantenimiento`, que permite conocer cuándo fue realizada la intervención más reciente sobre el dispositivo. Este atributo puede no tener valor cuando el sensor aún no ha recibido mantenimiento y, cuando exista, su fecha no puede ser anterior a la fecha de instalación. El sistema no mantiene un historial completo de mantenimientos, ya que para el alcance del proyecto se conserva únicamente la última fecha registrada. La relación `GENERA` vincula `SENSOR` con `LECTURA_ONDA` y presenta una multiplicidad uno a muchos. La participación de `SENSOR` es parcial, ya que un sensor puede existir sin haber generado lecturas, mientras que la participación de `LECTURA_ONDA` es total debido a que toda lectura debe haber sido producida por un único sensor.

`LECTURA_ONDA` representa las mediciones individuales obtenidas por los sensores y utiliza `id_lectura` como atributo llave. Además, incorpora el atributo `es_valida`, que permite distinguir las mediciones aceptadas para el análisis de aquellas que presentan anomalías o errores instrumentales. Las lecturas no válidas se conservan para mantener la trazabilidad de los datos, pero no se utilizan como sustento de una detección. La entidad se modela de forma independiente debido al elevado volumen de registros que puede generar el monitoreo continuo y a la necesidad de conservar las mediciones para consultas y análisis posteriores.

La entidad `EVENTO_SISMICO` representa cada sismo registrado dentro del sistema y se identifica mediante `id_evento`. Sus atributos almacenan información como la fecha y hora de origen, ubicación del epicentro, profundidad, magnitud y otros parámetros descriptivos del evento.

La relación `DETECTA` conecta `SENSOR` con `EVENTO_SISMICO` y posee multiplicidad muchos a muchos. Un sensor puede participar en la detección de múltiples eventos y un mismo evento puede encontrarse asociado a varios sensores. La participación es parcial en ambos lados, ya que un sensor puede no haber participado todavía en ninguna detección y un evento puede existir en la base de datos sin disponer de detecciones instrumentales asociadas.

`DETECTA` posee los atributos `hora_deteccion`, `aceleracion_maxima` y `calidad`, debido a que estos valores dependen específicamente de la asociación entre un sensor y un evento

determinado. En la transformación al modelo relacional, esta relación muchos a muchos dará origen a la relación DETECCION.

La entidad ALERTA representa las alertas registradas a partir de eventos sísmicos y se identifica mediante id_alerta. La relación GENERA_ALERTA, entre EVENTO_SISMICO y ALERTA, presenta multiplicidad uno a muchos. La participación de EVENTO_SISMICO es parcial, debido a que un evento puede no generar ninguna alerta, mientras que la participación de ALERTA es total, ya que toda alerta debe estar asociada obligatoriamente a un único evento.

El atributo magnitud_estimada de ALERTA se mantiene separado de magnitud en EVENTO_SISMICO, ya que representa el valor disponible al momento de emitir la alerta, mientras que la magnitud del evento puede ser revisada o actualizada posteriormente.

En cuanto a la notación utilizada, los atributos llave se encuentran subrayados. Las líneas dobles representan participación total y las líneas simples participación parcial. Las flechas representan restricciones de clave; en las relaciones uno a muchos se dibujan desde la entidad del lado muchos hacia la relación, indicando que cada instancia de dicha entidad puede asociarse, como máximo, con una instancia de la entidad ubicada en el lado uno. Cuando existe simultáneamente participación total y restricción de clave, se mantienen las dos líneas paralelas pero se utiliza una sola punta de flecha. Las relaciones muchos a muchos, como DETECTA, no presentan flechas.

Matriz de trazabilidad del modelo conceptual. En la tabla siguiente se vinculan las reglas semánticas con las multiplicidades y participaciones representadas. La expresión $a..b$ indica el mínimo y máximo de participaciones de una instancia de la entidad en la relación correspondiente.

Relación	Restricción conceptual	Reglas
CONTIENE	Una estación participa en $1..N$ asociaciones y un sensor en exactamente una. Participación total de ambas entidades; restricción de clave desde SENSOR.	2–4
GENERA	Un sensor participa en $0..N$ asociaciones y una lectura en exactamente una. Participación parcial de SENSOR y total de LECTURA_ONDA.	9–11
DETECTA	Un sensor participa en $0..N$ detecciones y un evento en $0..N$. Participación parcial de ambos extremos; sin flechas de restricción de clave.	15–19
GENERA_ALERTA	Un evento participa en $0..N$ asociaciones y una alerta en exactamente una. Participación parcial del evento y total de la alerta.	20–21

Correspondencia entre entidades, atributos y requerimientos. La siguiente tabla complementa la matriz de multiplicidades: muestra cómo los atributos y las restricciones del modelo responden a las funcionalidades especificadas en la Sección 1.5.2. La denominación de cada entidad y relación es la misma que se utiliza en el diagrama y en el modelo relacional.

Elemento del modelo	Atributos y restricciones relevantes	Requerimientos y reglas
ESTACION_SISMICA	id_estacion identifica cada estación; codigo es único. Latitud y longitud tienen los rangos geográficos indicados; la estación participa totalmente en CONTIENE.	RF–01; reglas 1–3 y 22.
SENSOR	id_sensor identifica cada sensor; numero_serie es único. La instalación y el mantenimiento siguen un orden temporal; la relación con su estación es obligatoria.	RF–02, RF–03; reglas 3–8 y 23–24.
LECTURA_ONDA	id_lectura identifica la lectura; sensor, instante y canal no pueden repetirse como combinación. es_valida conserva la distinción entre lecturas aptas y no aptas para análisis.	RF–04, RF–05; reglas 9–13 y 24.
EVENTO_SISMICO	id_evento identifica el evento; el epicentro respeta los rangos geográficos y la profundidad no puede ser negativa.	RF–06; reglas 14, 22–23.
DETECTA	La pareja sensor–evento no se repite; hora, aceleración máxima y calidad dependen de esa asociación. Al transformar esta relación se obtiene DETECCION.	RF–07, RF–10; reglas 15–19.
ALERTA	id_alerta identifica la alerta. Cada alerta pertenece a un evento; magnitud_estimada conserva el valor utilizado en la emisión, independientemente de revisiones posteriores.	RF–08, RF–09; reglas 20–21 y 25–26.

Alcance de las restricciones. La regla que exige al menos un sensor por estación constituye una participación total del lado de ESTACION_SISMICA, pero no queda garantizada por una llave foránea en SENSOR; su cumplimiento requiere una operación de alta controlada. Por otro lado, el modelo registra la relación entre sensor y evento en DETECTA, pero no registra cuáles lecturas individuales sustentaron esa detección. Por tanto, la regla que exige utilizar lecturas válidas es una condición del proceso de análisis/captura y no puede verificarse retrospectivamente a partir de una llave entre DETECCION y LECTURA_ONDA, pues esa referencia no existe en el modelo actual. Este límite se explicita para no atribuir al diagrama una restricción que no representa.

El modelo no contempla entidades débiles, relaciones ternarias ni jerarquías de generalización o especialización, ya que estas estructuras no son necesarias para representar el alcance definido para el sistema.

3. Modelo Relacional

3.1. Modelo Relacional

A partir del modelo Entidad–Relación definido previamente, se obtiene el siguiente conjunto de relaciones. Para cada atributo se indica su dominio conceptual utilizando los tipos generales trabajados en clase: `Int`, `String`, `Float`, `Bool` y `Date`. Los atributos que forman la llave primaria aparecen subrayados. Las llaves foráneas se indican mediante sus referencias y no se subrayan por el solo hecho de ser foráneas. En `DETECCION`, `id_sensor` e `id_evento` aparecen subrayados porque, además de ser llaves foráneas, forman conjuntamente la llave primaria compuesta.

El tipo conceptual `Date` representa información temporal; los atributos que contienen fecha y hora se implementarán como `TIMESTAMP` en PostgreSQL, mientras que las fechas sin componente horario se implementarán como `DATE`. De modo similar, el identificador de lecturas podrá utilizar `BIGINT` por el volumen previsto. Estas elecciones no modifican las entidades ni sus llaves.

`ESTACION_SISMICA(`<u>`id_estacion`</u>`:Int, codigo:String, nombre:String, latitud:Float, longitud:Float, altitud:Float, departamento:String, provincia:String, distrito:String, fecha_instalacion:Date, estado:String)`

- Llave primaria: `id_estacion`.
- Restricción de unicidad: `codigo`.

`SENSOR(`<u>`id_sensor`</u>`:Int, numero_serie:String, tipo:String, modelo:String, unidad_medida:String, frecuencia_muestreo:Float, fecha_instalacion:Date, fecha_ultimo_mantenimiento:Date, estado:String, id_estacion:Int)`

- Llave primaria: `id_sensor`.
- Llave foránea: `id_estacion` referencia a `ESTACION_SISMICA(id_estacion)`.
- Restricción de unicidad: `numero_serie`.
- `id_estacion` no puede ser nulo, debido a la participación total de `SENSOR` en la relación `CONTIENE`.
- `fecha_instalacion` no puede ser nula, porque constituye un dato requerido del sensor según la regla semántica 6.
- `fecha_ultimo_mantenimiento` puede no tener valor si el sensor todavía no ha recibido mantenimiento. Cuando exista, no puede ser anterior a `fecha_instalacion`.

`LECTURA_ONDA(`<u>`id_lectura`</u>`:Int, id_sensor:Int, fecha_hora:Date, canal:String, valor:Float, es_valida:Bool)`

- Llave primaria: `id_lectura`.
- Llave foránea: `id_sensor` referencia a `SENSOR(id_sensor)`.
- `id_sensor` no puede ser nulo, debido a la participación total de `LECTURA_ONDA` en la relación `GENERA`.
- La combinación `(id_sensor, fecha_hora, canal)` debe ser única para evitar almacenar dos veces una misma medición del mismo canal e instante.
- `es_valida` indica si la medición puede utilizarse en los análisis y detecciones del sistema.

```
EVENTO_SISMICO(id_evento:Int, fecha_hora_origen:Date,
latitud_epicentro:Float, longitud_epicentro:Float,
profundidad_km:Float, magnitud:Float, tipo_magnitud:String,
intensidad_maxima:String, referencia_geografica:String,
estado_revision:String)
```

- Llave primaria: `id_evento`.

```
DETECCION(id_sensor:Int, id_evento:Int,
hora_deteccion:Date, aceleracion_maxima:Float, calidad:String)
```

- Llave primaria compuesta: `(id_sensor, id_evento)`.
- Llave foránea: `id_sensor` referencia a `SENSOR(id_sensor)`.
- Llave foránea: `id_evento` referencia a `EVENTO_SISMICO(id_evento)`.

La relación `DETECCION` surge de la transformación de la relación muchos a muchos `DETECTA`. La llave primaria compuesta garantiza, de acuerdo con las reglas semánticas establecidas, que un mismo sensor no registre más de una detección para un mismo evento sísmico.

```
ALERTA(id_alerta:Int, id_evento:Int,
fecha_hora_emision:Date, nivel:String, mensaje:String,
magnitud_estimada:Float, estado:String)
```

- Llave primaria: `id_alerta`.
- Llave foránea: `id_evento` referencia a `EVENTO_SISMICO(id_evento)`.
- `id_evento` no puede ser nulo, debido a la participación total de `ALERTA` en la relación `GENERA_ALERTA`.

Las relaciones anteriores conservan las dependencias definidas en el modelo Entidad–Relación y evitan almacenar información derivable de otras relaciones. Por ejemplo, la estación asociada a una lectura puede obtenerse a través de `LECTURA_ONDA`, `SENSOR` y

ESTACION_SISMICA, por lo que no es necesario almacenar id_estacion dentro de cada lectura.

De manera similar, la unidad de medida de una lectura se obtiene a partir del sensor que la generó mediante SENSOR.unidad_medida, evitando repetir este valor en cada registro de LECTURA_ONDA.

La restricción de que toda estación registrada debe poseer al menos un sensor no puede garantizarse únicamente mediante la llave foránea almacenada en SENSOR. Esta condición será considerada durante la implementación mediante mecanismos adicionales de integridad o control de las operaciones de registro.

3.2. Especificaciones de transformación

La transformación del modelo Entidad–Relación al modelo relacional se realizó aplicando las reglas generales de conversión entre ambos modelos. Cada entidad fuerte se convirtió en una relación que conserva sus atributos y utiliza como llave primaria el atributo identificador definido en el modelo conceptual.

Las relaciones uno a muchos se transformaron incorporando la llave primaria de la entidad ubicada en el lado uno como llave foránea en la relación correspondiente al lado muchos. Cuando la participación de esta última es total, dicha llave foránea debe ser obligatoria.

La relación muchos a muchos DETECTA, existente entre SENSOR y EVENTO_SISMICO, se transformó en una nueva relación denominada DETECCION. Esta incorpora las llaves primarias de ambas entidades como llaves foráneas, utiliza su combinación como llave primaria compuesta y conserva los atributos propios de la relación.

La relación uno a muchos entre EVENTO_SISMICO y ALERTA se representó incorporando id_evento como llave foránea dentro de ALERTA.

El modelo no requiere transformaciones asociadas a entidades débiles, relaciones ternarias ni estructuras de generalización o especialización, ya que estos elementos no forman parte del modelo conceptual propuesto.

3.2.1. Entidades

Las entidades fuertes del modelo Entidad–Relación se transformaron directamente en relaciones del modelo relacional, conservando sus atributos y utilizando como llave primaria el atributo identificador definido en el modelo conceptual.

De esta manera, ESTACION_SISMICA, SENSOR, LECTURA_ONDA, EVENTO_SISMICO y ALERTA dieron origen a relaciones con el mismo nombre.

En los casos en que una entidad participa en el lado muchos de una relación uno a muchos, la relación resultante incorpora además la llave primaria de la entidad ubicada en el lado uno como llave foránea. Por ello, SENSOR incorpora id_estacion, LECTURA_ONDA incorpora id_sensor y ALERTA incorpora id_evento.

3.2.2. Entidades débiles

El modelo Entidad–Relación propuesto no contempla entidades débiles.

Todas las entidades principales poseen un atributo identificador propio que permite distinguir de manera única cada instancia sin depender de la llave de otra entidad para su identificación.

Aunque algunas entidades presentan dependencia de existencia respecto de otras, como `LECTURA_ONDA` respecto de `SENSOR` o `ALERTA` respecto de `EVENTO_SISMICO`, esta condición no implica que sean entidades débiles. En ambos casos, su identificación se realiza mediante `id_lectura` e `id_alerta`, respectivamente.

3.2.3. Entidades superclase/subclases

El modelo Entidad–Relación propuesto no contempla estructuras de generalización o especialización mediante superclases y subclases.

Aunque existen distintos tipos de sensores, como acelerómetros o sismómetros, sus diferencias se representan mediante el atributo `tipo` de `SENSOR`, ya que para el alcance del proyecto no requieren atributos o relaciones adicionales que justifiquen la creación de subclases independientes.

De manera similar, los distintos niveles o estados de una alerta se representan mediante atributos de `ALERTA`, sin necesidad de definir una jerarquía de especialización.

3.2.4. Relaciones binarias

El modelo Entidad–Relación contiene únicamente relaciones binarias, es decir, relaciones que vinculan dos entidades.

La relación `CONTIENE`, entre `ESTACION_SISMICA` y `SENSOR`, presenta una multiplicidad uno a muchos. Su transformación se realiza incorporando `id_estacion` como llave foránea en `SENSOR`. Debido a la participación total de `SENSOR`, esta referencia es obligatoria.

La relación `GENERA`, entre `SENSOR` y `LECTURA_ONDA`, también presenta una multiplicidad uno a muchos. Por ello, `id_sensor` se incorpora como llave foránea en `LECTURA_ONDA`. La participación total de `LECTURA_ONDA` implica que toda lectura debe estar asociada a un sensor.

La relación `DETECTA`, entre `SENSOR` y `EVENTO_SISMICO`, presenta una multiplicidad muchos a muchos y posee atributos propios. Por esta razón se transforma en la relación `DETECCION`, que contiene las llaves primarias de ambas entidades como llaves foráneas y utiliza su combinación como llave primaria compuesta.

Finalmente, la relación `GENERA_ALERTA`, entre `EVENTO_SISMICO` y `ALERTA`, presenta una multiplicidad uno a muchos. Su transformación consiste en incorporar `id_evento` como llave foránea en `ALERTA`. Debido a la participación total de `ALERTA`, esta referencia es obligatoria.

3.2.5. Relaciones ternarias

El modelo Entidad–Relación propuesto no contempla relaciones ternarias.

Todas las relaciones definidas vinculan únicamente dos entidades. En particular, no es necesario establecer una relación ternaria entre `ESTACION_SISMICA`, `SENSOR` y `EVENTO_SISMICO`, debido a que cada sensor ya se encuentra asociado a una única estación mediante la relación `CONTIENE`.

Por ello, la estación correspondiente a una detección puede obtenerse a partir del sensor involucrado, sin necesidad de almacenar nuevamente dicha información en la relación `DETECTA` o en su equivalente relacional `DETECCION`.

3.3. Diccionario de datos

El diccionario de datos describe los atributos que conforman cada relación del modelo relacional, indicando su tipo de dato, principales restricciones y significado dentro del sistema.

ESTACION_SISMICA

Atributo	Tipo	Restricciones	Descripción
`id_estacion`	INTEGER	PK, NOT NULL	Identificador único de la estación sísmica.
`codigo`	VARCHAR	UNIQUE, NOT NULL	Código único asignado a la estación.
`nombre`	VARCHAR	NOT NULL	Nombre de la estación sísmica.
`latitud`	DECIMAL	NOT NULL, entre -90 y 90	Coordenada geográfica de latitud.
`longitud`	DECIMAL	NOT NULL, entre -180 y 180	Coordenada geográfica de longitud.
`altitud`	DECIMAL	—	Altitud de la estación respecto al nivel del mar.
`departamento`	VARCHAR	—	Departamento donde se encuentra ubicada la estación.
`provincia`	VARCHAR	—	Provincia donde se encuentra ubicada la estación.
`distrito`	VARCHAR	—	Distrito donde se encuentra ubicada la estación.
`fecha_instalacion`	DATE	—	Fecha de instalación de la estación.

Atributo	Tipo	Restricciones	Descripción
estado	VARCHAR	NOT NULL	Estado operativo actual de la estación.

SENSOR

Atributo	Tipo	Restricciones	Descripción
id_sensor	INTEGER	PK, NOT NULL	Identificador único del sensor.
numero_serie	VARCHAR	UNIQUE, NOT NULL	Número de serie único del dispositivo.
tipo	VARCHAR	NOT NULL	Tipo de sensor utilizado para el monitoreo.
modelo	VARCHAR	—	Modelo del dispositivo.
unidad_medida	VARCHAR	NOT NULL	Unidad utilizada para expresar las mediciones generadas por el sensor.
frecuencia_muestreo	DECIMAL	NOT NULL, > 0	Frecuencia con la que el sensor genera mediciones.
fecha_instalacion	DATE	NOT NULL	Fecha de instalación del sensor en su estación actual.
fecha_ultimo_mantenimiento	DATE	Puede ser nulo; si existe, no puede ser anterior a fecha_instalacion	Fecha del mantenimiento más reciente realizado al sensor.
estado	VARCHAR	NOT NULL	Estado operativo actual del sensor.
id_estacion	INTEGER	FK, NOT NULL	Estación sísmica en la que se encuentra instalado actualmente el sensor.

id_estacion → ESTACION_SISMICA(id_estacion)

LECTURA_ONDA

Atributo	Tipo	Restricciones	Descripción
id_lectura	BIGINT	PK, NOT NULL	Identificador único de la lectura.

Atributo	Tipo	Restricciones	Descripción
id_sensor	INTEGER	FK, NOT NULL	Sensor que generó la lectura.
fecha_hora	TIMESTAMP	NOT NULL	Fecha y hora en la que fue registrada la medición.
canal	VARCHAR	NOT NULL	Canal o componente de medición correspondiente a la lectura.
valor	DECIMAL	NOT NULL	Valor numérico registrado por el sensor.
es_valida	BOOLEAN	NOT NULL	Indica si la lectura puede utilizarse en los análisis y detecciones del sistema.

id_sensor → SENSOR(id_sensor)

La combinación (id_sensor, fecha_hora, canal) debe ser única para evitar el registro duplicado de una misma medición. Se utiliza BIGINT para id_lectura debido al elevado volumen de registros que esta relación puede alcanzar.

La unidad correspondiente a una lectura se obtiene mediante el atributo unidad_medida del sensor que la generó, por lo que no se almacena nuevamente en LECTURA_ONDA.

EVENTO_SISMICO

Atributo	Tipo	Restricciones	Descripción
id_evento	INTEGER	PK, NOT NULL	Identificador único del evento sísmico.
fecha_hora_origen	TIMESTAMP	NOT NULL	Fecha y hora estimada de origen del evento.
latitud_epicentro	DECIMAL	NOT NULL, entre −90 y 90	Latitud estimada del epicentro.
longitud_epicentro	DECIMAL	NOT NULL, entre −180 y 180	Longitud estimada del epicentro.
profundidad_km	DECIMAL	NOT NULL, ≥ 0	Profundidad estimada del evento en kilómetros.
magnitud	DECIMAL	NOT NULL	Magnitud registrada para el evento.
tipo_magnitud	VARCHAR	—	Tipo o escala utilizada para expresar la magnitud.

Atributo	Tipo	Restricciones	Descripción
intensidad_maxima	VARCHAR	—	Intensidad máxima reportada para el evento.
referencia_geografica	VARCHAR	—	Descripción geográfica aproximada de la ubicación del evento.
estado_revision	VARCHAR	NOT NULL	Estado de revisión de la información del evento.

DETECCION

Atributo	Tipo	Restricciones	Descripción
id_sensor	INTEGER	PK, FK, NOT NULL	Sensor que participó en la detección.
id_evento	INTEGER	PK, FK, NOT NULL	Evento sísmico asociado a la detección.
hora_deteccion	TIMESTAMP	NOT NULL	Momento en que el sensor registró la detección del evento.
aceleracion_maxima	DECIMAL	Si existe, ≥ 0	Magnitud máxima de aceleración registrada durante la detección.
calidad	VARCHAR	—	Indicador de calidad asociado a la detección.

La llave primaria es compuesta:

`PK(id_sensor, id_evento)`

Las referencias son:

`id_sensor` → `SENSOR(id_sensor)`

`id_evento` → `EVENTO_SISMICO(id_evento)`

La llave compuesta garantiza que un mismo sensor registre como máximo una detección asociada a un mismo evento sísmico dentro del modelo planteado.

ALERTA

Atributo	Tipo	Restricciones	Descripción
id_alerta	INTEGER	PK, NOT NULL	Identificador único de la alerta.
id_evento	INTEGER	FK, NOT NULL	Evento sísmico asociado a la alerta.
fecha_hora_emision	TIMESTAMP	NOT NULL	Fecha y hora de emisión de la alerta.
nivel	VARCHAR	NOT NULL	Nivel asignado a la alerta.
mensaje	VARCHAR	NOT NULL	Contenido informativo de la alerta.
magnitud_estimada	DECIMAL	Puede ser nulo	Magnitud estimada disponible al momento de emitir la alerta.
estado	VARCHAR	NOT NULL	Estado actual de la alerta.

id_evento → EVENTO_SISMICO(id_evento)

La magnitud sísmica y la magnitud estimada no tienen en este diseño una restricción general de no negatividad: las escalas de magnitud pueden expresar valores negativos. En cambio, la profundidad y la aceleración máxima expresada como magnitud no pueden ser negativas. Los valores permitidos para los estados categóricos se precisarán en la implementación sin contradecir las reglas de operación del sensor.

Las restricciones descritas en este diccionario establecen los principales dominios y condiciones de integridad del modelo relacional. Durante la implementación en PostgreSQL se materializarán mediante tipos de datos específicos, restricciones `PRIMARY KEY`, `FOREIGN KEY`, `UNIQUE`, `NOT NULL` y `CHECK`, según corresponda.

4. Implementación de la base de datos

4.1. Creación de Tablas en PostgreSQL

La estructura física del sistema se implementó en PostgreSQL 18.6, dentro del esquema `monitoreo_sismologico`. Se conservaron las seis relaciones del modelo relacional de la Sección 3.1 y se definieron sus claves, restricciones de dominio y referencias entre tablas. La relación `DETECCION` mantiene la clave primaria compuesta `(id_sensor, id_evento)`, correspondiente a la relación muchos a muchos del modelo conceptual.

Los identificadores emplean `INTEGER`, con excepción de `id_lectura`, definido como `BIGINT` por el volumen de mediciones. Las claves simples utilizan `GENERATED BY DEFAULT AS IDENTITY`; los instantes se almacenan mediante `TIMESTAMP`, y los atributos cuantitativos mediante `NUMERIC`. La creación de las seis tablas precede a la incorporación de restricciones con `ALTER TABLE`.

Relación	Clave primaria	Integridad principal
`estacion_sismica`	`id_estacion`	Código único y coordenadas válidas.
`sensor`	`id_sensor`	Referencia a estación, serie única y frecuencia positiva.
`lectura_onda`	`id_lectura`	Referencia al sensor y combinación sensor–fecha–canal única.
`evento_sismico`	`id_evento`	Rango de coordenadas y profundidad no negativa.
`deteccion`	`(id_sensor, id_evento)`	Referencias a sensor y evento; aceleración no negativa si se registra.
`alerta`	`id_alerta`	Referencia al evento sísmico.

Cuadro 12: Claves y principales restricciones implementadas en PostgreSQL.

Definición de tablas y restricciones

El siguiente fragmento muestra la definición de una estación y el uso de `ALTER TABLE` para incorporar las restricciones que verifican sus coordenadas y la unicidad del código.

Bloque de código 1: Estructura y restricciones de estación (01_estructura.sql)

```
CREATE TABLE estacion_sismica (
    id_estacion INTEGER GENERATED BY DEFAULT AS IDENTITY,
    codigo VARCHAR(50) NOT NULL,
    nombre VARCHAR(150) NOT NULL,
    latitud NUMERIC NOT NULL,
    longitud NUMERIC NOT NULL,
    altitud NUMERIC,
    departamento VARCHAR(100),
    provincia VARCHAR(100),
    distrito VARCHAR(100),
    fecha_instalacion DATE,
    estado VARCHAR(40) NOT NULL
);

ALTER TABLE estacion_sismica
    ADD CONSTRAINT pk_estacion_sismica PRIMARY KEY (id_estacion);
ALTER TABLE estacion_sismica
```

```
    ADD CONSTRAINT uq_estacion_codigo UNIQUE (codigo);
ALTER TABLE estacion_sismica
    ADD CONSTRAINT ck_estacion_latitud CHECK (latitud BETWEEN -90 AND 90),
    ADD CONSTRAINT ck_estacion_longitud CHECK (longitud BETWEEN -180 AND 180);
```

En `DETECCION` se implementaron dos claves foráneas y una clave primaria compuesta, de modo que no se puede repetir la asociación entre el mismo sensor y evento sísmico.

Bloque de código 2: Clave compuesta y referencias de detección (01_estructura.sql)

```
CREATE TABLE deteccion (
    id_sensor INTEGER NOT NULL,
    id_evento INTEGER NOT NULL,
    hora_deteccion TIMESTAMP NOT NULL,
    aceleracion_maxima NUMERIC,
    calidad VARCHAR(40)
);

ALTER TABLE deteccion
    ADD CONSTRAINT pk_deteccion PRIMARY KEY (id_sensor, id_evento);
ALTER TABLE deteccion
    ADD CONSTRAINT fk_deteccion_sensor
    FOREIGN KEY (id_sensor) REFERENCES sensor(id_sensor),
    ADD CONSTRAINT fk_deteccion_evento
    FOREIGN KEY (id_evento) REFERENCES evento_sismico(id_evento);
ALTER TABLE deteccion
    ADD CONSTRAINT ck_deteccion_aceleracion
    CHECK (aceleracion_maxima IS NULL OR aceleracion_maxima >= 0);
```

Las definiciones restantes, con sus atributos y restricciones completas, se encuentran en el integrador `sql/01_estructura.sql` del repositorio del proyecto.

Verificación de la estructura y la integridad

La comprobación registrada con PostgreSQL identificó seis tablas y seis claves primarias, cinco foráneas, tres restricciones de unicidad y ocho de tipo `CHECK`. El catálogo registró además 36 restricciones de obligatoriedad asociadas a `NOT NULL`. El integrador incorporó pruebas de inserción para comprobar tanto la aceptación de registros válidos como el rechazo de operaciones que vulneraban las restricciones.

Prueba de integridad	Resultado
Latitud de estación fuera del intervalo permitido	Rechazada
Frecuencia de muestreo igual a cero	Rechazada
Duplicación de sensor, instante y canal de lectura	Rechazada
Alerta referida a un evento inexistente	Rechazada
Duplicación del par sensor–evento en detección	Rechazada

Cuadro 13: Rechazos de integridad comprobados durante la validación de la estructura.

Estas inserciones se realizaron en una transacción de comprobación finalizada con `ROLLBACK`, por lo que los datos de prueba no permanecieron almacenados.

Alcance de las restricciones

Las claves foráneas impiden, por ejemplo, registrar un sensor sin estación, pero no garantizan que toda estación cuente con al menos un sensor. Esta última regla requiere un mecanismo adicional de registro o validación. Asimismo, impedir nuevas lecturas válidas de sensores inoperativos depende del estado de otra relación y se aborda como regla de negocio en la Sección 4.6, no mediante una restricción `CHECK` sobre `LECTURA_ONDA`. El modelo tampoco registra una referencia directa de cada detección hacia las lecturas individuales que la sustentan.

Para la experimentación del Capítulo 5 se diferencian los índices necesarios para `PRIMARY KEY` y `UNIQUE` de los índices secundarios candidatos a optimizar las consultas.

4.2. Carga de datos

La carga de datos se realizó mediante el integrador `sql/02_datos.sql`, ejecutado con `psql` sobre las seis relaciones creadas en PostgreSQL. El procedimiento verifica previamente que las tablas estén vacías y efectúa, dentro de una misma transacción, la carga inicial, la importación del catálogo sísmico del Instituto Geofísico del Perú (IGP) y la simulación complementaria descrita en la Sección 4.3. La confirmación mediante `COMMIT` ocurre al finalizar las tres etapas y sus comprobaciones.

Carga inicial de datos sintéticos

La primera etapa incorporó registros definidos en el integrador mediante `INSERT INTO`, respetando las dependencias entre estaciones, sensores, lecturas, eventos, detecciones y alertas. Esta carga proporciona casos de prueba de las relaciones y restricciones, sin representar operaciones de una red sísmica real.

Relación	Esperados	Verificados
ESTACION_SISMICA	5	5
SENSOR	10	10
LECTURA_ONDA	190	190
EVENTO_SISMICO	8	8
DETECCION	14	14
ALERTA	5	5
Total	**232**	**232**

Cuadro 14: Recuentos verificados de la carga sintética inicial.

De las 190 lecturas insertadas, 166 se registraron como válidas y 24 como inválidas. Los controles del integrador no detectaron estaciones sin sensores, lecturas huérfanas, detecciones sin lectura válida asociada ni lecturas válidas provenientes de sensores no operativos. Estos resultados corresponden al conjunto cargado y no sustituyen las restricciones permanentes descritas en las Secciones 4.1, 4.5 y 4.6.

Importación del catálogo sísmico del IGP

La fuente externa es el recurso *Dataset del Catálogo Sísmico desde 1960 al 2025*, publicado por el IGP en la Plataforma Nacional de Datos Abiertos[1]. El archivo contiene 24 289 eventos en ocho columnas, con separador punto y coma. Para reproducir la importación, el CSV de ese recurso se ubica como `sql/datos/igp_catalogo.csv`, según las instrucciones de preparación del repositorio.

El integrador emplea `\copy` de `psql` para cargar las ocho columnas como texto en una tabla temporal, `igp_catalogo_42`. Antes de su inserción se comprueban los campos obligatorios, los identificadores repetidos, los formatos de fecha y hora y los rangos de coordenadas y profundidad. La tabla intermedia se elimina al confirmar la transacción mediante `ON COMMIT DROP`.

Bloque de código 3: Carga del catálogo IGP (02_datos.sql)

```
\copy igp_catalogo_42 (id, fecha_utc, hora_utc, latitud, longitud, profundidad, magnitud,
    fecha_corte) FROM 'datos/igp_catalogo.csv' WITH (FORMAT csv, HEADER true, DELIMITER ';',
    ENCODING 'UTF8')
```

Los campos `FECHA_UTC` y `HORA_UTC` se combinan en `fecha_hora_origen`; las horas se completan a seis dígitos para preservar los ceros iniciales. La columna se define como `TIMESTAMP` sin zona horaria, por lo que se conserva la interpretación UTC de la fuente. Las coordenadas, la profundidad y la magnitud se convierten a tipos numéricos.

El identificador interno se construye como `200000000 + ID` del catálogo, separándolo de los ocho eventos sintéticos originales. Los campos no disponibles en el CSV (`tipo_magnitud`, `intensidad_maxima` y `referencia_geografica`) permanecen en `NULL`. El valor `pendiente_revision` es un estado administrativo del sistema y no un atributo proporcionado por el IGP.

Comprobación	Resultado
Filas del CSV incorporadas a la tabla temporal	24 289
Eventos del IGP insertados en `EVENTO_SISMICO`	24 289
Eventos sintéticos de la etapa inicial conservados	8
Eventos acumulados antes de la simulación complementaria	24 297
Eventos importados con coordenadas o profundidad fuera de rango	0
Eventos con atributos ausentes rellenados artificialmente	0

Cuadro 15: Controles de la importación del catálogo sísmico del IGP.

La salida de la ejecución registró `COPY 24289` y `INSERT 0 24289`. En el conjunto importado, el primer evento corresponde al 13 de enero de 1960 a las 15:40:34 UTC y el último al 31 de diciembre de 2025 a las 20:34:41 UTC; las magnitudes registradas van de 3 a 8,4. Estos valores describen la versión del catálogo empleada en las pruebas.

[1]Recurso utilizado: https://www.datosabiertos.gob.pe/dataset/catalogo-sismico/resource/f9629081-0721-4101-a2c2-d0622e046f8b.

Estado de la carga antes de la simulación complementaria

Al finalizar estas dos etapas, los controles internos de la transacción registraron **24 521 filas**: 232 del lote sintético inicial y 24 289 eventos del IGP. Ninguna detección ni alerta sintética fue asociada a los eventos oficiales durante esta importación. La cifra corresponde a un estado intermedio de la ejecución; la transacción se confirma después de la simulación complementaria de la Sección 4.3.

4.3. Simulación de datos faltantes

Procedencia y propósito de la simulación

El catálogo del Instituto Geofísico del Perú (IGP) aporta información sobre los eventos sísmicos, pero no incluye las estaciones, los sensores, las lecturas instrumentales, las detecciones ni las alertas requeridas para evaluar las operaciones del sistema. Por ello, se incorporó un lote **sintético y reproducible** que complementa los registros de la Sección 4.2.

Las estaciones simuladas se identifican mediante códigos `SIM43-EST-`, los sensores mediante `SIM43-SEN-` y los demás registros mediante rangos de identificadores reservados. Los 24 289 eventos importados del IGP permanecen diferenciados: no se les asocian detecciones ni alertas ficticias, ni se completan arbitrariamente atributos no publicados por la fuente.

Generación de los datos

La tercera etapa del integrador `sql/02_datos.sql` emplea `generate_series` y expresiones deterministas de PostgreSQL. No requiere bibliotecas externas ni valores aleatorios. El siguiente fragmento ilustra la generación de las diez estaciones adicionales:

Bloque de código 4: Estaciones simuladas mediante generate_series (02_datos.sql)

```sql
INSERT INTO estacion_sismica (
    id_estacion, codigo, nombre, latitud, longitud, altitud,
    departamento, provincia, distrito, fecha_instalacion, estado
)
SELECT
    820300001 + g,
    'SIM43-EST-' || LPAD((g + 1)::TEXT, 2, '0'),
    'Estacion simulada de prueba ' || LPAD((g + 1)::TEXT, 2, '0'),
    -5.5::NUMERIC - g * 1.1::NUMERIC,
    -78.4::NUMERIC + g * 0.21::NUMERIC,
    100::NUMERIC + g * 85::NUMERIC,
    NULL, NULL, NULL, DATE '2024-01-01', 'operativa'
FROM generate_series(0, 9) AS serie(g);
```

Cada estación adicional recibe tres sensores operativos. Las 1 000 lecturas se generan para esos treinta sensores, con fechas y canales calculados de forma determinista; 947 se marcan como válidas y 53 como inválidas. Los valores de la señal pueden ser negativos, porque representan mediciones con signo. Por separado, se generan treinta eventos sintéticos, noventa detecciones (tres por evento) y veinticuatro alertas. Las detecciones corresponden a lecturas válidas del sensor en el mismo instante, y las alertas se emiten después del origen

de los eventos. Se conservan eventos sin alerta, de acuerdo con la participación opcional prevista en el modelo.

Relación	Antes	Añadidos	Después
ESTACION_SISMICA	5	10	15
SENSOR	10	30	40
LECTURA_ONDA	190	1 000	1 190
EVENTO_SISMICO	24 297	30	24 327
DETECCION	14	90	104
ALERTA	5	24	29
Total	**24 521**	**1 184**	**25 705**

Cuadro 16: Balance de registros antes y después de la simulación complementaria.

Controles de integridad y confirmación

Antes de insertar los datos, el integrador comprueba las cantidades de la carga inicial y del catálogo IGP, además de verificar que los identificadores reservados estén libres. Las tres etapas de carga pertenecen a una **única transacción**, que se confirma al final de `02_datos.sql` después de las validaciones. Ante un error que impida confirmar esa transacción, los cambios no confirmados pueden revertirse.

El integrador comprueba los 1 184 registros nuevos, la conservación de los 24 289 eventos oficiales y el total final de 25 705 filas. Los controles de consistencia del lote complementario registraron:

Control	Incidencias
Estaciones simuladas sin sensor	0
Lecturas simuladas sin sensor	0
Lecturas válidas de sensores no operativos	0
Detecciones sin lectura válida correspondiente	0
Alertas anteriores a su evento	0

Cuadro 17: Controles de integridad aplicados al lote sintético complementario.

Las fechas de las nuevas lecturas abarcan desde `2026-08-01 00:00:00` hasta `2026-08-30 00:00:29`. La composición final de la base se verifica mediante:

$$N_{\text{final}} = N_{\text{inicial}} + N_{\text{IGP}} + N_{\text{complementario}} = 232 + 24\,289 + 1\,184 = 25\,705.$$

Estos controles respaldan la coherencia del *conjunto generado*, pero no establecen por sí solos todas las reglas de negocio como restricciones permanentes. En particular, la relación entre una detección y una lectura válida se verifica por sensor e instante, sin una clave foránea directa hacia `LECTURA_ONDA`. Los mecanismos operativos adicionales se presentan en las Secciones 4.5 y 4.6. La simulación de este apartado es diferente de las bases experimentales utilizadas para medir rendimiento en el Capítulo 5.

4.4. Vistas (Views)

Diseño y finalidad

Se implementaron cinco vistas sobre las seis relaciones base mediante `CREATE OR REPLACE VIEW`. Su finalidad es facilitar la recuperación de información de estaciones, sensores, eventos y alertas, así como la elaboración de resúmenes del catálogo sísmico del Instituto Geofísico del Perú (IGP). Las vistas son convencionales: sus resultados se calculan al consultarlas y no requieren almacenar nuevas copias de los registros.

Vista	Información proporcionada
`vw_sensores_actividad`	Actividad, estado y lecturas válidas e inválidas por sensor.
`vw_estaciones_resumen`	Cantidad de sensores y lecturas acumuladas por estación.
`vw_eventos_seguimiento`	Procedencia, detecciones y alertas asociadas a cada evento sísmico.
`vw_alertas_detalle`	Información de cada alerta y tiempo transcurrido desde el origen del evento.
`vw_igp_resumen_mensual`	Recuento mensual y estadísticas de magnitud y profundidad de los eventos del IGP.

Cuadro 18: Vistas implementadas en PostgreSQL y su finalidad.

Las vistas de sensores y estaciones conservan, mediante `LEFT JOIN`, los registros sin lecturas o sin sensores asociados. En el seguimiento de eventos, las detecciones y las alertas se agrupan por separado antes de unir sus recuentos con cada evento; esto impide que una combinación de múltiples detecciones y alertas multiplique artificialmente los resultados. La procedencia oficial o simulada se identifica mediante los rangos de identificadores definidos durante la carga de datos, una convención del proyecto y no un atributo original del catálogo del IGP.

Implementación SQL

Las cinco definiciones se encuentran en el integrador `sql/03_logica_consultas.sql`. Se presenta el fragmento de `vw_eventos_seguimiento` donde se realizan las agregaciones por separado. Su definición completa, con la clasificación de procedencia y los atributos del evento, se conserva en el integrador.

Bloque de código 5: Agregaciones independientes de eventos (03_logica_consultas.sql)

```
LEFT JOIN (
    SELECT id_evento, COUNT(*) AS total_detecciones
    FROM deteccion
    GROUP BY id_evento
) d ON d.id_evento = ev.id_evento
LEFT JOIN (
    SELECT id_evento, COUNT(*) AS total_alertas
    FROM alerta
    GROUP BY id_evento
) a ON a.id_evento = ev.id_evento;
```

La vista `vw_igp_resumen_mensual` utiliza `DATE_TRUNC('month', fecha_hora_origen)` para agrupar los eventos y `FILTER` para contar los de magnitud mayor o igual que seis. Las vistas no garantizan un orden de presentación: las consultas que requieren un orden específico emplean `ORDER BY`.

Comprobación de resultados

El integrador verifica la existencia de las cinco vistas y compara su contenido agregado con las tablas base de `bd_sismologico`. Las verificaciones contemplan el recuento de registros, la integridad de las agregaciones, la procedencia de los eventos y el intervalo temporal del catálogo.

Vista	**Esperadas**	**Obtenidas**
`vw_sensores_actividad`	40	40
`vw_estaciones_resumen`	15	15
`vw_eventos_seguimiento`	24 327	24 327
`vw_alertas_detalle`	29	29
`vw_igp_resumen_mensual` (meses con eventos)	—	771

Cuadro 19: Cardinalidades de las vistas en la base operativa.

Comprobación de consistencia	**Resultado**
Sensores contabilizados por estación	40
Lecturas agregadas por estación	1 190
Lecturas válidas / inválidas	1 113 / 77
Detecciones / alertas contabilizadas por evento	104 / 29
Eventos del IGP / eventos simulados	24 289 / 38
Detecciones / alertas atribuidas a eventos del IGP	0 / 0
Eventos del IGP incluidos en el resumen mensual	24 289
Total de registros en las seis relaciones base	25 705

Cuadro 20: Totales derivados y controles de consistencia de las vistas.

El resumen mensual registró **771 meses con eventos** entre enero de 1960 y diciembre de 2025. La cifra de meses corresponde a la ejecución documentada: la validación SQL exige que se conserven los 24 289 eventos del IGP y los extremos del intervalo temporal, pero no establece una cantidad fija de meses con eventos. El detalle de alertas presenta el tiempo transcurrido desde el origen del evento como un valor de tipo `INTERVAL`, no como un número de segundos.

Las comprobaciones estrictas del integrador generan una excepción si difieren las cantidades esenciales, los recuentos derivados o la clasificación de procedencia. Los resultados documentados corresponden a la base operativa y permiten comprobar la coherencia de las vistas; no representan mediciones de rendimiento ni sustituyen las pruebas experimentales del Capítulo 5.

4.5. Procedimientos almacenados

Se implementaron seis procedimientos en PL/pgSQL para registrar estaciones, sensores, eventos, lecturas, detecciones y alertas, así como para actualizar el estado de los sensores. Se invocan mediante CALL; cada definición establece el esquema de búsqueda y opera con los permisos del usuario invocador. Sus operaciones complementan las restricciones declaradas en las tablas.

Procedimiento	Función y validación principal
sp_registrar_estacion_sensor	Inserta conjuntamente una estación y su primer sensor.
sp_registrar_evento	Registra un evento sin asignar valores a atributos opcionales no proporcionados.
sp_registrar_lectura	Verifica con FOR SHARE el estado del sensor antes de incorporar una lectura válida.
sp_registrar_deteccion	Exige una lectura válida del mismo sensor y hora, posterior o simultánea al origen del evento.
sp_emitir_alerta	Exige que la emisión no anteceda al origen; no impone un mínimo de detecciones.
sp_actualizar_estado_sensor	Actualiza el estado y mantenimiento; la restricción CHECK impide fechas anteriores a la instalación.

Cuadro 21: Procedimientos almacenados y reglas verificadas.

Implementación y controles

Los procedimientos de registro efectúan sus modificaciones en la transacción del cliente: ninguno ejecuta COMMIT ni ROLLBACK internamente. La operación conjunta de estación y sensor evita un alta incompleta cuando se utiliza esa interfaz. La participación mínima también queda protegida ante cambios directos por los disparadores de restricción diferidos descritos en la Sección 4.6.

El registro de lecturas comprueba si el sensor está operativo, sin invalidar las lecturas históricas al cambiar su estado. Para registrar una detección, se valida la fecha de origen del evento y la existencia de una lectura válida del mismo sensor en la hora indicada. La relación entre detección y lectura no es una clave foránea: estas validaciones se aplican en el procedimiento y, para las operaciones directas, mediante los disparadores que protegen el sustento de las detecciones. El siguiente fragmento pertenece a sp_registrar_deteccion:

Bloque de código 6: Validación de la lectura que sustenta una detección

```
PERFORM 1
FROM lectura_onda
WHERE id_sensor = p_id_sensor
  AND fecha_hora = p_hora_deteccion
  AND es_valida IS TRUE
FOR SHARE;
```

Las seis definiciones completas, incluidas sus firmas y controles, se encuentran en `sql/03_logica_consultas.sql`. El integrador las instala en una transacción y verifica su presencia en `pg_proc` antes de confirmar los cambios.

Pruebas y reversión

Sobre `bd_sismologico` se ejecutó una transacción de prueba con los 25 705 registros de partida. Se probaron las seis interfaces mediante seis llamadas iniciales y una llamada adicional válida para registrar una lectura inválida mientras el sensor estaba en mantenimiento. También se comprobaron los siguientes cinco rechazos:

Condición probada	**Control responsable**
Lectura nueva válida con sensor en mantenimiento	Procedimiento
Detección sin lectura válida a la hora indicada	Procedimiento
Segunda detección del mismo par sensor–evento	`PRIMARY KEY`
Alerta con fecha anterior al origen del evento	Procedimiento
Mantenimiento fechado antes de instalar el sensor	`CHECK`

Cuadro 22: Cinco rechazos verificados en las pruebas de procedimientos.

Antes de ejecutar `ROLLBACK` se observaron una estación, un sensor, un evento, dos lecturas, una detección y una alerta temporales. Tras la reversión, los seis controles retornaron cero registros para los identificadores reservados. La prueba no modificó los 25 705 registros permanentes. Los disparadores que hacen cumplir las reglas frente a modificaciones directas se detallan en la sección siguiente.

4.6. Triggers

Reglas automatizadas

Los disparadores (*triggers*) complementan las restricciones del esquema y los procedimientos almacenados descritos en la Sección 4.5. Su finalidad es controlar las modificaciones realizadas directamente sobre las tablas, incluso cuando no se utilizan las interfaces `CALL`. El integrador `sql/03_logica_consultas.sql` define **siete funciones PL/pgSQL y nueve disparadores**; seis efectúan comprobaciones inmediatas y tres verifican, de forma diferida, la participación mínima de las estaciones.

Disparador	Control	Regla implementada
`trg_lectura_operativa`	`BEFORE`	Rechaza nuevas lecturas válidas si su sensor no está operativo.
`trg_proteger_sustento_lectura`	`BEFORE`	Impide eliminar, invalidar o reasignar la última lectura válida que sustenta una detección.
`trg_deteccion_valida`	`BEFORE`	Exige lectura válida del mismo sensor y hora, y detección posterior o igual al origen.
`trg_alerta_temporal`	`BEFORE`	Impide alertas emitidas antes del origen del evento.
`trg_revisar_origen_evento`	`BEFORE`	Impide cambiar el origen a una hora posterior a detecciones o alertas asociadas.
`trg_bloquear_estacion_origen`	`BEFORE`	Bloquea la estación de origen al trasladar o eliminar sensores, para coordinar cambios concurrentes.
`ct_estacion_sensor_alta`	Diferido	Comprueba que una estación recién insertada tenga al menos un sensor.
`ct_estacion_sensor_baja`	Diferido	Comprueba que la baja de un sensor no deje una estación vacía.
`ct_estacion_sensor_traslado`	Diferido	Comprueba que el traslado no deje sin sensores a la estación de origen.

Cuadro 23: Disparadores y controles de integridad implementados.

Comprobaciones inmediatas y diferidas

Los seis disparadores `BEFORE` actúan sobre las operaciones `INSERT`, `UPDATE` o `DELETE` especificadas en cada definición. Una lectura válida debe proceder de un sensor operativo al registrarse o modificarse; cambiar después su estado a `mantenimiento` no invalida las mediciones históricas. Las detecciones requieren una lectura válida del mismo sensor y fecha-hora. Esta correspondencia se verifica mediante consultas, no mediante una clave foránea a `LECTURA_ONDA`, pues `DETECCION` no almacena `id_lectura`. Si existen varias lecturas válidas de distintos canales en el mismo instante, la protección preserva al menos un sustento, sin identificar un canal científico único.

La participación mínima de estaciones se verifica mediante tres `CONSTRAINT TRIGGER AFTER` con `DEFERRABLE INITIALLY DEFERRED`. De este modo puede registrarse una estación y su primer sensor dentro de una misma transacción; la comprobación ocurre al

confirmarla. Durante las pruebas se emplea `SET CONSTRAINTS ALL IMMEDIATE` para forzar la evaluación y capturar los rechazos esperados.

Bloque de código 7: Ejemplos de instalación de disparadores

```
CREATE OR REPLACE TRIGGER trg_deteccion_valida
BEFORE INSERT OR UPDATE OF id_sensor, id_evento, hora_deteccion
ON deteccion
FOR EACH ROW
EXECUTE FUNCTION fn_trg_deteccion_valida();

CREATE CONSTRAINT TRIGGER ct_estacion_sensor_alta
AFTER INSERT ON estacion_sismica
DEFERRABLE INITIALLY DEFERRED
FOR EACH ROW
EXECUTE FUNCTION fn_trg_estacion_con_sensor();
```

Pruebas de ejecución y reversión

La instalación registrada en PostgreSQL 18.6 confirmó los nueve disparadores mediante `COMMIT`. En una transacción de prueba independiente se insertaron dos estaciones, dos sensores, un evento, dos lecturas, una detección y una alerta; se cambió un sensor a `mantenimiento` y se verificó que su lectura histórica válida permaneciera disponible.

Se comprobaron **nueve rechazos**: lectura válida de un sensor no operativo; detección sin lectura válida; alerta anterior al origen; invalidación y eliminación de la última lectura de sustento; cambio de origen incompatible; y alta, baja o traslado que dejarían una estación sin sensores. Las excepciones se capturaron en bloques de prueba independientes para continuar la verificación sin confirmar cambios inválidos.

Comprobación	Esperado	Observado
Disparadores instalados	9	9
Pruebas negativas reconocidas	9	9
Estaciones y sensores temporales	2 y 2	2 y 2
Eventos, lecturas, detecciones y alertas temporales	1, 2, 1 y 1	1, 2, 1 y 1
Registros temporales restantes tras `ROLLBACK`	0	0
Registros permanentes de las seis tablas	25 705	25 705

Cuadro 24: Resultados documentados de las pruebas de disparadores.

Tras ejecutar `ROLLBACK`, las seis verificaciones de los identificadores reservados devolvieron cero registros y el total de la base permaneció en **25 705 filas**. El control completo del integrador incluye las consultas y los mensajes de conformidad.

Alcance. Estas reglas operan sobre modificaciones ordinarias de filas y no equivalen a una llave foránea entre una detección y una lectura específica. No cubren operaciones administrativas como deshabilitar disparadores o emplear `TRUNCATE`; las pruebas de rendimiento se examinan por separado en el Capítulo 5.

5. Optimización y experimentación

Se evaluó el rendimiento de cinco consultas relevantes para el sistema de monitoreo sismológico mediante bases experimentales de diferentes tamaños. La preparación de los datos, la evaluación de índices candidatos y las mediciones se implementaron en los integradores `04_bases_experimentales.sql`, `05_planes_indices.sql` y `06_mediciones.sql`.

Los experimentos combinaron registros del catálogo sísmico oficial del Instituto Geofísico del Perú (IGP) con datos sintéticos de estaciones y operaciones de monitoreo. Las procedencias se distinguen a través de los identificadores utilizados en la carga. Cada consulta se comparó con y sin índices secundarios candidatos, conservando los mismos filtros y resultados dentro de una escala; el diseño del experimento y las mediciones se detallan en las siguientes secciones.

5.1. Consultas SQL para el experimento

5.1.1. Descripción del tipo de consultas seleccionadas

La selección considera la utilidad de la información para el monitoreo, la complejidad de las operaciones SQL y la posibilidad de comparar una misma sentencia en las cuatro escalas experimentales. Se eligieron cinco consultas de nivel intermedio que combinan filtros temporales, agrupaciones, uniones entre relaciones, agregaciones, consultas correlacionadas y cálculos de intervalos. Los filtros, parámetros y límites de salida se fijan de antemano para no cambiar el problema consultado entre escenarios.

N.º	Necesidad de información	Operaciones principales
1	Evolución anual de eventos del IGP con magnitud mayor o igual a seis, entre 2010 y 2025.	`WHERE`, `GROUP BY`, `HAVING`, `COUNT`, `AVG`, `MAX`.
2	Conteo de epicentros del IGP que caen en una ventana rectangular alrededor de cada estación.	`JOIN` con condiciones geográficas, filtros de magnitud y tiempo, agregación y ordenamiento.
3	Porcentaje de lecturas inválidas y cobertura de sensores por estación, entre agosto y noviembre de 2026.	Tres tablas mediante `JOIN`, `COUNT(DISTINCT)`, agregados `FILTER` y `HAVING`.
4	Tiempo transcurrido hasta la primera detección y primera alerta de los eventos simulados.	Dos expresiones `WITH`, agregación previa, `LEFT JOIN`, `COALESCE` y `EXTRACT(EPOCH)`.
5	Sensores con lecturas en agosto de 2026 que no registraron detecciones en ese mismo período.	`JOIN`, subconsulta correlacionada `NOT EXISTS`, filtros temporales, agregación y `HAVING`.

Cuadro 25: Consultas seleccionadas y operaciones SQL involucradas.

Las consultas se seleccionaron por su utilidad operativa y por requerir distintas estrategias de acceso. C1 resume la actividad sísmica del catálogo importado por año, mientras que C2 relaciona los epicentros del IGP con las *estaciones simuladas* mediante ventanas rec-

tangulares de ±1.5 grados de latitud y longitud. Estas ventanas no representan distancias geodésicas ni prueban que una estación haya detectado los eventos incluidos.

C3 permite examinar la validez de las lecturas por estación; C4 calcula, exclusivamente con datos simulados, los intervalos desde el origen de cada evento hasta su primera detección y alerta; y C5 identifica sensores con lecturas, pero sin detecciones, durante agosto de 2026. La ausencia de detecciones no demuestra una avería del sensor. Las consultas C1 y C2 restringen los eventos a los registros del IGP; las restantes utilizan datos operativos sintéticos sin atribuir sus resultados a una red sismológica real.

5.1.2. Implementación de consultas en SQL

El integrador `sql/03_logica_consultas.sql` contiene las cinco sentencias y sus verificaciones. Las consultas aquí reproducidas también coinciden con las utilizadas en `05_planes_indices.sql` y `06_mediciones.sql`, sin cambios en filtros, parámetros ni ordenamiento. El integrador establece el esquema de trabajo con `SET search_path TO monitoreo_sismologico, public`.

Bloque de código 8: Consulta C1: actividad sísmica anual

```sql
SELECT
    EXTRACT(YEAR FROM ev.fecha_hora_origen)::INTEGER AS anio_utc,
    COUNT(*) AS cantidad_eventos,
    ROUND(AVG(ev.magnitud), 2) AS magnitud_promedio,
    MAX(ev.magnitud) AS magnitud_maxima,
    ROUND(AVG(ev.profundidad_km), 2) AS profundidad_promedio_km
FROM evento_sismico ev
WHERE ev.id_evento BETWEEN 200000000 AND 299999999
  AND ev.fecha_hora_origen >= TIMESTAMP '2010-01-01 00:00:00'
  AND ev.fecha_hora_origen < TIMESTAMP '2026-01-01 00:00:00'
  AND ev.magnitud >= 6
GROUP BY EXTRACT(YEAR FROM ev.fecha_hora_origen)::INTEGER
HAVING COUNT(*) >= 1
ORDER BY anio_utc;
```

Bloque de código 9: Consulta C2: eventos en ventanas geográficas

```sql
SELECT
    e.id_estacion,
    e.codigo AS codigo_estacion,
    e.nombre AS nombre_estacion,
    COUNT(*) AS eventos_en_ventana,
    ROUND(AVG(ev.magnitud), 2) AS magnitud_promedio,
    MAX(ev.magnitud) AS magnitud_maxima
FROM estacion_sismica e
JOIN evento_sismico ev
  ON ev.latitud_epicentro BETWEEN e.latitud - 1.5 AND e.latitud + 1.5
 AND ev.longitud_epicentro BETWEEN e.longitud - 1.5 AND e.longitud + 1.5
WHERE ev.id_evento BETWEEN 200000000 AND 299999999
  AND ev.fecha_hora_origen >= TIMESTAMP '2000-01-01 00:00:00'
  AND ev.fecha_hora_origen < TIMESTAMP '2026-01-01 00:00:00'
  AND ev.magnitud >= 5
GROUP BY e.id_estacion, e.codigo, e.nombre
HAVING COUNT(*) >= 1
ORDER BY eventos_en_ventana DESC, e.id_estacion
LIMIT 10;
```

Bloque de código 10: Consulta C3: calidad de lecturas por estación

```sql
SELECT
    e.id_estacion,
    e.codigo AS codigo_estacion,
    COUNT(DISTINCT s.id_sensor) AS sensores_con_lecturas,
    COUNT(*) AS total_lecturas,
    COUNT(*) FILTER (WHERE l.es_valida) AS lecturas_validas,
    COUNT(*) FILTER (WHERE NOT l.es_valida) AS lecturas_invalidas,
    ROUND(100.0 * COUNT(*) FILTER (WHERE NOT l.es_valida)
          / NULLIF(COUNT(*), 0), 2) AS porcentaje_invalidas
FROM estacion_sismica e
JOIN sensor s ON s.id_estacion = e.id_estacion
JOIN lectura_onda l ON l.id_sensor = s.id_sensor
WHERE l.fecha_hora >= TIMESTAMP '2026-08-01 00:00:00'
  AND l.fecha_hora < TIMESTAMP '2026-12-01 00:00:00'
GROUP BY e.id_estacion, e.codigo
HAVING COUNT(*) >= 10
ORDER BY porcentaje_invalidas DESC, total_lecturas DESC, e.id_estacion;
```

Bloque de código 11: Consulta C4: primera detección y alerta

```sql
WITH detecciones_resumen AS (
    SELECT id_evento, COUNT(*) AS total_detecciones,
           MIN(hora_deteccion) AS primera_deteccion
    FROM deteccion
    GROUP BY id_evento
), alertas_resumen AS (
    SELECT id_evento, COUNT(*) AS total_alertas,
           MIN(fecha_hora_emision) AS primera_alerta
    FROM alerta
    GROUP BY id_evento
)
SELECT
    ev.id_evento,
    ev.fecha_hora_origen,
    ev.magnitud,
    COALESCE(d.total_detecciones, 0) AS total_detecciones,
    COALESCE(a.total_alertas, 0) AS total_alertas,
    ROUND(EXTRACT(EPOCH FROM (d.primera_deteccion - ev.fecha_hora_origen))::NUMERIC, 2)
        AS segundos_primera_deteccion,
    ROUND(EXTRACT(EPOCH FROM (a.primera_alerta - ev.fecha_hora_origen))::NUMERIC, 2)
        AS segundos_primera_alerta
FROM evento_sismico ev
LEFT JOIN detecciones_resumen d ON d.id_evento = ev.id_evento
LEFT JOIN alertas_resumen a ON a.id_evento = ev.id_evento
WHERE (ev.id_evento BETWEEN 810000001 AND 810000008
    OR ev.id_evento BETWEEN 820300001 AND 820300030)
  AND (COALESCE(d.total_detecciones, 0) > 0
    OR COALESCE(a.total_alertas, 0) > 0)
ORDER BY ev.fecha_hora_origen DESC, ev.id_evento
LIMIT 15;
```

Bloque de código 12: Consulta C5: sensores con lecturas sin detecciones

```sql
SELECT
    s.id_sensor,
    s.numero_serie,
    e.codigo AS codigo_estacion,
    s.estado AS estado_sensor,
    COUNT(*) AS lecturas_periodo,
    COUNT(*) FILTER (WHERE NOT l.es_valida) AS lecturas_invalidas,
    MAX(l.fecha_hora) AS ultima_lectura
FROM sensor s
JOIN estacion_sismica e ON e.id_estacion = s.id_estacion
JOIN lectura_onda l ON l.id_sensor = s.id_sensor
WHERE l.fecha_hora >= TIMESTAMP '2026-08-01 00:00:00'
  AND l.fecha_hora < TIMESTAMP '2026-09-01 00:00:00'
  AND NOT EXISTS (
      SELECT 1
      FROM deteccion d
      WHERE d.id_sensor = s.id_sensor
        AND d.hora_deteccion >= TIMESTAMP '2026-08-01 00:00:00'
        AND d.hora_deteccion < TIMESTAMP '2026-09-01 00:00:00'
  )
GROUP BY s.id_sensor, s.numero_serie, e.codigo, s.estado
HAVING COUNT(*) >= 20
ORDER BY lecturas_invalidas DESC, lecturas_periodo DESC, s.id_sensor
LIMIT 15;
```

Control de ejecución y resultados verificados

En PostgreSQL 18 y sobre `bd_sismologico` se ejecutaron los bloques de consultas y verificación incorporados en `sql/03_logica_consultas.sql`. Se produjeron los cinco conjuntos de resultados y se comprobaron sus recuentos, junto con los totales de las seis relaciones y del catálogo oficial del IGP. La verificación utilizó una tabla temporal y concluyó con `ROLLBACK`; no modificó los datos permanentes de la base.

Consulta	Filas	Estado
1. Actividad sísmica anual del IGP (2010–2025)	15	Conforme
2. Coincidencia geográfica por ventanas rectangulares	10	Conforme
3. Calidad de mediciones por estación	14	Conforme
4. Tiempo de detección y alerta en eventos simulados	15	Conforme
5. Sensores con lecturas y sin detecciones	15	Conforme

Cuadro 26: Filas obtenidas en las cinco consultas: ejecución verificada.

Comprobación	Esperado	Observado
Consultas que retornan filas	5	5
Estaciones sísmicas	15	15
Sensores	40	40
Lecturas de onda	1 190	1 190
Eventos sísmicos	24 327	24 327
Detecciones	104	104
Alertas	29	29
Eventos del IGP	24 289	24 289
Registros totales	25 705	25 705

Cuadro 27: Integridad de los datos tras la ejecución de las consultas.

Interpretación de los resultados. La consulta 1 presentó quince agrupaciones anuales, con un total de 70 eventos que cumplen los filtros de período y magnitud ≥ 6; 2024 no aparece porque el conjunto importado no contiene eventos que satisfagan simultáneamente esos predicados en dicho año. Esto no equivale a afirmar que no ocurrieran sismos en 2024. En la consulta 2, el mayor recuento mostrado fue de 378 eventos dentro de la ventana rectangular de una estación *simulada*; no se interpreta como distancia geodésica.

La consulta 3 produjo catorce agrupaciones de estaciones que cumplen el umbral `HAVING COUNT(*) >= 10`. La mayor proporción de lecturas inválidas mostrada fue del 13,04 %. En la consulta 4 se observaron tiempos de primera detección de cero segundos para algunos eventos generados; son consecuencias de las marcas temporales de simulación y *no* tiempos operativos observados en una red sismológica real. Por último, las consultas 4 y 5 utilizan `LIMIT 15`: el número de filas devuelto refleja ese límite y no permite inferir el total de casos que satisfacen sus condiciones.

Alcance de la verificación. Estos resultados corresponden a la base operativa `bd_sismologico` y acreditan que las cinco consultas devolvieron filas sin alterar los datos. No deben confundirse con las mediciones de tiempo realizadas sobre las cuatro bases experimentales, descritas a partir de la Sección 5.2.

5.2. Metodología del experimento

5.2.1. Diseño experimental y escalas

Se evaluaron las cinco consultas de la Sección 5.1 sobre cuatro bases de datos independientes de 1 000, 10 000, 100 000 y 1 000 000 de registros: **20 combinaciones consulta–escala**. En cada escala, el volumen corresponde a la **suma de filas de las seis relaciones base**, no solamente a eventos sísmicos. Se conservaron el modelo relacional, las restricciones y los nueve triggers. La preparación de las bases se mantuvo separada de las mediciones de rendimiento.

Escala	Estaciones	Sensores	Eventos	Lecturas	Detecciones	Alertas	Total
1 000	5	15	100	750	90	40	1 000
10 000	10	30	2 000	7 550	300	110	10 000
100 000	25	75	24 000	74 700	900	300	100 000
1 000 000	100	300	120 000	875 000	3 500	1 100	1 000 000

Cuadro 28: Distribución de registros de las cuatro bases experimentales.

La base operativa `bd_sismologico` no se utilizó para recrear los escenarios. Los datos experimentales se organizaron mediante el integrador `sql/04_bases_experimentales.sql`; se comprobaron las cardinalidades e integridad de cada escala antes de medir las consultas.

5.2.2. Procedencia y alcance de los datos

Los eventos combinan registros del catálogo del Instituto Geofísico del Perú (IGP) y registros sintéticos. Para las tres escalas menores, el integrador ordena los eventos reales mediante criterios de fecha, magnitud e identificador, priorizando casos relevantes para las consultas C1 y C2. Esta selección es determinista, pero no aleatoria; por ello, **no constituye una muestra estadísticamente representativa** de la sismicidad peruana. La base de un millón de registros incorpora los 24 289 eventos reales del catálogo utilizado.

Escala	IGP reales	Sintéticos	Total de eventos
1 000	50	50	100
10 000	800	1 200	2 000
100 000	12 000	12 000	24 000
1 000 000	24 289	95 711	120 000

Cuadro 29: Procedencia de los eventos sísmicos por escala.

La generación determinista utilizó `generate_series`, códigos diferenciados para estaciones y sensores simulados y restricciones que preservan la unicidad de las lecturas. Las detecciones se vincularon con lecturas válidas coincidentes en sensor y fecha-hora; ninguna detección ni alerta sintética se asoció a eventos oficiales del IGP.

5.2.3. Condiciones comparadas y protocolo de medición

Cada consulta se ejecutó bajo dos condiciones: **sin índices secundarios experimentales**, conservando los índices obligatorios asociados a claves primarias y restricciones `UNIQUE`; y **con los índices candidatos** de la Sección 5.3. Dentro de cada escala se conservaron los mismos datos y la misma sentencia SQL en ambos escenarios. No se deshabilitó `enable_seqscan` ni se impuso un plan de ejecución.

Para cada combinación consulta–escala se aplicó el siguiente protocolo:

1. Verificar los recuentos y la procedencia de los eventos; disponer de estadísticas actualizadas mediante `ANALYZE` antes de las mediciones. Tras crear los índices candidatos, actualizar las estadísticas de las tablas afectadas.
2. Comprobar que ambas condiciones devuelven igual cantidad de filas y la misma firma MD5 de resultados, sin incluir esta comprobación en los tiempos medidos.
3. Ejecutar dos calentamientos y cinco repeticiones medidas por condición mediante `EXPLAIN (ANALYZE, BUFFERS, FORMAT JSON)`.
4. Calcular media aritmética, desviación estándar muestral, mínimo y máximo de los cinco tiempos medidos por condición.

En total se programaron **80 calentamientos y 200 mediciones** (280 ejecuciones instrumentadas) para las veinte combinaciones. La métrica temporal es `Execution Time` de PostgreSQL, no el tiempo total mostrado por el cliente.

El integrador `sql/06_mediciones.sql` obtiene internamente los planes en JSON, pero exporta en los CSV **tiempos, búferes, indicadores de uso de índices y firmas de resultados**, no los árboles completos. El integrador `sql/05_planes_indices.sql` realiza por separado los diagnósticos de los cinco planes sobre la base de un millón de registros. Se distingue así entre **índice creado** e **índice utilizado**; tampoco se interpretan los costos estimados como tiempos de ejecución.

5.2.4. Controles realizados y reproducibilidad

Las cuatro bases superaron la verificación de las cantidades indicadas en el Cuadro 28. Asimismo, los controles de estaciones sin sensor, detecciones sin lectura válida, alertas anteriores al origen y nuevas lecturas válidas de sensores inoperativos registraron **cero incumplimientos**; se confirmó la instalación de nueve triggers en cada escala.

Escala	Filas verificadas	Espacio	Triggers	Incumplimientos
1 000	1 000	8 678 kB	9	0
10 000	10 000	9 830 kB	9	0
100 000	100 000	21 MB	9	0
1 000 000	1 000 000	134 MB	9	0

Cuadro 30: Resumen de las verificaciones de las bases experimentales.

El espacio corresponde a `pg_database_size` e incluye objetos y sobrecargas de almacenamiento, no solo los datos de las relaciones. Se generaron cuatro respaldos independientes en formato `pg_dump -Fc`, publicados como activos de la versión `v1.0.0` del repositorio. Se verificó la restauración del respaldo de 1 000 registros; **no se afirma** haber restaurado los otros tres en esta comprobación. La generación desde cero requiere obtener el CSV oficial del IGP, cuyo origen se documenta en `sql/datos/ORIGEN_IGP.txt`; este archivo fuente no se incluye en el repositorio.

5.2.5. Limitaciones de la comparación

Los calentamientos pueden mantener búferes en caché, por lo que los tiempos no representan condiciones de caché fría. La selectividad de la Consulta 1 cambia con el tamaño de la muestra IGP; la Consulta 2 utiliza ventanas de coordenadas, no distancias en kilómetros; y las Consultas 4 y 5 tienen un `LIMIT 15`. Estas condiciones impiden atribuir cualquier diferencia de tiempo exclusivamente al número total de filas.

Además, el informe conserva los tiempos de la *serie principal* y reconoce una *segunda ejecución* con tiempos distintos, registrada en los archivos de medición del repositorio. Ambas series no se mezclan al calcular estadísticas ni porcentajes.

5.3. Optimización de consultas

Se comparan los dos escenarios de cada una de las cinco consultas en la base experimental con **1 000 000 de registros**. La comparación conserva las sentencias origi-

nales y los índices obligatorios de `PRIMARY KEY` y `UNIQUE`; "sin índices" significa *sin índices secundarios experimentales*. Los índices candidatos se definen en el integrador `sql/05_planes_indices.sql`, y las mediciones repetidas se documentan por separado en la Sección 5.2 y en la Sección 5.

Cada tabla de este apartado resume **dos diagnósticos puntuales** obtenidos con `EXPLAIN (ANALYZE, BUFFERS, SETTINGS)`: uno antes y otro después de crear el índice candidato. No son promedios ni demuestran por sí solos mejoras reproducibles. El criterio principal es comprobar *qué plan escogió PostgreSQL*, si utilizó el índice y si el resultado de la consulta se conservó. El análisis estadístico de las cinco repeticiones por escenario corresponde a las Secciones 5.5–5.7.

Las cinco figuras comparan extractos de los planes obtenidos en una comprobación adicional con `EXPLAIN ANALYZE` en pgAdmin 4. Se conservan los diez SVG originales junto a las figuras del proyecto. Los extractos permiten identificar los operadores sin confundir esta verificación visual con las mediciones repetidas de las Secciones 5.5 y 5.6.

5.3.1. Plan de índices para la Consulta 1

Objetivo y diagnóstico. La Consulta 1 agrupa por año los eventos del IGP de magnitud ≥ 6 entre 2010 y 2025. Sin índice secundario, PostgreSQL empleó el índice obligatorio `pk_evento_sismico`, recorrió 24 289 eventos del IGP y descartó 24 219 después de aplicar fecha y magnitud. Se propuso un B-tree parcial con clave temporal y columnas incluidas para suministrar las agregaciones desde el índice.

Bloque de código 13: Índice candidato de C1 (05_planes_indices.sql)

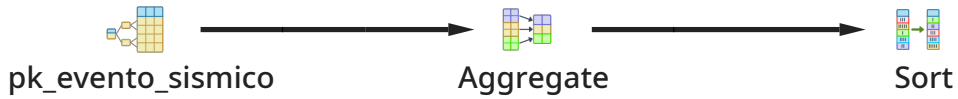
```
1  CREATE INDEX idx_ev_igp_m6_fecha
2  ON monitoreo_sismologico.evento_sismico (fecha_hora_origen)
3  INCLUDE (magnitud, profundidad_km)
4  WHERE id_evento BETWEEN 200000000 AND 299999999
5    AND magnitud >= 6;
```

Indicador	Sin índice secundario	Con índice C1
Acceso a eventos	`Index Scan` (PK)	`Index Only Scan`
Filas seleccionadas / agrupaciones anuales	70 / 15	70 / 15
Filas descartadas durante el acceso	24 219	0 reportadas
`shared hit` / `shared read`	24 189 / 79	1 / 2
`Heap Fetches`	No reportado	0
Planificación / ejecución (ms)	0,567 / 25,536	0,499 / 0,146

Cuadro 31: Diagnóstico de la Consulta 1: escenario base frente a índice candidato (un millón de registros).

Interpretación. El índice parcial (32 kB) fue realmente seleccionado y produjo las mismas 15 agrupaciones, con `Heap Fetches = 0`. El contraste de 25,536 frente a 0,146 ms muestra un cambio favorable en dos ejecuciones aisladas; la conveniencia del índice se juzga con las mediciones repetidas.

(a) Sin índices secundarios experimentales



(b) Con índices candidatos disponibles



Figura 3: Consulta 1: extractos de los planes de ejecución, sin índices secundarios y con índices candidatos (un millón de registros).

5.3.2. Plan de índices para la Consulta 2

Objetivo y diagnóstico. La Consulta 2 cuenta eventos del IGP en ventanas rectangulares de $\pm 1,5°$ de latitud y longitud alrededor de las estaciones; estas ventanas no son distancias geodésicas. El plan base combinó `Parallel Index Scan` de eventos con `Nested Loop` y `Seq Scan` de 100 estaciones. Se probó un índice B-tree parcial sobre las dos coordenadas para evitar recorrer eventos no elegibles.

Bloque de código 14: Índice candidato de C2 (05_planes_indices.sql)

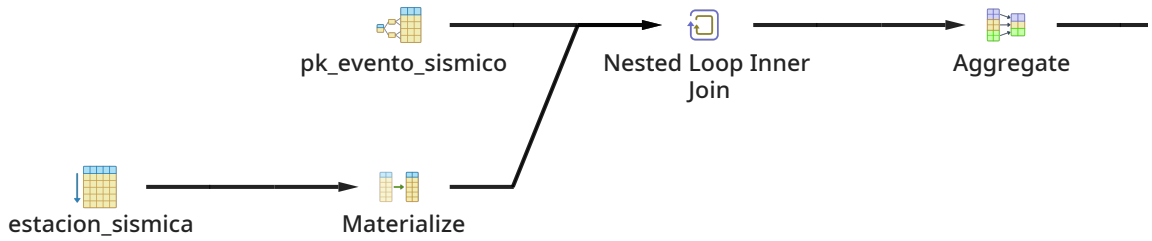
```
CREATE INDEX idx_ev_igp_geo_c2
ON monitoreo_sismologico.evento_sismico
    (latitud_epicentro, longitud_epicentro)
INCLUDE (magnitud)
WHERE id_evento BETWEEN 200000000 AND 299999999
  AND fecha_hora_origen >= TIMESTAMP '2000-01-01 00:00:00'
  AND fecha_hora_origen < TIMESTAMP '2026-01-01 00:00:00'
  AND magnitud >= 5;
```

Indicador	Sin índice secundario	Con índice C2
Acceso a eventos	`Parallel Index Scan` (PK)	`Index Only Scan`
Combinación de relaciones	`Nested Loop`	`Nested Loop`
Filas finales / coincidencias de las diez estaciones	10 / 3 737	10 / 3 737
`shared hit` / `shared read`	24 303 / 0	399 / 12
`Heap Fetches`	No reportado	0
Planificación / ejecución (ms)	0,402 / 48,640	0,550 / 11,144

Cuadro 32: Diagnóstico de la Consulta 2: comparación de planes y resultados.

Interpretación. El índice de 112 kB se utilizó en 100 búsquedas sobre eventos, sin visitas al *heap*. Las diez filas conservaron su firma MD5 `6ad084197533fbdf68204aecd874533b`. La reducción puntual de 48,640 a 11,144 ms es coherente con el cambio de acceso, pero debe distinguirse de un beneficio medio demostrado.

(a) Sin índices secundarios experimentales



(b) Con índices candidatos disponibles

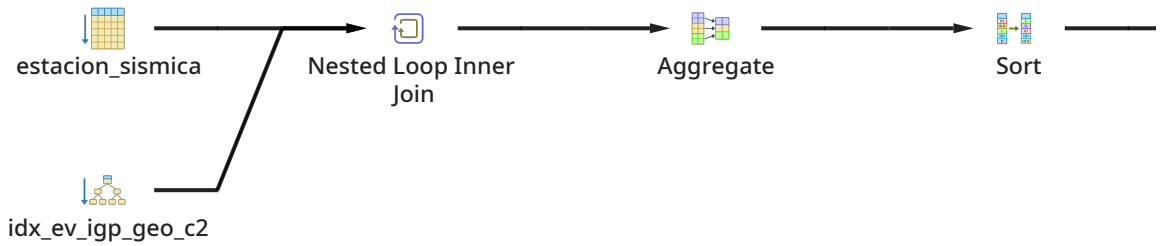


Figura 4: Consulta 2: extractos de los planes de ejecución, sin índices secundarios y con índices candidatos (un millón de registros).

5.3.3. Plan de índices para la Consulta 3

Objetivo y diagnóstico. La Consulta 3 calcula la proporción de lecturas inválidas por estación entre agosto y noviembre de 2026. En la escala mayor, **las 875 000 lecturas**

cumplen el filtro temporal. En el escenario base PostgreSQL recurrió a `Seq Scan`, dos `Hash Join` y un ordenamiento `external merge` de 30 032 kB antes de agrupar. Se evaluó un índice compuesto que aporta sensor, fecha e indicador de validez.

Bloque de código 15: Índice candidato de C3 (05_planes_indices.sql)

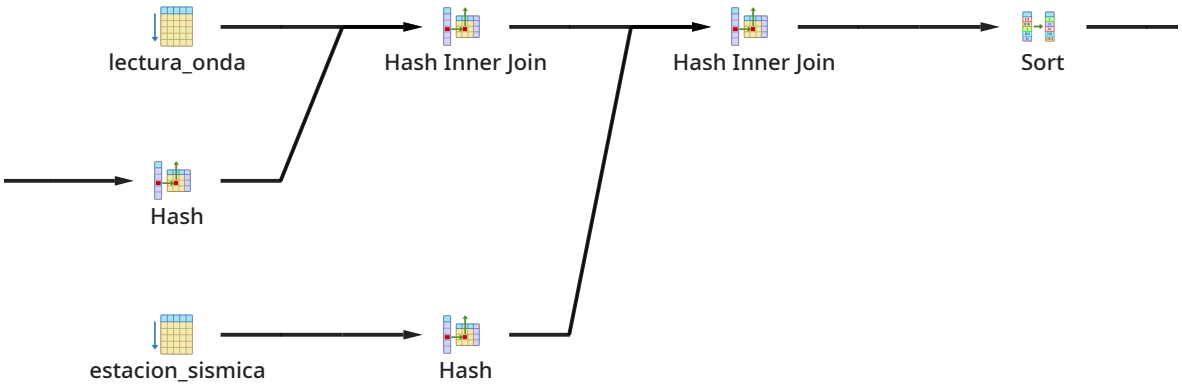
```
CREATE INDEX idx_lectura_sensor_fecha_c3
ON monitoreo_sismologico.lectura_onda
    (id_sensor, fecha_hora)
INCLUDE (es_valida);
```

Indicador	**Sin índice secundario**	**Con índice C3**
Acceso a lecturas	`Seq Scan`	`Index Only Scan`
Unión y ordenamiento	`Hash Join`, `external merge`	`Nested Loop`, `Incremental Sort`
Bloques temporales leídos / escritos	7 501 / 7 516	No reportados
Lecturas agrupadas / estaciones	875 000 / 100	875 000 / 100
Lecturas inválidas	25 732	25 732
`shared hit` / `shared read`	638 / 5 802	888 / 4 330
Planificación / ejecución (ms)	1,620 / 1 935,022	0,539 / 582,745

Cuadro 33: Diagnóstico de la Consulta 3: operación dominante y costo observado.

Interpretación. El índice de 34 MB permitió recorridos `Index Only Scan` (`Heap Fetches = 0`) y evitó el ordenamiento externo registrado en el plan base. El resultado se conservó (firma MD5 `907f40a44731c36635d0629b32149f02`). El ahorro de tiempo en este diagnóstico es importante, pero el índice se solapa parcialmente con el `UNIQUE` existente y aumenta el almacenamiento y el costo de actualización.

(a) Sin índices secundarios experimentales



(b) Con índices candidatos disponibles

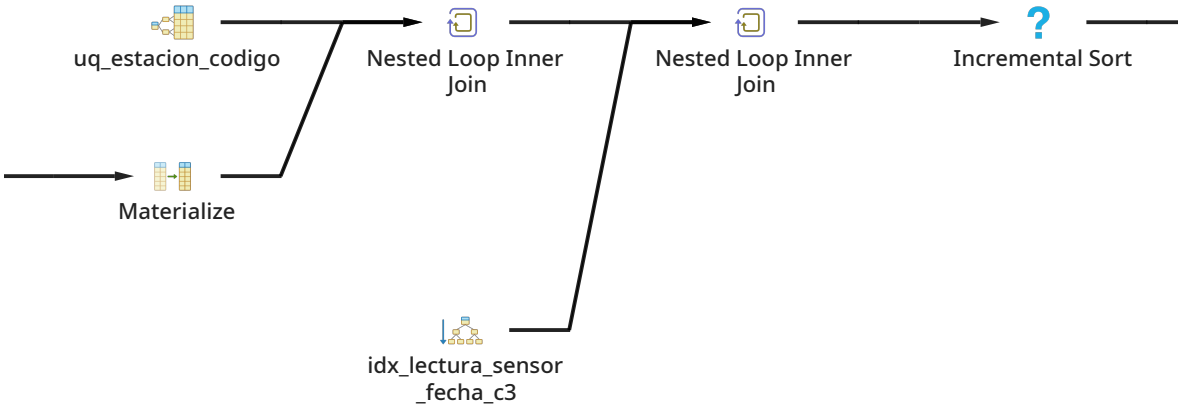


Figura 5: Consulta 3: extractos de los planes de ejecución, sin índices secundarios y con índices candidatos (un millón de registros).

5.3.4. Plan de índices para la Consulta 4

Objetivo y diagnóstico. La Consulta 4 obtiene la primera detección y alerta de cada evento *simulado*. Con 3 500 detecciones y 1 100 alertas, el plan base utilizó `Seq Scan` para ambas relaciones y `Bitmap Heap Scan` para los eventos. Los índices compuestos propuestos ordenan por evento y hora, pero no se presupone que superen a los recorridos secuenciales de tablas relativamente pequeñas.

Bloque de código 16: Índices candidatos de C4 (05_planes_indices.sql)

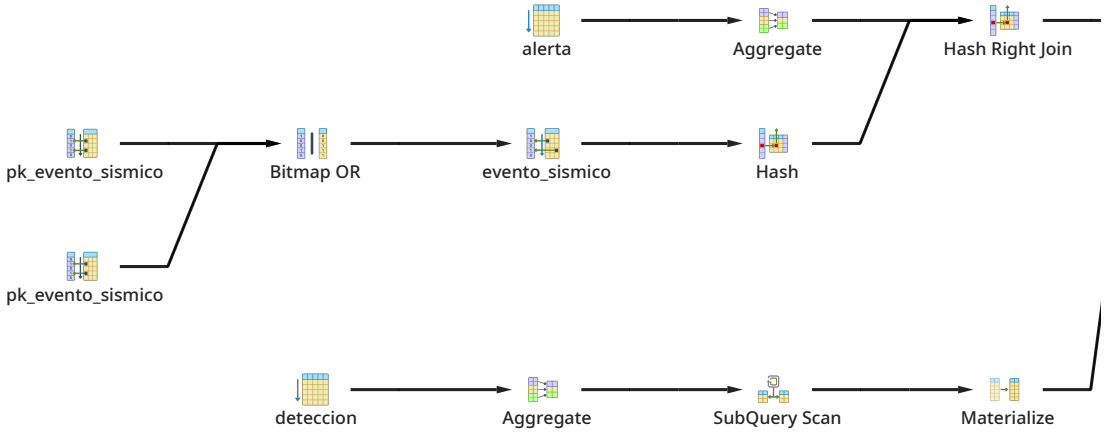
```
1  CREATE INDEX idx_det_evento_hora_c4
2  ON monitoreo_sismologico.deteccion
3      (id_evento, hora_deteccion);
4
5  CREATE INDEX idx_alerta_evento_hora_c4
6  ON monitoreo_sismologico.alerta
7      (id_evento, fecha_hora_emision);
```

Indicador	Sin índices secundarios	Con índices C4
Acceso a detecciones y alertas	`Seq Scan`	`Seq Scan`
Acceso a eventos	`Bitmap Heap Scan`	`Bitmap Heap Scan`
Filas finales / detecciones y alertas en esas filas	15 / 1 250 y 15	15 / 1 250 y 15
`shared hit`	49	49
Índices candidatos utilizados	No disponibles	Ninguno
Planificación / ejecución (ms)	0,303 / 1,067	0,249 / 0,968

Cuadro 34: Diagnóstico de la Consulta 4: disponibilidad y uso de índices.

Interpretación. Ambos índices sumaron 184 kB, pero PostgreSQL no utilizó ninguno. Aunque los tiempos aislados difieren en 0,099 ms, *no existe evidencia de una mejora atribuible a estos índices*. Los resultados fueron idénticos (firma MD5 `e2db687b17ebc96a1468a48f95c4e945`). Los tiempos de respuesta corresponden a eventos simulados, no a un sistema de alerta operativo.

(a) Sin índices secundarios experimentales



(b) Con índices candidatos disponibles

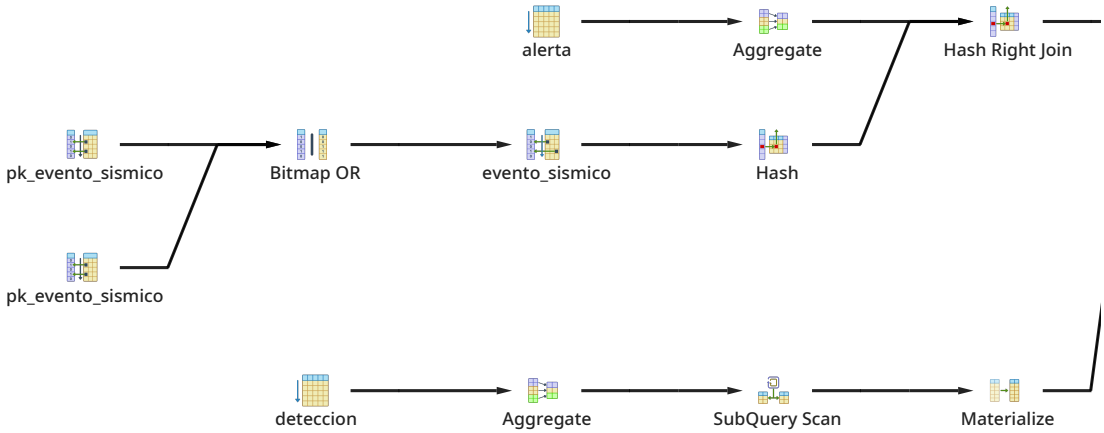


Figura 6: Consulta 4: extractos de los planes de ejecución, sin índices secundarios y con índices candidatos (un millón de registros).

5.3.5. Plan de índices para la Consulta 5

Objetivo y diagnóstico. La Consulta 5 identifica sensores con al menos veinte lecturas de agosto de 2026 y ninguna detección en ese mismo período. El filtro temporal selecciona todas las 875 000 lecturas de la escala mayor. El plan base ejecutó `Parallel Seq Scan` y `Nested Loop Anti Join`, con búsqueda sobre la llave primaria de detecciones. Se probaron dos índices para cubrir la lectura por sensor y la exclusión por sensor y hora.

Bloque de código 17: Índices candidatos de C5 (05_planes_indices.sql)

```
1  CREATE INDEX idx_lectura_sensor_hora_c5
2  ON monitoreo_sismologico.lectura_onda
3      (id_sensor, fecha_hora)
4  INCLUDE (es_valida);
5
```

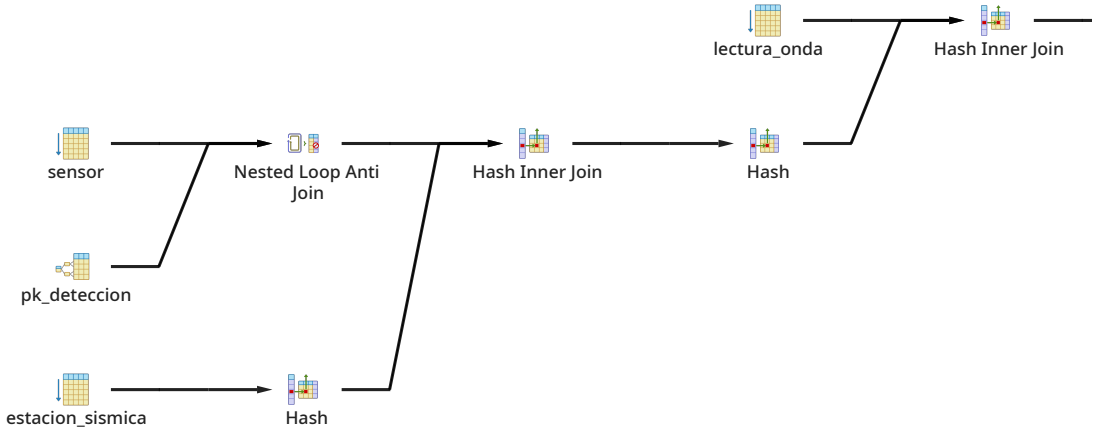
```sql
CREATE INDEX idx_det_sensor_hora_c5
ON monitoreo_sismologico.deteccion
    (id_sensor, hora_deteccion);
```

Indicador	**Sin índices secundarios**	**Con índices C5**
Acceso a lecturas	`Parallel Seq Scan`	`Index Only Scan`
Anti-unión de detecciones	`Nested Loop Anti Join` con PK	`Nested Loop Anti Join` con índice C5
Lecturas / lecturas inválidas en las 15 filas	43 755 / 2 580	43 755 / 2 580
`shared hit` / `shared read`	4 631 / 4 073	1 035 / 2 178
`Heap Fetches`	No reportados	0
Planificación / ejecución (ms)	0,663 / 113,493	0,938 / 266,999

Cuadro 35: Diagnóstico de la Consulta 5: efecto observado de los índices.

Interpretación. Los índices fueron utilizados y conservaron las quince filas (firma MD5 `eff5050db18aa1801e7f6f5a062f7948`), pero el plan con índices *tardó más*, pese a consultar menos búferes y a reportar cero visitas al *heap*. La propuesta ocupa unos 34 MB para lecturas y 128 kB para detecciones: solo tendría sentido conservarla si otras cargas justificasen ese costo. La ausencia de detecciones no se interpreta por sí sola como falla de sensor.

(a) Sin índices secundarios experimentales



(b) Con índices candidatos disponibles

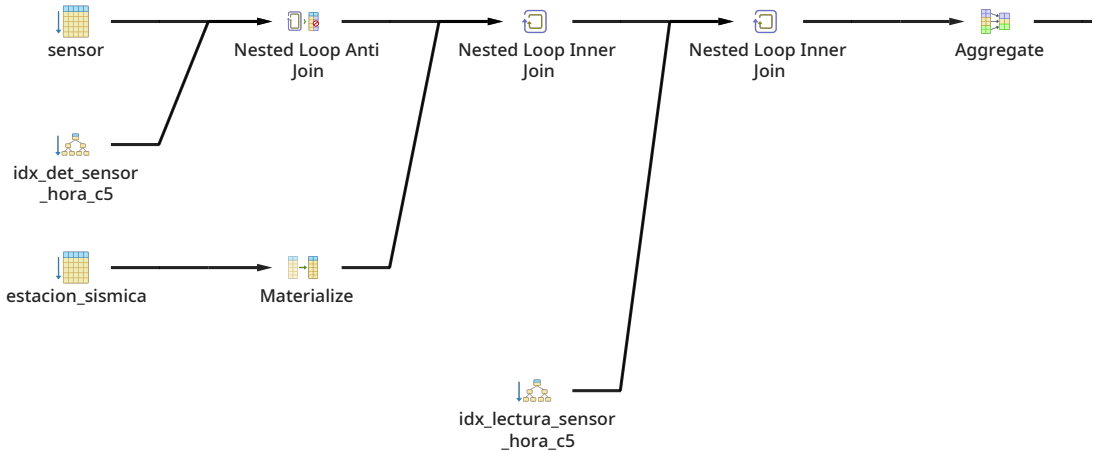


Figura 7: Consulta 5: extractos de los planes de ejecución, sin índices secundarios y con índices candidatos (un millón de registros).

Síntesis de la decisión. Los diagnósticos muestran por qué *usar un índice* no equivale necesariamente a *acelerar una consulta*: C1–C3 presentan cambios de plan favorables, C4 no utiliza los índices ofrecidos y C5 sí los utiliza, pero resulta más lenta en la ejecución observada. La selección de índices se contrasta con las cinco mediciones válidas de cada escenario, analizadas en las secciones siguientes.

5.4. Plataforma de pruebas

Los experimentos se ejecutaron en un mismo equipo con Windows 11 y PostgreSQL 18.6 de 64 bits. Se utilizaron cuatro bases independientes, cuya construcción, integridad y distribución de filas se describen en la Sección 5.2. La caracterización permite identificar las condiciones de ejecución; no pretende establecer que los tiempos observados sean generalizables a otros equipos o configuraciones.

Componente	Valor registrado
Procesador	Intel Core i5-12450H (12.ª generación)
Núcleos físicos / hilos lógicos	8 / 12
Memoria RAM instalada	15,63 GiB
Sistema operativo	Windows 11 Home (10.0.26300), 64 bits
Motor de base de datos	PostgreSQL 18.6, `x86_64-windows`

Cuadro 36: Características del equipo empleado en las pruebas.

Las características del equipo proceden del diagnóstico realizado en Windows mediante `Get-CimInstance`. El procedimiento de consulta se conserva en `herramientas/02_diagnostico_equipo.ps1` del repositorio. No se registró el tipo ni el modelo de la unidad de almacenamiento, por lo que no se presupone que se empleara SSD o HDD. Los parámetros del servidor se consultaron en `pg_settings`; los integradores no fuerzan el uso de escaneos secuenciales o de índices. Tampoco se documentó un control constante de la carga de otros procesos del equipo durante todas las mediciones.

Parámetro	Valor observado
`shared_buffers` / `work_mem`	128 MiB / 4 MiB
`maintenance_work_mem` / `effective_cache_size`	64 MiB / 4 GiB
`max_parallel_workers`	8
`max_parallel_workers_per_gather`	2
`random_page_cost` / `seq_page_cost`	4 / 1
`enable_seqscan` / `enable_indexscan`	`on` / `on`
`enable_indexonlyscan` / `enable_bitmapscan`	`on` / `on`
`jit` / `track_io_timing`	`on` / `off`
`server_encoding` / zona horaria	`UTF8` / `America/Bogota`

Cuadro 37: Configuración observada de PostgreSQL relevante para la comparación.

`effective_cache_size` es una estimación del optimizador, no memoria reservada; con `track_io_timing=off` no se registraron tiempos detallados de E/S. La zona horaria del servidor no modifica automáticamente la convención UTC empleada al almacenar el catálogo IGP en columnas `TIMESTAMP`.

Total de filas	Tamaño reportado
1 000	8 678 kB
10 000	9 846 kB
100 000	21 MB
1 000 000	168 MB

Cuadro 38: Espacio ocupado por cada base en el diagnóstico de plataforma.

Los valores de `pg_database_size` incluyen datos, índices y demás objetos, y corresponden al momento del diagnóstico. La base de un millón tenía los índices candidatos de C5 (34 MB y 128 kB), mientras que el tamaño inicial citado en la Sección 5.2 era de 134 MB. Son estados distintos: la diferencia no demuestra crecimiento de filas ni puede atribuirse íntegramente a los índices.

5.5. Medición de tiempos

Se compararon cinco consultas (C1–C5) en cuatro escalas: 1 000, 10 000, 100 000 y 1 000 000 registros, bajo dos escenarios. El primero conserva únicamente los índices necesarios para las restricciones de integridad; el segundo dispone de los índices candidatos definidos en la Sección 5.3. Cada combinación incluyó dos ejecuciones de calentamiento, excluidas de los cálculos, y cinco mediciones válidas mediante `EXPLAIN (ANALYZE, BUFFERS, FORMAT JSON)`. Se midió el *Execution Time* de PostgreSQL, no el tiempo de respuesta de una aplicación o de la red. El procedimiento general se expuso en la Sección 5.2.

Las 40 medias de la **serie principal** y los recuentos de uso de índices se contrastaron con el consolidado `59_resumen_global_5_6.csv`. Las desviaciones estándar se conservan del registro histórico, pero el CSV no incluye las mediciones individuales necesarias para recalcularlas. Las reejecuciones de `06_mediciones.sql` se tratan por separado.

5.5.1. Sin índices secundarios

Este escenario sirve como referencia de tiempo sin índices secundarios propuestos para cada consulta. El Cuadro 39 resume sus 20 combinaciones de consulta y volumen; cada cifra es una media de cinco mediciones, acompañada por su desviación estándar muestral (ambas en milisegundos).

Consulta	Registros	Media (ms)	Desv. est. (ms)
C1	1 000	0,1100	0,0046
C1	10 000	0,2508	0,0701
C1	100 000	2,4396	0,1940
C1	1 000 000	5,1576	0,4280
C2	1 000	0,1086	0,0067
C2	10 000	1,8676	0,0389
C2	100 000	20,9324	6,6785
C2	1 000 000	51,2426	1,0160
C3	1 000	0,6578	0,0161
C3	10 000	5,8068	0,2222
C3	100 000	111,4626	26,0896
C3	1 000 000	2 013,4210	44,4828
C4	1 000	0,1192	0,0034
C4	10 000	0,1824	0,0076
C4	100 000	0,5064	0,0212
C4	1 000 000	0,9538	0,0648
C5	1 000	0,2596	0,0066
C5	10 000	2,4626	0,4314
C5	100 000	21,2996	6,3979
C5	1 000 000	113,3538	4,0454

Cuadro 39: Serie principal: tiempo medio y desviación estándar sin índices secundarios.

Los controles previos exigieron la cantidad de filas prevista y la ausencia de índices secundarios adicionales; los índices vinculados a claves primarias y restricciones `UNIQUE` permanecieron habilitados. La secuencia de calentamiento seguida de medición se aplicó también al escenario con índices, aunque este orden fijo puede introducir efectos de caché y de condiciones del equipo.

5.5.2. Con índices candidatos

En el segundo escenario se crearon los índices candidatos de la consulta correspondiente, se actualizaron las estadísticas del planificador y se repitió la sentencia sin forzar recorridos por índice. El Cuadro 40 resume los tiempos y el uso observado de cada índice candidato durante las cinco mediciones.

Consulta	Registros	Media (ms)	Desv. est. (ms)	Uso
C1	1 000	0,0582	0,0051	0/5
C1	10 000	0,0670	0,0200	5/5
C1	100 000	0,0614	0,0054	5/5
C1	1 000 000	0,0608	0,0062	5/5
C2	1 000	0,1156	0,0046	0/5
C2	10 000	0,3522	0,0428	5/5
C2	100 000	3,2476	0,2236	5/5
C2	1 000 000	11,1818	0,1213	5/5
C3	1 000	0,9096	0,0130	0/5
C3	10 000	5,9190	0,4895	0/5
C3	100 000	131,7908	4,0412	0/5
C3	1 000 000	512,4504	15,6541	5/5
C4	1 000	0,1242	0,0196	0/5, 0/5
C4	10 000	0,1932	0,0131	0/5, 0/5
C4	100 000	0,5042	0,0209	0/5, 0/5
C4	1 000 000	1,4148	0,0659	0/5, 0/5
C5	1 000	0,3978	0,0282	0/5, 0/5
C5	10 000	1,8190	0,1068	5/5, 0/5
C5	100 000	17,7076	1,2741	5/5, 5/5
C5	1 000 000	202,8902	7,5625	5/5, 5/5

Cuadro 40: Serie principal: tiempo medio, dispersión y uso de los índices candidatos.

Nota: "Uso" indica en cuántas de las cinco mediciones el plan seleccionó el índice candidato. En C4 y C5, ambos recuentos siguen el orden de los índices presentados en 5.3. Un índice disponible no necesariamente se utiliza: en C4 los dos registraron 0/5 en todas las escalas.

Control experimental y alcance. El protocolo contempla 20 pares consulta–escala y 14 ejecuciones por par (siete por escenario): **280 ejecuciones**, distribuidas en **80 calentamientos** y **200 mediciones**. El integrador `06_mediciones.sql` comprueba la cantidad y firma de los resultados, exporta un CSV por par y revierte los índices con `ROLLBACK`. Esta comprobación forma parte del procedimiento de reejecución y no sustituye las observaciones individuales de la serie principal.

Los cuadros presentan resultados descriptivos, sin atribuir todas las diferencias de tiempo a los índices. Las variaciones relativas y su interpretación aparecen en las Secciones 5.6 y 5.7, junto con los gráficos comparativos.

5.6. Resultados

Las Figuras 8–12 comparan las medias de ejecución de la **serie principal histórica** en las cuatro escalas experimentales. Se utilizó el consolidado `59_resumen_global_5_6.csv`, cuyas 20 comparaciones se cotejaron con las medias de la Sección 5.5. Los dos calentamientos de cada condición se excluyeron del promedio. El eje horizontal emplea una escala logarítmica para los volúmenes (1 000 a 1 000 000 registros); el eje vertical es logarítmico en C1, C2, C3 y C5, y lineal en C4. Por ello, la inclinación visual de las curvas no debe interpretarse de igual manera entre consultas. Las tablas de 5.5 contienen las medias y desviaciones, mientras que el resumen consolidado de 5.7 presenta las variaciones relativas.

5.6.1. Consulta 1: Actividad sísmica anual del catálogo del IGP

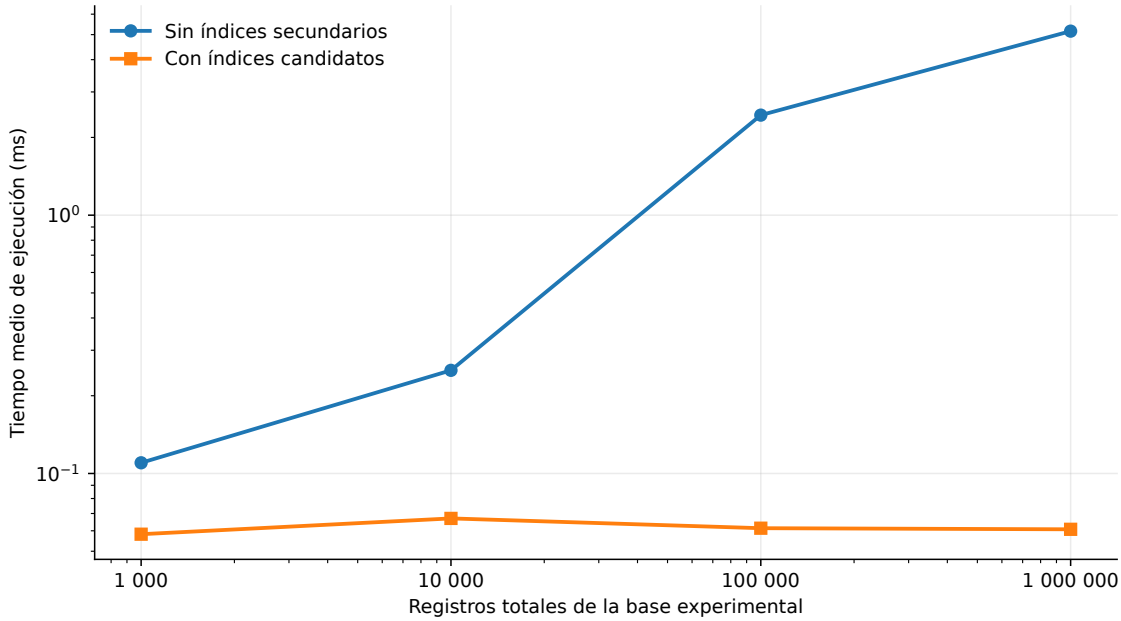


Figura 8: C1: tiempo medio por volumen, sin índices secundarios y con el índice candidato.

En la escala de 1 000 registros, PostgreSQL no empleó el índice candidato (0/5). Aunque el escenario con el índice disponible registró una media menor, esa diferencia no demuestra un beneficio del acceso indexado. Desde 10 000 registros el índice se utilizó en las cinco mediciones de cada escala. En un millón, el tiempo descendió de 5,1576 a 0,0608 ms (**98,82 % menos**). Esta diferencia corresponde a los datos y filtros del experimento; no debe generalizarse a toda distribución de eventos sísmicos.

5.6.2. Consulta 2: Eventos sísmicos en ventanas geográficas

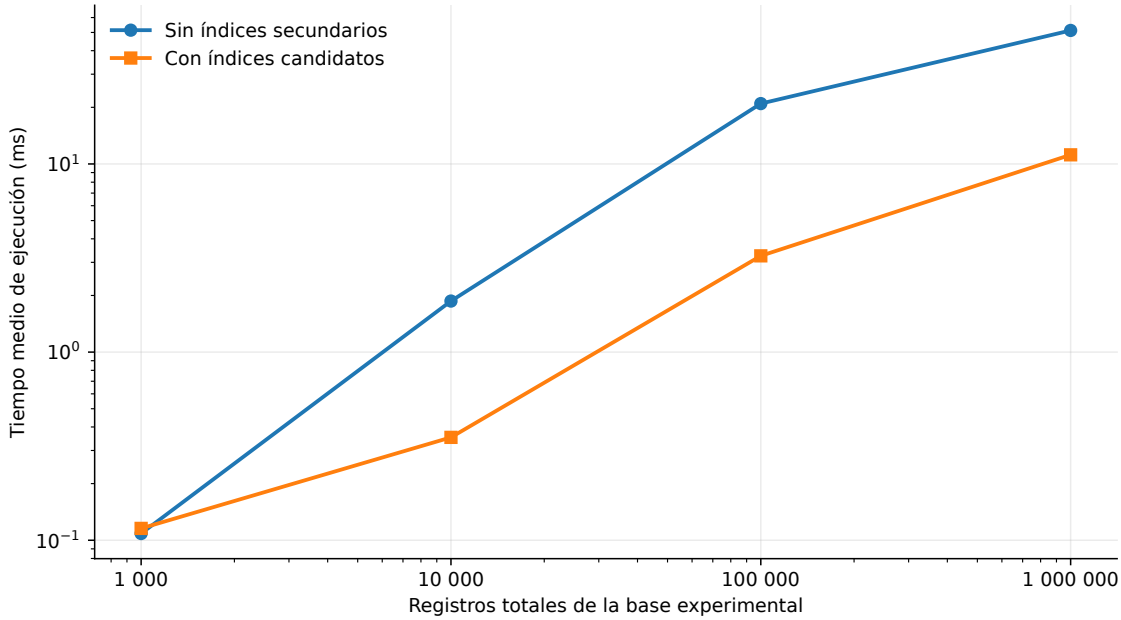


Figura 9: C2: tiempos medios de la consulta espacial con y sin el índice candidato.

Con 1 000 registros no se eligió el índice y la media fue ligeramente mayor en el escenario indexado (0,1156 frente a 0,1086 ms). Desde 10 000 registros hubo uso del índice en 5/5 mediciones, junto con menores medias. Para un millón, la consulta pasó de 51,2426 a 11,1818 ms (**78,18 % menos**). La consulta emplea ventanas rectangulares de latitud y longitud, no distancias geodésicas.

5.6.3. Consulta 3: Calidad de lecturas por estación

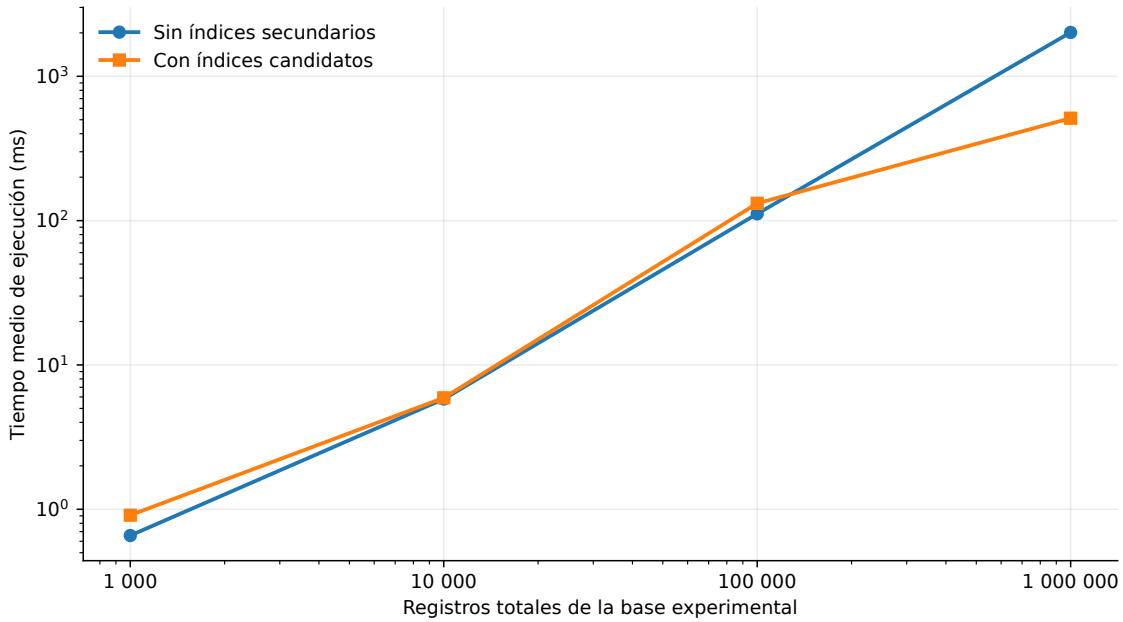


Figura 10: C3: evolución del tiempo medio según el tamaño de la base.

El índice candidato no fue seleccionado en las tres escalas menores; en ellas, las medias con el índice disponible fueron superiores a las del escenario base. En un millón sí se utilizó (5/5), y la media se redujo de 2 013,4210 a 512,4504 ms (**74,55 % menos**). El diagnóstico de la Sección 5.3 documenta un cambio de plan relevante para el ordenamiento; el beneficio de lectura debe considerarse junto con el tamaño y mantenimiento del índice.

5.6.4. Consulta 4: Tiempos de primera detección y primera alerta

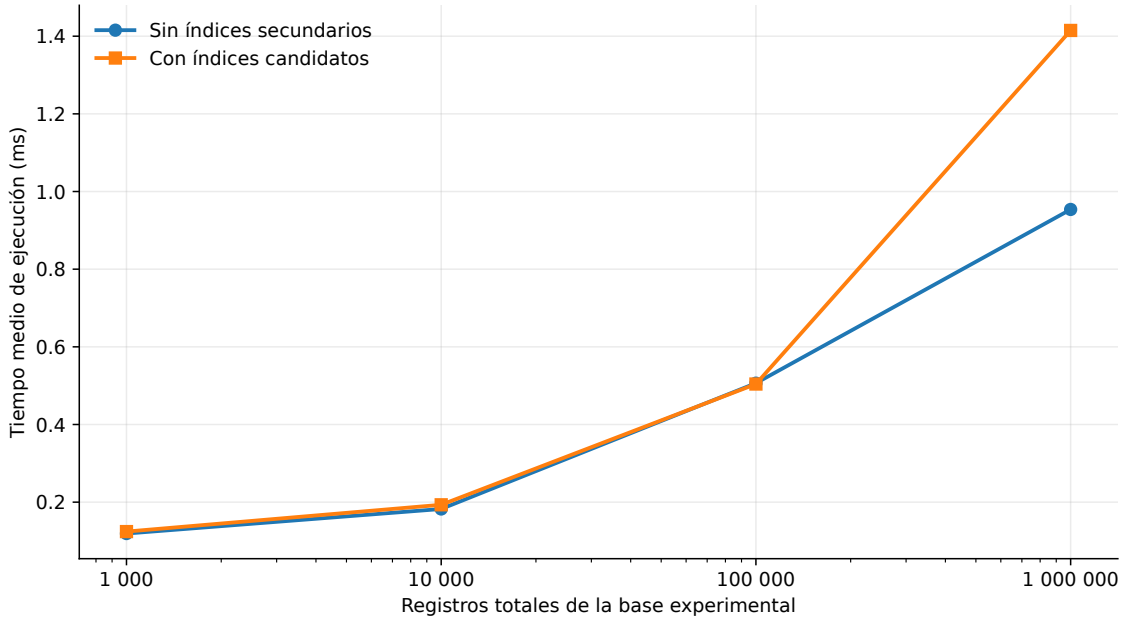


Figura 11: C4: tiempos medios con los dos índices candidatos disponibles o ausentes.

PostgreSQL no utilizó ninguno de los dos índices candidatos en ninguna escala (0/5 para ambos). En un millón, las medias fueron 0,9538 y 1,4148 ms sin y con los índices disponibles, respectivamente: una diferencia absoluta de 0,4610 ms (48,33 % más en el segundo escenario). Dado que ninguno se seleccionó, **no corresponde atribuir esa diferencia al acceso mediante los índices candidatos**.

5.6.5. Consulta 5: Sensores con lecturas y sin detecciones

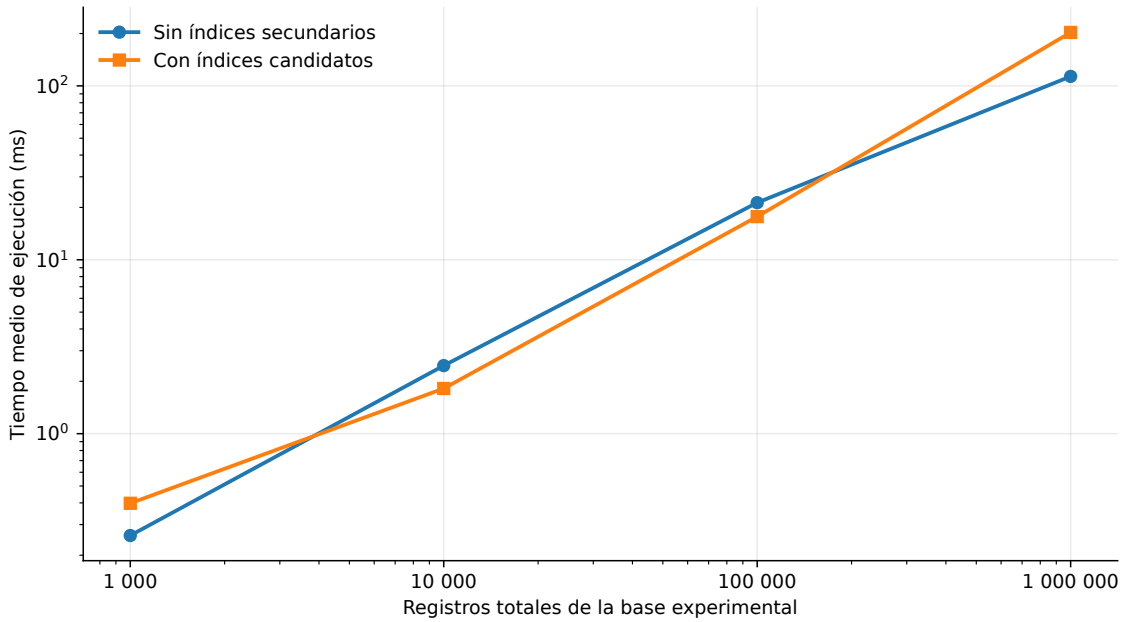


Figura 12: C5: contraste de las medias y efecto no uniforme de la disponibilidad de índices.

Con 1 000 registros, ninguno de los dos índices se utilizó y la media fue mayor al tenerlos disponibles (0,3978 frente a 0,2596 ms). En 10 000 registros se utilizó uno de ellos y la media disminuyó 26,13 %; en 100 000 se utilizaron ambos, con una reducción de 16,86 %. Sin embargo, con un millón los dos índices se usaron en 5/5 mediciones y la media aumentó de 113,3538 a 202,8902 ms (**78,99 % más**). Este caso demuestra que la selección de índices no garantiza un menor tiempo de ejecución.

5.7. Análisis y Discusión

5.7.1. Balance de las veinte comparaciones y decisiones de indexación

Se evaluaron cinco consultas en cuatro escalas, bajo dos escenarios por combinación. El protocolo contempló dos calentamientos y cinco mediciones por escenario, con **200 mediciones registradas y 80 calentamientos** en total. La comprobación de equivalencia de resultados se documenta en el procedimiento de medición.

El Cuadro 41 sintetiza *las decisiones para la escala de un millón de registros*; las veinte comparaciones y sus medidas descriptivas figuran en las Secciones 5.5 y 5.6. Una variación negativa indica menor media de ejecución en el escenario con índices. El uso corresponde a las cinco mediciones de la serie principal, no a su mera existencia.

C	Uso efectivo	Variación	Implicación para la decisión
C1	5/5	-98,82 %	Índice parcial favorable para el filtro estudiado.
C2	5/5	-78,18 %	Índice geográfico favorable en esta distribución.
C3	5/5	-74,55 %	Índice de cobertura útil a gran escala; evaluar su costo de mantenimiento.
C4	0/5 y 0/5	+48,33 %	Ninguno de los dos índices fue elegido; no hay beneficio atribuible a ellos.
C5	5/5 y 5/5	+78,99 %	Ambos se usaron, pero el plan resultó más lento en la serie principal.

Cuadro 41: Síntesis de la indexación en la escala de un millón de registros (serie principal).

Lectura del conjunto. C1 y C2 utilizaron sus índices desde 10 000 registros y fueron más rápidas en las tres escalas mayores. En C3 el índice sólo se seleccionó con un millón de registros, cuando el tiempo medio bajó de 2 013,4210 a 512,4504 ms; su ocupación aproximada de 34 MB debe considerarse junto con el costo de inserción y actualización. C4 no utilizó ninguno de sus índices en ninguna escala; su diferencia de tiempos no prueba un efecto de indexación. C5, por último, mejoró en 10 000 y 100 000 registros, pero empeoró en el millón aun utilizando ambos índices. **Utilización y beneficio no son equivalentes**: la decisión debe basarse en el costo del plan completo y en una carga de trabajo representativa.

5.7.2. Trazabilidad y alcance de las mediciones

Las medias de la serie principal proceden del consolidado histórico `59_resumen_global_5_6.csv`. Sus desviaciones estándar se conservan de los registros anteriores, ya que no se dispone de las observaciones individuales necesarias para recalcularlas.

Una reejecución independiente generó veinte exportaciones previstas por el integrador, cada una con cuatro calentamientos y diez mediciones distribuidos entre los dos escenarios. El resumen `auditoria_dos_series_20_comparaciones.csv` identifica los archivos locales y las firmas de resultados. No se adjuntan en este informe las veinte series individuales de la reejecución; el integrador `06_mediciones.sql` permite regenerarlas.

Por ejemplo, para C5 con un millón de registros, las medias de la serie principal fueron 113,3538 y 202,8902 ms (sin y con índices), mientras que la reejecución registró 110,3134 y 117,3334 ms. **Las dos series se analizan por separado**: no se promedian entre sí ni se atribuyen sus diferencias a una causa no verificada. No se afirma significancia estadística a partir de cinco mediciones por condición.

5.7.3. Análisis adicional: crecimiento más allá del millón

Se examina el posible comportamiento del sistema por encima del millón de registros y la suficiencia de la arquitectura cliente–servidor. **El mayor tamaño construido y medido fue de 1 000 000 de filas**; no se ejecutaron consultas sobre diez millones. Para disponer de una referencia cuantitativa, la Cuadro 42 multiplica *hipotéticamente* por diez la composición de la base máxima. No es una predicción del crecimiento real del catálogo del IGP ni una medición de rendimiento.

Relación	1 millón: medido	10 millones: supuesto
`estacion_sismica`	100	1 000
`sensor`	300	3 000
`evento_sismico`	120 000	1 200 000
`lectura_onda`	875 000	8 750 000
`deteccion`	3 500	35 000
`alerta`	1 100	11 000
Total	**1 000 000**	**10 000 000**

Cuadro 42: Volumen verificado y ejemplo hipotético de crecimiento proporcional.

El crecimiento afectaría de forma distinta a cada consulta. En C1, la selectividad del filtro temporal determinaría cuánto aporta el índice parcial. En C2, las búsquedas por rangos de latitud y longitud pueden producir numerosos candidatos aun usando un índice compuesto. C3 depende principalmente del volumen de lecturas que debe agrupar; C4 resume detecciones y alertas, por lo que un recorrido amplio puede seguir siendo preferible a búsquedas indexadas. En C5, la anti-unión y las búsquedas reiteradas por sensor pueden aumentar el costo incluso cuando PostgreSQL utiliza los índices propuestos.

No sería válido multiplicar por diez los milisegundos observados: pueden cambiar el plan, la tasa de aciertos en caché, los accesos a disco y la concurrencia. En particular,

el caso C5 muestra que incluso un plan que utiliza índices puede ser más lento que el plan sin ellos. Este análisis no supone que los 24 289 eventos reales del catálogo IGP se multiplicarán automáticamente; se requerirían nuevas fuentes verificadas o datos sintéticos identificados como tales.

¿Es suficiente cliente–servidor? PostgreSQL 18.6 respondió las cinco consultas con un millón de registros en el equipo descrito en 5.4. Esto respalda una arquitectura cliente–servidor como punto de partida, **no garantiza** latencias aceptables con diez millones ni con múltiples usuarios. Antes de cambiar de arquitectura correspondería (i) medir consultas y escrituras concurrentes con `EXPLAIN (ANALYZE, BUFFERS)` y estadísticas actualizadas, (ii) revisar índices y reescrituras de consultas, especialmente C5, y (iii) evaluar particionamiento temporal, vistas materializadas o más recursos únicamente si aparecen cuellos de botella medidos. Las referencias técnicas utilizadas son la documentación de https://www.postgresql.org/docs/18/using-explain.html y de https://www.postgresql.org/docs/18/ddl-partitioning.html.

5.7.4. Limitaciones y reproducibilidad

Las comparaciones se realizaron en un orden fijo (primero sin y luego con índices) y con sólo cinco mediciones válidas por condición. Los calentamientos, la caché y las distintas selectividades de las cuatro bases pueden influir en los tiempos; `shared hit` y `shared read` son contadores de búferes, no mediciones directas de operaciones físicas de disco. Los resultados son, por tanto, **descriptivos para esta carga y plataforma**, no una prueba causal o universal del beneficio de un índice.

Los cuatro archivos `.dump` se conservaron antes de crear los índices experimentales. Está documentada la restauración de la base de 1 000 registros en `bd_sismologico_restauracion_1000`, incluidos sus recuentos y controles de integridad; **no equivale a comprobar la restauración de los otros tres respaldos**. Los árboles reales de ejecución se presentan mediante extractos en la Sección 5.3; los diez SVG íntegros se conservan como respaldo. Esas verificaciones visuales posteriores no sustituyen las mediciones repetidas de la serie principal.

6. Conclusiones

1. **Implementación e integridad.** Se implementó en PostgreSQL un esquema de seis tablas consistente con el modelo relacional, con claves y restricciones de integridad. Las cinco vistas, seis procedimientos almacenados y nueve disparadores permiten consultar y controlar operaciones sobre el dominio sismológico. Las pruebas documentadas respaldan las reglas verificadas, sin extender ese resultado a todos los casos posibles.

2. **Procedencia y volumen de los datos.** Se integraron 24 289 eventos del catálogo oficial del Instituto Geofísico del Perú y registros operativos simulados, identificando claramente su procedencia. Se construyeron cuatro bases experimentales de 1 000, 10 000, 100 000 y 1 000 000 de registros, sin presentar los datos sintéticos como observaciones sismológicas reales.

3. **Optimización condicionada por la carga.** En la serie principal, los índices candidatos de C1 y C2 se utilizaron y mejoraron los tiempos desde 10 000 registros. En C3, el índice sólo fue utilizado en la escala de un millón, donde la media disminuyó de 2 013,4210 a 512,4504 ms (74,55 %). Los resultados evidencian que la utilidad de cada índice depende de la consulta, el volumen y el plan elegido por PostgreSQL.

4. **Ausencia de mejora automática.** C4 no utilizó sus índices candidatos en ninguna escala, por lo que la diferencia temporal observada no demuestra un beneficio de indexación. En C5, ambos índices fueron utilizados con un millón de registros, pero la media aumentó de 113,3538 a 202,8902 ms (78,99 %). La utilización de índices no equivale necesariamente a menor tiempo de respuesta.

5. **Alcance de la experimentación.** Se realizaron dos calentamientos y cinco mediciones por escenario en cada una de las veinte combinaciones de consulta y escala. Los tiempos son descriptivos para el equipo y los datos utilizados: el orden fijo de los escenarios, los efectos de caché y la falta de mediciones individuales de la primera serie limitan las inferencias. Las reejecuciones se mantienen separadas del consolidado principal.

6. **Validación del prototipo web.** El MVP de consulta en modo de sólo lectura se conectó localmente a la base experimental de un millón de registros y permitió ejecutar C1–C5 desde una interfaz web. Esta prueba demuestra integración funcional en el entorno utilizado, pero no constituye una validación de despliegue público, concurrencia o latencia de un sistema de alerta temprana real.

7. **Reproducibilidad y crecimiento.** Se publicaron cuatro respaldos experimentales y se documentó la restauración independiente del de 1 000 registros; no se verificó individualmente la restauración de los otros tres. El mayor volumen evaluado fue de un millón de registros. La operación con diez millones o con múltiples usuarios requiere pruebas adicionales antes de formular conclusiones sobre escalabilidad o infraestructura.

7. Anexos

7.1. Código fuente y reproducibilidad de la implementación

La implementación está disponible en el repositorio del proyecto. Se organiza en siete integradores SQL que permiten reproducir la estructura relacional, la carga de datos, la lógica de negocio y los experimentos de rendimiento.

Integrador	Responsabilidad
`01_estructura.sql`	Tablas, claves, restricciones y comprobaciones de integridad.
`02_datos.sql`	Carga del catálogo IGP, datos complementarios y verificaciones.
`03_logica_consultas.sql`	Vistas, procedimientos, triggers y cinco consultas intermedias.
`04_bases_experimentales.sql`	Preparación y validación de las cuatro escalas experimentales.
`05_planes_indices.sql`	Planes de ejecución y evaluación de índices por consulta.
`06_mediciones.sql`	Mediciones repetidas y controles de equivalencia de resultados.
`07_demostracion_video.sql`	Demostración reproducible en la escala de un millón de registros.

Cuadro 43: Integradores definitivos utilizados como referencia técnica del proyecto.

Para reproducir la implementación se utiliza `psql` y, en la etapa de carga, el CSV oficial del IGP obtenido de la fuente indicada en el repositorio. El integrador 07 presupone el estado de índices C5 generado por las pruebas finales; no se ejecuta directamente sobre un respaldo recién restaurado sin preparar dicho estado.

El archivo `59_resumen_global_5_6.csv` contiene las veinte comparaciones de la serie principal. La segunda serie experimental consta en `auditoria_dos_series_20_comparaciones.csv` y se analizó por separado, sin combinar sus medias con las de la serie principal.

7.2. Respaldo de las cuatro bases experimentales

Se generaron cuatro respaldos en formato personalizado mediante `pg_dump -Fc` tras verificar los recuentos e integridad de cada base. Los archivos conservan el estado anterior a la incorporación de los índices secundarios de las pruebas de rendimiento. Los cuatro respaldos están publicados como archivos adjuntos de la versión v1.0.0 en GitHub Releases.

Escala	Archivo de respaldo
1 000	`bd_sismologico_exp_1000.dump`
10 000	`bd_sismologico_exp_10000.dump`
100 000	`bd_sismologico_exp_100000.dump`
1 000 000	`bd_sismologico_exp_1000000.dump`

Cuadro 44: Respaldos de las cuatro bases experimentales publicados en la versión v1.0.0.

Restauración independiente verificada. La prueba de restauración y la verificación de integridad se ejecutaron sobre el respaldo `bd_sismologico_exp_1000.dump`, sin sobrescribir las bases experimentales. El procedimiento terminó correctamente. `pg_restore` reconoció el archivo en formato personalizado y finalizó sin errores dentro de una única transacción. La restauración se realizó sobre la base independiente `bd_sismologico_restauracion_1000`.

Relación	Esperado	Restaurado	Control
`estacion_sismica`	5	5	OK
`sensor`	15	15	OK
`evento_sismico`	100	100	OK
`lectura_onda`	750	750	OK
`deteccion`	90	90	OK
`alerta`	40	40	OK
Total	**1 000**	**1 000**	**OK**

Cuadro 45: Recuentos de relaciones tras restaurar el respaldo de 1 000 registros.

El verificador informó **50 eventos del IGP y 50 eventos sintéticos**, **9 triggers** y **cero incumplimientos** en los cuatro controles: estaciones sin sensores, detecciones sin lectura válida, alertas anteriores a su evento y lecturas válidas de sensores inoperativos. Además, no se encontraron detecciones ni alertas vinculadas a los eventos reales del IGP. La copia restaurada ocupó **8 534 kB** al finalizar.

El script reportó para el archivo original la siguiente huella **SHA-256**, útil para comprobar su identidad:

```
FDB9280617F3B20CD3229B671CD8AF06
ADFD08094E3F4003A13FA786B42D90F7
```

La salida terminó con el aviso *«OK: respaldo restaurado y verificado en base independiente»* y confirmó que las bases `bd_sismologico_exp_1000`, `bd_sismologico_exp_10000`, `bd_sismologico_exp_100000` y `bd_sismologico_exp_1000000` no fueron alteradas. **Alcance:** este resultado certifica la recuperación del respaldo de 1 000 registros; la recuperación de los otros tres archivos todavía no se ha comprobado individualmente.

7.3. Demostración SQL sobre un millón de registros

Ejecución demostrativa. Se ejecutó `sql/07_demostracion_video.sql` sobre `bd_sismologico_exp_1000000`. El control inicial verificó la distribución de **1 000 000 de registros** en seis tablas y la presencia exclusiva de los dos índices secundarios de C5. Se ejecutó una vez cada consulta en ambos escenarios; los diez resultados figuran en el Cuadro 46.

Consulta	Sin índice	Con índice	Índice candidato usado
C1	13,690	0,105	Sí, parcial
C2	46,298	11,090	Sí, geográfico
C3	1 819,563	567,325	Sí, cobertura
C4	1,543	1,435	Ninguno
C5	107,379	231,111	Ambos

Cuadro 46: Tiempos de la ejecución demostrativa del integrador 07, con un millón de registros (ms).

La demostración de C3 informó **7 501 bloques temporales leídos** y **7 516 escritos** en el plan sin índice, frente a cero en el plan indexado. En C4, los dos índices candidatos no fueron usados. En C5, ambos se utilizaron, aunque el tiempo demostrativo fue mayor con ellos. Los diez tiempos anteriores corresponden a *una sola ejecución por escenario*; no reemplazan las medias de cinco mediciones de la Sección 5.5.

Al finalizar, el archivo devolvió `ROLLBACK` y una consulta posterior confirmó que permanecían exclusivamente los índices `idx_det_sensor_hora_c5` y `idx_lectura_sensor_hora_c5`. Por tanto, la demostración SQL y el restablecimiento del estado original quedaron **verificados**.

Organización de la evidencia audiovisual. La evidencia audiovisual se organiza en un único recorrido por la base `bd_sismologico_exp_1000000`: verificación de los seis recuentos, consultas C1–C5 sin índices secundarios y con índices candidatos, lectura de `Execution Time` y verificación final de la restauración del estado mediante `ROLLBACK`. Las diez condiciones se identifican mediante marcas de tiempo para facilitar su consulta, sin sustituir las cinco mediciones repetidas de cada escenario presentadas en el informe. La carpeta de acceso a la evidencia audiovisual es Video de experimentación en Google Drive.

7.4. Prototipo web de consultas

Se preparó el código fuente de `MVP_Web_Sismologico`, una aplicación web de consulta que integra un servidor HTTP en `Node.js`, el controlador `pg` y una interfaz HTML, CSS y JavaScript. El servidor se conecta a PostgreSQL a través de la configuración privada `.env`; el navegador no recibe la contraseña. La implementación utiliza las cinco sentencias `SELECT` del integrador `03_logica_consultas.sql`, sin alterar sus filtros ni los identificadores correctos de la Consulta 4.

La interfaz incorpora una pantalla de bienvenida y un menú lateral plegable, que facilita la visualización de resultados en pantallas de distinta anchura. Permite visualizar

los recuentos de las seis tablas, seleccionar cualquiera de las cinco consultas intermedias, inspeccionar los resultados en una tabla, filtrar, ordenar, representar una métrica y exportar las filas visibles a CSV. Los servicios disponibles son `GET /api/health`, `GET /api/overview`, `GET /api/queries` y `GET /api/queries/C1` hasta `C5`. Las consultas se ejecutan mediante transacciones `BEGIN READ ONLY`, con tiempo máximo configurado y `ROLLBACK` al terminar. No se admiten sentencias SQL arbitrarias proporcionadas por el navegador ni operaciones de inserción, actualización o eliminación.

Validación técnica y funcional. Se verificó la sintaxis del servidor y del cliente, se contrastaron las cinco sentencias SQL con su fuente original y se aprobaron pruebas automatizadas del manejo HTTP y del modo de solo lectura mediante un controlador simulado. Posteriormente, se inició la aplicación en el equipo local y se confirmó la conexión con `bd_sismologico_exp_1000000`, la base experimental realizada en PostgreSQL con exactamente 1 000 000 de registros. El panel mostró 24 289 eventos provenientes del archivo del IGP, 875 000 lecturas y 300 sensores; dichas cantidades coincidieron con las verificaciones SQL de la base experimental.

Resultados de las consultas desde la interfaz web. Las pruebas locales permitieron observar los resultados de C1–C4 y exportar la respuesta de C5 a un archivo CSV. Se obtuvieron los siguientes resultados:

Consulta	Filas	Respuesta web	Resultado observado
C1	15	8,95 ms	Actividad sísmica anual agrupada.
C2	10	286,21 ms	Eventos clasificados por ventana geográfica.
C3	100	332,59 ms	Indicadores de validez de lecturas por estación.
C4	15	7,96 ms	Detecciones y alertas de eventos simulados.
C5	15	185,14 ms	Exportación CSV: 15 sensores distintos, 43 755 lecturas y 2 580 lecturas inválidas.

Cuadro 47: Resultados de las consultas C1–C5 desde el prototipo web local.

El archivo `C5_resultados_mvp.csv` fue comprobado: contiene 15 registros con 15 identificadores de sensor distintos, y las sumas de las columnas de lecturas e inválidas corresponden a los totales de la Consulta 5 documentados en la experimentación. Las cinco consultas demostraron funcionamiento conectado a la base local, con acceso `READ ONLY`, y se verificó también la exportación de resultados a CSV.

Los tiempos de *respuesta del servidor* de la aplicación web no se equiparan directamente con `Execution Time` de `EXPLAIN ANALYZE`: corresponden a mecanismos de medición y capas de ejecución diferentes. Los tiempos de detección y alerta observados en C4 proceden de registros simulados y no constituyen una evaluación de latencia operativa de un sistema público de alerta temprana.

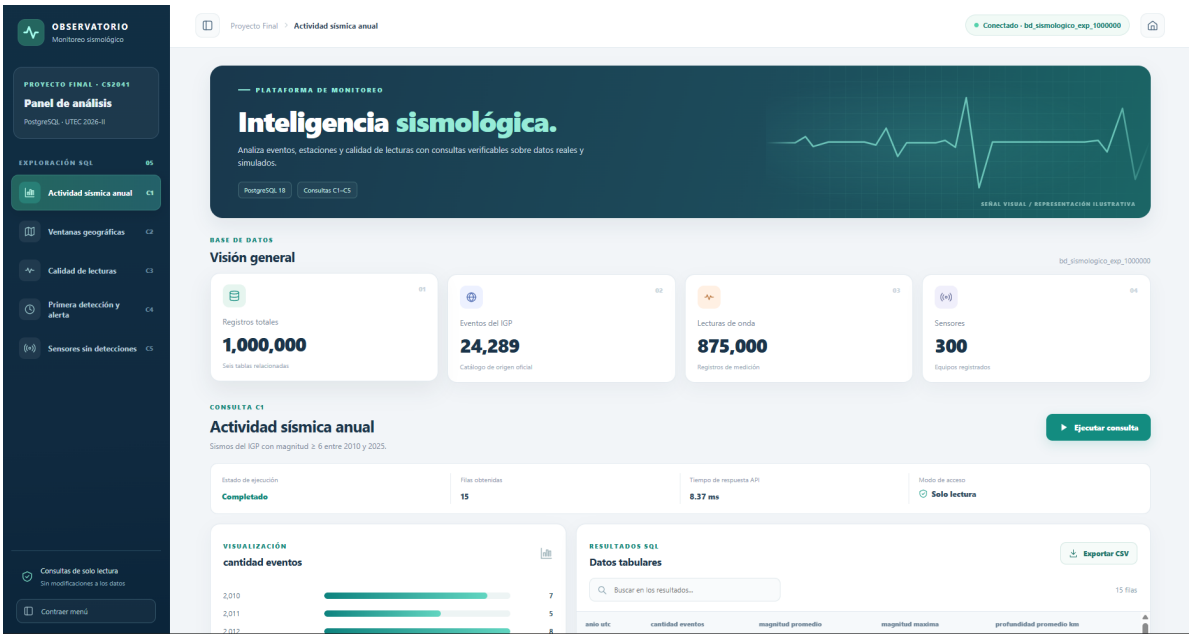


Figura 13: Panel principal del MVP web, conectado a la base experimental de un millón de registros. Se muestran el resumen de la base y la consulta C1 ejecutada.

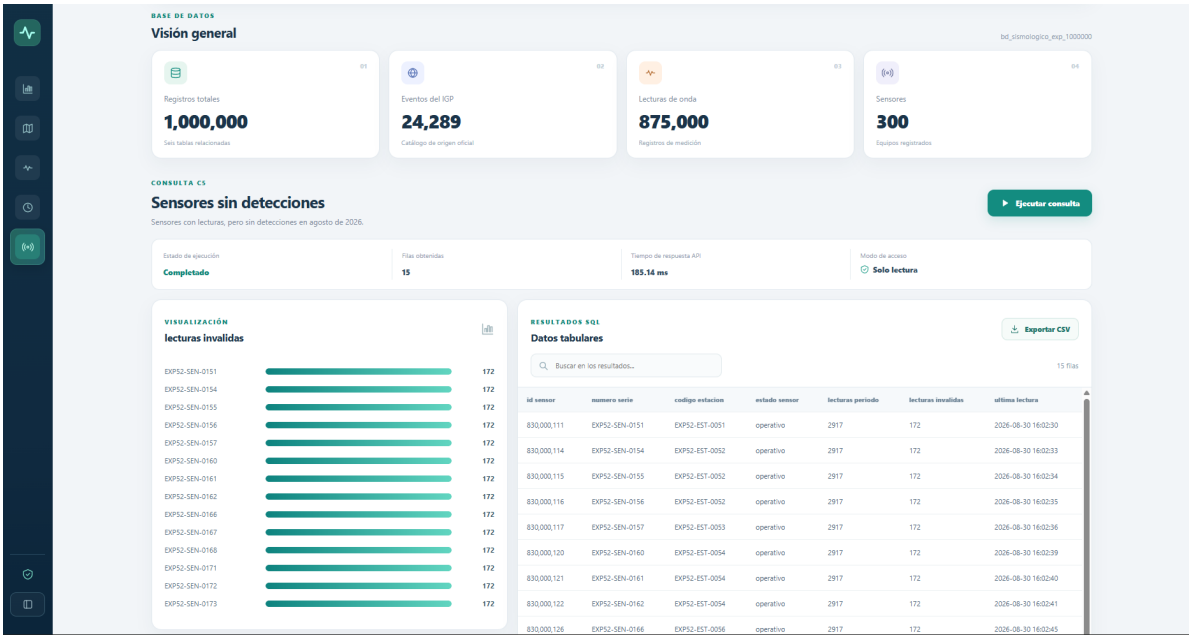


Figura 14: Consulta C5 ejecutada en el MVP, con quince sensores identificados, visualización de lecturas inválidas y tabla de resultados.

7.5. Verificación tabular de la consulta geográfica

La Figura 15 registra la ejecución original de C2 en la base temporal de comprobación `bd_sismologico_prueba_exp_1000000`, distinta de la base experimental principal, con las diez filas devueltas por la consulta y los campos de identificación, cantidad de eventos y magnitudes.

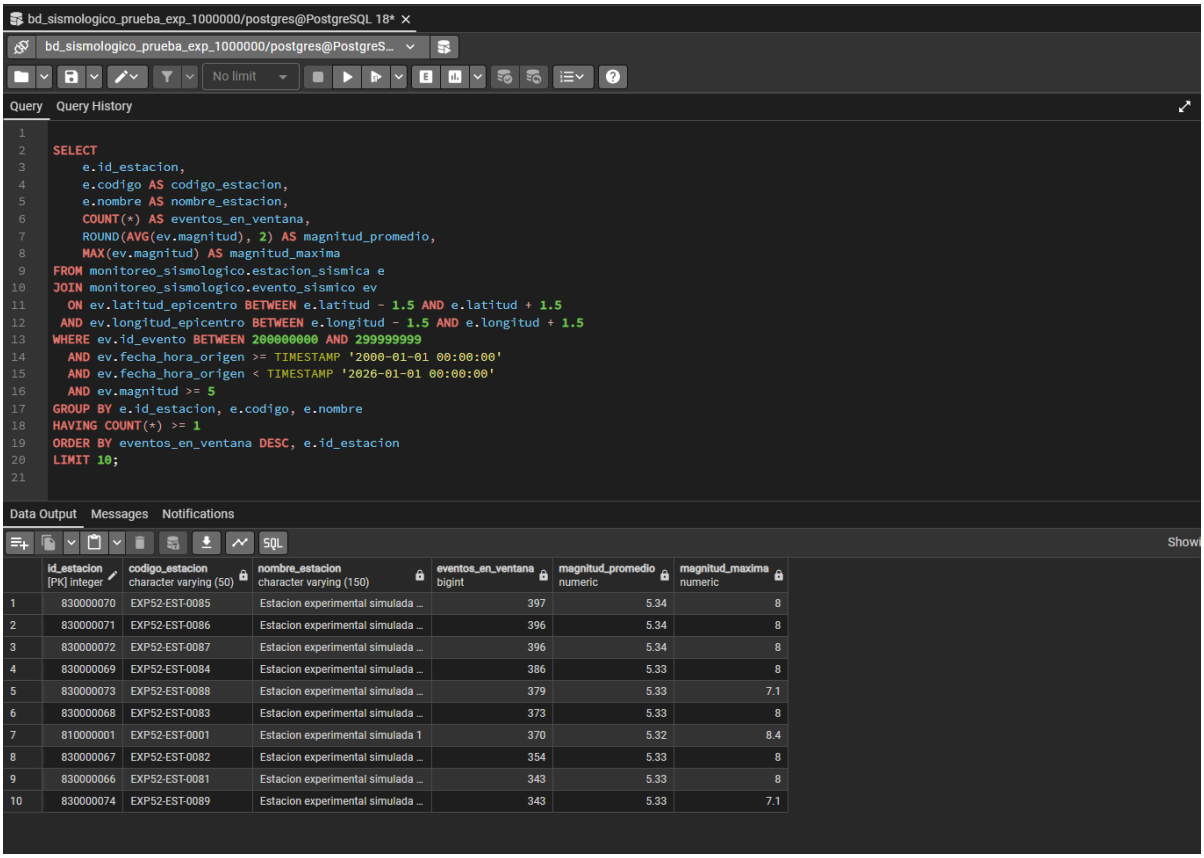


Figura 15: Consulta C2 en pgAdmin: resultados de ventanas geográficas en una copia de prueba de la escala de un millón de registros.

7.6. Código SQL de la implementación y los experimentos

Los siguientes listados reúnen los integradores SQL de la estructura, carga de datos, lógica de negocio, experimentación y demostración. Los fundamentos de las operaciones y el análisis de resultados se desarrollan en los Capítulos 4 y 5. El repositorio contiene los módulos complementarios y los archivos de resultados correspondientes.

7.6.1. Estructura y restricciones

```
-- 01. ESTRUCTURA DE LA BASE DE DATOS
\set ON_ERROR_STOP on
\pset pager off

\echo '=== ESTRUCTURA: VALIDACION DE BASE ==='
SELECT current_database() AS base_actual;

DO $$
BEGIN
    IF current_database() IN ('postgres', 'template0', 'template1') THEN
        RAISE EXCEPTION 'Seleccione una base de proyecto vacia, no una base del sistema.';
    END IF;

    IF EXISTS (
        SELECT 1
        FROM pg_class c
        JOIN pg_namespace n ON n.oid = c.relnamespace
        WHERE n.nspname = 'monitoreo_sismologico'
          AND c.relkind IN ('r', 'p', 'v', 'm', 'f', 'S')
    ) THEN
        RAISE EXCEPTION 'El esquema ya contiene objetos. No se modifico ningun dato.';
    END IF;
END;
$$;

\echo '=== ESTRUCTURA: CREACION DE TABLAS Y RESTRICCIONES ==='
BEGIN;
CREATE SCHEMA IF NOT EXISTS monitoreo_sismologico;
SET LOCAL search_path TO monitoreo_sismologico, public;

-- Tablas
CREATE TABLE estacion_sismica (
    id_estacion INTEGER GENERATED BY DEFAULT AS IDENTITY,
    codigo VARCHAR(50) NOT NULL,
    nombre VARCHAR(150) NOT NULL,
    latitud NUMERIC NOT NULL,
    longitud NUMERIC NOT NULL,
    altitud NUMERIC,
    departamento VARCHAR(100),
    provincia VARCHAR(100),
    distrito VARCHAR(100),
    fecha_instalacion DATE,
    estado VARCHAR(40) NOT NULL
);

CREATE TABLE sensor (
    id_sensor INTEGER GENERATED BY DEFAULT AS IDENTITY,
    numero_serie VARCHAR(80) NOT NULL,
    tipo VARCHAR(80) NOT NULL,
    modelo VARCHAR(100),
    unidad_medida VARCHAR(40) NOT NULL,
    frecuencia_muestreo NUMERIC NOT NULL,
    fecha_instalacion DATE NOT NULL,
    fecha_ultimo_mantenimiento DATE,
    estado VARCHAR(40) NOT NULL,
    id_estacion INTEGER NOT NULL
);

CREATE TABLE lectura_onda (
    id_lectura BIGINT GENERATED BY DEFAULT AS IDENTITY,
    id_sensor INTEGER NOT NULL,
    fecha_hora TIMESTAMP NOT NULL,
```

```
    canal VARCHAR(40) NOT NULL,
    valor NUMERIC NOT NULL,
    es_valida BOOLEAN NOT NULL
);

CREATE TABLE evento_sismico (
    id_evento INTEGER GENERATED BY DEFAULT AS IDENTITY,
    fecha_hora_origen TIMESTAMP NOT NULL,
    latitud_epicentro NUMERIC NOT NULL,
    longitud_epicentro NUMERIC NOT NULL,
    profundidad_km NUMERIC NOT NULL,
    magnitud NUMERIC NOT NULL,
    tipo_magnitud VARCHAR(30),
    intensidad_maxima VARCHAR(40),
    referencia_geografica VARCHAR(250),
    estado_revision VARCHAR(40) NOT NULL
);

CREATE TABLE deteccion (
    id_sensor INTEGER NOT NULL,
    id_evento INTEGER NOT NULL,
    hora_deteccion TIMESTAMP NOT NULL,
    aceleracion_maxima NUMERIC,
    calidad VARCHAR(40)
);

CREATE TABLE alerta (
    id_alerta INTEGER GENERATED BY DEFAULT AS IDENTITY,
    id_evento INTEGER NOT NULL,
    fecha_hora_emision TIMESTAMP NOT NULL,
    nivel VARCHAR(40) NOT NULL,
    mensaje TEXT NOT NULL,
    magnitud_estimada NUMERIC,
    estado VARCHAR(40) NOT NULL
);

-- Claves primarias
ALTER TABLE estacion_sismica
    ADD CONSTRAINT pk_estacion_sismica PRIMARY KEY (id_estacion);

ALTER TABLE sensor
    ADD CONSTRAINT pk_sensor PRIMARY KEY (id_sensor);

ALTER TABLE lectura_onda
    ADD CONSTRAINT pk_lectura_onda PRIMARY KEY (id_lectura);

ALTER TABLE evento_sismico
    ADD CONSTRAINT pk_evento_sismico PRIMARY KEY (id_evento);

ALTER TABLE deteccion
    ADD CONSTRAINT pk_deteccion PRIMARY KEY (id_sensor, id_evento);

ALTER TABLE alerta
    ADD CONSTRAINT pk_alerta PRIMARY KEY (id_alerta);

-- Claves foraneas
ALTER TABLE sensor
    ADD CONSTRAINT fk_sensor_estacion
    FOREIGN KEY (id_estacion) REFERENCES estacion_sismica(id_estacion);

ALTER TABLE lectura_onda
    ADD CONSTRAINT fk_lectura_sensor
    FOREIGN KEY (id_sensor) REFERENCES sensor(id_sensor);

ALTER TABLE deteccion
    ADD CONSTRAINT fk_deteccion_sensor
    FOREIGN KEY (id_sensor) REFERENCES sensor(id_sensor),
    ADD CONSTRAINT fk_deteccion_evento
    FOREIGN KEY (id_evento) REFERENCES evento_sismico(id_evento);

ALTER TABLE alerta
    ADD CONSTRAINT fk_alerta_evento
    FOREIGN KEY (id_evento) REFERENCES evento_sismico(id_evento);

-- Restricciones de unicidad
ALTER TABLE estacion_sismica
    ADD CONSTRAINT uq_estacion_codigo UNIQUE (codigo);
```

```
ALTER TABLE sensor
    ADD CONSTRAINT uq_sensor_numero_serie UNIQUE (numero_serie);

ALTER TABLE lectura_onda
    ADD CONSTRAINT uq_lectura_sensor_fecha_canal
    UNIQUE (id_sensor, fecha_hora, canal);

-- Restricciones de dominio
ALTER TABLE estacion_sismica
    ADD CONSTRAINT ck_estacion_latitud CHECK (latitud BETWEEN -90 AND 90),
    ADD CONSTRAINT ck_estacion_longitud CHECK (longitud BETWEEN -180 AND 180);

ALTER TABLE sensor
    ADD CONSTRAINT ck_sensor_frecuencia CHECK (frecuencia_muestreo > 0),
    ADD CONSTRAINT ck_sensor_mantenimiento
    CHECK (fecha_ultimo_mantenimiento IS NULL
           OR fecha_ultimo_mantenimiento >= fecha_instalacion);

ALTER TABLE evento_sismico
    ADD CONSTRAINT ck_evento_latitud
    CHECK (latitud_epicentro BETWEEN -90 AND 90),
    ADD CONSTRAINT ck_evento_longitud
    CHECK (longitud_epicentro BETWEEN -180 AND 180),
    ADD CONSTRAINT ck_evento_profundidad CHECK (profundidad_km >= 0);

ALTER TABLE deteccion
    ADD CONSTRAINT ck_deteccion_aceleracion
    CHECK (aceleracion_maxima IS NULL OR aceleracion_maxima >= 0);

COMMIT;

\echo '=== ESTRUCTURA: COMPROBACION ==='
SELECT table_name AS tabla
FROM information_schema.tables
WHERE table_schema = 'monitoreo_sismologico'
  AND table_type = 'BASE TABLE'
ORDER BY table_name;

SELECT CASE contype
           WHEN 'p' THEN 'PRIMARY KEY'
           WHEN 'f' THEN 'FOREIGN KEY'
           WHEN 'u' THEN 'UNIQUE'
           WHEN 'c' THEN 'CHECK'
           WHEN 'n' THEN 'NOT NULL'
       END AS restriccion, COUNT(*) AS cantidad
FROM pg_constraint
WHERE connamespace = 'monitoreo_sismologico'::regnamespace
GROUP BY contype
ORDER BY restriccion;

DO $$
BEGIN
    IF (SELECT COUNT(*) FROM information_schema.tables
        WHERE table_schema = 'monitoreo_sismologico'
          AND table_type = 'BASE TABLE') <> 6 THEN
        RAISE EXCEPTION 'No se encontraron las seis tablas esperadas.';
    END IF;

    IF (SELECT COUNT(*) FROM pg_constraint
        WHERE connamespace = 'monitoreo_sismologico'::regnamespace
          AND contype = 'p') <> 6
       OR (SELECT COUNT(*) FROM pg_constraint
        WHERE connamespace = 'monitoreo_sismologico'::regnamespace
          AND contype = 'f') <> 5
       OR (SELECT COUNT(*) FROM pg_constraint
        WHERE connamespace = 'monitoreo_sismologico'::regnamespace
          AND contype = 'u') <> 3
       OR (SELECT COUNT(*) FROM pg_constraint
        WHERE connamespace = 'monitoreo_sismologico'::regnamespace
          AND contype = 'c') <> 8 THEN
        RAISE EXCEPTION 'Las restricciones no coinciden con el modelo.';
    END IF;

    RAISE NOTICE 'OK: seis tablas y restricciones comprobadas.';
END;
$$;
```

```
\echo '=== ESTRUCTURA: PRUEBAS DE INTEGRIDAD ==='
BEGIN;
SET LOCAL search_path TO monitoreo_sismologico, public;

INSERT INTO estacion_sismica (
    id_estacion, codigo, nombre, latitud, longitud, estado
) VALUES (900000001, 'TEST-EST-01', 'Estacion de prueba', -12.1, -77.1, 'operativa');

INSERT INTO sensor (
    id_sensor, numero_serie, tipo, unidad_medida, frecuencia_muestreo,
    fecha_instalacion, estado, id_estacion
) VALUES (900000001, 'TEST-SEN-01', 'acelerometro', 'm/s2', 100,
          DATE '2026-01-01', 'operativo', 900000001);

INSERT INTO lectura_onda (
    id_lectura, id_sensor, fecha_hora, canal, valor, es_valida
) VALUES (900000001, 900000001, TIMESTAMP '2026-01-02 00:00:00', 'X', 0.25, TRUE);

INSERT INTO evento_sismico (
    id_evento, fecha_hora_origen, latitud_epicentro,
    longitud_epicentro, profundidad_km, magnitud, estado_revision
) VALUES (900000001, TIMESTAMP '2026-01-02 00:00:00',
          -12.2, -77.2, 10, -0.5, 'preliminar');

INSERT INTO deteccion (
    id_sensor, id_evento, hora_deteccion, aceleracion_maxima, calidad
) VALUES (900000001, 900000001, TIMESTAMP '2026-01-02 00:00:01', 0.25, 'alta');

INSERT INTO alerta (
    id_alerta, id_evento, fecha_hora_emision,
    nivel, mensaje, magnitud_estimada, estado
) VALUES (900000001, 900000001, TIMESTAMP '2026-01-02 00:00:05',
          'informativo', 'Prueba de alerta', -0.5, 'emitida');

-- Inserciones de prueba
SELECT 'estacion_sismica' AS tabla, COUNT(*) AS registros
FROM estacion_sismica WHERE id_estacion = 900000001
UNION ALL
SELECT 'sensor', COUNT(*) FROM sensor WHERE id_sensor = 900000001
UNION ALL
SELECT 'lectura_onda', COUNT(*) FROM lectura_onda WHERE id_lectura = 900000001
UNION ALL
SELECT 'evento_sismico', COUNT(*) FROM evento_sismico WHERE id_evento = 900000001
UNION ALL
SELECT 'deteccion', COUNT(*) FROM deteccion
WHERE id_sensor = 900000001 AND id_evento = 900000001
UNION ALL
SELECT 'alerta', COUNT(*) FROM alerta WHERE id_alerta = 900000001;

-- Rechazos esperados
DO $$
BEGIN
    BEGIN
        INSERT INTO estacion_sismica (
            id_estacion, codigo, nombre, latitud, longitud, estado
        ) VALUES (900000002, 'TEST-EST-02', 'Error latitud', -95, -77, 'operativa');
        RAISE EXCEPTION 'FALLO: no se rechazo la latitud invalida';
    EXCEPTION WHEN check_violation THEN
        RAISE NOTICE 'OK: latitud fuera de rango rechazada';
    END;

    BEGIN
        INSERT INTO sensor (
            id_sensor, numero_serie, tipo, unidad_medida, frecuencia_muestreo,
            fecha_instalacion, estado, id_estacion
        ) VALUES (900000002, 'TEST-SEN-02', 'acelerometro', 'm/s2', 0,
                  DATE '2026-01-01', 'operativo', 900000001);
        RAISE EXCEPTION 'FALLO: no se rechazo frecuencia 0';
    EXCEPTION WHEN check_violation THEN
        RAISE NOTICE 'OK: frecuencia 0 rechazada';
    END;

    BEGIN
        INSERT INTO lectura_onda (
            id_lectura, id_sensor, fecha_hora, canal, valor, es_valida
        ) VALUES (900000002, 900000001,
```

```
                TIMESTAMP '2026-01-02 00:00:00', 'X', 0.4, TRUE);
        RAISE EXCEPTION 'FALLO: se admitio lectura duplicada';
    EXCEPTION WHEN unique_violation THEN
        RAISE NOTICE 'OK: duplicado sensor-fecha-canal rechazado';
    END;

    BEGIN
        INSERT INTO alerta (
            id_alerta, id_evento, fecha_hora_emision,
            nivel, mensaje, estado
        ) VALUES (900000002, 900000002,
                TIMESTAMP '2026-01-02 00:00:05', 'informativo',
                'Evento inexistente', 'emitida');
        RAISE EXCEPTION 'FALLO: se admitio un evento inexistente';
    EXCEPTION WHEN foreign_key_violation THEN
        RAISE NOTICE 'OK: referencia inexistente rechazada';
    END;

    BEGIN
        INSERT INTO deteccion (
            id_sensor, id_evento, hora_deteccion, aceleracion_maxima
        ) VALUES (900000001, 900000001,
                TIMESTAMP '2026-01-02 00:00:02', 0.5);
        RAISE EXCEPTION 'FALLO: se admitio deteccion duplicada';
    EXCEPTION WHEN unique_violation THEN
        RAISE NOTICE 'OK: llave primaria compuesta comprobada';
    END;
END;
$$;

ROLLBACK;


\echo '=== ESTRUCTURA: COMPROBACION POSTERIOR ==='
SELECT 'estacion_sismica' AS tabla,
       COUNT(*) AS registros_prueba
FROM monitoreo_sismologico.estacion_sismica
WHERE id_estacion = 900000001
UNION ALL
SELECT 'sensor', COUNT(*)
FROM monitoreo_sismologico.sensor
WHERE id_sensor = 900000001
UNION ALL
SELECT 'lectura_onda', COUNT(*)
FROM monitoreo_sismologico.lectura_onda
WHERE id_lectura = 900000001
UNION ALL
SELECT 'evento_sismico', COUNT(*)
FROM monitoreo_sismologico.evento_sismico
WHERE id_evento = 900000001
UNION ALL
SELECT 'deteccion', COUNT(*)
FROM monitoreo_sismologico.deteccion
WHERE id_sensor = 900000001 AND id_evento = 900000001
UNION ALL
SELECT 'alerta', COUNT(*)
FROM monitoreo_sismologico.alerta
WHERE id_alerta = 900000001;

\echo '=== ETAPA 1 FINALIZADA ==='
```

Listing 1: Integrador 01: estructura y restricciones.

7.6.2. Carga, catálogo IGP y simulación

```
-- 02. CARGA Y SIMULACION DE DATOS
-- CSV oficial: datos/igp_catalogo.csv (UTF-8, separador ;).
\set ON_ERROR_STOP on
\pset pager off

\echo '=== DATOS: BASE DE TRABAJO ==='
SELECT current_database() AS base_actual;
SET search_path TO monitoreo_sismologico, public;
```

```
-- Base con seis tablas y sin registros.
DO $$
DECLARE
    objeto TEXT;
    cantidad BIGINT;
BEGIN
    IF current_database() IN ('postgres', 'template0', 'template1') THEN
        RAISE EXCEPTION 'Seleccione una base de trabajo, no una base del sistema.';
    END IF;

    FOREACH objeto IN ARRAY ARRAY[
        'estacion_sismica', 'sensor', 'lectura_onda',
        'evento_sismico', 'deteccion', 'alerta'
    ] LOOP
        IF to_regclass(format('monitoreo_sismologico.%I', objeto)) IS NULL THEN
            RAISE EXCEPTION 'Falta la tabla % en monitoreo_sismologico.', objeto;
        END IF;
        EXECUTE format('SELECT COUNT(*) FROM monitoreo_sismologico.%I', objeto) INTO cantidad;
        IF cantidad <> 0 THEN
            RAISE EXCEPTION 'La tabla % tiene % filas. La carga requiere tablas vacias.', objeto, cantidad;
        END IF;
    END LOOP;
    RAISE NOTICE 'OK: seis tablas vacias.';
END;
$$;

BEGIN;
SET LOCAL search_path TO monitoreo_sismologico, public;


\echo '=== CARGA INICIAL ==='
DO $$
BEGIN
    IF EXISTS (SELECT 1 FROM estacion_sismica WHERE id_estacion BETWEEN 810000001 AND 810000999)
       OR EXISTS (SELECT 1 FROM sensor WHERE id_sensor BETWEEN 810000001 AND 810000999)
       OR EXISTS (SELECT 1 FROM lectura_onda WHERE id_lectura BETWEEN 810000001 AND 810000999)
       OR EXISTS (SELECT 1 FROM evento_sismico WHERE id_evento BETWEEN 810000001 AND 810000999)
       OR EXISTS (SELECT 1 FROM deteccion WHERE id_sensor BETWEEN 810000001 AND 810000999)
       OR EXISTS (SELECT 1 FROM alerta WHERE id_alerta BETWEEN 810000001 AND 810000999) THEN
        RAISE EXCEPTION 'La carga inicial ya existe. No repetir la importacion.';
    END IF;
END;
$$;

INSERT INTO estacion_sismica (id_estacion, codigo, nombre, latitud, longitud, altitud, departamento, provincia,
     distrito, fecha_instalacion, estado) VALUES
    (810000001, 'SIM42-EST-01', 'Estacion Simulada Costa Norte', -6.9, -79.7, 65, 'Lambayeque', 'Chiclayo', 'Chiclayo
     ', '2023-03-15', 'operativa'),
    (810000002, 'SIM42-EST-02', 'Estacion Simulada Costa Centro', -12.05, -77.05, 130, 'Lima', 'Lima', 'Lima', '
     2023-03-15', 'operativa'),
    (810000003, 'SIM42-EST-03', 'Estacion Simulada Sierra Sur', -16.4, -71.5, 2350, 'Arequipa', 'Arequipa', 'Arequipa
     ', '2023-03-15', 'operativa'),
    (810000004, 'SIM42-EST-04', 'Estacion Simulada Sierra Centro', -11.99, -75.1, 3250, 'Junin', 'Huancayo', '
     Huancayo', '2023-03-15', 'operativa'),
    (810000005, 'SIM42-EST-05', 'Estacion Simulada Selva Alta', -9.5, -75.9, 1700, 'Huanuco', 'Huanuco', 'Huanuco', '
     2023-03-15', 'operativa');

INSERT INTO sensor (id_sensor, numero_serie, tipo, modelo, unidad_medida, frecuencia_muestreo, fecha_instalacion,
     fecha_ultimo_mantenimiento, estado, id_estacion) VALUES
    (810000001, 'SIM42-SEN-01', 'acelerometro', 'Modelo-SIM-2', 'm/s2', 50, '2024-06-01', NULL, 'operativo',
     810000001),
    (810000002, 'SIM42-SEN-02', 'acelerometro', 'Modelo-SIM-3', 'm/s2', 100, '2024-06-01', NULL, 'operativo',
     810000001),
    (810000003, 'SIM42-SEN-03', 'acelerometro', 'Modelo-SIM-1', 'm/s2', 50, '2024-06-01', NULL, 'operativo',
     810000002),
    (810000004, 'SIM42-SEN-04', 'acelerometro', 'Modelo-SIM-2', 'm/s2', 100, '2024-06-01', '2026-09-12', 'operativo',
      810000002),
    (810000005, 'SIM42-SEN-05', 'acelerometro', 'Modelo-SIM-3', 'm/s2', 50, '2024-06-01', NULL, 'operativo',
     810000003),
    (810000006, 'SIM42-SEN-06', 'acelerometro', 'Modelo-SIM-1', 'm/s2', 100, '2024-06-01', NULL, 'operativo',
     810000003),
    (810000007, 'SIM42-SEN-07', 'acelerometro', 'Modelo-SIM-2', 'm/s2', 50, '2024-06-01', '2026-09-12', 'operativo',
     810000004),
    (810000008, 'SIM42-SEN-08', 'acelerometro', 'Modelo-SIM-3', 'm/s2', 100, '2024-06-01', NULL, 'operativo',
     810000004),
```

```
    (810000009, 'SIM42-SEN-09', 'acelerometro', 'Modelo-SIM-1', 'm/s2', 50, '2024-06-01', NULL, 'mantenimiento',
     810000005),
    (810000010, 'SIM42-SEN-10', 'acelerometro', 'Modelo-SIM-2', 'm/s2', 100, '2024-06-01', NULL, 'fuera_servicio',
     810000005);

INSERT INTO lectura_onda (id_lectura, id_sensor, fecha_hora, canal, valor, es_valida) VALUES
    (810000001, 810000001, '2026-10-26 16:22:03', 'X', 0.18, TRUE),
    (810000002, 810000002, '2026-10-26 16:22:06', 'X', 0.22, TRUE),
    (810000003, 810000003, '2026-10-26 16:22:09', 'X', 0.26, TRUE),
    (810000004, 810000004, '2026-10-28 00:29:12', 'X', 0.32, TRUE),
    (810000005, 810000005, '2026-10-28 00:29:15', 'X', 0.36, TRUE),
    (810000006, 810000002, '2026-10-29 08:36:06', 'X', 0.26, TRUE),
    (810000007, 810000006, '2026-10-29 08:36:18', 'X', 0.42, TRUE),
    (810000008, 810000003, '2026-10-30 16:43:09', 'X', 0.32, TRUE),
    (810000009, 810000007, '2026-10-30 16:43:21', 'X', 0.48, TRUE),
    (810000010, 810000004, '2026-11-01 00:50:12', 'X', 0.38, TRUE),
    (810000011, 810000008, '2026-11-01 00:50:24', 'X', 0.54, TRUE),
    (810000012, 810000005, '2026-11-02 08:57:15', 'X', 0.44, TRUE),
    (810000013, 810000001, '2026-11-03 17:04:03', 'X', 0.3, TRUE),
    (810000014, 810000006, '2026-11-03 17:04:18', 'X', 0.5, TRUE),
    (810000015, 810000001, '2026-10-25 02:00:00', 'X', -0.5198, FALSE),
    (810000016, 810000001, '2026-10-25 14:00:03', 'Y', 0.254, TRUE),
    (810000017, 810000001, '2026-10-26 02:00:06', 'Z', -0.4886, TRUE),
    (810000018, 810000001, '2026-10-26 14:00:09', 'X', 0.4804, TRUE),
    (810000019, 810000001, '2026-10-27 02:00:12', 'Y', -0.5624, TRUE),
    (810000020, 810000001, '2026-10-27 14:00:15', 'Z', 0.1608, TRUE),
    (810000021, 810000001, '2026-10-28 02:00:18', 'X', -0.4622, TRUE),
    (810000022, 810000001, '2026-10-28 14:00:21', 'Y', 0.5153, TRUE),
    (810000023, 810000001, '2026-10-29 02:00:24', 'Z', -0.4615, TRUE),
    (810000024, 810000001, '2026-10-29 14:00:27', 'X', 0.2279, FALSE),
    (810000025, 810000001, '2026-10-30 02:00:30', 'Y', 0.3469, TRUE),
    (810000026, 810000001, '2026-10-30 14:00:33', 'Z', -0.0614, TRUE),
    (810000027, 810000001, '2026-10-31 02:00:36', 'X', 0.4053, TRUE),
    (810000028, 810000001, '2026-10-31 14:00:39', 'Y', -0.4418, TRUE),
    (810000029, 810000001, '2026-11-01 02:00:42', 'Z', -0.6627, TRUE),
    (810000030, 810000001, '2026-11-01 14:00:45', 'X', -0.3244, TRUE),
    (810000031, 810000001, '2026-11-02 02:00:48', 'Y', 0.6333, TRUE),
    (810000032, 810000001, '2026-11-02 14:00:51', 'Z', 0.5195, TRUE),
    (810000033, 810000001, '2026-11-03 02:00:54', 'X', -0.0205, FALSE),
    (810000034, 810000001, '2026-11-03 14:00:57', 'Y', -0.6054, TRUE),
    (810000035, 810000001, '2026-11-04 02:01:00', 'Z', 0.0898, TRUE),
    (810000036, 810000001, '2026-11-04 14:01:03', 'X', 0.6385, TRUE),
    (810000037, 810000002, '2026-10-25 04:00:00', 'X', -0.0281, FALSE),
    (810000038, 810000002, '2026-10-25 16:00:03', 'Y', 0.4676, TRUE),
    (810000039, 810000002, '2026-10-26 04:00:06', 'Z', -0.1713, TRUE),
    (810000040, 810000002, '2026-10-26 16:00:09', 'X', -0.0032, TRUE),
    (810000041, 810000002, '2026-10-27 04:00:12', 'Y', 0.0654, TRUE),
    (810000042, 810000002, '2026-10-27 16:00:15', 'Z', 0.6742, TRUE),
    (810000043, 810000002, '2026-10-28 04:00:18', 'X', -0.6346, TRUE),
    (810000044, 810000002, '2026-10-28 16:00:21', 'Y', 0.6047, TRUE),
    (810000045, 810000002, '2026-10-29 04:00:24', 'Z', 0.0232, TRUE),
    (810000046, 810000002, '2026-10-29 16:00:27', 'X', 0.3426, FALSE),
    (810000047, 810000002, '2026-10-30 04:00:30', 'Y', 0.558, TRUE),
    (810000048, 810000002, '2026-10-30 16:00:33', 'Z', -0.0431, TRUE),
    (810000049, 810000002, '2026-10-31 04:00:36', 'X', -0.6243, TRUE),
    (810000050, 810000002, '2026-10-31 16:00:39', 'Y', -0.0733, TRUE),
    (810000051, 810000002, '2026-11-01 04:00:42', 'Z', 0.173, TRUE),
    (810000052, 810000002, '2026-11-01 16:00:45', 'X', 0.4654, TRUE),
    (810000053, 810000002, '2026-11-02 04:00:48', 'Y', -0.5216, TRUE),
    (810000054, 810000002, '2026-11-02 16:00:51', 'Z', -0.3784, TRUE),
    (810000055, 810000002, '2026-11-03 04:00:54', 'X', 0.4895, FALSE),
    (810000056, 810000002, '2026-11-03 16:00:57', 'Y', -0.6581, TRUE),
    (810000057, 810000002, '2026-11-04 04:01:00', 'Z', 0.1554, TRUE),
    (810000058, 810000002, '2026-11-04 16:01:03', 'X', 0.4247, TRUE),
    (810000059, 810000003, '2026-10-25 06:00:00', 'X', 0.6769, FALSE),
    (810000060, 810000003, '2026-10-25 18:00:03', 'Y', -0.6232, TRUE),
    (810000061, 810000003, '2026-10-26 06:00:06', 'Z', 0.3131, TRUE),
    (810000062, 810000003, '2026-10-26 18:00:09', 'X', -0.4169, TRUE),
    (810000063, 810000003, '2026-10-27 06:00:12', 'Y', -0.6012, TRUE),
    (810000064, 810000003, '2026-10-27 18:00:15', 'Z', -0.1383, TRUE),
    (810000065, 810000003, '2026-10-28 06:00:18', 'X', 0.3527, TRUE),
    (810000066, 810000003, '2026-10-28 18:00:21', 'Y', -0.6422, TRUE),
    (810000067, 810000003, '2026-10-29 06:00:24', 'Z', -0.0764, TRUE),
    (810000068, 810000003, '2026-10-29 18:00:27', 'X', 0.2685, FALSE),
    (810000069, 810000003, '2026-10-30 06:00:30', 'Y', 0.025, TRUE),
    (810000070, 810000003, '2026-10-30 18:00:33', 'Z', 0.2436, TRUE),
    (810000071, 810000003, '2026-10-31 06:00:36', 'X', 0.6003, TRUE),
```

```
    (810000072, 810000003, '2026-10-31 18:00:39', 'Y', -0.2409, TRUE),
    (810000073, 810000003, '2026-11-01 06:00:42', 'Z', 0.2552, TRUE),
    (810000074, 810000003, '2026-11-01 18:00:45', 'X', 0.4439, TRUE),
    (810000075, 810000003, '2026-11-02 06:00:48', 'Y', -0.139, TRUE),
    (810000076, 810000003, '2026-11-02 18:00:51', 'Z', 0.1981, TRUE),
    (810000077, 810000003, '2026-11-03 06:00:54', 'X', -0.581, FALSE),
    (810000078, 810000003, '2026-11-03 18:00:57', 'Y', 0.4671, TRUE),
    (810000079, 810000003, '2026-11-04 06:01:00', 'Z', 0.1697, TRUE),
    (810000080, 810000003, '2026-11-04 18:01:03', 'X', 0.2891, TRUE),
    (810000081, 810000004, '2026-10-25 08:00:00', 'X', -0.5985, FALSE),
    (810000082, 810000004, '2026-10-25 20:00:03', 'Y', -0.0932, TRUE),
    (810000083, 810000004, '2026-10-26 08:00:06', 'Z', 0.5727, TRUE),
    (810000084, 810000004, '2026-10-26 20:00:09', 'X', -0.4985, TRUE),
    (810000085, 810000004, '2026-10-27 08:00:12', 'Y', 0.1143, TRUE),
    (810000086, 810000004, '2026-10-27 20:00:15', 'Z', -0.0463, TRUE),
    (810000087, 810000004, '2026-10-28 08:00:18', 'X', -0.5224, TRUE),
    (810000088, 810000004, '2026-10-28 20:00:21', 'Y', 0.6983, TRUE),
    (810000089, 810000004, '2026-10-29 08:00:24', 'Z', -0.6352, TRUE),
    (810000090, 810000004, '2026-10-29 20:00:27', 'X', 0.5609, FALSE),
    (810000091, 810000004, '2026-10-30 08:00:30', 'Y', -0.5025, TRUE),
    (810000092, 810000004, '2026-10-30 20:00:33', 'Z', -0.2209, TRUE),
    (810000093, 810000004, '2026-10-31 08:00:36', 'X', -0.6856, TRUE),
    (810000094, 810000004, '2026-10-31 20:00:39', 'Y', -0.1531, TRUE),
    (810000095, 810000004, '2026-11-01 08:00:42', 'Z', 0.4692, TRUE),
    (810000096, 810000004, '2026-11-01 20:00:45', 'X', -0.0101, TRUE),
    (810000097, 810000004, '2026-11-02 08:00:48', 'Y', 0.3083, TRUE),
    (810000098, 810000004, '2026-11-02 20:00:51', 'Z', -0.1846, TRUE),
    (810000099, 810000004, '2026-11-03 08:00:54', 'X', -0.539, FALSE),
    (810000100, 810000004, '2026-11-03 20:00:57', 'Y', 0.3753, TRUE),
    (810000101, 810000004, '2026-11-04 08:01:00', 'Z', -0.3566, TRUE),
    (810000102, 810000004, '2026-11-04 20:01:03', 'X', -0.0015, TRUE),
    (810000103, 810000005, '2026-10-25 10:00:00', 'X', 0.1325, FALSE),
    (810000104, 810000005, '2026-10-25 22:00:03', 'Y', -0.6839, TRUE),
    (810000105, 810000005, '2026-10-26 10:00:06', 'Z', -0.52, TRUE),
    (810000106, 810000005, '2026-10-26 22:00:09', 'X', 0.1401, TRUE),
    (810000107, 810000005, '2026-10-27 10:00:12', 'Y', 0.5514, TRUE),
    (810000108, 810000005, '2026-10-27 22:00:15', 'Z', -0.4811, TRUE),
    (810000109, 810000005, '2026-10-28 10:00:18', 'X', 0.2344, TRUE),
    (810000110, 810000005, '2026-10-28 22:00:21', 'Y', -0.0496, TRUE),
    (810000111, 810000005, '2026-10-29 10:00:24', 'Z', 0.4348, TRUE),
    (810000112, 810000005, '2026-10-29 22:00:27', 'X', 0.065, FALSE),
    (810000113, 810000005, '2026-10-30 10:00:30', 'Y', -0.3129, TRUE),
    (810000114, 810000005, '2026-10-30 22:00:33', 'Z', -0.3321, TRUE),
    (810000115, 810000005, '2026-10-31 10:00:36', 'X', 0.2575, TRUE),
    (810000116, 810000005, '2026-10-31 22:00:39', 'Y', -0.1241, TRUE),
    (810000117, 810000005, '2026-11-01 10:00:42', 'Z', 0.6445, TRUE),
    (810000118, 810000005, '2026-11-01 22:00:45', 'X', -0.4053, TRUE),
    (810000119, 810000005, '2026-11-02 10:00:48', 'Y', 0.3246, TRUE),
    (810000120, 810000005, '2026-11-02 22:00:51', 'Z', -0.5748, TRUE),
    (810000121, 810000005, '2026-11-03 10:00:54', 'X', -0.6522, FALSE),
    (810000122, 810000005, '2026-11-03 22:00:57', 'Y', 0.0033, TRUE),
    (810000123, 810000005, '2026-11-04 10:01:00', 'Z', -0.102, TRUE),
    (810000124, 810000005, '2026-11-04 22:01:03', 'X', 0.3942, TRUE),
    (810000125, 810000006, '2026-10-25 12:00:00', 'X', 0.0924, FALSE),
    (810000126, 810000006, '2026-10-26 00:00:03', 'Y', 0.5115, TRUE),
    (810000127, 810000006, '2026-10-26 12:00:06', 'Z', 0.1725, TRUE),
    (810000128, 810000006, '2026-10-27 00:00:09', 'X', -0.0742, TRUE),
    (810000129, 810000006, '2026-10-27 12:00:12', 'Y', 0.0264, TRUE),
    (810000130, 810000006, '2026-10-28 00:00:15', 'Z', 0.0625, TRUE),
    (810000131, 810000006, '2026-10-28 12:00:18', 'X', 0.4732, TRUE),
    (810000132, 810000006, '2026-10-29 00:00:21', 'Y', -0.6773, TRUE),
    (810000133, 810000006, '2026-10-29 12:00:24', 'Z', -0.3799, TRUE),
    (810000134, 810000006, '2026-10-30 00:00:27', 'X', 0.1736, FALSE),
    (810000135, 810000006, '2026-10-30 12:00:30', 'Y', 0.6712, TRUE),
    (810000136, 810000006, '2026-10-31 00:00:33', 'Z', 0.0415, TRUE),
    (810000137, 810000006, '2026-10-31 12:00:36', 'X', 0.379, TRUE),
    (810000138, 810000006, '2026-11-01 00:00:39', 'Y', 0.3164, TRUE),
    (810000139, 810000006, '2026-11-01 12:00:42', 'Z', -0.5086, TRUE),
    (810000140, 810000006, '2026-11-02 00:00:45', 'X', -0.3274, TRUE),
    (810000141, 810000006, '2026-11-02 12:00:48', 'Y', 0.6818, TRUE),
    (810000142, 810000006, '2026-11-03 00:00:51', 'Z', -0.3537, TRUE),
    (810000143, 810000006, '2026-11-03 12:00:54', 'X', -0.2317, FALSE),
    (810000144, 810000006, '2026-11-04 00:00:57', 'Y', 0.532, TRUE),
    (810000145, 810000006, '2026-11-04 12:01:00', 'Z', 0.1948, TRUE),
    (810000146, 810000006, '2026-11-05 00:01:03', 'X', 0.1959, TRUE),
    (810000147, 810000007, '2026-10-25 14:00:00', 'X', 0.5015, FALSE),
    (810000148, 810000007, '2026-10-26 02:00:03', 'Y', 0.1314, TRUE),
```

```
    (810000149, 810000007, '2026-10-26 14:00:06', 'Z', -0.4236, TRUE),
    (810000150, 810000007, '2026-10-27 02:00:09', 'X', -0.2342, TRUE),
    (810000151, 810000007, '2026-10-27 14:00:12', 'Y', -0.1644, TRUE),
    (810000152, 810000007, '2026-10-28 02:00:15', 'Z', 0.0052, TRUE),
    (810000153, 810000007, '2026-10-28 14:00:18', 'X', -0.36, TRUE),
    (810000154, 810000007, '2026-10-29 02:00:21', 'Y', -0.4459, TRUE),
    (810000155, 810000007, '2026-10-29 14:00:24', 'Z', -0.2547, TRUE),
    (810000156, 810000007, '2026-10-30 02:00:27', 'X', -0.317, FALSE),
    (810000157, 810000007, '2026-10-30 14:00:30', 'Y', -0.0104, TRUE),
    (810000158, 810000007, '2026-10-31 02:00:33', 'Z', 0.5708, TRUE),
    (810000159, 810000007, '2026-10-31 14:00:36', 'X', -0.2835, TRUE),
    (810000160, 810000007, '2026-11-01 02:00:39', 'Y', 0.4389, TRUE),
    (810000161, 810000007, '2026-11-01 14:00:42', 'Z', -0.5705, TRUE),
    (810000162, 810000007, '2026-11-02 02:00:45', 'X', -0.345, TRUE),
    (810000163, 810000007, '2026-11-02 14:00:48', 'Y', -0.2703, TRUE),
    (810000164, 810000007, '2026-11-03 02:00:51', 'Z', 0.1181, TRUE),
    (810000165, 810000007, '2026-11-03 14:00:54', 'X', 0.3142, FALSE),
    (810000166, 810000007, '2026-11-04 02:00:57', 'Y', 0.3343, TRUE),
    (810000167, 810000007, '2026-11-04 14:01:00', 'Z', -0.0707, TRUE),
    (810000168, 810000007, '2026-11-05 02:01:03', 'X', -0.1328, TRUE),
    (810000169, 810000008, '2026-10-25 16:00:00', 'X', -0.5984, FALSE),
    (810000170, 810000008, '2026-10-26 04:00:03', 'Y', -0.1302, TRUE),
    (810000171, 810000008, '2026-10-26 16:00:06', 'Z', 0.1942, TRUE),
    (810000172, 810000008, '2026-10-27 04:00:09', 'X', 0.0142, TRUE),
    (810000173, 810000008, '2026-10-27 16:00:12', 'Y', -0.1908, TRUE),
    (810000174, 810000008, '2026-10-28 04:00:15', 'Z', 0.357, TRUE),
    (810000175, 810000008, '2026-10-28 16:00:18', 'X', 0.4735, TRUE),
    (810000176, 810000008, '2026-10-29 04:00:21', 'Y', 0.4169, TRUE),
    (810000177, 810000008, '2026-10-29 16:00:24', 'Z', -0.0853, TRUE),
    (810000178, 810000008, '2026-10-30 04:00:27', 'X', -0.0498, FALSE),
    (810000179, 810000008, '2026-10-30 16:00:30', 'Y', -0.3445, TRUE),
    (810000180, 810000008, '2026-10-31 04:00:33', 'Z', -0.1446, TRUE),
    (810000181, 810000008, '2026-10-31 16:00:36', 'X', -0.265, TRUE),
    (810000182, 810000008, '2026-11-01 04:00:39', 'Y', 0.3159, TRUE),
    (810000183, 810000008, '2026-11-01 16:00:42', 'Z', 0.515, TRUE),
    (810000184, 810000008, '2026-11-02 04:00:45', 'X', -0.3058, TRUE),
    (810000185, 810000008, '2026-11-02 16:00:48', 'Y', -0.2683, TRUE),
    (810000186, 810000008, '2026-11-03 04:00:51', 'Z', 0.1938, TRUE),
    (810000187, 810000008, '2026-11-03 16:00:54', 'X', -0.4002, FALSE),
    (810000188, 810000008, '2026-11-04 04:00:57', 'Y', 0.3507, TRUE),
    (810000189, 810000008, '2026-11-04 16:01:00', 'Z', 0.6698, TRUE),
    (810000190, 810000008, '2026-11-05 04:01:03', 'X', -0.4935, TRUE);

INSERT INTO evento_sismico (id_evento, fecha_hora_origen, latitud_epicentro, longitud_epicentro, profundidad_km,
     magnitud, tipo_magnitud, intensidad_maxima, referencia_geografica, estado_revision) VALUES
    (810000001, '2026-10-26 16:22:00', -15.95, -75.45, 16, 3.02, 'Mw', NULL, 'SIMULACION: epicentro no verificado', '
     preliminar'),
    (810000002, '2026-10-28 00:29:00', -15.1, -74.4, 22, 3.34, 'Mw', NULL, 'SIMULACION: epicentro no verificado', '
     preliminar'),
    (810000003, '2026-10-29 08:36:00', -14.25, -73.35, 28, 3.66, 'Mw', NULL, 'SIMULACION: epicentro no verificado', '
     preliminar'),
    (810000004, '2026-10-30 16:43:00', -13.4, -76.5, 34, 3.98, 'Mw', NULL, 'SIMULACION: epicentro no verificado', '
     preliminar'),
    (810000005, '2026-11-01 00:50:00', -12.55, -75.45, 40, 4.3, 'Mw', NULL, 'SIMULACION: epicentro no verificado', '
     preliminar'),
    (810000006, '2026-11-02 08:57:00', -11.7, -74.4, 46, 4.62, 'Mw', NULL, 'SIMULACION: epicentro no verificado', '
     preliminar'),
    (810000007, '2026-11-03 17:04:00', -10.85, -73.35, 52, 4.94, 'Mw', NULL, 'SIMULACION: epicentro no verificado', '
     preliminar'),
    (810000008, '2026-11-05 01:11:00', -10.0, -76.5, 58, 5.26, 'Mw', NULL, 'SIMULACION: epicentro no verificado', '
     preliminar');

INSERT INTO deteccion (id_sensor, id_evento, hora_deteccion, aceleracion_maxima, calidad) VALUES
    (810000001, 810000001, '2026-10-26 16:22:03', 0.18, 'alta'),
    (810000002, 810000001, '2026-10-26 16:22:06', 0.22, 'alta'),
    (810000003, 810000001, '2026-10-26 16:22:09', 0.26, 'alta'),
    (810000004, 810000002, '2026-10-28 00:29:12', 0.32, 'alta'),
    (810000005, 810000002, '2026-10-28 00:29:15', 0.36, 'alta'),
    (810000002, 810000003, '2026-10-29 08:36:06', 0.26, 'alta'),
    (810000006, 810000003, '2026-10-29 08:36:18', 0.42, 'alta'),
    (810000003, 810000004, '2026-10-30 16:43:09', 0.32, 'alta'),
    (810000007, 810000004, '2026-10-30 16:43:21', 0.48, 'alta'),
    (810000004, 810000005, '2026-11-01 00:50:12', 0.38, 'alta'),
    (810000008, 810000005, '2026-11-01 00:50:24', 0.54, 'alta'),
    (810000005, 810000006, '2026-11-02 08:57:15', 0.44, 'alta'),
    (810000001, 810000007, '2026-11-03 17:04:03', 0.3, 'alta'),
    (810000006, 810000007, '2026-11-03 17:04:18', 0.5, 'alta');
```

```
INSERT INTO alerta (id_alerta, id_evento, fecha_hora_emision, nivel, mensaje, magnitud_estimada, estado) VALUES
    ('810000001', 810000001, '2026-10-26 16:23:05', 'informativa', 'SIMULACION: alerta sin difusion real', 3.02, '
     registrada'),
    ('810000002', 810000002, '2026-10-28 00:30:05', 'informativa', 'SIMULACION: alerta sin difusion real', 3.34, '
     registrada'),
    ('810000003', 810000003, '2026-10-29 08:37:05', 'informativa', 'SIMULACION: alerta sin difusion real', 3.66, '
     registrada'),
    ('810000004', 810000005, '2026-11-01 00:51:05', 'informativa', 'SIMULACION: alerta sin difusion real', 4.3, '
     registrada'),
    ('810000005', 810000007, '2026-11-03 17:05:05', 'informativa', 'SIMULACION: alerta sin difusion real', 4.94, '
     registrada');

DO $$
DECLARE
    control RECORD;
    encontrado BIGINT;
BEGIN
    FOR control IN
        SELECT * FROM (VALUES
            ('estacion_sismica', 5, 'id_estacion'),
            ('sensor', 10, 'id_sensor'),
            ('lectura_onda', 190, 'id_lectura'),
            ('evento_sismico', 8, 'id_evento'),
            ('alerta', 5, 'id_alerta')
        ) AS e(tabla, esperado, columna)
    LOOP
        EXECUTE format('SELECT COUNT(*) FROM %I WHERE %I BETWEEN 810000001 AND 810000999',
                       control.tabla, control.columna) INTO encontrado;
        IF encontrado <> control.esperado THEN
            RAISE EXCEPTION 'Cantidad incorrecta en %: esperaba %, hay %',
                            control.tabla, control.esperado, encontrado;
        END IF;
    END LOOP;

    SELECT COUNT(*) INTO encontrado FROM deteccion
    WHERE id_sensor BETWEEN 810000001 AND 810000999
      AND id_evento BETWEEN 810000001 AND 810000999;
    IF encontrado <> 14 THEN
        RAISE EXCEPTION 'Cantidad incorrecta en deteccion: esperaba 14, hay %', encontrado;
    END IF;

    RAISE NOTICE 'OK: 232 registros sintéticos verificados antes del COMMIT';
END;
$$;


\echo '=== CONTROL DE CARGA INICIAL ==='
SELECT 'estacion_sismica' AS tabla, 5 AS esperado, COUNT(*) AS encontrado, CASE WHEN COUNT(*)=5 THEN 'OK' ELSE '
     REVISAR' END AS resultado FROM estacion_sismica WHERE id_estacion BETWEEN 810000001 AND 810000999
UNION ALL
SELECT 'sensor' AS tabla, 10 AS esperado, COUNT(*) AS encontrado, CASE WHEN COUNT(*)=10 THEN 'OK' ELSE 'REVISAR' END
     AS resultado FROM sensor WHERE id_sensor BETWEEN 810000001 AND 810000999
UNION ALL
SELECT 'lectura_onda' AS tabla, 190 AS esperado, COUNT(*) AS encontrado, CASE WHEN COUNT(*)=190 THEN 'OK' ELSE '
     REVISAR' END AS resultado FROM lectura_onda WHERE id_lectura BETWEEN 810000001 AND 810000999
UNION ALL
SELECT 'evento_sismico' AS tabla, 8 AS esperado, COUNT(*) AS encontrado, CASE WHEN COUNT(*)=8 THEN 'OK' ELSE 'REVISAR
     ' END AS resultado FROM evento_sismico WHERE id_evento BETWEEN 810000001 AND 810000999
UNION ALL
SELECT 'deteccion' AS tabla, 14 AS esperado, COUNT(*) AS encontrado, CASE WHEN COUNT(*)=14 THEN 'OK' ELSE 'REVISAR'
     END AS resultado FROM deteccion WHERE id_sensor BETWEEN 810000001 AND 810000999 AND id_evento BETWEEN 810000001
      AND 810000999
UNION ALL
SELECT 'alerta' AS tabla, 5 AS esperado, COUNT(*) AS encontrado, CASE WHEN COUNT(*)=5 THEN 'OK' ELSE 'REVISAR' END AS
      resultado FROM alerta WHERE id_alerta BETWEEN 810000001 AND 810000999
ORDER BY tabla;


SELECT 'estaciones_sin_sensores' AS control, COUNT(*) AS encontrados, 0 AS esperado
FROM estacion_sismica e
WHERE e.id_estacion BETWEEN 810000001 AND 810000999
AND NOT EXISTS (SELECT 1 FROM sensor s WHERE s.id_estacion=e.id_estacion)
UNION ALL
SELECT 'lecturas_huerfanas', COUNT(*), 0
FROM lectura_onda l LEFT JOIN sensor s ON l.id_sensor=s.id_sensor
WHERE l.id_lectura BETWEEN 810000001 AND 810000999 AND s.id_sensor IS NULL
```

```
UNION ALL
SELECT 'detecciones_sin_lectura_valida', COUNT(*), 0
FROM deteccion d
WHERE d.id_sensor BETWEEN 810000001 AND 810000999
AND NOT EXISTS (SELECT 1 FROM lectura_onda l WHERE l.id_sensor=d.id_sensor AND l.fecha_hora=d.hora_deteccion AND l.
    es_valida)
UNION ALL
SELECT 'lecturas_validas_sensor_no_operativo', COUNT(*), 0
FROM lectura_onda l JOIN sensor s ON l.id_sensor=s.id_sensor
WHERE l.id_lectura BETWEEN 810000001 AND 810000999 AND l.es_valida AND s.estado <> 'operativo';

SELECT COUNT(*) FILTER (WHERE es_valida) AS lecturas_validas,
       COUNT(*) FILTER (WHERE NOT es_valida) AS lecturas_invalidas
FROM lectura_onda
WHERE id_lectura BETWEEN 810000001 AND 810000999;

SELECT e.id_evento, COUNT(DISTINCT d.id_sensor) AS sensores_que_detectaron,
       COUNT(DISTINCT a.id_alerta) AS alertas_emitidas
FROM evento_sismico e
LEFT JOIN deteccion d ON d.id_evento=e.id_evento
LEFT JOIN alerta a ON a.id_evento=e.id_evento
WHERE e.id_evento BETWEEN 810000001 AND 810000999
GROUP BY e.id_evento
ORDER BY e.id_evento;


\echo '=== IMPORTACION DEL CATALOGO IGP ==='
CREATE TEMP TABLE igp_catalogo_42 (
    id TEXT,
    fecha_utc TEXT,
    hora_utc TEXT,
    latitud TEXT,
    longitud TEXT,
    profundidad TEXT,
    magnitud TEXT,
    fecha_corte TEXT
) ON COMMIT DROP;

\copy igp_catalogo_42 (id, fecha_utc, hora_utc, latitud, longitud, profundidad, magnitud, fecha_corte) FROM 'datos/
    igp_catalogo.csv' WITH (FORMAT csv, HEADER true, DELIMITER ';', ENCODING 'UTF8')

DO $$
DECLARE
    total_fuente BIGINT;
    filas_incorrectas BIGINT;
BEGIN
    SELECT COUNT(*) INTO total_fuente FROM igp_catalogo_42;
    IF total_fuente = 0 THEN
        RAISE EXCEPTION 'El archivo IGP no contiene registros.';
    END IF;

    SELECT COUNT(*) INTO filas_incorrectas
    FROM igp_catalogo_42
    WHERE COALESCE(BTRIM(id), '') !~ '^[0-9]{1,8}$'
       OR COALESCE(BTRIM(fecha_utc), '') !~ '^[0-9]{8}$'
       OR COALESCE(BTRIM(hora_utc), '') !~ '^[0-9]{1,6}$'
       OR COALESCE(BTRIM(latitud), '') !~ '^[+-]?([0-9]+([.][0-9]*)?|[.][0-9]+)$'
       OR COALESCE(BTRIM(longitud), '') !~ '^[+-]?([0-9]+([.][0-9]*)?|[.][0-9]+)$'
       OR COALESCE(BTRIM(profundidad), '') !~ '^[+-]?([0-9]+([.][0-9]*)?|[.][0-9]+)$'
       OR COALESCE(BTRIM(magnitud), '') !~ '^[+-]?([0-9]+([.][0-9]*)?|[.][0-9]+)$';

    IF filas_incorrectas > 0 THEN
        RAISE EXCEPTION 'Hay % filas con datos obligatorios vacios o formatos invalidos. No se importo ningun evento.
     ', filas_incorrectas;
    END IF;

    IF EXISTS (
        SELECT 1
        FROM igp_catalogo_42
        GROUP BY BTRIM(id)::INTEGER
        HAVING COUNT(*) > 1
    ) THEN
        RAISE EXCEPTION 'El archivo IGP contiene ID repetidos. No se importo ningun evento.';
    END IF;

    SELECT COUNT(*) INTO filas_incorrectas
    FROM igp_catalogo_42
```

```
    WHERE BTRIM(latitud)::NUMERIC NOT BETWEEN -90 AND 90
       OR BTRIM(longitud)::NUMERIC NOT BETWEEN -180 AND 180
       OR BTRIM(profundidad)::NUMERIC < 0
       OR SUBSTRING(BTRIM(fecha_utc) FROM 5 FOR 2)::INTEGER NOT BETWEEN 1 AND 12
       OR SUBSTRING(BTRIM(fecha_utc) FROM 7 FOR 2)::INTEGER NOT BETWEEN 1 AND 31
       OR SUBSTRING(LPAD(BTRIM(hora_utc), 6, '0') FROM 1 FOR 2)::INTEGER NOT BETWEEN 0 AND 23
       OR SUBSTRING(LPAD(BTRIM(hora_utc), 6, '0') FROM 3 FOR 2)::INTEGER NOT BETWEEN 0 AND 59
       OR SUBSTRING(LPAD(BTRIM(hora_utc), 6, '0') FROM 5 FOR 2)::INTEGER NOT BETWEEN 0 AND 59;

    IF filas_incorrectas > 0 THEN
        RAISE EXCEPTION 'Hay % filas con fechas, horas, ubicaciones o profundidades fuera de rango. No se importo
     ningun evento.', filas_incorrectas;
    END IF;

    RAISE NOTICE 'Filas del CSV oficial IGP leidas: %', total_fuente;
END;
$$;

INSERT INTO evento_sismico (
    id_evento,
    fecha_hora_origen,
    latitud_epicentro,
    longitud_epicentro,
    profundidad_km,
    magnitud,
    tipo_magnitud,
    intensidad_maxima,
    referencia_geografica,
    estado_revision
)
SELECT
    200000000 + BTRIM(id)::INTEGER,
    MAKE_TIMESTAMP(
        SUBSTRING(BTRIM(fecha_utc) FROM 1 FOR 4)::INTEGER,
        SUBSTRING(BTRIM(fecha_utc) FROM 5 FOR 2)::INTEGER,
        SUBSTRING(BTRIM(fecha_utc) FROM 7 FOR 2)::INTEGER,
        SUBSTRING(LPAD(BTRIM(hora_utc), 6, '0') FROM 1 FOR 2)::INTEGER,
        SUBSTRING(LPAD(BTRIM(hora_utc), 6, '0') FROM 3 FOR 2)::INTEGER,
        SUBSTRING(LPAD(BTRIM(hora_utc), 6, '0') FROM 5 FOR 2)::INTEGER
    ),
    BTRIM(latitud)::NUMERIC,
    BTRIM(longitud)::NUMERIC,
    BTRIM(profundidad)::NUMERIC,
    BTRIM(magnitud)::NUMERIC,
    NULL,
    NULL,
    NULL,
    'pendiente_revision'
FROM igp_catalogo_42
ORDER BY BTRIM(id)::INTEGER
ON CONFLICT (id_evento) DO NOTHING;

DO $$
DECLARE
    total_fuente BIGINT;
    total_en_bd BIGINT;
    sinteticos BIGINT;
BEGIN
    SELECT COUNT(*) INTO total_fuente FROM igp_catalogo_42;
    SELECT COUNT(*) INTO total_en_bd
    FROM evento_sismico e
    JOIN igp_catalogo_42 c ON e.id_evento = 200000000 + BTRIM(c.id)::INTEGER;
    SELECT COUNT(*) INTO sinteticos
    FROM evento_sismico
    WHERE id_evento BETWEEN 810000001 AND 810000008;

    IF total_en_bd <> total_fuente THEN
        RAISE EXCEPTION 'Se encontraron % eventos IGP en la BD, pero hay % filas en el CSV. No se confirma la
     transaccion.', total_en_bd, total_fuente;
    END IF;
    IF sinteticos <> 8 THEN
        RAISE EXCEPTION 'Los 8 eventos sinteticos anteriores no estan completos. No se confirma la transaccion.';
    END IF;

    RAISE NOTICE 'OK: % eventos del archivo IGP y % eventos sinteticos conservados.', total_en_bd, sinteticos;
END;
$$;
```

```
SELECT
    (SELECT COUNT(*) FROM igp_catalogo_42) AS filas_igp_en_csv,
    (SELECT COUNT(*) FROM evento_sismico WHERE id_evento BETWEEN 200000000 AND 299999999) AS eventos_igp_en_bd,
    (SELECT COUNT(*) FROM evento_sismico WHERE id_evento BETWEEN 810000001 AND 810000008) AS eventos_sinteticos;


\echo '=== CONTROL DEL CATALOGO IGP ==='
SELECT 'estacion_sismica' AS tabla, COUNT(*) AS encontrados, 5 AS minimo_inicial
FROM estacion_sismica
UNION ALL SELECT 'sensor', COUNT(*), 10 FROM sensor
UNION ALL SELECT 'lectura_onda', COUNT(*), 190 FROM lectura_onda
UNION ALL SELECT 'deteccion', COUNT(*), 14 FROM deteccion
UNION ALL SELECT 'alerta', COUNT(*), 5 FROM alerta
ORDER BY tabla;

SELECT
    COUNT(*) FILTER (WHERE id_evento BETWEEN 810000001 AND 810000008) AS eventos_sinteticos,
    COUNT(*) FILTER (WHERE id_evento BETWEEN 200000000 AND 299999999) AS eventos_igp,
    COUNT(*) AS eventos_totales
FROM evento_sismico;

SELECT
    COUNT(*) FILTER (
        WHERE latitud_epicentro NOT BETWEEN -90 AND 90
           OR longitud_epicentro NOT BETWEEN -180 AND 180
           OR profundidad_km < 0
    ) AS valores_fuera_de_rango,
    COUNT(*) FILTER (
        WHERE tipo_magnitud IS NOT NULL
           OR intensidad_maxima IS NOT NULL
           OR referencia_geografica IS NOT NULL
    ) AS registros_con_datos_no_informados_rellenados
FROM evento_sismico
WHERE id_evento BETWEEN 200000000 AND 299999999;

SELECT
    MIN(fecha_hora_origen) AS primer_evento_igp_utc,
    MAX(fecha_hora_origen) AS ultimo_evento_igp_utc,
    MIN(magnitud) AS magnitud_minima,
    MAX(magnitud) AS magnitud_maxima
FROM evento_sismico
WHERE id_evento BETWEEN 200000000 AND 299999999;

SELECT id_evento, fecha_hora_origen, latitud_epicentro,
       longitud_epicentro, profundidad_km, magnitud, estado_revision
FROM evento_sismico
WHERE id_evento BETWEEN 200000000 AND 299999999
ORDER BY id_evento
LIMIT 5;


\echo '=== SIMULACION COMPLEMENTARIA ==='
DO $$
BEGIN
    IF (SELECT COUNT(*) FROM evento_sismico
        WHERE id_evento BETWEEN 200000000 AND 299999999) <> 24289
       OR (SELECT COUNT(*) FROM evento_sismico
        WHERE id_evento BETWEEN 810000001 AND 810000008) <> 8
       OR (SELECT COUNT(*) FROM estacion_sismica
        WHERE id_estacion BETWEEN 810000001 AND 810000005) <> 5
       OR (SELECT COUNT(*) FROM sensor
        WHERE id_sensor BETWEEN 810000001 AND 810000010) <> 10
       OR (SELECT COUNT(*) FROM lectura_onda
        WHERE id_lectura BETWEEN 810000001 AND 810000190) <> 190
       OR (SELECT COUNT(*) FROM deteccion
        WHERE id_evento BETWEEN 810000001 AND 810000008) <> 14
       OR (SELECT COUNT(*) FROM alerta
        WHERE id_alerta BETWEEN 810000001 AND 810000005) <> 5 THEN
        RAISE EXCEPTION 'La carga inicial o los eventos IGP no coinciden con lo esperado.';
    END IF;

    IF EXISTS (SELECT 1 FROM estacion_sismica
               WHERE id_estacion BETWEEN 820300001 AND 820300010)
       OR EXISTS (SELECT 1 FROM sensor
               WHERE id_sensor BETWEEN 820300001 AND 820300030)
       OR EXISTS (SELECT 1 FROM evento_sismico
```

```
                WHERE id_evento BETWEEN 820300001 AND 820300030)
       OR EXISTS (SELECT 1 FROM lectura_onda
                WHERE id_lectura BETWEEN 820300000001 AND 820300001000)
       OR EXISTS (SELECT 1 FROM deteccion
                WHERE id_sensor BETWEEN 820300001 AND 820300030)
       OR EXISTS (SELECT 1 FROM alerta
                WHERE id_alerta BETWEEN 820300001 AND 820300024) THEN
        RAISE EXCEPTION 'La simulacion ya contiene registros. No repetir la carga.';
    END IF;
END;
$$;

INSERT INTO estacion_sismica (
    id_estacion, codigo, nombre, latitud, longitud, altitud,
    departamento, provincia, distrito, fecha_instalacion, estado
)
SELECT
    820300001 + g,
    'SIM43-EST-' || LPAD((g + 1)::TEXT, 2, '0'),
    'Estacion simulada de prueba ' || LPAD((g + 1)::TEXT, 2, '0'),
    -5.5::NUMERIC - g * 1.1::NUMERIC,
    -78.4::NUMERIC + g * 0.21::NUMERIC,
    100::NUMERIC + g * 85::NUMERIC,
    NULL, NULL, NULL, DATE '2024-01-01', 'operativa'
FROM generate_series(0, 9) AS serie(g);

INSERT INTO sensor (
    id_sensor, numero_serie, tipo, modelo, unidad_medida,
    frecuencia_muestreo, fecha_instalacion, fecha_ultimo_mantenimiento,
    estado, id_estacion
)
SELECT
    820300001 + g,
    'SIM43-SEN-' || LPAD((g + 1)::TEXT, 2, '0'),
    'acelerometro', 'SIM-43', 'm/s2',
    CASE WHEN g % 2 = 0 THEN 50 ELSE 100 END,
    DATE '2025-01-01', NULL,
    'operativo', 820300001 + (g / 3)
FROM generate_series(0, 29) AS serie(g);

INSERT INTO evento_sismico (
    id_evento, fecha_hora_origen, latitud_epicentro,
    longitud_epicentro, profundidad_km, magnitud,
    tipo_magnitud, intensidad_maxima, referencia_geografica,
    estado_revision
)
SELECT
    820300001 + g,
    TIMESTAMP '2026-08-01 00:00:00' + g * INTERVAL '1 day',
    -8::NUMERIC - g * 0.15::NUMERIC,
    -77.5::NUMERIC + g * 0.10::NUMERIC,
    5::NUMERIC + g,
    3.4::NUMERIC + (g % 15) * 0.1::NUMERIC,
    NULL, NULL, NULL, 'simulado'
FROM generate_series(0, 29) AS serie(g);

INSERT INTO lectura_onda (
    id_lectura, id_sensor, fecha_hora, canal, valor, es_valida
)
SELECT
    820300000001::BIGINT + g,
    820300001 + (g % 30),
    TIMESTAMP '2026-08-01 00:00:00'
        + ((g / 30) % 30) * INTERVAL '1 day'
        + (g % 30) * INTERVAL '1 second'
        + (g / 900) * INTERVAL '1 hour',
    CASE g % 3 WHEN 0 THEN 'X' WHEN 1 THEN 'Y' ELSE 'Z' END,
    (((g * 37) % 401) - 200)::NUMERIC / 1000,
    (g % 30 < 3 OR g % 17 <> 0)
FROM generate_series(0, 999) AS serie(g);

INSERT INTO deteccion (
    id_sensor, id_evento, hora_deteccion, aceleracion_maxima, calidad
)
SELECT
    820300001 + s,
    820300001 + e,
```

```
    TIMESTAMP '2026-08-01 00:00:00'
        + e * INTERVAL '1 day' + s * INTERVAL '1 second',
    (e + s + 1)::NUMERIC / 100,
    CASE WHEN (e + s) % 2 = 0 THEN 'alta' ELSE 'media' END
FROM generate_series(0, 29) AS eventos(e)
CROSS JOIN generate_series(0, 2) AS sensores(s);

INSERT INTO alerta (
    id_alerta, id_evento, fecha_hora_emision,
    nivel, mensaje, magnitud_estimada, estado
)
SELECT
    820300001 + g,
    820300001 + g,
    TIMESTAMP '2026-08-01 00:00:00'
        + g * INTERVAL '1 day' + INTERVAL '90 seconds',
    CASE WHEN g % 2 = 0 THEN 'preventiva' ELSE 'informativa' END,
    'Alerta de prueba para el evento simulado ' || (820300001 + g)::TEXT,
    3.4::NUMERIC + (g % 15) * 0.1::NUMERIC,
    'simulada'
FROM generate_series(0, 23) AS serie(g);

DO $$
DECLARE
    numero_incorrecto INTEGER;
BEGIN
    IF (SELECT COUNT(*) FROM estacion_sismica
        WHERE id_estacion BETWEEN 820300001 AND 820300010) <> 10
       OR (SELECT COUNT(*) FROM sensor
        WHERE id_sensor BETWEEN 820300001 AND 820300030) <> 30
       OR (SELECT COUNT(*) FROM evento_sismico
        WHERE id_evento BETWEEN 820300001 AND 820300030) <> 30
       OR (SELECT COUNT(*) FROM lectura_onda
        WHERE id_lectura BETWEEN 820300000001 AND 820300001000) <> 1000
       OR (SELECT COUNT(*) FROM deteccion
        WHERE id_evento BETWEEN 820300001 AND 820300030) <> 90
       OR (SELECT COUNT(*) FROM alerta
        WHERE id_alerta BETWEEN 820300001 AND 820300024) <> 24 THEN
        RAISE EXCEPTION 'Los recuentos de los datos simulados no coinciden. Se cancela.';
    END IF;

    SELECT COUNT(*) INTO numero_incorrecto
    FROM deteccion d
    WHERE d.id_evento BETWEEN 820300001 AND 820300030
      AND NOT EXISTS (
          SELECT 1 FROM lectura_onda l
          WHERE l.id_sensor = d.id_sensor
            AND l.fecha_hora = d.hora_deteccion
            AND l.es_valida
      );
    IF numero_incorrecto <> 0 THEN
        RAISE EXCEPTION 'Hay % detecciones sin lectura valida vinculable.', numero_incorrecto;
    END IF;

    IF EXISTS (SELECT 1 FROM deteccion
               WHERE id_evento BETWEEN 200000000 AND 299999999)
       OR EXISTS (SELECT 1 FROM alerta
               WHERE id_evento BETWEEN 200000000 AND 299999999) THEN
        RAISE EXCEPTION 'Se encontraron detecciones o alertas de eventos IGP. Verificar antes de confirmar.';
    END IF;
    RAISE NOTICE 'OK: 1184 registros simulados generados y verificados antes de COMMIT.';
END;
$$;

SELECT 'estaciones' AS tipo, 10 AS generados
UNION ALL SELECT 'sensores', 30
UNION ALL SELECT 'eventos', 30
UNION ALL SELECT 'lecturas', 1000
UNION ALL SELECT 'detecciones', 90
UNION ALL SELECT 'alertas', 24;


\echo '=== CONTROL DE SIMULACION ==='
WITH cantidades AS (
    SELECT 'estacion_sismica'::TEXT AS tabla, COUNT(*) AS encontrados, 15::BIGINT AS esperado FROM estacion_sismica
    UNION ALL SELECT 'sensor', COUNT(*), 40 FROM sensor
    UNION ALL SELECT 'lectura_onda', COUNT(*), 1190 FROM lectura_onda
```

```
    UNION ALL SELECT 'evento_sismico', COUNT(*), 24327 FROM evento_sismico
    UNION ALL SELECT 'deteccion', COUNT(*), 104 FROM deteccion
    UNION ALL SELECT 'alerta', COUNT(*), 29 FROM alerta
)
SELECT tabla, esperado, encontrados,
       CASE WHEN encontrados = esperado THEN 'OK' ELSE 'REVISAR' END AS resultado
FROM cantidades
ORDER BY tabla;

SELECT
    (SELECT COUNT(*) FROM evento_sismico
     WHERE id_evento BETWEEN 200000000 AND 299999999) AS eventos_igp,
    (SELECT COUNT(*) FROM evento_sismico
     WHERE id_evento BETWEEN 810000001 AND 810000008) AS eventos_simulados_iniciales,
    (SELECT COUNT(*) FROM evento_sismico
     WHERE id_evento BETWEEN 820300001 AND 820300030) AS eventos_simulados_nuevos,
    (SELECT COUNT(*) FROM deteccion
     WHERE id_evento BETWEEN 200000000 AND 299999999) AS detecciones_sobre_igp,
    (SELECT COUNT(*) FROM alerta
     WHERE id_evento BETWEEN 200000000 AND 299999999) AS alertas_sobre_igp;

SELECT 'estaciones' AS clase, COUNT(*) AS generados FROM estacion_sismica
WHERE id_estacion BETWEEN 820300001 AND 820300010
UNION ALL SELECT 'sensores', COUNT(*) FROM sensor WHERE id_sensor BETWEEN 820300001 AND 820300030
UNION ALL SELECT 'eventos', COUNT(*) FROM evento_sismico WHERE id_evento BETWEEN 820300001 AND 820300030
UNION ALL SELECT 'lecturas', COUNT(*) FROM lectura_onda WHERE id_lectura BETWEEN 820300000001 AND 820300001000
UNION ALL SELECT 'detecciones', COUNT(*) FROM deteccion WHERE id_evento BETWEEN 820300001 AND 820300030
UNION ALL SELECT 'alertas', COUNT(*) FROM alerta WHERE id_alerta BETWEEN 820300001 AND 820300024
ORDER BY clase;

SELECT 'estaciones_sin_sensor' AS control, COUNT(*) AS encontrados, 0 AS esperado
FROM estacion_sismica e
WHERE e.id_estacion BETWEEN 820300001 AND 820300010
  AND NOT EXISTS (SELECT 1 FROM sensor s WHERE s.id_estacion = e.id_estacion)
UNION ALL
SELECT 'lecturas_sin_sensor', COUNT(*), 0
FROM lectura_onda l
LEFT JOIN sensor s ON s.id_sensor = l.id_sensor
WHERE l.id_lectura BETWEEN 820300000001 AND 820300001000
  AND s.id_sensor IS NULL
UNION ALL
SELECT 'lecturas_validas_sensor_no_operativo', COUNT(*), 0
FROM lectura_onda l
JOIN sensor s ON s.id_sensor = l.id_sensor
WHERE l.id_lectura BETWEEN 820300000001 AND 820300001000
  AND l.es_valida AND s.estado <> 'operativo'
UNION ALL
SELECT 'detecciones_sin_lectura_valida', COUNT(*), 0
FROM deteccion d
WHERE d.id_evento BETWEEN 820300001 AND 820300030
  AND NOT EXISTS (
      SELECT 1 FROM lectura_onda l
      WHERE l.id_sensor = d.id_sensor
        AND l.fecha_hora = d.hora_deteccion
        AND l.es_valida
  )
UNION ALL
SELECT 'alertas_anteriores_al_evento', COUNT(*), 0
FROM alerta a
JOIN evento_sismico e ON e.id_evento = a.id_evento
WHERE a.id_alerta BETWEEN 820300001 AND 820300024
  AND a.fecha_hora_emision < e.fecha_hora_origen
ORDER BY control;

SELECT
    COUNT(*) AS nuevas_lecturas,
    COUNT(*) FILTER (WHERE es_valida) AS validas,
    COUNT(*) FILTER (WHERE NOT es_valida) AS invalidas,
    MIN(fecha_hora) AS primera_lectura,
    MAX(fecha_hora) AS ultima_lectura
FROM lectura_onda
WHERE id_lectura BETWEEN 820300000001 AND 820300001000;

SELECT
    (SELECT COUNT(*) FROM estacion_sismica)
    + (SELECT COUNT(*) FROM sensor)
    + (SELECT COUNT(*) FROM lectura_onda)
```

```
    + (SELECT COUNT(*) FROM evento_sismico)
    + (SELECT COUNT(*) FROM deteccion)
    + (SELECT COUNT(*) FROM alerta) AS registros_totales,
    25705 AS esperado;

\echo '=== RESULTADOS FINALES ==='
WITH conteos AS (
    SELECT 'estacion_sismica'::TEXT AS tabla, COUNT(*) AS obtenido, 15::BIGINT AS esperado FROM estacion_sismica
    UNION ALL SELECT 'sensor', COUNT(*), 40 FROM sensor
    UNION ALL SELECT 'lectura_onda', COUNT(*), 1190 FROM lectura_onda
    UNION ALL SELECT 'evento_sismico', COUNT(*), 24327 FROM evento_sismico
    UNION ALL SELECT 'deteccion', COUNT(*), 104 FROM deteccion
    UNION ALL SELECT 'alerta', COUNT(*), 29 FROM alerta
)
SELECT tabla, esperado, obtenido,
       CASE WHEN esperado = obtenido THEN 'OK' ELSE 'REVISAR' END AS estado
FROM conteos
ORDER BY tabla;

DO $$
DECLARE
    total BIGINT;
    oficiales BIGINT;
BEGIN
    SELECT
        (SELECT COUNT(*) FROM estacion_sismica)
        + (SELECT COUNT(*) FROM sensor)
        + (SELECT COUNT(*) FROM lectura_onda)
        + (SELECT COUNT(*) FROM evento_sismico)
        + (SELECT COUNT(*) FROM deteccion)
        + (SELECT COUNT(*) FROM alerta)
    INTO total;
    SELECT COUNT(*) INTO oficiales
    FROM evento_sismico
    WHERE id_evento BETWEEN 200000000 AND 299999999;
    IF total <> 25705 OR oficiales <> 24289 THEN
        RAISE EXCEPTION 'Recuentos incorrectos: total % (esperado 25705), IGP % (esperado 24289).', total, oficiales
     ;
    END IF;
    RAISE NOTICE 'OK: 25705 registros, de los cuales 24289 son eventos oficiales IGP.';
END;
$$;

COMMIT;
\echo '=== ETAPA 2 FINALIZADA ==='
```

Listing 2: Integrador 02: carga y verificación de datos.

7.6.3. Vistas, procedimientos, disparadores y consultas

```
-- 03. LOGICA Y CONSULTAS
\set ON_ERROR_STOP on
\pset pager off

\echo '=== VALIDACION INICIAL ==='
SELECT current_database() AS base_actual;
SET search_path TO monitoreo_sismologico, public;

DO $$
DECLARE
    tabla TEXT;
    cantidad BIGINT;
    total BIGINT := 0;
BEGIN
    IF current_database() IN ('postgres', 'template0', 'template1') THEN
        RAISE EXCEPTION 'Seleccione una base de proyecto.';
    END IF;

    FOR tabla, cantidad IN
        SELECT nombre, esperado
        FROM (VALUES
            ('estacion_sismica', 15::BIGINT),
            ('sensor', 40),
```

```
            ('lectura_onda', 1190),
            ('evento_sismico', 24327),
            ('deteccion', 104),
            ('alerta', 29)
        ) AS t(nombre, esperado)
    LOOP
        IF to_regclass(format('monitoreo_sismologico.%I', tabla)) IS NULL THEN
            RAISE EXCEPTION 'Falta la tabla %.', tabla;
        END IF;
        EXECUTE format('SELECT COUNT(*) FROM monitoreo_sismologico.%I', tabla) INTO total;
        IF total <> cantidad THEN
            RAISE EXCEPTION 'La tabla % contiene % registros; se esperaban %.', tabla, total, cantidad;
        END IF;
    END LOOP;
    RAISE NOTICE 'OK: seis tablas y 25705 registros verificados.';
END;
$$;


\echo '=== VISTAS ==='
BEGIN;
SET LOCAL search_path TO monitoreo_sismologico, public;

-- 1. Actividad y validez de lecturas para cada sensor.
-- LEFT JOIN conserva sensores que aun no tienen lecturas.
CREATE OR REPLACE VIEW vw_sensores_actividad AS
SELECT
    s.id_sensor,
    s.numero_serie,
    s.tipo,
    s.estado AS estado_sensor,
    s.frecuencia_muestreo,
    e.id_estacion,
    e.codigo AS codigo_estacion,
    e.nombre AS nombre_estacion,
    COALESCE(l.total_lecturas, 0) AS total_lecturas,
    COALESCE(l.lecturas_validas, 0) AS lecturas_validas,
    COALESCE(l.lecturas_invalidas, 0) AS lecturas_invalidas,
    l.primera_lectura,
    l.ultima_lectura
FROM sensor s
JOIN estacion_sismica e ON e.id_estacion = s.id_estacion
LEFT JOIN (
    SELECT
        id_sensor,
        COUNT(*) AS total_lecturas,
        COUNT(*) FILTER (WHERE es_valida IS TRUE) AS lecturas_validas,
        COUNT(*) FILTER (WHERE es_valida IS FALSE) AS lecturas_invalidas,
        MIN(fecha_hora) AS primera_lectura,
        MAX(fecha_hora) AS ultima_lectura
    FROM lectura_onda
    GROUP BY id_sensor
) l ON l.id_sensor = s.id_sensor;

-- 2. Cobertura y lecturas por estacion.
-- Se agregan sensores ya resumidos para no multiplicar lecturas por JOIN.
CREATE OR REPLACE VIEW vw_estaciones_resumen AS
SELECT
    e.id_estacion,
    e.codigo,
    e.nombre,
    e.departamento,
    e.estado AS estado_estacion,
    COUNT(sa.id_sensor) AS total_sensores,
    COUNT(sa.id_sensor) FILTER (
        WHERE LOWER(sa.estado_sensor) IN ('operativo', 'operativa')
    ) AS sensores_operativos,
    COALESCE(SUM(sa.total_lecturas), 0) AS total_lecturas,
    COALESCE(SUM(sa.lecturas_validas), 0) AS lecturas_validas,
    COALESCE(SUM(sa.lecturas_invalidas), 0) AS lecturas_invalidas
FROM estacion_sismica e
LEFT JOIN vw_sensores_actividad sa ON sa.id_estacion = e.id_estacion
GROUP BY e.id_estacion, e.codigo, e.nombre, e.departamento, e.estado;

-- 3. Seguimiento de eventos con recuentos independientes.
CREATE OR REPLACE VIEW vw_eventos_seguimiento AS
SELECT
```

```
    ev.id_evento,
    CASE
        WHEN ev.id_evento BETWEEN 200000000 AND 299999999 THEN 'igp'
        WHEN ev.id_evento BETWEEN 810000001 AND 810000008
          OR ev.id_evento BETWEEN 820300001 AND 820300030 THEN 'simulado'
        ELSE 'sin_clasificar'
    END AS procedencia,
    ev.fecha_hora_origen,
    ev.latitud_epicentro,
    ev.longitud_epicentro,
    ev.profundidad_km,
    ev.magnitud,
    ev.estado_revision,
    COALESCE(d.total_detecciones, 0) AS total_detecciones,
    COALESCE(a.total_alertas, 0) AS total_alertas,
    COALESCE(d.total_detecciones, 0) > 0 AS tiene_deteccion,
    COALESCE(a.total_alertas, 0) > 0 AS tiene_alerta
FROM evento_sismico ev
LEFT JOIN (
    SELECT id_evento, COUNT(*) AS total_detecciones
    FROM deteccion
    GROUP BY id_evento
) d ON d.id_evento = ev.id_evento
LEFT JOIN (
    SELECT id_evento, COUNT(*) AS total_alertas
    FROM alerta
    GROUP BY id_evento
) a ON a.id_evento = ev.id_evento;

-- 4. Detalle de alertas y tiempo transcurrido desde el origen.
-- La resta de TIMESTAMP produce un INTERVAL (no segundos numericos).
CREATE OR REPLACE VIEW vw_alertas_detalle AS
SELECT
    a.id_alerta,
    a.id_evento,
    ev.procedencia,
    ev.fecha_hora_origen,
    a.fecha_hora_emision,
    a.fecha_hora_emision - ev.fecha_hora_origen AS tiempo_desde_origen,
    ev.magnitud AS magnitud_evento,
    a.magnitud_estimada,
    a.nivel,
    a.estado AS estado_alerta,
    a.mensaje,
    ev.total_detecciones AS sensores_detectores
FROM alerta a
JOIN vw_eventos_seguimiento ev ON ev.id_evento = a.id_evento;

-- 5. Resumen mensual exclusivo del catalogo oficial importado del IGP.
-- TIMESTAMP se interpreta como UTC porque esa fue la zona de la fuente.
CREATE OR REPLACE VIEW vw_igp_resumen_mensual AS
SELECT
    DATE_TRUNC('month', fecha_hora_origen)::DATE AS mes_utc,
    COUNT(*) AS total_eventos,
    COUNT(*) FILTER (WHERE magnitud >= 6) AS eventos_magnitud_6_mas,
    MIN(magnitud) AS magnitud_minima,
    MAX(magnitud) AS magnitud_maxima,
    AVG(magnitud) AS magnitud_promedio,
    AVG(profundidad_km) AS profundidad_promedio_km
FROM evento_sismico
WHERE id_evento BETWEEN 200000000 AND 299999999
GROUP BY DATE_TRUNC('month', fecha_hora_origen)::DATE;


SELECT table_name AS vista
FROM information_schema.views
WHERE table_schema = 'monitoreo_sismologico'
  AND table_name IN (
      'vw_sensores_actividad',
      'vw_estaciones_resumen',
      'vw_eventos_seguimiento',
      'vw_alertas_detalle',
      'vw_igp_resumen_mensual'
  )
ORDER BY table_name;

COMMIT;
```

```
\echo '=== VERIFICACION DE VISTAS ==='
SET search_path TO monitoreo_sismologico, public;

-- A. Existencia y cardinalidad de cuatro vistas de detalle.
WITH recuentos AS (
    SELECT 'vw_alertas_detalle'::TEXT AS vista, 29::BIGINT AS esperado,
           (SELECT COUNT(*) FROM vw_alertas_detalle) AS encontrado
    UNION ALL
    SELECT 'vw_estaciones_resumen', 15,
           (SELECT COUNT(*) FROM vw_estaciones_resumen)
    UNION ALL
    SELECT 'vw_eventos_seguimiento', 24327,
           (SELECT COUNT(*) FROM vw_eventos_seguimiento)
    UNION ALL
    SELECT 'vw_sensores_actividad', 40,
           (SELECT COUNT(*) FROM vw_sensores_actividad)
)
SELECT vista, esperado, encontrado,
       CASE WHEN encontrado = esperado THEN 'OK' ELSE 'REVISAR' END AS resultado
FROM recuentos
ORDER BY vista;

-- B. Sumas derivadas: no se deben multiplicar filas al combinar relaciones.
SELECT
    (SELECT SUM(total_sensores) FROM vw_estaciones_resumen) AS sensores_por_estacion,
    (SELECT SUM(total_lecturas) FROM vw_estaciones_resumen) AS lecturas_por_estacion,
    (SELECT SUM(lecturas_validas) FROM vw_sensores_actividad) AS lecturas_validas,
    (SELECT SUM(lecturas_invalidas) FROM vw_sensores_actividad) AS lecturas_invalidas,
    (SELECT SUM(total_detecciones) FROM vw_eventos_seguimiento) AS detecciones_por_evento,
    (SELECT SUM(total_alertas) FROM vw_eventos_seguimiento) AS alertas_por_evento;

-- Esperado: 40, 1190, 1113, 77, 104, 29.

-- C. Procedencia tecnica del evento y detecciones/alertas asociadas.
SELECT procedencia, COUNT(*) AS eventos,
       SUM(total_detecciones) AS detecciones,
       SUM(total_alertas) AS alertas
FROM vw_eventos_seguimiento
GROUP BY procedencia
ORDER BY procedencia;

-- Esperado: 'igp' 24289 / 0 / 0; 'simulado' 38 / 104 / 29.

-- D. Resumen mensual, sin asumir una cifra fija de meses con eventos.
SELECT COUNT(*) AS meses_con_eventos,
       SUM(total_eventos) AS eventos_igp,
       MIN(mes_utc) AS primer_mes_utc,
       MAX(mes_utc) AS ultimo_mes_utc
FROM vw_igp_resumen_mensual;

-- Esperado: 24289 eventos; primer mes 1960-01-01, ultimo 2025-12-01.

-- E. Cinco alertas como ejemplo. Las vistas no garantizan orden por si solas.
SELECT id_alerta, id_evento, procedencia, nivel,
       fecha_hora_emision, tiempo_desde_origen, sensores_detectores
FROM vw_alertas_detalle
ORDER BY fecha_hora_emision, id_alerta
LIMIT 5;

-- F. Verificacion estricta; si algo difiere se obtiene ERROR (sin borrar datos).
DO $$
BEGIN
    IF (SELECT COUNT(*) FROM information_schema.views
        WHERE table_schema = 'monitoreo_sismologico'
          AND table_name IN (
              'vw_sensores_actividad', 'vw_estaciones_resumen',
              'vw_eventos_seguimiento', 'vw_alertas_detalle',
              'vw_igp_resumen_mensual'
          )) <> 5
       OR ((SELECT COUNT(*) FROM estacion_sismica)
           + (SELECT COUNT(*) FROM sensor)
           + (SELECT COUNT(*) FROM lectura_onda)
           + (SELECT COUNT(*) FROM evento_sismico)
           + (SELECT COUNT(*) FROM deteccion)
           + (SELECT COUNT(*) FROM alerta)) <> 25705
```

```
       OR (SELECT COUNT(*) FROM vw_sensores_actividad) <> 40
       OR (SELECT COUNT(*) FROM vw_estaciones_resumen) <> 15
       OR (SELECT COUNT(*) FROM vw_eventos_seguimiento) <> 24327
       OR (SELECT COUNT(*) FROM vw_alertas_detalle) <> 29
       OR (SELECT COALESCE(SUM(total_eventos), 0) FROM vw_igp_resumen_mensual) <> 24289
       OR (SELECT COALESCE(SUM(total_sensores), 0) FROM vw_estaciones_resumen) <> 40
       OR (SELECT COALESCE(SUM(total_lecturas), 0) FROM vw_estaciones_resumen) <> 1190
       OR (SELECT COALESCE(SUM(lecturas_validas), 0) FROM vw_sensores_actividad) <> 1113
       OR (SELECT COALESCE(SUM(lecturas_invalidas), 0) FROM vw_sensores_actividad) <> 77
       OR (SELECT COALESCE(SUM(total_detecciones), 0) FROM vw_eventos_seguimiento) <> 104
       OR (SELECT COALESCE(SUM(total_alertas), 0) FROM vw_eventos_seguimiento) <> 29
       OR (SELECT COUNT(*) FROM vw_eventos_seguimiento WHERE procedencia = 'igp') <> 24289
       OR (SELECT COUNT(*) FROM vw_eventos_seguimiento WHERE procedencia = 'simulado') <> 38
       OR EXISTS (SELECT 1 FROM vw_eventos_seguimiento WHERE procedencia = 'sin_clasificar')
       OR EXISTS (SELECT 1 FROM vw_eventos_seguimiento
                  WHERE procedencia = 'igp' AND (total_detecciones <> 0 OR total_alertas <> 0))
       OR (SELECT MIN(mes_utc) FROM vw_igp_resumen_mensual) <> DATE '1960-01-01'
       OR (SELECT MAX(mes_utc) FROM vw_igp_resumen_mensual) <> DATE '2025-12-01' THEN
        RAISE EXCEPTION 'REVISAR: alguna vista o total derivado no coincide con los datos auditados.';
    END IF;
    RAISE NOTICE 'OK: 5 vistas, 25705 registros base y controles derivados consistentes.';
END;
$$;


\echo '=== PROCEDIMIENTOS ==='
BEGIN;
SET LOCAL search_path TO monitoreo_sismologico, public;

-- P1. Registrar conjuntamente una estacion y su primer sensor.
-- Evita dejar una estacion sin sensores durante el alta por esta interfaz.
CREATE OR REPLACE PROCEDURE sp_registrar_estacion_sensor(
    IN p_id_estacion INTEGER,
    IN p_codigo VARCHAR,
    IN p_nombre VARCHAR,
    IN p_latitud NUMERIC,
    IN p_longitud NUMERIC,
    IN p_estado_estacion VARCHAR,
    IN p_id_sensor INTEGER,
    IN p_numero_serie VARCHAR,
    IN p_tipo_sensor VARCHAR,
    IN p_unidad_medida VARCHAR,
    IN p_frecuencia_muestreo NUMERIC,
    IN p_fecha_instalacion DATE,
    IN p_estado_sensor VARCHAR
)
LANGUAGE plpgsql
SET search_path = monitoreo_sismologico, pg_temp
AS $$
BEGIN
    IF NULLIF(BTRIM(p_codigo), '') IS NULL
       OR NULLIF(BTRIM(p_nombre), '') IS NULL
       OR NULLIF(BTRIM(p_numero_serie), '') IS NULL
       OR NULLIF(BTRIM(p_tipo_sensor), '') IS NULL
       OR NULLIF(BTRIM(p_unidad_medida), '') IS NULL
       OR NULLIF(BTRIM(p_estado_estacion), '') IS NULL
       OR NULLIF(BTRIM(p_estado_sensor), '') IS NULL THEN
        RAISE EXCEPTION 'Los textos obligatorios de estacion y sensor no pueden estar vacios';
    END IF;

    INSERT INTO estacion_sismica (
        id_estacion, codigo, nombre, latitud, longitud,
        fecha_instalacion, estado
    ) VALUES (
        p_id_estacion, p_codigo, p_nombre, p_latitud, p_longitud,
        p_fecha_instalacion, p_estado_estacion
    );

    INSERT INTO sensor (
        id_sensor, numero_serie, tipo, unidad_medida,
        frecuencia_muestreo, fecha_instalacion, estado, id_estacion
    ) VALUES (
        p_id_sensor, p_numero_serie, p_tipo_sensor, p_unidad_medida,
        p_frecuencia_muestreo, p_fecha_instalacion,
        p_estado_sensor, p_id_estacion
    );
END;
```

```
$$;

-- P2. Registrar un evento sismico, sin fabricar atributos opcionales.
CREATE OR REPLACE PROCEDURE sp_registrar_evento(
    IN p_id_evento INTEGER,
    IN p_fecha_hora_origen TIMESTAMP,
    IN p_latitud_epicentro NUMERIC,
    IN p_longitud_epicentro NUMERIC,
    IN p_profundidad_km NUMERIC,
    IN p_magnitud NUMERIC,
    IN p_estado_revision VARCHAR
)
LANGUAGE plpgsql
SET search_path = monitoreo_sismologico, pg_temp
AS $$
BEGIN
    IF NULLIF(BTRIM(p_estado_revision), '') IS NULL THEN
        RAISE EXCEPTION 'El estado de revision es obligatorio';
    END IF;

    INSERT INTO evento_sismico (
        id_evento, fecha_hora_origen, latitud_epicentro,
        longitud_epicentro, profundidad_km, magnitud, estado_revision
    ) VALUES (
        p_id_evento, p_fecha_hora_origen, p_latitud_epicentro,
        p_longitud_epicentro, p_profundidad_km, p_magnitud, p_estado_revision
    );
END;
$$;

-- P3. Registrar una lectura. Solo admite nuevas lecturas validas
-- cuando el sensor se encuentra operativo en ese instante.
-- FOR SHARE evita un cambio concurrente de estado durante esta comprobacion.
CREATE OR REPLACE PROCEDURE sp_registrar_lectura(
    IN p_id_lectura BIGINT,
    IN p_id_sensor INTEGER,
    IN p_fecha_hora TIMESTAMP,
    IN p_canal VARCHAR,
    IN p_valor NUMERIC,
    IN p_es_valida BOOLEAN
)
LANGUAGE plpgsql
SET search_path = monitoreo_sismologico, pg_temp
AS $$
DECLARE
    v_estado VARCHAR(40);
BEGIN
    SELECT LOWER(BTRIM(estado)) INTO v_estado
    FROM sensor
    WHERE id_sensor = p_id_sensor
    FOR SHARE;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'No existe el sensor %', p_id_sensor;
    END IF;

    IF p_es_valida IS TRUE AND v_estado <> 'operativo' THEN
        RAISE EXCEPTION 'El sensor % no esta operativo; no puede registrar una nueva lectura valida', p_id_sensor;
    END IF;

    IF NULLIF(BTRIM(p_canal), '') IS NULL THEN
        RAISE EXCEPTION 'El canal de lectura no puede estar vacio';
    END IF;

    INSERT INTO lectura_onda (
        id_lectura, id_sensor, fecha_hora, canal, valor, es_valida
    ) VALUES (
        p_id_lectura, p_id_sensor, p_fecha_hora,
        p_canal, p_valor, p_es_valida
    );
END;
$$;

-- P4. Registrar una deteccion sustentada por una lectura valida.
-- Se exige coincidencia de sensor y hora; el modelo no posee id_lectura
-- en DETECCION, por lo que este control es procedimental y no una FK.
CREATE OR REPLACE PROCEDURE sp_registrar_deteccion(
```

```
    IN p_id_sensor INTEGER,
    IN p_id_evento INTEGER,
    IN p_hora_deteccion TIMESTAMP,
    IN p_aceleracion_maxima NUMERIC,
    IN p_calidad VARCHAR
)
LANGUAGE plpgsql
SET search_path = monitoreo_sismologico, pg_temp
AS $$
DECLARE
    v_fecha_origen TIMESTAMP;
BEGIN
    SELECT fecha_hora_origen INTO v_fecha_origen
    FROM evento_sismico
    WHERE id_evento = p_id_evento
    FOR SHARE;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'No existe el evento %', p_id_evento;
    END IF;

    IF p_hora_deteccion < v_fecha_origen THEN
        RAISE EXCEPTION 'La deteccion no puede anteceder al origen del evento %', p_id_evento;
    END IF;

    PERFORM 1
    FROM lectura_onda
    WHERE id_sensor = p_id_sensor
      AND fecha_hora = p_hora_deteccion
      AND es_valida IS TRUE
    FOR SHARE;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'No existe lectura valida para el sensor % en la hora indicada', p_id_sensor;
    END IF;

    INSERT INTO deteccion (
        id_sensor, id_evento, hora_deteccion, aceleracion_maxima, calidad
    ) VALUES (
        p_id_sensor, p_id_evento, p_hora_deteccion,
        p_aceleracion_maxima, p_calidad
    );
END;
$$;

-- P5. Emitir una alerta para un evento existente.
-- No exige una deteccion: el modelo permite alertas para un evento
-- sin imponer un numero minimo de detecciones.
CREATE OR REPLACE PROCEDURE sp_emitir_alerta(
    IN p_id_alerta INTEGER,
    IN p_id_evento INTEGER,
    IN p_fecha_hora_emision TIMESTAMP,
    IN p_nivel VARCHAR,
    IN p_mensaje TEXT,
    IN p_magnitud_estimada NUMERIC,
    IN p_estado VARCHAR
)
LANGUAGE plpgsql
SET search_path = monitoreo_sismologico, pg_temp
AS $$
DECLARE
    v_fecha_origen TIMESTAMP;
BEGIN
    SELECT fecha_hora_origen INTO v_fecha_origen
    FROM evento_sismico
    WHERE id_evento = p_id_evento
    FOR SHARE;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'No existe el evento %', p_id_evento;
    END IF;

    IF p_fecha_hora_emision < v_fecha_origen THEN
        RAISE EXCEPTION 'La alerta no puede emitirse antes del origen del evento %', p_id_evento;
    END IF;

    IF NULLIF(BTRIM(p_nivel), '') IS NULL
```

```
       OR NULLIF(BTRIM(p_mensaje), '') IS NULL
       OR NULLIF(BTRIM(p_estado), '') IS NULL THEN
        RAISE EXCEPTION 'Nivel, mensaje y estado de alerta son obligatorios';
    END IF;

    INSERT INTO alerta (
        id_alerta, id_evento, fecha_hora_emision,
        nivel, mensaje, magnitud_estimada, estado
    ) VALUES (
        p_id_alerta, p_id_evento, p_fecha_hora_emision,
        p_nivel, p_mensaje, p_magnitud_estimada, p_estado
    );
END;
$$;

-- P6. Actualizar estado y ultimo mantenimiento de un sensor.
-- NULL en la fecha elimina la ultima fecha (no implica nuevo mantenimiento).
-- El CHECK existente valida que no sea anterior a la instalacion.
CREATE OR REPLACE PROCEDURE sp_actualizar_estado_sensor(
    IN p_id_sensor INTEGER,
    IN p_estado VARCHAR,
    IN p_fecha_ultimo_mantenimiento DATE
)
LANGUAGE plpgsql
SET search_path = monitoreo_sismologico, pg_temp
AS $$
BEGIN
    IF NULLIF(BTRIM(p_estado), '') IS NULL THEN
        RAISE EXCEPTION 'El estado del sensor es obligatorio';
    END IF;

    UPDATE sensor
    SET estado = p_estado,
        fecha_ultimo_mantenimiento = p_fecha_ultimo_mantenimiento
    WHERE id_sensor = p_id_sensor;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'No existe el sensor %', p_id_sensor;
    END IF;
END;
$$;

SELECT proname AS procedimiento
FROM pg_proc
WHERE pronamespace = 'monitoreo_sismologico'::regnamespace
  AND prokind = 'p'
  AND proname IN (
      'sp_registrar_estacion_sensor',
      'sp_registrar_evento',
      'sp_registrar_lectura',
      'sp_registrar_deteccion',
      'sp_emitir_alerta',
      'sp_actualizar_estado_sensor'
  )
ORDER BY proname;

COMMIT;


\echo '=== PRUEBAS DE PROCEDIMIENTOS ==='
BEGIN;
SET LOCAL search_path TO monitoreo_sismologico, public;

DO $$
BEGIN
    IF (SELECT COUNT(*) FROM estacion_sismica) <> 15
       OR (SELECT COUNT(*) FROM sensor) <> 40
       OR (SELECT COUNT(*) FROM evento_sismico) <> 24327
       OR (SELECT COUNT(*) FROM lectura_onda) <> 1190
       OR (SELECT COUNT(*) FROM deteccion) <> 104
       OR (SELECT COUNT(*) FROM alerta) <> 29 THEN
        RAISE EXCEPTION 'Los recuentos de la base difieren de la distribucion esperada';
    END IF;

    IF EXISTS (SELECT 1 FROM estacion_sismica WHERE id_estacion = 930450001)
       OR EXISTS (SELECT 1 FROM sensor WHERE id_sensor = 930450001)
       OR EXISTS (SELECT 1 FROM evento_sismico WHERE id_evento = 930450001)
```

```
       OR EXISTS (SELECT 1 FROM lectura_onda WHERE id_lectura BETWEEN 930450000001 AND 930450000003)
       OR EXISTS (SELECT 1 FROM deteccion WHERE id_evento = 930450001)
       OR EXISTS (SELECT 1 FROM alerta WHERE id_alerta = 930450001) THEN
        RAISE EXCEPTION 'El rango de IDs reservado para las pruebas ya esta ocupado';
    END IF;
END;
$$;

-- 1. Altas validas: estacion + sensor, evento, lectura, deteccion y alerta.
CALL sp_registrar_estacion_sensor(
    930450001, 'TEST45-EST-01', 'Estacion de prueba transaccional',
    -12.05, -77.04, 'operativa',
    930450001, 'TEST45-SEN-01', 'acelerometro', 'm/s2', 50,
    DATE '2026-07-01', 'operativo'
);

CALL sp_registrar_evento(
    930450001, TIMESTAMP '2026-08-02 00:00:00',
    -12.08, -77.05, 28, 4.2, 'simulado'
);

CALL sp_registrar_lectura(
    930450000001, 930450001, TIMESTAMP '2026-08-02 00:00:10',
    'X', 0.18, TRUE
);

CALL sp_registrar_deteccion(
    930450001, 930450001, TIMESTAMP '2026-08-02 00:00:10',
    0.18, 'alta'
);

CALL sp_emitir_alerta(
    930450001, 930450001, TIMESTAMP '2026-08-02 00:01:30',
    'preventiva', 'Alerta de prueba, no destinada a difusion', 4.2, 'simulada'
);

CALL sp_actualizar_estado_sensor(
    930450001, 'mantenimiento', DATE '2026-08-05'
);

-- 2. Verificaciones negativas. Cada bloque con EXCEPTION se revierte
-- parcialmente sin comprometer la transaccion principal.
DO $$
DECLARE
    v_rechazos INTEGER := 0;
BEGIN
    -- Nueva lectura valida con sensor en mantenimiento: rechazo esperado.
    BEGIN
        CALL sp_registrar_lectura(
            930450000002, 930450001, TIMESTAMP '2026-08-02 00:00:11',
            'X', 0.20, TRUE
        );
    EXCEPTION WHEN raise_exception THEN
        IF SQLERRM NOT LIKE 'El sensor % no esta operativo%' THEN RAISE; END IF;
        v_rechazos := v_rechazos + 1;
    END;
    IF v_rechazos <> 1 THEN
        RAISE EXCEPTION 'No se rechazo la lectura valida de un sensor en mantenimiento';
    END IF;

    -- Deteccion sin lectura valida en la hora: rechazo esperado.
    BEGIN
        CALL sp_registrar_deteccion(
            930450001, 930450001, TIMESTAMP '2026-08-02 00:00:12',
            0.3, 'media'
        );
    EXCEPTION WHEN raise_exception THEN
        IF SQLERRM NOT LIKE 'No existe lectura valida%' THEN RAISE; END IF;
        v_rechazos := v_rechazos + 1;
    END;
    IF v_rechazos <> 2 THEN
        RAISE EXCEPTION 'No se rechazo la deteccion sin lectura valida';
    END IF;

    -- Segunda deteccion del mismo par sensor-evento: rechazo por PK.
    BEGIN
        CALL sp_registrar_deteccion(
```

```
            930450001, 930450001, TIMESTAMP '2026-08-02 00:00:10',
            0.18, 'alta'
        );
    EXCEPTION WHEN unique_violation THEN
        v_rechazos := v_rechazos + 1;
    END;
    IF v_rechazos <> 3 THEN
        RAISE EXCEPTION 'No se rechazo la deteccion duplicada';
    END IF;

    -- Emision de alerta anterior al evento: rechazo esperado.
    BEGIN
        CALL sp_emitir_alerta(
            930450002, 930450001, TIMESTAMP '2026-08-01 23:59:59',
            'informativa', 'Alerta no valida', NULL, 'simulada'
        );
    EXCEPTION WHEN raise_exception THEN
        IF SQLERRM NOT LIKE 'La alerta no puede emitirse antes%' THEN RAISE; END IF;
        v_rechazos := v_rechazos + 1;
    END;
    IF v_rechazos <> 4 THEN
        RAISE EXCEPTION 'No se rechazo la alerta anterior al evento';
    END IF;

    -- Fecha de mantenimiento anterior a instalacion: rechazo por CHECK.
    BEGIN
        CALL sp_actualizar_estado_sensor(
            930450001, 'mantenimiento', DATE '2026-06-01'
        );
    EXCEPTION WHEN check_violation THEN
        v_rechazos := v_rechazos + 1;
    END;
    IF v_rechazos <> 5 THEN
        RAISE EXCEPTION 'No se rechazo el mantenimiento anterior a la instalacion';
    END IF;

    -- Lecturas invalidas se pueden conservar en estado de mantenimiento.
    CALL sp_registrar_lectura(
        930450000003, 930450001, TIMESTAMP '2026-08-02 00:00:13',
        'X', 0.22, FALSE
    );

    IF (SELECT COUNT(*) FROM estacion_sismica WHERE id_estacion = 930450001) <> 1
       OR (SELECT COUNT(*) FROM sensor WHERE id_sensor = 930450001 AND estado = 'mantenimiento') <> 1
       OR (SELECT COUNT(*) FROM evento_sismico WHERE id_evento = 930450001) <> 1
       OR (SELECT COUNT(*) FROM lectura_onda WHERE id_lectura BETWEEN 930450000001 AND 930450000003) <> 2
       OR (SELECT COUNT(*) FROM deteccion WHERE id_sensor = 930450001 AND id_evento = 930450001) <> 1
       OR (SELECT COUNT(*) FROM alerta WHERE id_alerta = 930450001) <> 1 THEN
        RAISE EXCEPTION 'La prueba positiva o el control de rechazos no coincide';
    END IF;

    RAISE NOTICE 'OK: 6 procedimientos operativos, 5 rechazos esperados y lecturas historicas conservadas.';
END;
$$;

SELECT 'estaciones_prueba' AS control, COUNT(*) AS resultado
FROM estacion_sismica WHERE id_estacion = 930450001
UNION ALL SELECT 'sensores_prueba', COUNT(*) FROM sensor WHERE id_sensor = 930450001
UNION ALL SELECT 'eventos_prueba', COUNT(*) FROM evento_sismico WHERE id_evento = 930450001
UNION ALL SELECT 'lecturas_prueba', COUNT(*) FROM lectura_onda WHERE id_lectura BETWEEN 930450000001 AND 930450000003
UNION ALL SELECT 'detecciones_prueba', COUNT(*) FROM deteccion WHERE id_evento = 930450001
UNION ALL SELECT 'alertas_prueba', COUNT(*) FROM alerta WHERE id_alerta = 930450001
ORDER BY control;

ROLLBACK;

-- Comprobar que la prueba no haya dejado registros persistidos.
SELECT 'estaciones_prueba' AS control, COUNT(*) AS restantes
FROM monitoreo_sismologico.estacion_sismica WHERE id_estacion = 930450001
UNION ALL SELECT 'sensores_prueba', COUNT(*) FROM monitoreo_sismologico.sensor WHERE id_sensor = 930450001
UNION ALL SELECT 'eventos_prueba', COUNT(*) FROM monitoreo_sismologico.evento_sismico WHERE id_evento = 930450001
UNION ALL SELECT 'lecturas_prueba', COUNT(*) FROM monitoreo_sismologico.lectura_onda WHERE id_lectura BETWEEN
      930450000001 AND 930450000003
UNION ALL SELECT 'detecciones_prueba', COUNT(*) FROM monitoreo_sismologico.deteccion WHERE id_evento = 930450001
UNION ALL SELECT 'alertas_prueba', COUNT(*) FROM monitoreo_sismologico.alerta WHERE id_alerta = 930450001
ORDER BY control;
```

```
\echo '=== TRIGGERS ==='
-- No inserta, modifica ni borra registros de las seis tablas base.
BEGIN;
SET LOCAL search_path TO monitoreo_sismologico, public;

-- T1. Una lectura nueva o reasignada solo puede marcarse valida
-- si su sensor se encuentra operativo. No se invalidan lecturas
-- historicas cuando el estado del sensor cambia posteriormente.
CREATE OR REPLACE FUNCTION fn_trg_lectura_operativa()
RETURNS TRIGGER
LANGUAGE plpgsql
SET search_path = monitoreo_sismologico, pg_temp
AS $$
DECLARE
    v_estado TEXT;
BEGIN
    IF NEW.es_valida IS TRUE THEN
        SELECT LOWER(BTRIM(estado)) INTO v_estado
        FROM sensor
        WHERE id_sensor = NEW.id_sensor
        FOR SHARE;

        IF NOT FOUND THEN
            RAISE EXCEPTION 'TRG: el sensor % no existe', NEW.id_sensor;
        END IF;

        IF v_estado IS DISTINCT FROM 'operativo' THEN
            RAISE EXCEPTION 'TRG: el sensor % no esta operativo para nuevas lecturas validas', NEW.id_sensor;
        END IF;
    END IF;
    RETURN NEW;
END;
$$;

CREATE OR REPLACE TRIGGER trg_lectura_operativa
BEFORE INSERT OR UPDATE OF id_sensor, fecha_hora, es_valida
ON lectura_onda
FOR EACH ROW
EXECUTE FUNCTION fn_trg_lectura_operativa();

-- T2. Al actualizar o eliminar una lectura que sustentaba una
-- deteccion, debe quedar otra lectura valida con igual sensor y hora.
-- El identificador de lectura no forma parte de DETECCION.
CREATE OR REPLACE FUNCTION fn_trg_proteger_sustento()
RETURNS TRIGGER
LANGUAGE plpgsql
SET search_path = monitoreo_sismologico, pg_temp
AS $$
BEGIN
    IF OLD.es_valida IS NOT TRUE THEN
        IF TG_OP = 'DELETE' THEN RETURN OLD; END IF;
        RETURN NEW;
    END IF;

    IF TG_OP = 'UPDATE' THEN
        IF NEW.es_valida IS TRUE
           AND NEW.id_sensor = OLD.id_sensor
           AND NEW.fecha_hora = OLD.fecha_hora THEN
            RETURN NEW;
        END IF;
    END IF;

    -- Bloquea la deteccion mientras se comprueba su sustento.
    PERFORM 1
    FROM deteccion
    WHERE id_sensor = OLD.id_sensor
      AND hora_deteccion = OLD.fecha_hora
    FOR SHARE;

    IF FOUND AND NOT EXISTS (
        SELECT 1
        FROM lectura_onda
        WHERE id_sensor = OLD.id_sensor
          AND fecha_hora = OLD.fecha_hora
          AND es_valida IS TRUE
          AND id_lectura <> OLD.id_lectura
```

```
    ) THEN
        RAISE EXCEPTION 'TRG: la lectura % es el ultimo sustento valido de una deteccion', OLD.id_lectura;
    END IF;

    IF TG_OP = 'DELETE' THEN RETURN OLD; END IF;
    RETURN NEW;
END;
$$;

CREATE OR REPLACE TRIGGER trg_proteger_sustento_lectura
BEFORE UPDATE OF id_sensor, fecha_hora, es_valida OR DELETE
ON lectura_onda
FOR EACH ROW
EXECUTE FUNCTION fn_trg_proteger_sustento();

-- T3. Las detecciones directas deben tener hora >= origen y
-- una lectura valida del mismo sensor y hora exacta.
CREATE OR REPLACE FUNCTION fn_trg_deteccion_valida()
RETURNS TRIGGER
LANGUAGE plpgsql
SET search_path = monitoreo_sismologico, pg_temp
AS $$
DECLARE
    v_origen TIMESTAMP;
BEGIN
    SELECT fecha_hora_origen INTO v_origen
    FROM evento_sismico
    WHERE id_evento = NEW.id_evento
    FOR SHARE;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'TRG: el evento % no existe', NEW.id_evento;
    END IF;
    IF NEW.hora_deteccion < v_origen THEN
        RAISE EXCEPTION 'TRG: deteccion anterior al origen del evento %', NEW.id_evento;
    END IF;

    PERFORM 1
    FROM lectura_onda
    WHERE id_sensor = NEW.id_sensor
      AND fecha_hora = NEW.hora_deteccion
      AND es_valida IS TRUE
    FOR SHARE;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'TRG: deteccion sin lectura valida del sensor % en la hora indicada', NEW.id_sensor;
    END IF;

    RETURN NEW;
END;
$$;

CREATE OR REPLACE TRIGGER trg_deteccion_valida
BEFORE INSERT OR UPDATE OF id_sensor, id_evento, hora_deteccion
ON deteccion
FOR EACH ROW
EXECUTE FUNCTION fn_trg_deteccion_valida();

-- T4. No se emiten alertas anteriores al origen de su evento.
CREATE OR REPLACE FUNCTION fn_trg_alerta_temporal()
RETURNS TRIGGER
LANGUAGE plpgsql
SET search_path = monitoreo_sismologico, pg_temp
AS $$
DECLARE
    v_origen TIMESTAMP;
BEGIN
    SELECT fecha_hora_origen INTO v_origen
    FROM evento_sismico
    WHERE id_evento = NEW.id_evento
    FOR SHARE;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'TRG: el evento % no existe', NEW.id_evento;
    END IF;
    IF NEW.fecha_hora_emision < v_origen THEN
        RAISE EXCEPTION 'TRG: alerta anterior al origen del evento %', NEW.id_evento;
```

```
    END IF;
    RETURN NEW;
END;
$$;

CREATE OR REPLACE TRIGGER trg_alerta_temporal
BEFORE INSERT OR UPDATE OF id_evento, fecha_hora_emision
ON alerta
FOR EACH ROW
EXECUTE FUNCTION fn_trg_alerta_temporal();

-- T5. Una revision del origen no puede dejar detecciones ni
-- alertas ya asociadas con una hora anterior al nuevo origen.
CREATE OR REPLACE FUNCTION fn_trg_revisar_origen_evento()
RETURNS TRIGGER
LANGUAGE plpgsql
SET search_path = monitoreo_sismologico, pg_temp
AS $$
BEGIN
    IF NEW.fecha_hora_origen IS DISTINCT FROM OLD.fecha_hora_origen THEN
        IF EXISTS (
            SELECT 1 FROM deteccion
            WHERE id_evento = OLD.id_evento
              AND hora_deteccion < NEW.fecha_hora_origen
        ) OR EXISTS (
            SELECT 1 FROM alerta
            WHERE id_evento = OLD.id_evento
              AND fecha_hora_emision < NEW.fecha_hora_origen
        ) THEN
            RAISE EXCEPTION 'TRG: el nuevo origen del evento % invalida detecciones o alertas existentes', OLD.
     id_evento;
        END IF;
    END IF;
    RETURN NEW;
END;
$$;

CREATE OR REPLACE TRIGGER trg_revisar_origen_evento
BEFORE UPDATE OF fecha_hora_origen
ON evento_sismico
FOR EACH ROW
EXECUTE FUNCTION fn_trg_revisar_origen_evento();

-- T6. Serializa traslados y bajas de sensores de una estacion
-- mediante un bloqueo sobre la fila de su estacion de origen.
CREATE OR REPLACE FUNCTION fn_trg_bloquear_estacion_origen()
RETURNS TRIGGER
LANGUAGE plpgsql
SET search_path = monitoreo_sismologico, pg_temp
AS $$
BEGIN
    IF TG_OP = 'DELETE' THEN
        PERFORM 1 FROM estacion_sismica
        WHERE id_estacion = OLD.id_estacion
        FOR UPDATE;
        RETURN OLD;
    END IF;
    IF OLD.id_estacion IS DISTINCT FROM NEW.id_estacion THEN
        PERFORM 1 FROM estacion_sismica
        WHERE id_estacion = OLD.id_estacion
        FOR UPDATE;
    END IF;
    RETURN NEW;
END;
$$;

CREATE OR REPLACE TRIGGER trg_bloquear_estacion_origen
BEFORE UPDATE OF id_estacion OR DELETE
ON sensor
FOR EACH ROW
EXECUTE FUNCTION fn_trg_bloquear_estacion_origen();

-- T7. Participacion minima: cada estacion existente posee
-- al menos un sensor al verificar restricciones diferidas.
-- Asi se admite INSERT estacion seguido de INSERT sensor
-- en la misma transaccion, pero se rechaza dejarla vacia.
CREATE OR REPLACE FUNCTION fn_trg_estacion_con_sensor()
```

```
RETURNS TRIGGER
LANGUAGE plpgsql
SET search_path = monitoreo_sismologico, pg_temp
AS $$
DECLARE
    v_id_estacion INTEGER;
BEGIN
    IF TG_TABLE_NAME = 'estacion_sismica' THEN
        v_id_estacion := NEW.id_estacion;
    ELSE
        v_id_estacion := OLD.id_estacion;
        IF TG_OP = 'UPDATE' THEN
            IF OLD.id_estacion IS NOT DISTINCT FROM NEW.id_estacion THEN
                RETURN NULL;
            END IF;
        END IF;
    END IF;

    -- La estacion puede haber sido eliminada en esta misma transaccion.
    IF EXISTS (
        SELECT 1 FROM estacion_sismica WHERE id_estacion = v_id_estacion
    ) AND NOT EXISTS (
        SELECT 1 FROM sensor WHERE id_estacion = v_id_estacion
    ) THEN
        RAISE EXCEPTION 'TRG: la estacion % debe conservar al menos un sensor', v_id_estacion;
    END IF;
    RETURN NULL;
END;
$$;

DROP TRIGGER IF EXISTS ct_estacion_sensor_alta ON estacion_sismica;
CREATE CONSTRAINT TRIGGER ct_estacion_sensor_alta
AFTER INSERT ON estacion_sismica
DEFERRABLE INITIALLY DEFERRED
FOR EACH ROW
EXECUTE FUNCTION fn_trg_estacion_con_sensor();

DROP TRIGGER IF EXISTS ct_estacion_sensor_baja ON sensor;
CREATE CONSTRAINT TRIGGER ct_estacion_sensor_baja
AFTER DELETE ON sensor
DEFERRABLE INITIALLY DEFERRED
FOR EACH ROW
EXECUTE FUNCTION fn_trg_estacion_con_sensor();

DROP TRIGGER IF EXISTS ct_estacion_sensor_traslado ON sensor;
CREATE CONSTRAINT TRIGGER ct_estacion_sensor_traslado
AFTER UPDATE OF id_estacion ON sensor
DEFERRABLE INITIALLY DEFERRED
FOR EACH ROW
EXECUTE FUNCTION fn_trg_estacion_con_sensor();

-- Verificacion estructural previa: datos ya existentes.
DO $$
BEGIN
    IF EXISTS (
        SELECT 1 FROM estacion_sismica e
        WHERE NOT EXISTS (SELECT 1 FROM sensor s WHERE s.id_estacion = e.id_estacion)
    ) OR EXISTS (
        SELECT 1 FROM deteccion d
        JOIN evento_sismico e USING (id_evento)
        WHERE d.hora_deteccion < e.fecha_hora_origen
           OR NOT EXISTS (
               SELECT 1 FROM lectura_onda l
               WHERE l.id_sensor = d.id_sensor
                 AND l.fecha_hora = d.hora_deteccion
                 AND l.es_valida IS TRUE
           )
    ) OR EXISTS (
        SELECT 1 FROM alerta a
        JOIN evento_sismico e USING (id_evento)
        WHERE a.fecha_hora_emision < e.fecha_hora_origen
    ) THEN
        RAISE EXCEPTION 'Existen datos previos incompatibles con las reglas de los triggers';
    END IF;

    IF (
        SELECT COUNT(*)
```

```
        FROM pg_trigger t
        JOIN pg_class c ON c.oid = t.tgrelid
        JOIN pg_namespace n ON n.oid = c.relnamespace
        WHERE n.nspname = 'monitoreo_sismologico'
          AND NOT t.tgisinternal
          AND t.tgname IN (
              'trg_lectura_operativa', 'trg_proteger_sustento_lectura',
              'trg_deteccion_valida', 'trg_alerta_temporal',
              'trg_revisar_origen_evento', 'trg_bloquear_estacion_origen',
              'ct_estacion_sensor_alta', 'ct_estacion_sensor_baja',
              'ct_estacion_sensor_traslado'
          )
    ) <> 9 THEN
        RAISE EXCEPTION 'No se encontraron los 9 triggers previstos';
    END IF;

    RAISE NOTICE 'OK: 9 triggers instalados; datos previos compatibles.';
END;
$$;

SELECT c.relname AS tabla, t.tgname AS trigger,
       CASE WHEN t.tgdeferrable THEN 'DIFERIDO' ELSE 'INMEDIATO' END AS control
FROM pg_trigger t
JOIN pg_class c ON c.oid = t.tgrelid
JOIN pg_namespace n ON n.oid = c.relnamespace
WHERE n.nspname = 'monitoreo_sismologico'
  AND NOT t.tgisinternal
  AND t.tgname IN (
      'trg_lectura_operativa', 'trg_proteger_sustento_lectura',
      'trg_deteccion_valida', 'trg_alerta_temporal',
      'trg_revisar_origen_evento', 'trg_bloquear_estacion_origen',
      'ct_estacion_sensor_alta', 'ct_estacion_sensor_baja',
      'ct_estacion_sensor_traslado'
  )
ORDER BY tabla, trigger;

COMMIT;


\echo '=== PRUEBAS DE TRIGGERS ==='
BEGIN;
SET LOCAL search_path TO monitoreo_sismologico, public;

DO $$
BEGIN
    IF (SELECT COUNT(*) FROM estacion_sismica) <> 15
       OR (SELECT COUNT(*) FROM sensor) <> 40
       OR (SELECT COUNT(*) FROM evento_sismico) <> 24327
       OR (SELECT COUNT(*) FROM lectura_onda) <> 1190
       OR (SELECT COUNT(*) FROM deteccion) <> 104
       OR (SELECT COUNT(*) FROM alerta) <> 29 THEN
        RAISE EXCEPTION 'Los recuentos iniciales no coinciden con los 25705 registros verificados';
    END IF;

    IF EXISTS (SELECT 1 FROM estacion_sismica WHERE id_estacion BETWEEN 930460001 AND 930460099)
       OR EXISTS (SELECT 1 FROM sensor WHERE id_sensor BETWEEN 930460001 AND 930460099)
       OR EXISTS (SELECT 1 FROM evento_sismico WHERE id_evento = 930460001)
       OR EXISTS (SELECT 1 FROM lectura_onda WHERE id_lectura BETWEEN 930460000001 AND 930460000099)
       OR EXISTS (SELECT 1 FROM deteccion WHERE id_evento = 930460001)
       OR EXISTS (SELECT 1 FROM alerta WHERE id_alerta = 930460001) THEN
        RAISE EXCEPTION 'Rango de IDs reservado para los triggers ya ocupado';
    END IF;
END;
$$;

-- 1. Escenario positivo: inserciones SQL directas (sin CALL).
INSERT INTO estacion_sismica (
    id_estacion, codigo, nombre, latitud, longitud, fecha_instalacion, estado
) VALUES
    (930460001, 'TEST46-EST-A', 'Estacion temporal A', -12.05, -77.04,
     DATE '2026-07-01', 'operativa'),
    (930460002, 'TEST46-EST-B', 'Estacion temporal B', -12.06, -77.05,
     DATE '2026-07-01', 'operativa');

INSERT INTO sensor (
    id_sensor, numero_serie, tipo, unidad_medida, frecuencia_muestreo,
    fecha_instalacion, estado, id_estacion
```

```
) VALUES
    (930460001, 'TEST46-SEN-A', 'acelerometro', 'm/s2', 50,
     DATE '2026-07-01', 'operativo', 930460001),
    (930460002, 'TEST46-SEN-B', 'acelerometro', 'm/s2', 50,
     DATE '2026-07-01', 'operativo', 930460002);

INSERT INTO evento_sismico (
    id_evento, fecha_hora_origen, latitud_epicentro,
    longitud_epicentro, profundidad_km, magnitud, estado_revision
) VALUES (
    930460001, TIMESTAMP '2026-08-02 00:00:00',
    -12.08, -77.05, 28, 4.2, 'simulado'
);

INSERT INTO lectura_onda (
    id_lectura, id_sensor, fecha_hora, canal, valor, es_valida
) VALUES (
    930460000001, 930460001, TIMESTAMP '2026-08-02 00:00:10', 'X', 0.18, TRUE
);

INSERT INTO deteccion (
    id_sensor, id_evento, hora_deteccion, aceleracion_maxima, calidad
) VALUES (
    930460001, 930460001, TIMESTAMP '2026-08-02 00:00:10', 0.18, 'alta'
);

INSERT INTO alerta (
    id_alerta, id_evento, fecha_hora_emision, nivel, mensaje,
    magnitud_estimada, estado
) VALUES (
    930460001, 930460001, TIMESTAMP '2026-08-02 00:01:30',
    'preventiva', 'Alerta de prueba no destinada a difusion', 4.2, 'simulada'
);

-- Las lecturas validas existentes no cambian si el sensor pasa a mantenimiento.
UPDATE sensor SET estado = 'mantenimiento' WHERE id_sensor = 930460001;
INSERT INTO lectura_onda (
    id_lectura, id_sensor, fecha_hora, canal, valor, es_valida
) VALUES (
    930460000002, 930460001, TIMESTAMP '2026-08-02 00:00:11', 'X', 0.20, FALSE
);

-- 2. Nueve rechazos esperados: se capturan en subtransacciones.
DO $$
DECLARE
    v_rechazos INTEGER := 0;
BEGIN
    -- Caso 1: el sensor en mantenimiento no genera otra lectura valida.
    BEGIN
        INSERT INTO lectura_onda (id_lectura, id_sensor, fecha_hora, canal, valor, es_valida)
        VALUES (930460000003, 930460001, TIMESTAMP '2026-08-02 00:00:12', 'X', 0.25, TRUE);
        RAISE EXCEPTION 'No se rechazo la lectura valida del sensor en mantenimiento';
    EXCEPTION WHEN raise_exception THEN
        IF SQLERRM NOT LIKE 'TRG: el sensor % no esta operativo%' THEN RAISE; END IF;
        v_rechazos := v_rechazos + 1;
    END;

    -- Caso 2: deteccion sin lectura valida a esa hora.
    BEGIN
        INSERT INTO deteccion (id_sensor, id_evento, hora_deteccion, aceleracion_maxima, calidad)
        VALUES (930460002, 930460001, TIMESTAMP '2026-08-02 00:00:13', 0.19, 'media');
        RAISE EXCEPTION 'No se rechazo la deteccion sin lectura';
    EXCEPTION WHEN raise_exception THEN
        IF SQLERRM NOT LIKE 'TRG: deteccion sin lectura valida%' THEN RAISE; END IF;
        v_rechazos := v_rechazos + 1;
    END;

    -- Caso 3: alerta anterior al origen del evento.
    BEGIN
        INSERT INTO alerta (id_alerta, id_evento, fecha_hora_emision, nivel, mensaje, estado)
        VALUES (930460002, 930460001, TIMESTAMP '2026-08-01 23:59:59',
                'informativa', 'Alerta de prueba incorrecta', 'simulada');
        RAISE EXCEPTION 'No se rechazo la alerta anterior al origen';
    EXCEPTION WHEN raise_exception THEN
        IF SQLERRM NOT LIKE 'TRG: alerta anterior al origen%' THEN RAISE; END IF;
        v_rechazos := v_rechazos + 1;
    END;
```

```
    -- Caso 4: no se puede invalidar el unico sustento de una deteccion.
    BEGIN
        UPDATE lectura_onda SET es_valida = FALSE WHERE id_lectura = 930460000001;
        RAISE EXCEPTION 'No se rechazo la invalidacion de lectura sustentadora';
    EXCEPTION WHEN raise_exception THEN
        IF SQLERRM NOT LIKE 'TRG: la lectura % es el ultimo sustento%' THEN RAISE; END IF;
        v_rechazos := v_rechazos + 1;
    END;

    -- Caso 5: tampoco se puede borrar ese unico sustento.
    BEGIN
        DELETE FROM lectura_onda WHERE id_lectura = 930460000001;
        RAISE EXCEPTION 'No se rechazo el borrado de lectura sustentadora';
    EXCEPTION WHEN raise_exception THEN
        IF SQLERRM NOT LIKE 'TRG: la lectura % es el ultimo sustento%' THEN RAISE; END IF;
        v_rechazos := v_rechazos + 1;
    END;

    -- Caso 6: una correccion de origen no debe invalidar detecciones previas.
    BEGIN
        UPDATE evento_sismico
        SET fecha_hora_origen = TIMESTAMP '2026-08-02 00:00:11'
        WHERE id_evento = 930460001;
        RAISE EXCEPTION 'No se rechazo el cambio inconsistente de origen';
    EXCEPTION WHEN raise_exception THEN
        IF SQLERRM NOT LIKE 'TRG: el nuevo origen%' THEN RAISE; END IF;
        v_rechazos := v_rechazos + 1;
    END;

    -- Caso 7: una nueva estacion no puede quedar sin sensores al validar.
    BEGIN
        INSERT INTO estacion_sismica (
            id_estacion, codigo, nombre, latitud, longitud, estado
        ) VALUES (930460099, 'TEST46-SOLA', 'Estacion sin sensor', -12, -77, 'operativa');
        SET CONSTRAINTS ALL IMMEDIATE;
        RAISE EXCEPTION 'No se rechazo la estacion sin sensor';
    EXCEPTION WHEN raise_exception THEN
        IF SQLERRM NOT LIKE 'TRG: la estacion % debe conservar%' THEN RAISE; END IF;
        v_rechazos := v_rechazos + 1;
    END;

    -- Caso 8: no se puede eliminar el unico sensor de una estacion.
    BEGIN
        DELETE FROM sensor WHERE id_sensor = 930460002;
        SET CONSTRAINTS ALL IMMEDIATE;
        RAISE EXCEPTION 'No se rechazo la baja del ultimo sensor';
    EXCEPTION WHEN raise_exception THEN
        IF SQLERRM NOT LIKE 'TRG: la estacion % debe conservar%' THEN RAISE; END IF;
        v_rechazos := v_rechazos + 1;
    END;

    -- Caso 9: tampoco puede trasladarse el unico sensor de una estacion.
    BEGIN
        UPDATE sensor SET id_estacion = 930460001 WHERE id_sensor = 930460002;
        SET CONSTRAINTS ALL IMMEDIATE;
        RAISE EXCEPTION 'No se rechazo el traslado del ultimo sensor';
    EXCEPTION WHEN raise_exception THEN
        IF SQLERRM NOT LIKE 'TRG: la estacion % debe conservar%' THEN RAISE; END IF;
        v_rechazos := v_rechazos + 1;
    END;

    IF v_rechazos <> 9 THEN
        RAISE EXCEPTION 'Se esperaban 9 rechazos, pero se comprobaron %', v_rechazos;
    END IF;

    IF (SELECT COUNT(*) FROM estacion_sismica WHERE id_estacion BETWEEN 930460001 AND 930460099) <> 2
       OR (SELECT COUNT(*) FROM sensor WHERE id_sensor BETWEEN 930460001 AND 930460099) <> 2
       OR (SELECT COUNT(*) FROM evento_sismico WHERE id_evento = 930460001) <> 1
       OR (SELECT COUNT(*) FROM lectura_onda WHERE id_lectura BETWEEN 930460000001 AND 930460000099) <> 2
       OR (SELECT COUNT(*) FROM lectura_onda WHERE id_lectura = 930460000001 AND es_valida) <> 1
       OR (SELECT COUNT(*) FROM deteccion WHERE id_evento = 930460001) <> 1
       OR (SELECT COUNT(*) FROM alerta WHERE id_alerta = 930460001) <> 1 THEN
        RAISE EXCEPTION 'Resultados positivos o reversiones parciales inconsistentes';
    END IF;

    RAISE NOTICE 'OK: 9 triggers, 9 rechazos esperados y lecturas historicas preservadas.';
```

```
END;
$$;

SELECT 'estaciones_prueba' AS control, COUNT(*) AS resultado
FROM estacion_sismica WHERE id_estacion BETWEEN 930460001 AND 930460099
UNION ALL SELECT 'sensores_prueba', COUNT(*) FROM sensor WHERE id_sensor BETWEEN 930460001 AND 930460099
UNION ALL SELECT 'eventos_prueba', COUNT(*) FROM evento_sismico WHERE id_evento = 930460001
UNION ALL SELECT 'lecturas_prueba', COUNT(*) FROM lectura_onda WHERE id_lectura BETWEEN 930460000001 AND 930460000099
UNION ALL SELECT 'detecciones_prueba', COUNT(*) FROM deteccion WHERE id_evento = 930460001
UNION ALL SELECT 'alertas_prueba', COUNT(*) FROM alerta WHERE id_alerta = 930460001
ORDER BY control;

ROLLBACK;

-- Confirmar que todos los IDs reservados siguen libres tras las pruebas.
SELECT 'estaciones_prueba' AS control, COUNT(*) AS restantes
FROM monitoreo_sismologico.estacion_sismica WHERE id_estacion BETWEEN 930460001 AND 930460099
UNION ALL SELECT 'sensores_prueba', COUNT(*) FROM monitoreo_sismologico.sensor WHERE id_sensor BETWEEN 930460001 AND
      930460099
UNION ALL SELECT 'eventos_prueba', COUNT(*) FROM monitoreo_sismologico.evento_sismico WHERE id_evento = 930460001
UNION ALL SELECT 'lecturas_prueba', COUNT(*) FROM monitoreo_sismologico.lectura_onda WHERE id_lectura BETWEEN
      930460000001 AND 930460000099
UNION ALL SELECT 'detecciones_prueba', COUNT(*) FROM monitoreo_sismologico.deteccion WHERE id_evento = 930460001
UNION ALL SELECT 'alertas_prueba', COUNT(*) FROM monitoreo_sismologico.alerta WHERE id_alerta = 930460001
ORDER BY control;

SELECT SUM(registros) AS registros_totales, 25705 AS esperado
FROM (
    SELECT COUNT(*) AS registros FROM monitoreo_sismologico.estacion_sismica
    UNION ALL SELECT COUNT(*) FROM monitoreo_sismologico.sensor
    UNION ALL SELECT COUNT(*) FROM monitoreo_sismologico.evento_sismico
    UNION ALL SELECT COUNT(*) FROM monitoreo_sismologico.lectura_onda
    UNION ALL SELECT COUNT(*) FROM monitoreo_sismologico.deteccion
    UNION ALL SELECT COUNT(*) FROM monitoreo_sismologico.alerta
) r;


\echo '=== CONSULTAS C1-C5 ==='
-- 5.1 - Cinco consultas SQL intermedias para la experimentacion

SET search_path TO monitoreo_sismologico, public;

\echo
\echo CONSULTA 1. Sismos IGP de magnitud >= 6, por anio (2010-2025)
SELECT
    EXTRACT(YEAR FROM ev.fecha_hora_origen)::INTEGER AS anio_utc,
    COUNT(*) AS cantidad_eventos,
    ROUND(AVG(ev.magnitud), 2) AS magnitud_promedio,
    MAX(ev.magnitud) AS magnitud_maxima,
    ROUND(AVG(ev.profundidad_km), 2) AS profundidad_promedio_km
FROM evento_sismico ev
WHERE ev.id_evento BETWEEN 200000000 AND 299999999
  AND ev.fecha_hora_origen >= TIMESTAMP '2010-01-01 00:00:00'
  AND ev.fecha_hora_origen < TIMESTAMP '2026-01-01 00:00:00'
  AND ev.magnitud >= 6
GROUP BY EXTRACT(YEAR FROM ev.fecha_hora_origen)::INTEGER
HAVING COUNT(*) >= 1
ORDER BY anio_utc;

\echo
\echo CONSULTA 2. Sismos IGP cercanos por ventana rectangular a estaciones
-- Ventana de +/- 1.5 grados latitud/longitud; NO es una distancia geodesica.
SELECT
    e.id_estacion,
    e.codigo AS codigo_estacion,
    e.nombre AS nombre_estacion,
    COUNT(*) AS eventos_en_ventana,
    ROUND(AVG(ev.magnitud), 2) AS magnitud_promedio,
    MAX(ev.magnitud) AS magnitud_maxima
FROM estacion_sismica e
JOIN evento_sismico ev
  ON ev.latitud_epicentro BETWEEN e.latitud - 1.5 AND e.latitud + 1.5
 AND ev.longitud_epicentro BETWEEN e.longitud - 1.5 AND e.longitud + 1.5
WHERE ev.id_evento BETWEEN 200000000 AND 299999999
  AND ev.fecha_hora_origen >= TIMESTAMP '2000-01-01 00:00:00'
  AND ev.fecha_hora_origen < TIMESTAMP '2026-01-01 00:00:00'
  AND ev.magnitud >= 5
```

```
GROUP BY e.id_estacion, e.codigo, e.nombre
HAVING COUNT(*) >= 1
ORDER BY eventos_en_ventana DESC, e.id_estacion
LIMIT 10;

\echo
\echo CONSULTA 3. Calidad de lecturas por estacion (agosto-noviembre 2026)
SELECT
    e.id_estacion,
    e.codigo AS codigo_estacion,
    COUNT(DISTINCT s.id_sensor) AS sensores_con_lecturas,
    COUNT(*) AS total_lecturas,
    COUNT(*) FILTER (WHERE l.es_valida) AS lecturas_validas,
    COUNT(*) FILTER (WHERE NOT l.es_valida) AS lecturas_invalidas,
    ROUND(100.0 * COUNT(*) FILTER (WHERE NOT l.es_valida)
          / NULLIF(COUNT(*), 0), 2) AS porcentaje_invalidas
FROM estacion_sismica e
JOIN sensor s ON s.id_estacion = e.id_estacion
JOIN lectura_onda l ON l.id_sensor = s.id_sensor
WHERE l.fecha_hora >= TIMESTAMP '2026-08-01 00:00:00'
  AND l.fecha_hora < TIMESTAMP '2026-12-01 00:00:00'
GROUP BY e.id_estacion, e.codigo
HAVING COUNT(*) >= 10
ORDER BY porcentaje_invalidas DESC, total_lecturas DESC, e.id_estacion;

\echo
\echo CONSULTA 4. Tiempo a primera deteccion y primera alerta en eventos simulados
WITH detecciones_resumen AS (
    SELECT id_evento, COUNT(*) AS total_detecciones,
           MIN(hora_deteccion) AS primera_deteccion
    FROM deteccion
    GROUP BY id_evento
), alertas_resumen AS (
    SELECT id_evento, COUNT(*) AS total_alertas,
           MIN(fecha_hora_emision) AS primera_alerta
    FROM alerta
    GROUP BY id_evento
)
SELECT
    ev.id_evento,
    ev.fecha_hora_origen,
    ev.magnitud,
    COALESCE(d.total_detecciones, 0) AS total_detecciones,
    COALESCE(a.total_alertas, 0) AS total_alertas,
    ROUND(EXTRACT(EPOCH FROM (d.primera_deteccion - ev.fecha_hora_origen))::NUMERIC, 2)
        AS segundos_primera_deteccion,
    ROUND(EXTRACT(EPOCH FROM (a.primera_alerta - ev.fecha_hora_origen))::NUMERIC, 2)
        AS segundos_primera_alerta
FROM evento_sismico ev
LEFT JOIN detecciones_resumen d ON d.id_evento = ev.id_evento
LEFT JOIN alertas_resumen a ON a.id_evento = ev.id_evento
WHERE (ev.id_evento BETWEEN 810000001 AND 810000008
    OR ev.id_evento BETWEEN 820300001 AND 820300030)
  AND (COALESCE(d.total_detecciones, 0) > 0
    OR COALESCE(a.total_alertas, 0) > 0)
ORDER BY ev.fecha_hora_origen DESC, ev.id_evento
LIMIT 15;

\echo
\echo CONSULTA 5. Sensores con lecturas pero sin detecciones en agosto de 2026
-- Sin detecciones no equivale por si solo a sensor defectuoso.
SELECT
    s.id_sensor,
    s.numero_serie,
    e.codigo AS codigo_estacion,
    s.estado AS estado_sensor,
    COUNT(*) AS lecturas_periodo,
    COUNT(*) FILTER (WHERE NOT l.es_valida) AS lecturas_invalidas,
    MAX(l.fecha_hora) AS ultima_lectura
FROM sensor s
JOIN estacion_sismica e ON e.id_estacion = s.id_estacion
JOIN lectura_onda l ON l.id_sensor = s.id_sensor
WHERE l.fecha_hora >= TIMESTAMP '2026-08-01 00:00:00'
  AND l.fecha_hora < TIMESTAMP '2026-09-01 00:00:00'
  AND NOT EXISTS (
      SELECT 1
      FROM deteccion d
```

```
      WHERE d.id_sensor = s.id_sensor
        AND d.hora_deteccion >= TIMESTAMP '2026-08-01 00:00:00'
        AND d.hora_deteccion < TIMESTAMP '2026-09-01 00:00:00'
  )
GROUP BY s.id_sensor, s.numero_serie, e.codigo, s.estado
HAVING COUNT(*) >= 20
ORDER BY lecturas_invalidas DESC, lecturas_periodo DESC, s.id_sensor
LIMIT 15;


\echo '=== VERIFICACION DE CONSULTAS ==='
BEGIN;
SET LOCAL search_path TO monitoreo_sismologico, public;

CREATE TEMP TABLE control_consultas_51 ON COMMIT DROP AS
SELECT 1 AS numero, 'Historico IGP por anio'::TEXT AS consulta,
       (SELECT COUNT(*) FROM (
      SELECT
          EXTRACT(YEAR FROM ev.fecha_hora_origen)::INTEGER AS anio_utc,
          COUNT(*) AS cantidad_eventos,
          ROUND(AVG(ev.magnitud), 2) AS magnitud_promedio,
          MAX(ev.magnitud) AS magnitud_maxima,
          ROUND(AVG(ev.profundidad_km), 2) AS profundidad_promedio_km
      FROM evento_sismico ev
      WHERE ev.id_evento BETWEEN 200000000 AND 299999999
        AND ev.fecha_hora_origen >= TIMESTAMP '2010-01-01 00:00:00'
        AND ev.fecha_hora_origen < TIMESTAMP '2026-01-01 00:00:00'
        AND ev.magnitud >= 6
      GROUP BY EXTRACT(YEAR FROM ev.fecha_hora_origen)::INTEGER
      HAVING COUNT(*) >= 1
      ORDER BY anio_utc
       ) AS resultado) AS filas_obtenidas
UNION ALL
SELECT 2 AS numero, 'Ventanas geograficas de estaciones'::TEXT AS consulta,
       (SELECT COUNT(*) FROM (
      SELECT
          e.id_estacion,
          e.codigo AS codigo_estacion,
          e.nombre AS nombre_estacion,
          COUNT(*) AS eventos_en_ventana,
          ROUND(AVG(ev.magnitud), 2) AS magnitud_promedio,
          MAX(ev.magnitud) AS magnitud_maxima
      FROM estacion_sismica e
      JOIN evento_sismico ev
        ON ev.latitud_epicentro BETWEEN e.latitud - 1.5 AND e.latitud + 1.5
       AND ev.longitud_epicentro BETWEEN e.longitud - 1.5 AND e.longitud + 1.5
      WHERE ev.id_evento BETWEEN 200000000 AND 299999999
        AND ev.fecha_hora_origen >= TIMESTAMP '2000-01-01 00:00:00'
        AND ev.fecha_hora_origen < TIMESTAMP '2026-01-01 00:00:00'
        AND ev.magnitud >= 5
      GROUP BY e.id_estacion, e.codigo, e.nombre
      HAVING COUNT(*) >= 1
      ORDER BY eventos_en_ventana DESC, e.id_estacion
      LIMIT 10
       ) AS resultado) AS filas_obtenidas
UNION ALL
SELECT 3 AS numero, 'Calidad de lecturas por estacion'::TEXT AS consulta,
       (SELECT COUNT(*) FROM (
      SELECT
          e.id_estacion,
          e.codigo AS codigo_estacion,
          COUNT(DISTINCT s.id_sensor) AS sensores_con_lecturas,
          COUNT(*) AS total_lecturas,
          COUNT(*) FILTER (WHERE l.es_valida) AS lecturas_validas,
          COUNT(*) FILTER (WHERE NOT l.es_valida) AS lecturas_invalidas,
          ROUND(100.0 * COUNT(*) FILTER (WHERE NOT l.es_valida)
                / NULLIF(COUNT(*), 0), 2) AS porcentaje_invalidas
      FROM estacion_sismica e
      JOIN sensor s ON s.id_estacion = e.id_estacion
      JOIN lectura_onda l ON l.id_sensor = s.id_sensor
      WHERE l.fecha_hora >= TIMESTAMP '2026-08-01 00:00:00'
        AND l.fecha_hora < TIMESTAMP '2026-12-01 00:00:00'
      GROUP BY e.id_estacion, e.codigo
      HAVING COUNT(*) >= 10
      ORDER BY porcentaje_invalidas DESC, total_lecturas DESC, e.id_estacion
       ) AS resultado) AS filas_obtenidas
UNION ALL
```

```sql
SELECT 4 AS numero, 'Latencia de deteccion y alerta'::TEXT AS consulta,
       (SELECT COUNT(*) FROM (
      WITH detecciones_resumen AS (
          SELECT id_evento, COUNT(*) AS total_detecciones,
                 MIN(hora_deteccion) AS primera_deteccion
          FROM deteccion
          GROUP BY id_evento
      ), alertas_resumen AS (
          SELECT id_evento, COUNT(*) AS total_alertas,
                 MIN(fecha_hora_emision) AS primera_alerta
          FROM alerta
          GROUP BY id_evento
      )
      SELECT
          ev.id_evento,
          ev.fecha_hora_origen,
          ev.magnitud,
          COALESCE(d.total_detecciones, 0) AS total_detecciones,
          COALESCE(a.total_alertas, 0) AS total_alertas,
          ROUND(EXTRACT(EPOCH FROM (d.primera_deteccion - ev.fecha_hora_origen))::NUMERIC, 2)
              AS segundos_primera_deteccion,
          ROUND(EXTRACT(EPOCH FROM (a.primera_alerta - ev.fecha_hora_origen))::NUMERIC, 2)
              AS segundos_primera_alerta
      FROM evento_sismico ev
      LEFT JOIN detecciones_resumen d ON d.id_evento = ev.id_evento
      LEFT JOIN alertas_resumen a ON a.id_evento = ev.id_evento
      WHERE (ev.id_evento BETWEEN 810000001 AND 810000008
          OR ev.id_evento BETWEEN 820300001 AND 820300030)
        AND (COALESCE(d.total_detecciones, 0) > 0
          OR COALESCE(a.total_alertas, 0) > 0)
      ORDER BY ev.fecha_hora_origen DESC, ev.id_evento
      LIMIT 15
       ) AS resultado) AS filas_obtenidas
UNION ALL
SELECT 5 AS numero, 'Sensores sin deteccion en agosto'::TEXT AS consulta,
       (SELECT COUNT(*) FROM (
      SELECT
          s.id_sensor,
          s.numero_serie,
          e.codigo AS codigo_estacion,
          s.estado AS estado_sensor,
          COUNT(*) AS lecturas_periodo,
          COUNT(*) FILTER (WHERE NOT l.es_valida) AS lecturas_invalidas,
          MAX(l.fecha_hora) AS ultima_lectura
      FROM sensor s
      JOIN estacion_sismica e ON e.id_estacion = s.id_estacion
      JOIN lectura_onda l ON l.id_sensor = s.id_sensor
      WHERE l.fecha_hora >= TIMESTAMP '2026-08-01 00:00:00'
        AND l.fecha_hora < TIMESTAMP '2026-09-01 00:00:00'
        AND NOT EXISTS (
            SELECT 1
            FROM deteccion d
            WHERE d.id_sensor = s.id_sensor
              AND d.hora_deteccion >= TIMESTAMP '2026-08-01 00:00:00'
              AND d.hora_deteccion < TIMESTAMP '2026-09-01 00:00:00'
        )
      GROUP BY s.id_sensor, s.numero_serie, e.codigo, s.estado
      HAVING COUNT(*) >= 20
      ORDER BY lecturas_invalidas DESC, lecturas_periodo DESC, s.id_sensor
      LIMIT 15
       ) AS resultado) AS filas_obtenidas;

SELECT numero, consulta, filas_obtenidas,
       CASE WHEN filas_obtenidas > 0 THEN 'OK' ELSE 'REVISAR' END AS resultado
FROM control_consultas_51
ORDER BY numero;

SELECT tabla, encontrados, esperado,
       CASE WHEN encontrados = esperado THEN 'OK' ELSE 'REVISAR' END AS resultado
FROM (
    SELECT 'estacion_sismica'::TEXT AS tabla, (SELECT COUNT(*) FROM estacion_sismica) AS encontrados, 15::BIGINT AS
     esperado
    UNION ALL SELECT 'sensor', (SELECT COUNT(*) FROM sensor), 40
    UNION ALL SELECT 'lectura_onda', (SELECT COUNT(*) FROM lectura_onda), 1190
    UNION ALL SELECT 'evento_sismico', (SELECT COUNT(*) FROM evento_sismico), 24327
    UNION ALL SELECT 'deteccion', (SELECT COUNT(*) FROM deteccion), 104
    UNION ALL SELECT 'alerta', (SELECT COUNT(*) FROM alerta), 29
```

```
) AS cantidades
ORDER BY tabla;

DO $$
DECLARE
    numero_consultas INTEGER;
    consultas_vacias INTEGER;
    registros_base BIGINT;
    eventos_igp BIGINT;
BEGIN
    SELECT COUNT(*), COUNT(*) FILTER (WHERE filas_obtenidas = 0)
      INTO numero_consultas, consultas_vacias
      FROM control_consultas_51;

    SELECT (SELECT COUNT(*) FROM estacion_sismica)
         + (SELECT COUNT(*) FROM sensor)
         + (SELECT COUNT(*) FROM lectura_onda)
         + (SELECT COUNT(*) FROM evento_sismico)
         + (SELECT COUNT(*) FROM deteccion)
         + (SELECT COUNT(*) FROM alerta)
      INTO registros_base;

    SELECT COUNT(*) INTO eventos_igp
    FROM evento_sismico
    WHERE id_evento BETWEEN 200000000 AND 299999999;

    IF numero_consultas <> 5 OR consultas_vacias <> 0
       OR registros_base <> 25705 OR eventos_igp <> 24289 THEN
        RAISE EXCEPTION 'Fallo en verificacion 5.1: consultas=%, vacias=%, registros=%, IGP=%',
            numero_consultas, consultas_vacias, registros_base, eventos_igp;
    END IF;

    RAISE NOTICE 'OK: 5 consultas devolvieron filas; 25705 registros y 24289 eventos IGP conservados.';
END;
$$;

ROLLBACK;

\echo '=== COMPROBACION FINAL ==='
DO $$
DECLARE
    v_vistas INTEGER;
    v_procedimientos INTEGER;
    v_triggers INTEGER;
    v_registros BIGINT;
BEGIN
    SELECT COUNT(*) INTO v_vistas
    FROM pg_views
    WHERE schemaname = 'monitoreo_sismologico'
      AND viewname IN (
        'vw_sensores_actividad', 'vw_estaciones_resumen',
        'vw_eventos_seguimiento', 'vw_alertas_detalle', 'vw_igp_resumen_mensual'
      );

    SELECT COUNT(*) INTO v_procedimientos
    FROM pg_proc p
    JOIN pg_namespace n ON n.oid = p.pronamespace
    WHERE n.nspname = 'monitoreo_sismologico' AND p.prokind = 'p'
      AND p.proname IN (
        'sp_registrar_estacion_sensor', 'sp_registrar_evento',
        'sp_registrar_lectura', 'sp_registrar_deteccion',
        'sp_emitir_alerta', 'sp_actualizar_estado_sensor'
      );

    SELECT COUNT(*) INTO v_triggers
    FROM pg_trigger t
    JOIN pg_class c ON c.oid = t.tgrelid
    JOIN pg_namespace n ON n.oid = c.relnamespace
    WHERE n.nspname = 'monitoreo_sismologico'
      AND NOT t.tgisinternal
      AND t.tgname IN (
        'trg_lectura_operativa', 'trg_proteger_sustento_lectura',
        'trg_deteccion_valida', 'trg_alerta_temporal',
        'trg_revisar_origen_evento', 'trg_bloquear_estacion_origen',
        'ct_estacion_sensor_alta', 'ct_estacion_sensor_baja',
        'ct_estacion_sensor_traslado'
      );
```

```
    SELECT SUM(registros) INTO v_registros
    FROM (
        SELECT COUNT(*) AS registros FROM monitoreo_sismologico.estacion_sismica
        UNION ALL SELECT COUNT(*) FROM monitoreo_sismologico.sensor
        UNION ALL SELECT COUNT(*) FROM monitoreo_sismologico.lectura_onda
        UNION ALL SELECT COUNT(*) FROM monitoreo_sismologico.evento_sismico
        UNION ALL SELECT COUNT(*) FROM monitoreo_sismologico.deteccion
        UNION ALL SELECT COUNT(*) FROM monitoreo_sismologico.alerta
    ) AS conteos;

    IF ROW(v_vistas, v_procedimientos, v_triggers, v_registros)
       IS DISTINCT FROM ROW(5, 6, 9, 25705::BIGINT) THEN
        RAISE EXCEPTION 'Control final incorrecto: vistas %, procedimientos %, triggers %, registros %',
            v_vistas, v_procedimientos, v_triggers, v_registros;
    END IF;
    RAISE NOTICE 'OK: 5 vistas, 6 procedimientos, 9 triggers y 25705 registros.';
END;
$$;
\echo '=== ETAPA 3 FINALIZADA ==='
```

Listing 3: Integrador 03: lógica de negocio y consultas C1–C5.

7.6.4. Bases experimentales

```
-- 04. BASES EXPERIMENTALES
\set ON_ERROR_STOP on
\pset pager off

\if :{?escala}
\else
\set escala 1000
\endif

\if :{?originales}
\else
\set originales false
\endif

\if :originales
\set prefijo bd_sismologico_exp_
\else
\set prefijo bd_sismologico_prueba_exp_
\endif

\echo '=== CONFIGURACION ==='
SELECT 1 / CASE WHEN :'escala' IN ('1000','10000','100000','1000000') THEN 1 ELSE 0 END AS escala_valida;
SELECT :'prefijo' || :'escala' AS base_destino \gset
SELECT :'base_destino' AS base_destino, :'escala'::INTEGER AS registros_objetivo;

\echo '=== CREACION DE BASE ==='
\connect postgres
SELECT CASE
    WHEN EXISTS (SELECT 1 FROM pg_database WHERE datname = :'base_destino') THEN
        format('DO $abort$ BEGIN RAISE EXCEPTION ''La base %s ya existe. No se modifico.''; END $abort$;', :'
     base_destino')
    ELSE format('CREATE DATABASE %I', :'base_destino')
END
\gexec
\connect :base_destino
\echo '=== GENERACION DE DATOS ==='
DO $$
BEGIN
    IF current_database() !~ '^bd_sismologico_(prueba_)?exp_(1000|10000|100000|1000000)$' THEN
        RAISE EXCEPTION 'Base incorrecta: %. No se modifica la base principal.', current_database();
    END IF;
    IF EXISTS (
        SELECT 1 FROM information_schema.tables
        WHERE table_schema = 'monitoreo_sismologico'
          AND table_name = 'evento_sismico'
    ) THEN
        RAISE EXCEPTION 'La base % contiene tablas y no admite una nueva generacion.', current_database();
    END IF;
```

```
END;
$$;

\echo '=== ESTRUCTURA ==='
CREATE SCHEMA IF NOT EXISTS monitoreo_sismologico;
SET search_path TO monitoreo_sismologico, public;

CREATE TABLE estacion_sismica (
    id_estacion INTEGER GENERATED BY DEFAULT AS IDENTITY,
    codigo VARCHAR(50) NOT NULL,
    nombre VARCHAR(150) NOT NULL,
    latitud NUMERIC NOT NULL,
    longitud NUMERIC NOT NULL,
    altitud NUMERIC,
    departamento VARCHAR(100),
    provincia VARCHAR(100),
    distrito VARCHAR(100),
    fecha_instalacion DATE,
    estado VARCHAR(40) NOT NULL
);

CREATE TABLE sensor (
    id_sensor INTEGER GENERATED BY DEFAULT AS IDENTITY,
    numero_serie VARCHAR(80) NOT NULL,
    tipo VARCHAR(80) NOT NULL,
    modelo VARCHAR(100),
    unidad_medida VARCHAR(40) NOT NULL,
    frecuencia_muestreo NUMERIC NOT NULL,
    fecha_instalacion DATE NOT NULL,
    fecha_ultimo_mantenimiento DATE,
    estado VARCHAR(40) NOT NULL,
    id_estacion INTEGER NOT NULL
);

CREATE TABLE lectura_onda (
    id_lectura BIGINT GENERATED BY DEFAULT AS IDENTITY,
    id_sensor INTEGER NOT NULL,
    fecha_hora TIMESTAMP NOT NULL,
    canal VARCHAR(40) NOT NULL,
    valor NUMERIC NOT NULL,
    es_valida BOOLEAN NOT NULL
);

CREATE TABLE evento_sismico (
    id_evento INTEGER GENERATED BY DEFAULT AS IDENTITY,
    fecha_hora_origen TIMESTAMP NOT NULL,
    latitud_epicentro NUMERIC NOT NULL,
    longitud_epicentro NUMERIC NOT NULL,
    profundidad_km NUMERIC NOT NULL,
    magnitud NUMERIC NOT NULL,
    tipo_magnitud VARCHAR(30),
    intensidad_maxima VARCHAR(40),
    referencia_geografica VARCHAR(250),
    estado_revision VARCHAR(40) NOT NULL
);

CREATE TABLE deteccion (
    id_sensor INTEGER NOT NULL,
    id_evento INTEGER NOT NULL,
    hora_deteccion TIMESTAMP NOT NULL,
    aceleracion_maxima NUMERIC,
    calidad VARCHAR(40)
);

CREATE TABLE alerta (
    id_alerta INTEGER GENERATED BY DEFAULT AS IDENTITY,
    id_evento INTEGER NOT NULL,
    fecha_hora_emision TIMESTAMP NOT NULL,
    nivel VARCHAR(40) NOT NULL,
    mensaje TEXT NOT NULL,
    magnitud_estimada NUMERIC,
    estado VARCHAR(40) NOT NULL
);

ALTER TABLE estacion_sismica
    ADD CONSTRAINT pk_estacion_sismica PRIMARY KEY (id_estacion);
```

```sql
ALTER TABLE sensor
    ADD CONSTRAINT pk_sensor PRIMARY KEY (id_sensor);

ALTER TABLE lectura_onda
    ADD CONSTRAINT pk_lectura_onda PRIMARY KEY (id_lectura);

ALTER TABLE evento_sismico
    ADD CONSTRAINT pk_evento_sismico PRIMARY KEY (id_evento);

ALTER TABLE deteccion
    ADD CONSTRAINT pk_deteccion PRIMARY KEY (id_sensor, id_evento);

ALTER TABLE alerta
    ADD CONSTRAINT pk_alerta PRIMARY KEY (id_alerta);

ALTER TABLE sensor
    ADD CONSTRAINT fk_sensor_estacion
    FOREIGN KEY (id_estacion) REFERENCES estacion_sismica(id_estacion);

ALTER TABLE lectura_onda
    ADD CONSTRAINT fk_lectura_sensor
    FOREIGN KEY (id_sensor) REFERENCES sensor(id_sensor);

ALTER TABLE deteccion
    ADD CONSTRAINT fk_deteccion_sensor
    FOREIGN KEY (id_sensor) REFERENCES sensor(id_sensor),
    ADD CONSTRAINT fk_deteccion_evento
    FOREIGN KEY (id_evento) REFERENCES evento_sismico(id_evento);

ALTER TABLE alerta
    ADD CONSTRAINT fk_alerta_evento
    FOREIGN KEY (id_evento) REFERENCES evento_sismico(id_evento);

ALTER TABLE estacion_sismica
    ADD CONSTRAINT uq_estacion_codigo UNIQUE (codigo);

ALTER TABLE sensor
    ADD CONSTRAINT uq_sensor_numero_serie UNIQUE (numero_serie);

ALTER TABLE lectura_onda
    ADD CONSTRAINT uq_lectura_sensor_fecha_canal
    UNIQUE (id_sensor, fecha_hora, canal);

ALTER TABLE estacion_sismica
    ADD CONSTRAINT ck_estacion_latitud CHECK (latitud BETWEEN -90 AND 90),
    ADD CONSTRAINT ck_estacion_longitud CHECK (longitud BETWEEN -180 AND 180);

ALTER TABLE sensor
    ADD CONSTRAINT ck_sensor_frecuencia CHECK (frecuencia_muestreo > 0),
    ADD CONSTRAINT ck_sensor_mantenimiento
    CHECK (fecha_ultimo_mantenimiento IS NULL
           OR fecha_ultimo_mantenimiento >= fecha_instalacion);

ALTER TABLE evento_sismico
    ADD CONSTRAINT ck_evento_latitud
    CHECK (latitud_epicentro BETWEEN -90 AND 90),
    ADD CONSTRAINT ck_evento_longitud
    CHECK (longitud_epicentro BETWEEN -180 AND 180),
    ADD CONSTRAINT ck_evento_profundidad CHECK (profundidad_km >= 0);

ALTER TABLE deteccion
    ADD CONSTRAINT ck_deteccion_aceleracion
    CHECK (aceleracion_maxima IS NULL OR aceleracion_maxima >= 0);

BEGIN;
SET LOCAL search_path TO monitoreo_sismologico, public;

CREATE TEMP TABLE exp_parametros_52 ON COMMIT DROP AS
SELECT *
FROM (VALUES
    ('bd_sismologico_exp_1000',       1000,   5,  15,    100,     50,    750,    90,   40),
    ('bd_sismologico_exp_10000',     10000,  10,  30,   2000,    800,   7550,   300,  110),
    ('bd_sismologico_exp_100000',   100000,  25,  75,  24000,  12000,  74700,   900,  300),
    ('bd_sismologico_exp_1000000', 1000000, 100, 300, 120000,  24289, 875000,  3500, 1100)
) AS v(nombre, total, n_est, n_sen, n_evt, n_igp, n_lec, n_det, n_ale)
WHERE nombre = 'bd_sismologico_exp_' || regexp_replace(current_database(), '^.*_', '');
```

```
DO $$
DECLARE p RECORD;
BEGIN
    SELECT * INTO p FROM exp_parametros_52;
    IF NOT FOUND OR p.n_est+p.n_sen+p.n_evt+p.n_lec+p.n_det+p.n_ale <> p.total
       OR p.n_evt-p.n_igp < 40
       OR p.n_det > (p.n_sen/2) * 30
       OR ((p.n_det-1)/(p.n_sen/2))*p.n_sen
            + ((p.n_det-1)%(p.n_sen/2)) >= p.n_lec THEN
        RAISE EXCEPTION 'Configuracion de escala inconsistente para %', current_database();
    END IF;
    RAISE NOTICE 'Iniciando % registros en %', p.total, current_database();
END;
$$;

CREATE FUNCTION exp_id_estacion(p_g INTEGER) RETURNS INTEGER
LANGUAGE SQL IMMUTABLE AS $$
    SELECT CASE WHEN p_g < 5 THEN 810000001+p_g
                WHEN p_g < 15 THEN 820300001+p_g-5
                ELSE 830000001+p_g-15 END;
$$;
CREATE FUNCTION exp_id_sensor(p_g INTEGER) RETURNS INTEGER
LANGUAGE SQL IMMUTABLE AS $$
    SELECT CASE WHEN p_g < 10 THEN 810000001+p_g
                WHEN p_g < 40 THEN 820300001+p_g-10
                ELSE 830000001+p_g-40 END;
$$;
CREATE FUNCTION exp_id_evento(p_g INTEGER) RETURNS INTEGER
LANGUAGE SQL IMMUTABLE AS $$
    SELECT CASE WHEN p_g < 8 THEN 810000001+p_g
                WHEN p_g < 38 THEN 820300001+p_g-8
                ELSE 830000001+p_g-38 END;
$$;
CREATE FUNCTION exp_origen_evento(p_g INTEGER) RETURNS TIMESTAMP
LANGUAGE SQL IMMUTABLE AS $$
    SELECT TIMESTAMP '2026-08-01 00:00:00'
         + (p_g % 30) * INTERVAL '1 day'
         + (p_g / 30) * INTERVAL '1 second';
$$;
CREATE FUNCTION exp_hora_lectura(p_g INTEGER, p_sensores INTEGER) RETURNS TIMESTAMP
LANGUAGE SQL IMMUTABLE AS $$
    SELECT TIMESTAMP '2026-08-01 00:00:00'
         + ((p_g / p_sensores) % 30) * INTERVAL '1 day'
         + (p_g % p_sensores) * INTERVAL '1 second'
         + (p_g / (p_sensores * 30)) * INTERVAL '10 minutes';
$$;

CREATE TEMP TABLE exp_igp_fuente_52 (
    id TEXT, fecha_utc TEXT, hora_utc TEXT,
    latitud TEXT, longitud TEXT, profundidad TEXT,
    magnitud TEXT, fecha_corte TEXT
) ON COMMIT DROP;

\copy exp_igp_fuente_52 (id, fecha_utc, hora_utc, latitud, longitud, profundidad, magnitud, fecha_corte) FROM 'datos/
     igp_catalogo.csv' WITH (FORMAT csv, HEADER true, DELIMITER ';', ENCODING 'UTF8')

DO $$
BEGIN
    IF (SELECT COUNT(*) FROM exp_igp_fuente_52) <> 24289 THEN
        RAISE EXCEPTION 'El CSV oficial debe contener 24289 filas; se cancela esta version del experimento.';
    END IF;
END;
$$;

WITH fuente AS (
    SELECT
        BTRIM(id)::INTEGER AS id_fuente,
        MAKE_TIMESTAMP(
            SUBSTRING(BTRIM(fecha_utc) FROM 1 FOR 4)::INTEGER,
            SUBSTRING(BTRIM(fecha_utc) FROM 5 FOR 2)::INTEGER,
            SUBSTRING(BTRIM(fecha_utc) FROM 7 FOR 2)::INTEGER,
            SUBSTRING(LPAD(BTRIM(hora_utc), 6, '0') FROM 1 FOR 2)::INTEGER,
            SUBSTRING(LPAD(BTRIM(hora_utc), 6, '0') FROM 3 FOR 2)::INTEGER,
            SUBSTRING(LPAD(BTRIM(hora_utc), 6, '0') FROM 5 FOR 2)::INTEGER
        ) AS fecha,
        BTRIM(latitud)::NUMERIC AS lat,
        BTRIM(longitud)::NUMERIC AS lon,
```

```
        BTRIM(profundidad)::NUMERIC AS prof,
        BTRIM(magnitud)::NUMERIC AS mag
    FROM exp_igp_fuente_52
), muestra AS (
    SELECT fuente.*,
       ROW_NUMBER() OVER (
         ORDER BY CASE
             WHEN fecha >= '2010-01-01' AND fecha < '2026-01-01' AND mag >= 6 THEN 0
             WHEN fecha >= '2000-01-01' AND fecha < '2026-01-01' AND mag >= 5 THEN 1
             ELSE 2
         END, md5(id_fuente::TEXT)
       ) AS orden
    FROM fuente
)
INSERT INTO evento_sismico (
    id_evento, fecha_hora_origen, latitud_epicentro,
    longitud_epicentro, profundidad_km, magnitud,
    tipo_magnitud, intensidad_maxima, referencia_geografica, estado_revision
)
SELECT 200000000 + i.id_fuente, i.fecha, i.lat, i.lon,
       i.prof, i.mag, NULL, NULL, NULL, 'pendiente_revision'
FROM muestra i CROSS JOIN exp_parametros_52 p
WHERE i.orden <= p.n_igp;

INSERT INTO estacion_sismica (
    id_estacion, codigo, nombre, latitud, longitud, altitud,
    departamento, provincia, distrito, fecha_instalacion, estado
)
SELECT
    exp_id_estacion(g),
    'EXP52-EST-' || LPAD((g+1)::TEXT, 4, '0'),
    'Estacion experimental simulada ' || (g+1),
    CASE WHEN g=0 THEN (
        SELECT latitud_epicentro FROM evento_sismico
        WHERE id_evento BETWEEN 200000000 AND 299999999
        ORDER BY magnitud DESC, id_evento LIMIT 1
    ) ELSE (-5.5 - g*0.11)::NUMERIC END,
    CASE WHEN g=0 THEN (
        SELECT longitud_epicentro FROM evento_sismico
        WHERE id_evento BETWEEN 200000000 AND 299999999
        ORDER BY magnitud DESC, id_evento LIMIT 1
    ) ELSE (-78.4 + g*0.03)::NUMERIC END,
    100 + g*15, NULL, NULL, NULL, DATE '2024-01-01', 'operativa'
FROM exp_parametros_52 p CROSS JOIN LATERAL generate_series(0,p.n_est-1) AS gs(g);

INSERT INTO sensor (
    id_sensor, numero_serie, tipo, modelo, unidad_medida,
    frecuencia_muestreo, fecha_instalacion, fecha_ultimo_mantenimiento,
    estado, id_estacion
)
SELECT exp_id_sensor(g),
       'EXP52-SEN-' || LPAD((g+1)::TEXT, 4, '0'),
       'acelerometro', 'SIM-52', 'm/s2',
       CASE WHEN g%2=0 THEN 50 ELSE 100 END,
       DATE '2025-01-01', NULL, 'operativo', exp_id_estacion(g/3)
FROM exp_parametros_52 p CROSS JOIN LATERAL generate_series(0,p.n_sen-1) AS gs(g);

INSERT INTO evento_sismico (
    id_evento, fecha_hora_origen, latitud_epicentro, longitud_epicentro,
    profundidad_km, magnitud, tipo_magnitud,
    intensidad_maxima, referencia_geografica, estado_revision
)
SELECT exp_id_evento(g), exp_origen_evento(g),
       (-8-(g%120)*0.07)::NUMERIC, (-77+(g%100)*0.04)::NUMERIC,
       5+(g%500), (3.4+(g%40)*0.1)::NUMERIC,
       NULL, NULL, NULL, 'simulado'
FROM exp_parametros_52 p
CROSS JOIN LATERAL generate_series(0,p.n_evt-p.n_igp-1) AS gs(g);

INSERT INTO lectura_onda (
    id_lectura, id_sensor, fecha_hora, canal, valor, es_valida
)
SELECT 900000000001::BIGINT + g,
       exp_id_sensor(g % p.n_sen), exp_hora_lectura(g, p.n_sen),
       CASE g%3 WHEN 0 THEN 'X' WHEN 1 THEN 'Y' ELSE 'Z' END,
       (((g*37::BIGINT)%401)-200)::NUMERIC / 1000,
       (g % p.n_sen < p.n_sen/2 OR g % 17 <> 0)
```

```
FROM exp_parametros_52 p
CROSS JOIN LATERAL generate_series(0,p.n_lec-1) AS gs(g);

INSERT INTO deteccion (
    id_sensor, id_evento, hora_deteccion, aceleracion_maxima, calidad
)
SELECT exp_id_sensor(g % (p.n_sen/2)),
       exp_id_evento((g / (p.n_sen/2)) % 30),
       exp_hora_lectura(
           (g/(p.n_sen/2))*p.n_sen + (g%(p.n_sen/2)), p.n_sen),
       ((g%80)+1)::NUMERIC / 100, 'alta'
FROM exp_parametros_52 p
CROSS JOIN LATERAL generate_series(0,p.n_det-1) AS gs(g);

INSERT INTO alerta (
    id_alerta, id_evento, fecha_hora_emision,
    nivel, mensaje, magnitud_estimada, estado
)
SELECT CASE WHEN g<5 THEN 810000001+g
            WHEN g<29 THEN 820300001+g-5
            ELSE 830000001+g-29 END,
       exp_id_evento(g), exp_origen_evento(g) + INTERVAL '90 seconds',
       CASE WHEN g%2=0 THEN 'preventiva' ELSE 'informativa' END,
       'Alerta experimental para evento simulado ' || exp_id_evento(g),
       3.4+(g%40)*0.1, 'simulada'
FROM exp_parametros_52 p
CROSS JOIN LATERAL generate_series(0,p.n_ale-1) AS gs(g);

DO $$
DECLARE p RECORD; v_total BIGINT;
BEGIN
    SELECT * INTO p FROM exp_parametros_52;
    SELECT (SELECT COUNT(*) FROM estacion_sismica)
         + (SELECT COUNT(*) FROM sensor)
         + (SELECT COUNT(*) FROM evento_sismico)
         + (SELECT COUNT(*) FROM lectura_onda)
         + (SELECT COUNT(*) FROM deteccion)
         + (SELECT COUNT(*) FROM alerta)
    INTO v_total;
    IF v_total <> p.total OR
       (SELECT COUNT(*) FROM evento_sismico WHERE id_evento BETWEEN 200000000 AND 299999999) <> p.n_igp OR
       (SELECT COUNT(*) FROM evento_sismico WHERE estado_revision='simulado') <> p.n_evt-p.n_igp OR
       EXISTS (SELECT 1 FROM deteccion WHERE id_evento BETWEEN 200000000 AND 299999999) OR
       EXISTS (SELECT 1 FROM alerta WHERE id_evento BETWEEN 200000000 AND 299999999) OR
       EXISTS (
          SELECT 1 FROM deteccion d
          WHERE NOT EXISTS (
              SELECT 1 FROM lectura_onda l
              WHERE l.id_sensor=d.id_sensor AND l.fecha_hora=d.hora_deteccion AND l.es_valida
          )
       ) OR
       EXISTS (
          SELECT 1 FROM estacion_sismica e
          WHERE NOT EXISTS (SELECT 1 FROM sensor s WHERE s.id_estacion=e.id_estacion)
       ) THEN
       RAISE EXCEPTION 'Fallaron controles de integridad para la escala %', p.total;
    END IF;
    RAISE NOTICE 'OK: % registros generados; IGP=% y simulados=%; integridad compatible.',
       p.total, p.n_igp, p.n_evt-p.n_igp;
END;
$$;

SELECT 'estacion_sismica' AS tabla, COUNT(*) AS filas FROM estacion_sismica
UNION ALL SELECT 'sensor',COUNT(*) FROM sensor
UNION ALL SELECT 'evento_sismico',COUNT(*) FROM evento_sismico
UNION ALL SELECT 'lectura_onda',COUNT(*) FROM lectura_onda
UNION ALL SELECT 'deteccion',COUNT(*) FROM deteccion
UNION ALL SELECT 'alerta',COUNT(*) FROM alerta
ORDER BY tabla;
COMMIT;

\echo '=== REGLAS AUTOMATICAS ==='
BEGIN;
SET LOCAL search_path TO monitoreo_sismologico, public;

CREATE OR REPLACE FUNCTION fn_trg_lectura_operativa()
RETURNS TRIGGER
```

```
LANGUAGE plpgsql
SET search_path = monitoreo_sismologico, pg_temp
AS $$
DECLARE
    v_estado TEXT;
BEGIN
    IF NEW.es_valida IS TRUE THEN
        SELECT LOWER(BTRIM(estado)) INTO v_estado
        FROM sensor
        WHERE id_sensor = NEW.id_sensor
        FOR SHARE;

        IF NOT FOUND THEN
            RAISE EXCEPTION 'TRG: el sensor % no existe', NEW.id_sensor;
        END IF;

        IF v_estado IS DISTINCT FROM 'operativo' THEN
            RAISE EXCEPTION 'TRG: el sensor % no esta operativo para nuevas lecturas validas', NEW.id_sensor;
        END IF;
    END IF;
    RETURN NEW;
END;
$$;

CREATE OR REPLACE TRIGGER trg_lectura_operativa
BEFORE INSERT OR UPDATE OF id_sensor, fecha_hora, es_valida
ON lectura_onda
FOR EACH ROW
EXECUTE FUNCTION fn_trg_lectura_operativa();

CREATE OR REPLACE FUNCTION fn_trg_proteger_sustento()
RETURNS TRIGGER
LANGUAGE plpgsql
SET search_path = monitoreo_sismologico, pg_temp
AS $$
BEGIN
    IF OLD.es_valida IS NOT TRUE THEN
        IF TG_OP = 'DELETE' THEN RETURN OLD; END IF;
        RETURN NEW;
    END IF;

    IF TG_OP = 'UPDATE' THEN
        IF NEW.es_valida IS TRUE
           AND NEW.id_sensor = OLD.id_sensor
           AND NEW.fecha_hora = OLD.fecha_hora THEN
            RETURN NEW;
        END IF;
    END IF;

    PERFORM 1
    FROM deteccion
    WHERE id_sensor = OLD.id_sensor
      AND hora_deteccion = OLD.fecha_hora
    FOR SHARE;

    IF FOUND AND NOT EXISTS (
        SELECT 1
        FROM lectura_onda
        WHERE id_sensor = OLD.id_sensor
          AND fecha_hora = OLD.fecha_hora
          AND es_valida IS TRUE
          AND id_lectura <> OLD.id_lectura
    ) THEN
        RAISE EXCEPTION 'TRG: la lectura % es el ultimo sustento valido de una deteccion', OLD.id_lectura;
    END IF;

    IF TG_OP = 'DELETE' THEN RETURN OLD; END IF;
    RETURN NEW;
END;
$$;

CREATE OR REPLACE TRIGGER trg_proteger_sustento_lectura
BEFORE UPDATE OF id_sensor, fecha_hora, es_valida OR DELETE
ON lectura_onda
FOR EACH ROW
EXECUTE FUNCTION fn_trg_proteger_sustento();
```

```
CREATE OR REPLACE FUNCTION fn_trg_deteccion_valida()
RETURNS TRIGGER
LANGUAGE plpgsql
SET search_path = monitoreo_sismologico, pg_temp
AS $$
DECLARE
    v_origen TIMESTAMP;
BEGIN
    SELECT fecha_hora_origen INTO v_origen
    FROM evento_sismico
    WHERE id_evento = NEW.id_evento
    FOR SHARE;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'TRG: el evento % no existe', NEW.id_evento;
    END IF;
    IF NEW.hora_deteccion < v_origen THEN
        RAISE EXCEPTION 'TRG: deteccion anterior al origen del evento %', NEW.id_evento;
    END IF;

    PERFORM 1
    FROM lectura_onda
    WHERE id_sensor = NEW.id_sensor
      AND fecha_hora = NEW.hora_deteccion
      AND es_valida IS TRUE
    FOR SHARE;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'TRG: deteccion sin lectura valida del sensor % en la hora indicada', NEW.id_sensor;
    END IF;

    RETURN NEW;
END;
$$;

CREATE OR REPLACE TRIGGER trg_deteccion_valida
BEFORE INSERT OR UPDATE OF id_sensor, id_evento, hora_deteccion
ON deteccion
FOR EACH ROW
EXECUTE FUNCTION fn_trg_deteccion_valida();

CREATE OR REPLACE FUNCTION fn_trg_alerta_temporal()
RETURNS TRIGGER
LANGUAGE plpgsql
SET search_path = monitoreo_sismologico, pg_temp
AS $$
DECLARE
    v_origen TIMESTAMP;
BEGIN
    SELECT fecha_hora_origen INTO v_origen
    FROM evento_sismico
    WHERE id_evento = NEW.id_evento
    FOR SHARE;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'TRG: el evento % no existe', NEW.id_evento;
    END IF;
    IF NEW.fecha_hora_emision < v_origen THEN
        RAISE EXCEPTION 'TRG: alerta anterior al origen del evento %', NEW.id_evento;
    END IF;
    RETURN NEW;
END;
$$;

CREATE OR REPLACE TRIGGER trg_alerta_temporal
BEFORE INSERT OR UPDATE OF id_evento, fecha_hora_emision
ON alerta
FOR EACH ROW
EXECUTE FUNCTION fn_trg_alerta_temporal();

CREATE OR REPLACE FUNCTION fn_trg_revisar_origen_evento()
RETURNS TRIGGER
LANGUAGE plpgsql
SET search_path = monitoreo_sismologico, pg_temp
AS $$
BEGIN
    IF NEW.fecha_hora_origen IS DISTINCT FROM OLD.fecha_hora_origen THEN
```

```
        IF EXISTS (
            SELECT 1 FROM deteccion
            WHERE id_evento = OLD.id_evento
              AND hora_deteccion < NEW.fecha_hora_origen
        ) OR EXISTS (
            SELECT 1 FROM alerta
            WHERE id_evento = OLD.id_evento
              AND fecha_hora_emision < NEW.fecha_hora_origen
        ) THEN
            RAISE EXCEPTION 'TRG: el nuevo origen del evento % invalida detecciones o alertas existentes', OLD.
     id_evento;
        END IF;
    END IF;
    RETURN NEW;
END;
$$;

CREATE OR REPLACE TRIGGER trg_revisar_origen_evento
BEFORE UPDATE OF fecha_hora_origen
ON evento_sismico
FOR EACH ROW
EXECUTE FUNCTION fn_trg_revisar_origen_evento();

CREATE OR REPLACE FUNCTION fn_trg_bloquear_estacion_origen()
RETURNS TRIGGER
LANGUAGE plpgsql
SET search_path = monitoreo_sismologico, pg_temp
AS $$
BEGIN
    IF TG_OP = 'DELETE' THEN
        PERFORM 1 FROM estacion_sismica
        WHERE id_estacion = OLD.id_estacion
        FOR UPDATE;
        RETURN OLD;
    END IF;
    IF OLD.id_estacion IS DISTINCT FROM NEW.id_estacion THEN
        PERFORM 1 FROM estacion_sismica
        WHERE id_estacion = OLD.id_estacion
        FOR UPDATE;
    END IF;
    RETURN NEW;
END;
$$;

CREATE OR REPLACE TRIGGER trg_bloquear_estacion_origen
BEFORE UPDATE OF id_estacion OR DELETE
ON sensor
FOR EACH ROW
EXECUTE FUNCTION fn_trg_bloquear_estacion_origen();

CREATE OR REPLACE FUNCTION fn_trg_estacion_con_sensor()
RETURNS TRIGGER
LANGUAGE plpgsql
SET search_path = monitoreo_sismologico, pg_temp
AS $$
DECLARE
    v_id_estacion INTEGER;
BEGIN
    IF TG_TABLE_NAME = 'estacion_sismica' THEN
        v_id_estacion := NEW.id_estacion;
    ELSE
        v_id_estacion := OLD.id_estacion;
        IF TG_OP = 'UPDATE' THEN
            IF OLD.id_estacion IS NOT DISTINCT FROM NEW.id_estacion THEN
                RETURN NULL;
            END IF;
        END IF;
    END IF;

    IF EXISTS (
        SELECT 1 FROM estacion_sismica WHERE id_estacion = v_id_estacion
    ) AND NOT EXISTS (
        SELECT 1 FROM sensor WHERE id_estacion = v_id_estacion
    ) THEN
        RAISE EXCEPTION 'TRG: la estacion % debe conservar al menos un sensor', v_id_estacion;
    END IF;
    RETURN NULL;
```

```
END;
$$;

DROP TRIGGER IF EXISTS ct_estacion_sensor_alta ON estacion_sismica;
CREATE CONSTRAINT TRIGGER ct_estacion_sensor_alta
AFTER INSERT ON estacion_sismica
DEFERRABLE INITIALLY DEFERRED
FOR EACH ROW
EXECUTE FUNCTION fn_trg_estacion_con_sensor();

DROP TRIGGER IF EXISTS ct_estacion_sensor_baja ON sensor;
CREATE CONSTRAINT TRIGGER ct_estacion_sensor_baja
AFTER DELETE ON sensor
DEFERRABLE INITIALLY DEFERRED
FOR EACH ROW
EXECUTE FUNCTION fn_trg_estacion_con_sensor();

DROP TRIGGER IF EXISTS ct_estacion_sensor_traslado ON sensor;
CREATE CONSTRAINT TRIGGER ct_estacion_sensor_traslado
AFTER UPDATE OF id_estacion ON sensor
DEFERRABLE INITIALLY DEFERRED
FOR EACH ROW
EXECUTE FUNCTION fn_trg_estacion_con_sensor();

DO $$
BEGIN
    IF EXISTS (
        SELECT 1 FROM estacion_sismica e
        WHERE NOT EXISTS (SELECT 1 FROM sensor s WHERE s.id_estacion = e.id_estacion)
    ) OR EXISTS (
        SELECT 1 FROM deteccion d
        JOIN evento_sismico e USING (id_evento)
        WHERE d.hora_deteccion < e.fecha_hora_origen
           OR NOT EXISTS (
                SELECT 1 FROM lectura_onda l
                WHERE l.id_sensor = d.id_sensor
                  AND l.fecha_hora = d.hora_deteccion
                  AND l.es_valida IS TRUE
           )
    ) OR EXISTS (
        SELECT 1 FROM alerta a
        JOIN evento_sismico e USING (id_evento)
        WHERE a.fecha_hora_emision < e.fecha_hora_origen
    ) THEN
        RAISE EXCEPTION 'Existen datos previos incompatibles con las reglas de los triggers';
    END IF;

    IF (
        SELECT COUNT(*)
        FROM pg_trigger t
        JOIN pg_class c ON c.oid = t.tgrelid
        JOIN pg_namespace n ON n.oid = c.relnamespace
        WHERE n.nspname = 'monitoreo_sismologico'
          AND NOT t.tgisinternal
          AND t.tgname IN (
              'trg_lectura_operativa', 'trg_proteger_sustento_lectura',
              'trg_deteccion_valida', 'trg_alerta_temporal',
              'trg_revisar_origen_evento', 'trg_bloquear_estacion_origen',
              'ct_estacion_sensor_alta', 'ct_estacion_sensor_baja',
              'ct_estacion_sensor_traslado'
          )
    ) <> 9 THEN
        RAISE EXCEPTION 'No se encontraron los 9 triggers previstos';
    END IF;

    RAISE NOTICE 'OK: 9 triggers instalados; datos previos compatibles.';
END;
$$;

SELECT c.relname AS tabla, t.tgname AS trigger,
       CASE WHEN t.tgdeferrable THEN 'DIFERIDO' ELSE 'INMEDIATO' END AS control
FROM pg_trigger t
JOIN pg_class c ON c.oid = t.tgrelid
JOIN pg_namespace n ON n.oid = c.relnamespace
WHERE n.nspname = 'monitoreo_sismologico'
  AND NOT t.tgisinternal
  AND t.tgname IN (
```

```
      'trg_lectura_operativa', 'trg_proteger_sustento_lectura',
      'trg_deteccion_valida', 'trg_alerta_temporal',
      'trg_revisar_origen_evento', 'trg_bloquear_estacion_origen',
      'ct_estacion_sensor_alta', 'ct_estacion_sensor_baja',
      'ct_estacion_sensor_traslado'
  )
ORDER BY tabla, trigger;

COMMIT;

ANALYZE monitoreo_sismologico.estacion_sismica;
ANALYZE monitoreo_sismologico.sensor;
ANALYZE monitoreo_sismologico.evento_sismico;
ANALYZE monitoreo_sismologico.lectura_onda;
ANALYZE monitoreo_sismologico.deteccion;
ANALYZE monitoreo_sismologico.alerta;

SELECT current_database() AS base_experimental,
       (SELECT COUNT(*) FROM monitoreo_sismologico.estacion_sismica)
     + (SELECT COUNT(*) FROM monitoreo_sismologico.sensor)
     + (SELECT COUNT(*) FROM monitoreo_sismologico.evento_sismico)
     + (SELECT COUNT(*) FROM monitoreo_sismologico.lectura_onda)
     + (SELECT COUNT(*) FROM monitoreo_sismologico.deteccion)
     + (SELECT COUNT(*) FROM monitoreo_sismologico.alerta) AS registros_totales;

\echo '=== VERIFICACION FINAL ==='
SET search_path TO monitoreo_sismologico, public;

WITH parametros AS (
SELECT * FROM (VALUES
    ('bd_sismologico_exp_1000',       1000,   5,  15,    100,     50,    750,    90,   40),
    ('bd_sismologico_exp_10000',     10000,  10,  30,   2000,    800,   7550,   300,  110),
    ('bd_sismologico_exp_100000',   100000,  25,  75,  24000,  12000,  74700,   900,  300),
    ('bd_sismologico_exp_1000000', 1000000, 100, 300, 120000,  24289, 875000, 3500, 1100)
) AS v(nombre,total,n_est,n_sen,n_evt,n_igp,n_lec,n_det,n_ale)
WHERE nombre = 'bd_sismologico_exp_' || regexp_replace(current_database(), '^.*_', '')
), cantidades AS (
SELECT 'estacion_sismica' AS tabla, n_est AS esperado,
       (SELECT COUNT(*) FROM estacion_sismica) AS encontrado FROM parametros
UNION ALL SELECT 'sensor',n_sen,(SELECT COUNT(*) FROM sensor) FROM parametros
UNION ALL SELECT 'evento_sismico',n_evt,(SELECT COUNT(*) FROM evento_sismico) FROM parametros
UNION ALL SELECT 'lectura_onda',n_lec,(SELECT COUNT(*) FROM lectura_onda) FROM parametros
UNION ALL SELECT 'deteccion',n_det,(SELECT COUNT(*) FROM deteccion) FROM parametros
UNION ALL SELECT 'alerta',n_ale,(SELECT COUNT(*) FROM alerta) FROM parametros
)
SELECT tabla, esperado, encontrado,
       CASE WHEN esperado=encontrado THEN 'OK' ELSE 'REVISAR' END AS resultado
FROM cantidades ORDER BY tabla;

SELECT (SELECT COUNT(*) FROM evento_sismico WHERE id_evento BETWEEN 200000000 AND 299999999) AS eventos_igp_reales,
       (SELECT COUNT(*) FROM evento_sismico WHERE estado_revision='simulado') AS eventos_ficticios,
       (SELECT COUNT(*) FROM deteccion WHERE id_evento BETWEEN 200000000 AND 299999999) AS detecciones_sobre_igp,
       (SELECT COUNT(*) FROM alerta WHERE id_evento BETWEEN 200000000 AND 299999999) AS alertas_sobre_igp;

SELECT 'estaciones_sin_sensor' AS control, COUNT(*) AS incumplimientos
FROM estacion_sismica e WHERE NOT EXISTS (SELECT 1 FROM sensor s WHERE s.id_estacion=e.id_estacion)
UNION ALL
SELECT 'detecciones_sin_lectura_valida',COUNT(*) FROM deteccion d
WHERE NOT EXISTS (SELECT 1 FROM lectura_onda l WHERE l.id_sensor=d.id_sensor AND l.fecha_hora=d.hora_deteccion AND l.
     es_valida)
UNION ALL
SELECT 'alertas_anteriores_al_evento',COUNT(*) FROM alerta a JOIN evento_sismico e USING(id_evento)
WHERE a.fecha_hora_emision < e.fecha_hora_origen
UNION ALL
SELECT 'lecturas_validas_sensor_inoperativo',COUNT(*) FROM lectura_onda l JOIN sensor s USING(id_sensor)
WHERE l.es_valida AND LOWER(TRIM(s.estado)) <> 'operativo';

SELECT COUNT(*) AS triggers_implementados
FROM pg_trigger t JOIN pg_class c ON c.oid=t.tgrelid
JOIN pg_namespace n ON n.oid=c.relnamespace
WHERE n.nspname='monitoreo_sismologico' AND NOT t.tgisinternal;

SELECT current_database() AS base_experimental,
       pg_size_pretty(pg_database_size(current_database())) AS espacio_ocupado;

DO $$
DECLARE v_total BIGINT; v_esperado BIGINT; v_igp INTEGER;
```

```
BEGIN
    SELECT total,n_igp INTO v_esperado,v_igp
    FROM (VALUES
      ('bd_sismologico_exp_1000',1000,50),
      ('bd_sismologico_exp_10000',10000,800),
      ('bd_sismologico_exp_100000',100000,12000),
      ('bd_sismologico_exp_1000000',1000000,24289)
    ) AS p(nombre,total,n_igp)
    WHERE nombre = 'bd_sismologico_exp_' || regexp_replace(current_database(), '^.*_', '');
    IF NOT FOUND THEN RAISE EXCEPTION 'Escala no reconocida'; END IF;
    SELECT (SELECT COUNT(*) FROM estacion_sismica)
         + (SELECT COUNT(*) FROM sensor)
         + (SELECT COUNT(*) FROM evento_sismico)
         + (SELECT COUNT(*) FROM lectura_onda)
         + (SELECT COUNT(*) FROM deteccion)
         + (SELECT COUNT(*) FROM alerta) INTO v_total;
    IF v_total <> v_esperado
       OR (SELECT COUNT(*) FROM evento_sismico WHERE id_evento BETWEEN 200000000 AND 299999999)<>v_igp
       OR (SELECT COUNT(*) FROM pg_trigger t JOIN pg_class c ON c.oid=t.tgrelid
           JOIN pg_namespace n ON n.oid=c.relnamespace
           WHERE n.nspname='monitoreo_sismologico' AND NOT t.tgisinternal)<>9
       OR EXISTS (SELECT 1 FROM estacion_sismica e
            WHERE NOT EXISTS (SELECT 1 FROM sensor s WHERE s.id_estacion=e.id_estacion))
       OR EXISTS (SELECT 1 FROM deteccion d
            WHERE NOT EXISTS (SELECT 1 FROM lectura_onda l
                WHERE l.id_sensor=d.id_sensor AND l.fecha_hora=d.hora_deteccion AND l.es_valida))
       OR EXISTS (SELECT 1 FROM deteccion WHERE id_evento BETWEEN 200000000 AND 299999999)
       OR EXISTS (SELECT 1 FROM alerta WHERE id_evento BETWEEN 200000000 AND 299999999) THEN
        RAISE EXCEPTION 'REVISAR: escala, catalogo, triggers o integridad';
    END IF;
    RAISE NOTICE 'OK: base %, % filas, % eventos IGP reales y 9 triggers.',
       current_database(),v_total,v_igp;
END;
$$;

\echo '=== ETAPA 4 FINALIZADA ==='
SELECT current_database() AS base_verificada;
```

Listing 4: Integrador 04: preparación de las cuatro escalas.

7.6.5. Planes de ejecución e índices

```
-- 05. PLANES E INDICES
-- Base de ensayo: 1 000 000 de registros.
-- La transaccion revierte los indices experimentales.

\set ON_ERROR_STOP on
\pset pager off
\timing off
\connect bd_sismologico_prueba_exp_1000000

\echo
\echo === VALIDACION INICIAL ===
DO $validacion$
DECLARE
    v_total BIGINT;
BEGIN
    IF current_database() <> 'bd_sismologico_prueba_exp_1000000' THEN
        RAISE EXCEPTION 'Base de datos incorrecta';
    END IF;

    SELECT (SELECT COUNT(*) FROM monitoreo_sismologico.estacion_sismica)
         + (SELECT COUNT(*) FROM monitoreo_sismologico.sensor)
         + (SELECT COUNT(*) FROM monitoreo_sismologico.evento_sismico)
         + (SELECT COUNT(*) FROM monitoreo_sismologico.lectura_onda)
         + (SELECT COUNT(*) FROM monitoreo_sismologico.deteccion)
         + (SELECT COUNT(*) FROM monitoreo_sismologico.alerta)
    INTO v_total;

    IF v_total <> 1000000
       OR (SELECT COUNT(*) FROM monitoreo_sismologico.estacion_sismica) <> 100
       OR (SELECT COUNT(*) FROM monitoreo_sismologico.sensor) <> 300
       OR (SELECT COUNT(*) FROM monitoreo_sismologico.evento_sismico) <> 120000
```

```
       OR (SELECT COUNT(*) FROM monitoreo_sismologico.lectura_onda) <> 875000
       OR (SELECT COUNT(*) FROM monitoreo_sismologico.deteccion) <> 3500
       OR (SELECT COUNT(*) FROM monitoreo_sismologico.alerta) <> 1100 THEN
        RAISE EXCEPTION 'Distribucion experimental incorrecta';
    END IF;

    IF (SELECT COUNT(*) FROM monitoreo_sismologico.evento_sismico
        WHERE id_evento BETWEEN 200000000 AND 299999999) <> 24289 THEN
        RAISE EXCEPTION 'Catalogo IGP diferente del esperado';
    END IF;

    IF EXISTS (
        SELECT 1
        FROM pg_index i
        JOIN pg_class t ON t.oid = i.indrelid
        JOIN pg_namespace n ON n.oid = t.relnamespace
        LEFT JOIN pg_constraint c ON c.conindid = i.indexrelid
        WHERE n.nspname = 'monitoreo_sismologico'
          AND c.oid IS NULL
    ) THEN
        RAISE EXCEPTION 'Hay indices secundarios previos; no se modifico nada';
    END IF;

    RAISE NOTICE 'OK: 1000000 registros, 24289 eventos IGP y sin indices secundarios';
END;
$validacion$;

BEGIN;
SET LOCAL search_path TO monitoreo_sismologico, public;

CREATE TEMP TABLE comparacion_planes (
    consulta TEXT PRIMARY KEY,
    filas_sin BIGINT NOT NULL,
    firma_sin TEXT NOT NULL,
    filas_con BIGINT,
    firma_con TEXT
) ON COMMIT DROP;

\echo
\echo === C1: ACTIVIDAD SISMICA ANUAL ===
\echo SIN INDICE
EXPLAIN (ANALYZE, BUFFERS, SETTINGS)
SELECT
    EXTRACT(YEAR FROM ev.fecha_hora_origen)::INTEGER AS anio_utc,
    COUNT(*) AS cantidad_eventos,
    ROUND(AVG(ev.magnitud), 2) AS magnitud_promedio,
    MAX(ev.magnitud) AS magnitud_maxima,
    ROUND(AVG(ev.profundidad_km), 2) AS profundidad_promedio_km
FROM evento_sismico ev
WHERE ev.id_evento BETWEEN 200000000 AND 299999999
  AND ev.fecha_hora_origen >= TIMESTAMP '2010-01-01 00:00:00'
  AND ev.fecha_hora_origen < TIMESTAMP '2026-01-01 00:00:00'
  AND ev.magnitud >= 6
GROUP BY EXTRACT(YEAR FROM ev.fecha_hora_origen)::INTEGER
HAVING COUNT(*) >= 1
ORDER BY anio_utc;

INSERT INTO comparacion_planes (consulta, filas_sin, firma_sin)
SELECT 'C1', COUNT(*),
       MD5(COALESCE(STRING_AGG(TO_JSONB(t)::TEXT, '|' ORDER BY TO_JSONB(t)::TEXT), ''))
FROM (
SELECT
    EXTRACT(YEAR FROM ev.fecha_hora_origen)::INTEGER AS anio_utc,
    COUNT(*) AS cantidad_eventos,
    ROUND(AVG(ev.magnitud), 2) AS magnitud_promedio,
    MAX(ev.magnitud) AS magnitud_maxima,
    ROUND(AVG(ev.profundidad_km), 2) AS profundidad_promedio_km
FROM evento_sismico ev
WHERE ev.id_evento BETWEEN 200000000 AND 299999999
  AND ev.fecha_hora_origen >= TIMESTAMP '2010-01-01 00:00:00'
  AND ev.fecha_hora_origen < TIMESTAMP '2026-01-01 00:00:00'
  AND ev.magnitud >= 6
GROUP BY EXTRACT(YEAR FROM ev.fecha_hora_origen)::INTEGER
HAVING COUNT(*) >= 1
ORDER BY anio_utc
) AS t;
```

```
\echo INDICES CANDIDATOS
CREATE INDEX idx_ev_igp_m6_fecha
ON monitoreo_sismologico.evento_sismico (fecha_hora_origen)
INCLUDE (magnitud, profundidad_km)
WHERE id_evento BETWEEN 200000000 AND 299999999
  AND magnitud >= 6;
ANALYZE monitoreo_sismologico.evento_sismico;

\echo CON INDICE
EXPLAIN (ANALYZE, BUFFERS, SETTINGS)
SELECT
    EXTRACT(YEAR FROM ev.fecha_hora_origen)::INTEGER AS anio_utc,
    COUNT(*) AS cantidad_eventos,
    ROUND(AVG(ev.magnitud), 2) AS magnitud_promedio,
    MAX(ev.magnitud) AS magnitud_maxima,
    ROUND(AVG(ev.profundidad_km), 2) AS profundidad_promedio_km
FROM evento_sismico ev
WHERE ev.id_evento BETWEEN 200000000 AND 299999999
  AND ev.fecha_hora_origen >= TIMESTAMP '2010-01-01 00:00:00'
  AND ev.fecha_hora_origen < TIMESTAMP '2026-01-01 00:00:00'
  AND ev.magnitud >= 6
GROUP BY EXTRACT(YEAR FROM ev.fecha_hora_origen)::INTEGER
HAVING COUNT(*) >= 1
ORDER BY anio_utc;

UPDATE comparacion_planes
SET (filas_con, firma_con) = (
    SELECT COUNT(*),
           MD5(COALESCE(STRING_AGG(TO_JSONB(t)::TEXT, '|' ORDER BY TO_JSONB(t)::TEXT), ''))
    FROM (
SELECT
    EXTRACT(YEAR FROM ev.fecha_hora_origen)::INTEGER AS anio_utc,
    COUNT(*) AS cantidad_eventos,
    ROUND(AVG(ev.magnitud), 2) AS magnitud_promedio,
    MAX(ev.magnitud) AS magnitud_maxima,
    ROUND(AVG(ev.profundidad_km), 2) AS profundidad_promedio_km
FROM evento_sismico ev
WHERE ev.id_evento BETWEEN 200000000 AND 299999999
  AND ev.fecha_hora_origen >= TIMESTAMP '2010-01-01 00:00:00'
  AND ev.fecha_hora_origen < TIMESTAMP '2026-01-01 00:00:00'
  AND ev.magnitud >= 6
GROUP BY EXTRACT(YEAR FROM ev.fecha_hora_origen)::INTEGER
HAVING COUNT(*) >= 1
ORDER BY anio_utc
    ) AS t
)
WHERE consulta = 'C1';

DROP INDEX monitoreo_sismologico.idx_ev_igp_m6_fecha;

\echo
\echo === C2: VENTANAS GEOGRAFICAS ===
\echo SIN INDICE
EXPLAIN (ANALYZE, BUFFERS, SETTINGS)
SELECT
    e.id_estacion,
    e.codigo AS codigo_estacion,
    e.nombre AS nombre_estacion,
    COUNT(*) AS eventos_en_ventana,
    ROUND(AVG(ev.magnitud), 2) AS magnitud_promedio,
    MAX(ev.magnitud) AS magnitud_maxima
FROM estacion_sismica e
JOIN evento_sismico ev
  ON ev.latitud_epicentro BETWEEN e.latitud - 1.5 AND e.latitud + 1.5
 AND ev.longitud_epicentro BETWEEN e.longitud - 1.5 AND e.longitud + 1.5
WHERE ev.id_evento BETWEEN 200000000 AND 299999999
  AND ev.fecha_hora_origen >= TIMESTAMP '2000-01-01 00:00:00'
  AND ev.fecha_hora_origen < TIMESTAMP '2026-01-01 00:00:00'
  AND ev.magnitud >= 5
GROUP BY e.id_estacion, e.codigo, e.nombre
HAVING COUNT(*) >= 1
ORDER BY eventos_en_ventana DESC, e.id_estacion
LIMIT 10;

INSERT INTO comparacion_planes (consulta, filas_sin, firma_sin)
SELECT 'C2', COUNT(*),
       MD5(COALESCE(STRING_AGG(TO_JSONB(t)::TEXT, '|' ORDER BY TO_JSONB(t)::TEXT), ''))
```

```
FROM (
SELECT
    e.id_estacion,
    e.codigo AS codigo_estacion,
    e.nombre AS nombre_estacion,
    COUNT(*) AS eventos_en_ventana,
    ROUND(AVG(ev.magnitud), 2) AS magnitud_promedio,
    MAX(ev.magnitud) AS magnitud_maxima
FROM estacion_sismica e
JOIN evento_sismico ev
  ON ev.latitud_epicentro BETWEEN e.latitud - 1.5 AND e.latitud + 1.5
 AND ev.longitud_epicentro BETWEEN e.longitud - 1.5 AND e.longitud + 1.5
WHERE ev.id_evento BETWEEN 200000000 AND 299999999
  AND ev.fecha_hora_origen >= TIMESTAMP '2000-01-01 00:00:00'
  AND ev.fecha_hora_origen < TIMESTAMP '2026-01-01 00:00:00'
  AND ev.magnitud >= 5
GROUP BY e.id_estacion, e.codigo, e.nombre
HAVING COUNT(*) >= 1
ORDER BY eventos_en_ventana DESC, e.id_estacion
LIMIT 10
) AS t;

\echo INDICES CANDIDATOS
CREATE INDEX idx_ev_igp_geo_c2
ON monitoreo_sismologico.evento_sismico
    (latitud_epicentro, longitud_epicentro)
INCLUDE (magnitud)
WHERE id_evento BETWEEN 200000000 AND 299999999
  AND fecha_hora_origen >= TIMESTAMP '2000-01-01 00:00:00'
  AND fecha_hora_origen < TIMESTAMP '2026-01-01 00:00:00'
  AND magnitud >= 5;
ANALYZE monitoreo_sismologico.evento_sismico;

\echo CON INDICE
EXPLAIN (ANALYZE, BUFFERS, SETTINGS)
SELECT
    e.id_estacion,
    e.codigo AS codigo_estacion,
    e.nombre AS nombre_estacion,
    COUNT(*) AS eventos_en_ventana,
    ROUND(AVG(ev.magnitud), 2) AS magnitud_promedio,
    MAX(ev.magnitud) AS magnitud_maxima
FROM estacion_sismica e
JOIN evento_sismico ev
  ON ev.latitud_epicentro BETWEEN e.latitud - 1.5 AND e.latitud + 1.5
 AND ev.longitud_epicentro BETWEEN e.longitud - 1.5 AND e.longitud + 1.5
WHERE ev.id_evento BETWEEN 200000000 AND 299999999
  AND ev.fecha_hora_origen >= TIMESTAMP '2000-01-01 00:00:00'
  AND ev.fecha_hora_origen < TIMESTAMP '2026-01-01 00:00:00'
  AND ev.magnitud >= 5
GROUP BY e.id_estacion, e.codigo, e.nombre
HAVING COUNT(*) >= 1
ORDER BY eventos_en_ventana DESC, e.id_estacion
LIMIT 10;

UPDATE comparacion_planes
SET (filas_con, firma_con) = (
    SELECT COUNT(*),
           MD5(COALESCE(STRING_AGG(TO_JSONB(t)::TEXT, '|' ORDER BY TO_JSONB(t)::TEXT), ''))
    FROM (
SELECT
    e.id_estacion,
    e.codigo AS codigo_estacion,
    e.nombre AS nombre_estacion,
    COUNT(*) AS eventos_en_ventana,
    ROUND(AVG(ev.magnitud), 2) AS magnitud_promedio,
    MAX(ev.magnitud) AS magnitud_maxima
FROM estacion_sismica e
JOIN evento_sismico ev
  ON ev.latitud_epicentro BETWEEN e.latitud - 1.5 AND e.latitud + 1.5
 AND ev.longitud_epicentro BETWEEN e.longitud - 1.5 AND e.longitud + 1.5
WHERE ev.id_evento BETWEEN 200000000 AND 299999999
  AND ev.fecha_hora_origen >= TIMESTAMP '2000-01-01 00:00:00'
  AND ev.fecha_hora_origen < TIMESTAMP '2026-01-01 00:00:00'
  AND ev.magnitud >= 5
GROUP BY e.id_estacion, e.codigo, e.nombre
HAVING COUNT(*) >= 1
```

```
ORDER BY eventos_en_ventana DESC, e.id_estacion
LIMIT 10
    ) AS t
)
WHERE consulta = 'C2';

DROP INDEX monitoreo_sismologico.idx_ev_igp_geo_c2;

\echo
\echo === C3: CALIDAD DE LECTURAS ===
\echo SIN INDICE
EXPLAIN (ANALYZE, BUFFERS, SETTINGS)
SELECT
    e.id_estacion,
    e.codigo AS codigo_estacion,
    COUNT(DISTINCT s.id_sensor) AS sensores_con_lecturas,
    COUNT(*) AS total_lecturas,
    COUNT(*) FILTER (WHERE l.es_valida) AS lecturas_validas,
    COUNT(*) FILTER (WHERE NOT l.es_valida) AS lecturas_invalidas,
    ROUND(100.0 * COUNT(*) FILTER (WHERE NOT l.es_valida)
          / NULLIF(COUNT(*), 0), 2) AS porcentaje_invalidas
FROM estacion_sismica e
JOIN sensor s ON s.id_estacion = e.id_estacion
JOIN lectura_onda l ON l.id_sensor = s.id_sensor
WHERE l.fecha_hora >= TIMESTAMP '2026-08-01 00:00:00'
  AND l.fecha_hora < TIMESTAMP '2026-12-01 00:00:00'
GROUP BY e.id_estacion, e.codigo
HAVING COUNT(*) >= 10
ORDER BY porcentaje_invalidas DESC, total_lecturas DESC, e.id_estacion;

INSERT INTO comparacion_planes (consulta, filas_sin, firma_sin)
SELECT 'C3', COUNT(*),
       MD5(COALESCE(STRING_AGG(TO_JSONB(t)::TEXT, '|' ORDER BY TO_JSONB(t)::TEXT), ''))
FROM (
SELECT
    e.id_estacion,
    e.codigo AS codigo_estacion,
    COUNT(DISTINCT s.id_sensor) AS sensores_con_lecturas,
    COUNT(*) AS total_lecturas,
    COUNT(*) FILTER (WHERE l.es_valida) AS lecturas_validas,
    COUNT(*) FILTER (WHERE NOT l.es_valida) AS lecturas_invalidas,
    ROUND(100.0 * COUNT(*) FILTER (WHERE NOT l.es_valida)
          / NULLIF(COUNT(*), 0), 2) AS porcentaje_invalidas
FROM estacion_sismica e
JOIN sensor s ON s.id_estacion = e.id_estacion
JOIN lectura_onda l ON l.id_sensor = s.id_sensor
WHERE l.fecha_hora >= TIMESTAMP '2026-08-01 00:00:00'
  AND l.fecha_hora < TIMESTAMP '2026-12-01 00:00:00'
GROUP BY e.id_estacion, e.codigo
HAVING COUNT(*) >= 10
ORDER BY porcentaje_invalidas DESC, total_lecturas DESC, e.id_estacion
) AS t;

\echo INDICES CANDIDATOS
CREATE INDEX idx_lectura_sensor_fecha_c3
ON monitoreo_sismologico.lectura_onda (id_sensor, fecha_hora)
INCLUDE (es_valida);
ANALYZE monitoreo_sismologico.lectura_onda;

\echo CON INDICE
EXPLAIN (ANALYZE, BUFFERS, SETTINGS)
SELECT
    e.id_estacion,
    e.codigo AS codigo_estacion,
    COUNT(DISTINCT s.id_sensor) AS sensores_con_lecturas,
    COUNT(*) AS total_lecturas,
    COUNT(*) FILTER (WHERE l.es_valida) AS lecturas_validas,
    COUNT(*) FILTER (WHERE NOT l.es_valida) AS lecturas_invalidas,
    ROUND(100.0 * COUNT(*) FILTER (WHERE NOT l.es_valida)
          / NULLIF(COUNT(*), 0), 2) AS porcentaje_invalidas
FROM estacion_sismica e
JOIN sensor s ON s.id_estacion = e.id_estacion
JOIN lectura_onda l ON l.id_sensor = s.id_sensor
WHERE l.fecha_hora >= TIMESTAMP '2026-08-01 00:00:00'
  AND l.fecha_hora < TIMESTAMP '2026-12-01 00:00:00'
GROUP BY e.id_estacion, e.codigo
HAVING COUNT(*) >= 10
```

```
ORDER BY porcentaje_invalidas DESC, total_lecturas DESC, e.id_estacion;

UPDATE comparacion_planes
SET (filas_con, firma_con) = (
    SELECT COUNT(*),
           MD5(COALESCE(STRING_AGG(TO_JSONB(t)::TEXT, '|' ORDER BY TO_JSONB(t)::TEXT), ''))
    FROM (
SELECT
    e.id_estacion,
    e.codigo AS codigo_estacion,
    COUNT(DISTINCT s.id_sensor) AS sensores_con_lecturas,
    COUNT(*) AS total_lecturas,
    COUNT(*) FILTER (WHERE l.es_valida) AS lecturas_validas,
    COUNT(*) FILTER (WHERE NOT l.es_valida) AS lecturas_invalidas,
    ROUND(100.0 * COUNT(*) FILTER (WHERE NOT l.es_valida)
          / NULLIF(COUNT(*), 0), 2) AS porcentaje_invalidas
FROM estacion_sismica e
JOIN sensor s ON s.id_estacion = e.id_estacion
JOIN lectura_onda l ON l.id_sensor = s.id_sensor
WHERE l.fecha_hora >= TIMESTAMP '2026-08-01 00:00:00'
  AND l.fecha_hora < TIMESTAMP '2026-12-01 00:00:00'
GROUP BY e.id_estacion, e.codigo
HAVING COUNT(*) >= 10
ORDER BY porcentaje_invalidas DESC, total_lecturas DESC, e.id_estacion
    ) AS t
)
WHERE consulta = 'C3';

DROP INDEX monitoreo_sismologico.idx_lectura_sensor_fecha_c3;

\echo
\echo === C4: DETECCIONES Y ALERTAS ===
\echo SIN INDICE
EXPLAIN (ANALYZE, BUFFERS, SETTINGS)
WITH detecciones_resumen AS (
    SELECT id_evento, COUNT(*) AS total_detecciones,
           MIN(hora_deteccion) AS primera_deteccion
    FROM deteccion
    GROUP BY id_evento
), alertas_resumen AS (
    SELECT id_evento, COUNT(*) AS total_alertas,
           MIN(fecha_hora_emision) AS primera_alerta
    FROM alerta
    GROUP BY id_evento
)
SELECT
    ev.id_evento,
    ev.fecha_hora_origen,
    ev.magnitud,
    COALESCE(d.total_detecciones, 0) AS total_detecciones,
    COALESCE(a.total_alertas, 0) AS total_alertas,
    ROUND(EXTRACT(EPOCH FROM (d.primera_deteccion - ev.fecha_hora_origen))::NUMERIC, 2)
        AS segundos_primera_deteccion,
    ROUND(EXTRACT(EPOCH FROM (a.primera_alerta - ev.fecha_hora_origen))::NUMERIC, 2)
        AS segundos_primera_alerta
FROM evento_sismico ev
LEFT JOIN detecciones_resumen d ON d.id_evento = ev.id_evento
LEFT JOIN alertas_resumen a ON a.id_evento = ev.id_evento
WHERE (ev.id_evento BETWEEN 810000001 AND 810000008
    OR ev.id_evento BETWEEN 820300001 AND 820300030)
  AND (COALESCE(d.total_detecciones, 0) > 0
    OR COALESCE(a.total_alertas, 0) > 0)
ORDER BY ev.fecha_hora_origen DESC, ev.id_evento
LIMIT 15;

INSERT INTO comparacion_planes (consulta, filas_sin, firma_sin)
SELECT 'C4', COUNT(*),
       MD5(COALESCE(STRING_AGG(TO_JSONB(t)::TEXT, '|' ORDER BY TO_JSONB(t)::TEXT), ''))
FROM (
WITH detecciones_resumen AS (
    SELECT id_evento, COUNT(*) AS total_detecciones,
           MIN(hora_deteccion) AS primera_deteccion
    FROM deteccion
    GROUP BY id_evento
), alertas_resumen AS (
    SELECT id_evento, COUNT(*) AS total_alertas,
           MIN(fecha_hora_emision) AS primera_alerta
```

```
    FROM alerta
    GROUP BY id_evento
)
SELECT
    ev.id_evento,
    ev.fecha_hora_origen,
    ev.magnitud,
    COALESCE(d.total_detecciones, 0) AS total_detecciones,
    COALESCE(a.total_alertas, 0) AS total_alertas,
    ROUND(EXTRACT(EPOCH FROM (d.primera_deteccion - ev.fecha_hora_origen))::NUMERIC, 2)
        AS segundos_primera_deteccion,
    ROUND(EXTRACT(EPOCH FROM (a.primera_alerta - ev.fecha_hora_origen))::NUMERIC, 2)
        AS segundos_primera_alerta
FROM evento_sismico ev
LEFT JOIN detecciones_resumen d ON d.id_evento = ev.id_evento
LEFT JOIN alertas_resumen a ON a.id_evento = ev.id_evento
WHERE (ev.id_evento BETWEEN 810000001 AND 810000008
    OR ev.id_evento BETWEEN 820300001 AND 820300030)
  AND (COALESCE(d.total_detecciones, 0) > 0
    OR COALESCE(a.total_alertas, 0) > 0)
ORDER BY ev.fecha_hora_origen DESC, ev.id_evento
LIMIT 15
) AS t;

\echo INDICES CANDIDATOS
CREATE INDEX idx_det_evento_hora_c4
    ON monitoreo_sismologico.deteccion (id_evento, hora_deteccion);
CREATE INDEX idx_alerta_evento_hora_c4
    ON monitoreo_sismologico.alerta (id_evento, fecha_hora_emision);
ANALYZE monitoreo_sismologico.deteccion;
ANALYZE monitoreo_sismologico.alerta;

\echo CON INDICE
EXPLAIN (ANALYZE, BUFFERS, SETTINGS)
WITH detecciones_resumen AS (
    SELECT id_evento, COUNT(*) AS total_detecciones,
           MIN(hora_deteccion) AS primera_deteccion
    FROM deteccion
    GROUP BY id_evento
), alertas_resumen AS (
    SELECT id_evento, COUNT(*) AS total_alertas,
           MIN(fecha_hora_emision) AS primera_alerta
    FROM alerta
    GROUP BY id_evento
)
SELECT
    ev.id_evento,
    ev.fecha_hora_origen,
    ev.magnitud,
    COALESCE(d.total_detecciones, 0) AS total_detecciones,
    COALESCE(a.total_alertas, 0) AS total_alertas,
    ROUND(EXTRACT(EPOCH FROM (d.primera_deteccion - ev.fecha_hora_origen))::NUMERIC, 2)
        AS segundos_primera_deteccion,
    ROUND(EXTRACT(EPOCH FROM (a.primera_alerta - ev.fecha_hora_origen))::NUMERIC, 2)
        AS segundos_primera_alerta
FROM evento_sismico ev
LEFT JOIN detecciones_resumen d ON d.id_evento = ev.id_evento
LEFT JOIN alertas_resumen a ON a.id_evento = ev.id_evento
WHERE (ev.id_evento BETWEEN 810000001 AND 810000008
    OR ev.id_evento BETWEEN 820300001 AND 820300030)
  AND (COALESCE(d.total_detecciones, 0) > 0
    OR COALESCE(a.total_alertas, 0) > 0)
ORDER BY ev.fecha_hora_origen DESC, ev.id_evento
LIMIT 15;

UPDATE comparacion_planes
SET (filas_con, firma_con) = (
    SELECT COUNT(*),
           MD5(COALESCE(STRING_AGG(TO_JSONB(t)::TEXT, '|' ORDER BY TO_JSONB(t)::TEXT), ''))
    FROM (
WITH detecciones_resumen AS (
    SELECT id_evento, COUNT(*) AS total_detecciones,
           MIN(hora_deteccion) AS primera_deteccion
    FROM deteccion
    GROUP BY id_evento
), alertas_resumen AS (
    SELECT id_evento, COUNT(*) AS total_alertas,
```

```
            MIN(fecha_hora_emision) AS primera_alerta
    FROM alerta
    GROUP BY id_evento
)
SELECT
    ev.id_evento,
    ev.fecha_hora_origen,
    ev.magnitud,
    COALESCE(d.total_detecciones, 0) AS total_detecciones,
    COALESCE(a.total_alertas, 0) AS total_alertas,
    ROUND(EXTRACT(EPOCH FROM (d.primera_deteccion - ev.fecha_hora_origen))::NUMERIC, 2)
        AS segundos_primera_deteccion,
    ROUND(EXTRACT(EPOCH FROM (a.primera_alerta - ev.fecha_hora_origen))::NUMERIC, 2)
        AS segundos_primera_alerta
FROM evento_sismico ev
LEFT JOIN detecciones_resumen d ON d.id_evento = ev.id_evento
LEFT JOIN alertas_resumen a ON a.id_evento = ev.id_evento
WHERE (ev.id_evento BETWEEN 810000001 AND 810000008
    OR ev.id_evento BETWEEN 820300001 AND 820300030)
  AND (COALESCE(d.total_detecciones, 0) > 0
    OR COALESCE(a.total_alertas, 0) > 0)
ORDER BY ev.fecha_hora_origen DESC, ev.id_evento
LIMIT 15
    ) AS t
)
WHERE consulta = 'C4';

DROP INDEX monitoreo_sismologico.idx_det_evento_hora_c4;
DROP INDEX monitoreo_sismologico.idx_alerta_evento_hora_c4;

\echo
\echo === C5: SENSORES SIN DETECCIONES ===
\echo SIN INDICE
EXPLAIN (ANALYZE, BUFFERS, SETTINGS)
SELECT
    s.id_sensor,
    s.numero_serie,
    e.codigo AS codigo_estacion,
    s.estado AS estado_sensor,
    COUNT(*) AS lecturas_periodo,
    COUNT(*) FILTER (WHERE NOT l.es_valida) AS lecturas_invalidas,
    MAX(l.fecha_hora) AS ultima_lectura
FROM sensor s
JOIN estacion_sismica e ON e.id_estacion = s.id_estacion
JOIN lectura_onda l ON l.id_sensor = s.id_sensor
WHERE l.fecha_hora >= TIMESTAMP '2026-08-01 00:00:00'
  AND l.fecha_hora < TIMESTAMP '2026-09-01 00:00:00'
  AND NOT EXISTS (
      SELECT 1
      FROM deteccion d
      WHERE d.id_sensor = s.id_sensor
        AND d.hora_deteccion >= TIMESTAMP '2026-08-01 00:00:00'
        AND d.hora_deteccion < TIMESTAMP '2026-09-01 00:00:00'
  )
GROUP BY s.id_sensor, s.numero_serie, e.codigo, s.estado
HAVING COUNT(*) >= 20
ORDER BY lecturas_invalidas DESC, lecturas_periodo DESC, s.id_sensor
LIMIT 15;

INSERT INTO comparacion_planes (consulta, filas_sin, firma_sin)
SELECT 'C5', COUNT(*),
       MD5(COALESCE(STRING_AGG(TO_JSONB(t)::TEXT, '|' ORDER BY TO_JSONB(t)::TEXT), ''))
FROM (
SELECT
    s.id_sensor,
    s.numero_serie,
    e.codigo AS codigo_estacion,
    s.estado AS estado_sensor,
    COUNT(*) AS lecturas_periodo,
    COUNT(*) FILTER (WHERE NOT l.es_valida) AS lecturas_invalidas,
    MAX(l.fecha_hora) AS ultima_lectura
FROM sensor s
JOIN estacion_sismica e ON e.id_estacion = s.id_estacion
JOIN lectura_onda l ON l.id_sensor = s.id_sensor
WHERE l.fecha_hora >= TIMESTAMP '2026-08-01 00:00:00'
  AND l.fecha_hora < TIMESTAMP '2026-09-01 00:00:00'
  AND NOT EXISTS (
```

```
      SELECT 1
      FROM deteccion d
      WHERE d.id_sensor = s.id_sensor
        AND d.hora_deteccion >= TIMESTAMP '2026-08-01 00:00:00'
        AND d.hora_deteccion < TIMESTAMP '2026-09-01 00:00:00'
  )
GROUP BY s.id_sensor, s.numero_serie, e.codigo, s.estado
HAVING COUNT(*) >= 20
ORDER BY lecturas_invalidas DESC, lecturas_periodo DESC, s.id_sensor
LIMIT 15
) AS t;

\echo INDICES CANDIDATOS
CREATE INDEX idx_lectura_sensor_hora_c5
    ON monitoreo_sismologico.lectura_onda (id_sensor, fecha_hora)
    INCLUDE (es_valida);
CREATE INDEX idx_det_sensor_hora_c5
    ON monitoreo_sismologico.deteccion (id_sensor, hora_deteccion);
ANALYZE monitoreo_sismologico.lectura_onda;
ANALYZE monitoreo_sismologico.deteccion;

\echo CON INDICE
EXPLAIN (ANALYZE, BUFFERS, SETTINGS)
SELECT
    s.id_sensor,
    s.numero_serie,
    e.codigo AS codigo_estacion,
    s.estado AS estado_sensor,
    COUNT(*) AS lecturas_periodo,
    COUNT(*) FILTER (WHERE NOT l.es_valida) AS lecturas_invalidas,
    MAX(l.fecha_hora) AS ultima_lectura
FROM sensor s
JOIN estacion_sismica e ON e.id_estacion = s.id_estacion
JOIN lectura_onda l ON l.id_sensor = s.id_sensor
WHERE l.fecha_hora >= TIMESTAMP '2026-08-01 00:00:00'
  AND l.fecha_hora < TIMESTAMP '2026-09-01 00:00:00'
  AND NOT EXISTS (
      SELECT 1
      FROM deteccion d
      WHERE d.id_sensor = s.id_sensor
        AND d.hora_deteccion >= TIMESTAMP '2026-08-01 00:00:00'
        AND d.hora_deteccion < TIMESTAMP '2026-09-01 00:00:00'
  )
GROUP BY s.id_sensor, s.numero_serie, e.codigo, s.estado
HAVING COUNT(*) >= 20
ORDER BY lecturas_invalidas DESC, lecturas_periodo DESC, s.id_sensor
LIMIT 15;

UPDATE comparacion_planes
SET (filas_con, firma_con) = (
    SELECT COUNT(*),
           MD5(COALESCE(STRING_AGG(TO_JSONB(t)::TEXT, '|' ORDER BY TO_JSONB(t)::TEXT), ''))
    FROM (
SELECT
    s.id_sensor,
    s.numero_serie,
    e.codigo AS codigo_estacion,
    s.estado AS estado_sensor,
    COUNT(*) AS lecturas_periodo,
    COUNT(*) FILTER (WHERE NOT l.es_valida) AS lecturas_invalidas,
    MAX(l.fecha_hora) AS ultima_lectura
FROM sensor s
JOIN estacion_sismica e ON e.id_estacion = s.id_estacion
JOIN lectura_onda l ON l.id_sensor = s.id_sensor
WHERE l.fecha_hora >= TIMESTAMP '2026-08-01 00:00:00'
  AND l.fecha_hora < TIMESTAMP '2026-09-01 00:00:00'
  AND NOT EXISTS (
      SELECT 1
      FROM deteccion d
      WHERE d.id_sensor = s.id_sensor
        AND d.hora_deteccion >= TIMESTAMP '2026-08-01 00:00:00'
        AND d.hora_deteccion < TIMESTAMP '2026-09-01 00:00:00'
  )
GROUP BY s.id_sensor, s.numero_serie, e.codigo, s.estado
HAVING COUNT(*) >= 20
ORDER BY lecturas_invalidas DESC, lecturas_periodo DESC, s.id_sensor
LIMIT 15
```

```
    ) AS t
)
WHERE consulta = 'C5';

DROP INDEX monitoreo_sismologico.idx_lectura_sensor_hora_c5;
DROP INDEX monitoreo_sismologico.idx_det_sensor_hora_c5;

\echo
\echo === EQUIVALENCIA DE RESULTADOS ===
SELECT consulta, filas_sin, filas_con,
       CASE WHEN filas_sin = filas_con AND firma_sin = firma_con
            THEN 'OK' ELSE 'ERROR' END AS resultado
FROM comparacion_planes
ORDER BY consulta;

DO $control$
BEGIN
    IF (SELECT COUNT(*) FROM comparacion_planes) <> 5
       OR EXISTS (
          SELECT 1 FROM comparacion_planes
          WHERE filas_con IS NULL OR firma_con IS NULL
             OR filas_sin <> filas_con OR firma_sin <> firma_con
       ) THEN
        RAISE EXCEPTION 'Diferencia en los resultados de las consultas';
    END IF;

    IF EXISTS (
        SELECT 1
        FROM pg_index i
        JOIN pg_class t ON t.oid = i.indrelid
        JOIN pg_namespace n ON n.oid = t.relnamespace
        LEFT JOIN pg_constraint c ON c.conindid = i.indexrelid
        WHERE n.nspname = 'monitoreo_sismologico'
          AND c.oid IS NULL
    ) THEN
        RAISE EXCEPTION 'Quedaron indices secundarios temporales';
    END IF;

    RAISE NOTICE 'OK: cinco consultas equivalentes; indices candidatos retirados';
END;
$control$;

\echo
\echo === RESTAURAR ESTADO INICIAL ===
ROLLBACK;

\echo
\echo === COMPROBACION FINAL ===
DO $final$
BEGIN
    IF EXISTS (
        SELECT 1
        FROM pg_index i
        JOIN pg_class t ON t.oid = i.indrelid
        JOIN pg_namespace n ON n.oid = t.relnamespace
        LEFT JOIN pg_constraint c ON c.conindid = i.indexrelid
        WHERE n.nspname = 'monitoreo_sismologico'
          AND c.oid IS NULL
    ) THEN
        RAISE EXCEPTION 'Hay indices secundarios tras ROLLBACK';
    END IF;
    RAISE NOTICE 'OK: indices originales preservados';
END;
$final$;

\echo === ETAPA 5 FINALIZADA ===
```

Listing 5: Integrador 05: planes y candidatos de indexación.

7.6.6. Medición de rendimiento

```
-- 06. MEDICIONES
-- Dos calentamientos y cinco mediciones por escenario.
```

```
\set ON_ERROR_STOP on
\pset pager off
\timing off

\echo === VALIDACION INICIAL ===
SELECT set_config('sismo.consulta', :'consulta', false) AS consulta,
       set_config('sismo.escala', :'escala', false) AS escala;

DO $validacion$
DECLARE
    v_c INTEGER := current_setting('sismo.consulta')::INTEGER;
    v_e INTEGER := current_setting('sismo.escala')::INTEGER;
    v_tablas BIGINT[];
    v_esperado BIGINT[];
    v_igp INTEGER;
BEGIN
    IF v_c NOT BETWEEN 1 AND 5 OR v_e NOT IN (1000, 10000, 100000, 1000000) THEN
        RAISE EXCEPTION 'Consulta o escala incorrecta';
    END IF;
    IF current_database() <> 'bd_sismologico_prueba_exp_' || v_e THEN
        RAISE EXCEPTION 'Base incorrecta: %; se requiere bd_sismologico_prueba_exp_%',
            current_database(), v_e;
    END IF;
    SELECT ARRAY[
        (SELECT COUNT(*) FROM monitoreo_sismologico.estacion_sismica),
        (SELECT COUNT(*) FROM monitoreo_sismologico.sensor),
        (SELECT COUNT(*) FROM monitoreo_sismologico.evento_sismico),
        (SELECT COUNT(*) FROM monitoreo_sismologico.lectura_onda),
        (SELECT COUNT(*) FROM monitoreo_sismologico.deteccion),
        (SELECT COUNT(*) FROM monitoreo_sismologico.alerta)
    ] INTO v_tablas;
    v_esperado := CASE v_e
        WHEN 1000 THEN ARRAY[5,15,100,750,90,40]::BIGINT[]
        WHEN 10000 THEN ARRAY[10,30,2000,7550,300,110]::BIGINT[]
        WHEN 100000 THEN ARRAY[25,75,24000,74700,900,300]::BIGINT[]
        WHEN 1000000 THEN ARRAY[100,300,120000,875000,3500,1100]::BIGINT[]
    END;
    IF v_tablas <> v_esperado THEN
        RAISE EXCEPTION 'Distribucion inesperada: %, esperado: %', v_tablas, v_esperado;
    END IF;
    v_igp := CASE v_e WHEN 1000 THEN 50 WHEN 10000 THEN 800
        WHEN 100000 THEN 12000 ELSE 24289 END;
    IF (SELECT COUNT(*) FROM monitoreo_sismologico.evento_sismico
        WHERE id_evento BETWEEN 200000000 AND 299999999) <> v_igp THEN
        RAISE EXCEPTION 'Cantidad IGP inesperada';
    END IF;
    IF EXISTS (
        SELECT 1 FROM pg_index i
        JOIN pg_class t ON t.oid = i.indrelid
        JOIN pg_namespace n ON n.oid = t.relnamespace
        LEFT JOIN pg_constraint c ON c.conindid = i.indexrelid
        WHERE n.nspname = 'monitoreo_sismologico' AND c.oid IS NULL
    ) THEN
        RAISE EXCEPTION 'Hay indices secundarios previos';
    END IF;
    RAISE NOTICE 'OK: C%, escala %, % registros, % eventos IGP, sin indices secundarios',
        v_c, v_e, v_e, v_igp;
END;
$validacion$;

BEGIN;
SET LOCAL search_path TO monitoreo_sismologico, public;

CREATE TEMP TABLE resultado_mediciones (
    consulta TEXT NOT NULL,
    escala INTEGER NOT NULL,
    escenario TEXT NOT NULL,
    repeticion INTEGER NOT NULL,
    fase TEXT NOT NULL,
    execution_ms NUMERIC NOT NULL,
    planning_ms NUMERIC NOT NULL,
    shared_hit BIGINT NOT NULL,
    shared_read BIGINT NOT NULL,
    temp_read BIGINT NOT NULL,
    temp_written BIGINT NOT NULL,
    uso_indice_1 BOOLEAN NOT NULL,
```

```
    uso_indice_2 BOOLEAN NOT NULL,
    filas_resultado INTEGER NOT NULL,
    firma_resultados TEXT NOT NULL,
    PRIMARY KEY (escenario, repeticion)
) ON COMMIT DROP;

\echo === MEDICIONES ===
DO $medicion$
DECLARE
    v_c INTEGER := current_setting('sismo.consulta')::INTEGER;
    v_e INTEGER := current_setting('sismo.escala')::INTEGER;
    v_sql TEXT;
    v_indices TEXT[];
    v_nombres TEXT[];
    v_tablas TEXT[];
    v_modo INTEGER;
    v_repeticion INTEGER;
    v_escenario TEXT;
    v_filas INTEGER;
    v_firma TEXT;
    v_filas_base INTEGER;
    v_firma_base TEXT;
    v_plan JSONB;
    v_raiz JSONB;
    v_uso_1 BOOLEAN;
    v_uso_2 BOOLEAN;
BEGIN
    CASE v_c
        WHEN 1 THEN
            v_sql := $consulta$
SELECT
    EXTRACT(YEAR FROM ev.fecha_hora_origen)::INTEGER AS anio_utc,
    COUNT(*) AS cantidad_eventos,
    ROUND(AVG(ev.magnitud), 2) AS magnitud_promedio,
    MAX(ev.magnitud) AS magnitud_maxima,
    ROUND(AVG(ev.profundidad_km), 2) AS profundidad_promedio_km
FROM evento_sismico ev
WHERE ev.id_evento BETWEEN 200000000 AND 299999999
  AND ev.fecha_hora_origen >= TIMESTAMP '2010-01-01 00:00:00'
  AND ev.fecha_hora_origen < TIMESTAMP '2026-01-01 00:00:00'
  AND ev.magnitud >= 6
GROUP BY EXTRACT(YEAR FROM ev.fecha_hora_origen)::INTEGER
HAVING COUNT(*) >= 1
ORDER BY anio_utc
$consulta$;
            v_indices := ARRAY[
                $indice$CREATE INDEX idx_ev_igp_m6_fecha
ON monitoreo_sismologico.evento_sismico (fecha_hora_origen)
INCLUDE (magnitud, profundidad_km)
WHERE id_evento BETWEEN 200000000 AND 299999999
  AND magnitud >= 6$indice$
            ];
            v_nombres := ARRAY['idx_ev_igp_m6_fecha'];
            v_tablas := ARRAY['evento_sismico'];
        WHEN 2 THEN
            v_sql := $consulta$
SELECT
    e.id_estacion,
    e.codigo AS codigo_estacion,
    e.nombre AS nombre_estacion,
    COUNT(*) AS eventos_en_ventana,
    ROUND(AVG(ev.magnitud), 2) AS magnitud_promedio,
    MAX(ev.magnitud) AS magnitud_maxima
FROM estacion_sismica e
JOIN evento_sismico ev
  ON ev.latitud_epicentro BETWEEN e.latitud - 1.5 AND e.latitud + 1.5
 AND ev.longitud_epicentro BETWEEN e.longitud - 1.5 AND e.longitud + 1.5
WHERE ev.id_evento BETWEEN 200000000 AND 299999999
  AND ev.fecha_hora_origen >= TIMESTAMP '2000-01-01 00:00:00'
  AND ev.fecha_hora_origen < TIMESTAMP '2026-01-01 00:00:00'
  AND ev.magnitud >= 5
GROUP BY e.id_estacion, e.codigo, e.nombre
HAVING COUNT(*) >= 1
ORDER BY eventos_en_ventana DESC, e.id_estacion
LIMIT 10
$consulta$;
            v_indices := ARRAY[
```

```
                $indice$CREATE INDEX idx_ev_igp_geo_c2
ON monitoreo_sismologico.evento_sismico
    (latitud_epicentro, longitud_epicentro)
INCLUDE (magnitud)
WHERE id_evento BETWEEN 200000000 AND 299999999
  AND fecha_hora_origen >= TIMESTAMP '2000-01-01 00:00:00'
  AND fecha_hora_origen < TIMESTAMP '2026-01-01 00:00:00'
  AND magnitud >= 5$indice$
            ];
            v_nombres := ARRAY['idx_ev_igp_geo_c2'];
            v_tablas := ARRAY['evento_sismico'];
        WHEN 3 THEN
            v_sql := $consulta$
SELECT
    e.id_estacion,
    e.codigo AS codigo_estacion,
    COUNT(DISTINCT s.id_sensor) AS sensores_con_lecturas,
    COUNT(*) AS total_lecturas,
    COUNT(*) FILTER (WHERE l.es_valida) AS lecturas_validas,
    COUNT(*) FILTER (WHERE NOT l.es_valida) AS lecturas_invalidas,
    ROUND(100.0 * COUNT(*) FILTER (WHERE NOT l.es_valida)
          / NULLIF(COUNT(*), 0), 2) AS porcentaje_invalidas
FROM estacion_sismica e
JOIN sensor s ON s.id_estacion = e.id_estacion
JOIN lectura_onda l ON l.id_sensor = s.id_sensor
WHERE l.fecha_hora >= TIMESTAMP '2026-08-01 00:00:00'
  AND l.fecha_hora < TIMESTAMP '2026-12-01 00:00:00'
GROUP BY e.id_estacion, e.codigo
HAVING COUNT(*) >= 10
ORDER BY porcentaje_invalidas DESC, total_lecturas DESC, e.id_estacion
$consulta$;
            v_indices := ARRAY[
                $indice$CREATE INDEX idx_lectura_sensor_fecha_c3
ON monitoreo_sismologico.lectura_onda (id_sensor, fecha_hora)
INCLUDE (es_valida)$indice$
            ];
            v_nombres := ARRAY['idx_lectura_sensor_fecha_c3'];
            v_tablas := ARRAY['lectura_onda'];
        WHEN 4 THEN
            v_sql := $consulta$
WITH detecciones_resumen AS (
    SELECT id_evento, COUNT(*) AS total_detecciones,
           MIN(hora_deteccion) AS primera_deteccion
    FROM deteccion
    GROUP BY id_evento
), alertas_resumen AS (
    SELECT id_evento, COUNT(*) AS total_alertas,
           MIN(fecha_hora_emision) AS primera_alerta
    FROM alerta
    GROUP BY id_evento
)
SELECT
    ev.id_evento,
    ev.fecha_hora_origen,
    ev.magnitud,
    COALESCE(d.total_detecciones, 0) AS total_detecciones,
    COALESCE(a.total_alertas, 0) AS total_alertas,
    ROUND(EXTRACT(EPOCH FROM (d.primera_deteccion - ev.fecha_hora_origen))::NUMERIC, 2)
        AS segundos_primera_deteccion,
    ROUND(EXTRACT(EPOCH FROM (a.primera_alerta - ev.fecha_hora_origen))::NUMERIC, 2)
        AS segundos_primera_alerta
FROM evento_sismico ev
LEFT JOIN detecciones_resumen d ON d.id_evento = ev.id_evento
LEFT JOIN alertas_resumen a ON a.id_evento = ev.id_evento
WHERE (ev.id_evento BETWEEN 810000001 AND 810000008
    OR ev.id_evento BETWEEN 820300001 AND 820300030)
  AND (COALESCE(d.total_detecciones, 0) > 0
    OR COALESCE(a.total_alertas, 0) > 0)
ORDER BY ev.fecha_hora_origen DESC, ev.id_evento
LIMIT 15
$consulta$;
            v_indices := ARRAY[
                $indice$CREATE INDEX idx_det_evento_hora_c4
    ON monitoreo_sismologico.deteccion (id_evento, hora_deteccion)$indice$,
                $indice$CREATE INDEX idx_alerta_evento_hora_c4
    ON monitoreo_sismologico.alerta (id_evento, fecha_hora_emision)$indice$
            ];
```

```
            v_nombres := ARRAY['idx_det_evento_hora_c4', 'idx_alerta_evento_hora_c4'];
            v_tablas := ARRAY['deteccion', 'alerta'];
        WHEN 5 THEN
            v_sql := $consulta$
SELECT
    s.id_sensor,
    s.numero_serie,
    e.codigo AS codigo_estacion,
    s.estado AS estado_sensor,
    COUNT(*) AS lecturas_periodo,
    COUNT(*) FILTER (WHERE NOT l.es_valida) AS lecturas_invalidas,
    MAX(l.fecha_hora) AS ultima_lectura
FROM sensor s
JOIN estacion_sismica e ON e.id_estacion = s.id_estacion
JOIN lectura_onda l ON l.id_sensor = s.id_sensor
WHERE l.fecha_hora >= TIMESTAMP '2026-08-01 00:00:00'
  AND l.fecha_hora < TIMESTAMP '2026-09-01 00:00:00'
  AND NOT EXISTS (
      SELECT 1
      FROM deteccion d
      WHERE d.id_sensor = s.id_sensor
        AND d.hora_deteccion >= TIMESTAMP '2026-08-01 00:00:00'
        AND d.hora_deteccion < TIMESTAMP '2026-09-01 00:00:00'
  )
GROUP BY s.id_sensor, s.numero_serie, e.codigo, s.estado
HAVING COUNT(*) >= 20
ORDER BY lecturas_invalidas DESC, lecturas_periodo DESC, s.id_sensor
LIMIT 15
$consulta$;
            v_indices := ARRAY[
                $indice$CREATE INDEX idx_lectura_sensor_hora_c5
    ON monitoreo_sismologico.lectura_onda (id_sensor, fecha_hora)
    INCLUDE (es_valida)$indice$,
                $indice$CREATE INDEX idx_det_sensor_hora_c5
    ON monitoreo_sismologico.deteccion (id_sensor, hora_deteccion)$indice$
            ];
            v_nombres := ARRAY['idx_lectura_sensor_hora_c5', 'idx_det_sensor_hora_c5'];
            v_tablas := ARRAY['lectura_onda', 'deteccion'];
    END CASE;

    FOREACH v_escenario IN ARRAY ARRAY['sin_indice', 'con_indice'] LOOP
        IF v_escenario = 'con_indice' THEN
            FOR v_modo IN 1..ARRAY_LENGTH(v_indices, 1) LOOP
                EXECUTE v_indices[v_modo];
            END LOOP;
            FOR v_modo IN 1..ARRAY_LENGTH(v_tablas, 1) LOOP
                EXECUTE 'ANALYZE monitoreo_sismologico.' || quote_ident(v_tablas[v_modo]);
            END LOOP;
        END IF;

        EXECUTE FORMAT(
            'SELECT COUNT(*)::INTEGER, MD5(COALESCE(STRING_AGG(TO_JSONB(t)::TEXT, ''|'' ORDER BY TO_JSONB(t)::TEXT),
     ''''')) FROM (%s) t',
            v_sql
        ) INTO v_filas, v_firma;
        IF v_filas < 1 THEN
            RAISE EXCEPTION 'Consulta % sin resultados', v_c;
        END IF;
        IF v_escenario = 'sin_indice' THEN
            v_filas_base := v_filas;
            v_firma_base := v_firma;
        ELSIF (v_filas, v_firma) IS DISTINCT FROM (v_filas_base, v_firma_base) THEN
            RAISE EXCEPTION 'Resultados diferentes entre escenarios para C%', v_c;
        END IF;

        FOR v_repeticion IN 1..7 LOOP
            EXECUTE 'EXPLAIN (ANALYZE, BUFFERS, FORMAT JSON) ' || v_sql INTO v_plan;
            v_raiz := v_plan->0->'Plan';
            v_uso_1 := POSITION(v_nombres[1] IN v_raiz::TEXT) > 0;
            v_uso_2 := CASE WHEN ARRAY_LENGTH(v_nombres, 1) > 1
                THEN POSITION(v_nombres[2] IN v_raiz::TEXT) > 0 ELSE FALSE END;
            INSERT INTO resultado_mediciones VALUES (
                'C' || v_c, v_e, v_escenario, v_repeticion,
                CASE WHEN v_repeticion <= 2 THEN 'calentamiento' ELSE 'medicion' END,
                (v_plan->0->>'Execution Time')::NUMERIC,
                (v_plan->0->>'Planning Time')::NUMERIC,
                COALESCE((v_raiz->>'Shared Hit Blocks')::BIGINT, 0),
```

```
                COALESCE((v_raiz->>'Shared Read Blocks')::BIGINT, 0),
                COALESCE((v_raiz->>'Temp Read Blocks')::BIGINT, 0),
                COALESCE((v_raiz->>'Temp Written Blocks')::BIGINT, 0),
                v_uso_1, v_uso_2, v_filas, v_firma
            );
        END LOOP;
    END LOOP;
    RAISE NOTICE 'OK: C%, escala %, 14 ejecuciones, resultados identicos', v_c, v_e;
END;
$medicion$;

\echo === RESUMEN ===
SELECT consulta, escala, escenario, COUNT(*) AS mediciones,
       ROUND(AVG(execution_ms), 4) AS media_ms,
       ROUND(STDDEV_SAMP(execution_ms), 4) AS desviacion_ms,
       MIN(execution_ms) AS minimo_ms,
       MAX(execution_ms) AS maximo_ms,
       ROUND(AVG(shared_hit), 2) AS media_shared_hit,
       ROUND(AVG(shared_read), 2) AS media_shared_read,
       ROUND(AVG(temp_read), 2) AS media_temp_read,
       ROUND(AVG(temp_written), 2) AS media_temp_written,
       COUNT(*) FILTER (WHERE uso_indice_1) AS usos_indice_1,
       COUNT(*) FILTER (WHERE uso_indice_2) AS usos_indice_2,
       MIN(filas_resultado) AS filas
FROM resultado_mediciones
WHERE fase = 'medicion'
GROUP BY consulta, escala, escenario
ORDER BY CASE escenario WHEN 'sin_indice' THEN 1 ELSE 2 END;

DO $verificacion$
BEGIN
    IF (SELECT COUNT(*) FROM resultado_mediciones) <> 14
       OR (SELECT COUNT(*) FROM resultado_mediciones WHERE fase='medicion') <> 10
       OR (SELECT COUNT(DISTINCT firma_resultados) FROM resultado_mediciones) <> 1
       OR (SELECT COUNT(DISTINCT filas_resultado) FROM resultado_mediciones) <> 1 THEN
        RAISE EXCEPTION 'Mediciones o resultados inconsistentes';
    END IF;
    RAISE NOTICE 'OK: 4 calentamientos, 10 mediciones, resultados equivalentes';
END;
$verificacion$;

\echo === EXPORTACION CSV ===
SELECT
    CASE WHEN current_setting('sismo.consulta')::INTEGER=1 AND current_setting('sismo.escala')::INTEGER=1000 THEN '
     true' ELSE 'false' END AS guardar_c1_1000,
    CASE WHEN current_setting('sismo.consulta')::INTEGER=1 AND current_setting('sismo.escala')::INTEGER=10000 THEN '
     true' ELSE 'false' END AS guardar_c1_10000,
    CASE WHEN current_setting('sismo.consulta')::INTEGER=1 AND current_setting('sismo.escala')::INTEGER=100000 THEN '
     true' ELSE 'false' END AS guardar_c1_100000,
    CASE WHEN current_setting('sismo.consulta')::INTEGER=1 AND current_setting('sismo.escala')::INTEGER=1000000 THEN
     'true' ELSE 'false' END AS guardar_c1_1000000,
    CASE WHEN current_setting('sismo.consulta')::INTEGER=2 AND current_setting('sismo.escala')::INTEGER=1000 THEN '
     true' ELSE 'false' END AS guardar_c2_1000,
    CASE WHEN current_setting('sismo.consulta')::INTEGER=2 AND current_setting('sismo.escala')::INTEGER=10000 THEN '
     true' ELSE 'false' END AS guardar_c2_10000,
    CASE WHEN current_setting('sismo.consulta')::INTEGER=2 AND current_setting('sismo.escala')::INTEGER=100000 THEN '
     true' ELSE 'false' END AS guardar_c2_100000,
    CASE WHEN current_setting('sismo.consulta')::INTEGER=2 AND current_setting('sismo.escala')::INTEGER=1000000 THEN
     'true' ELSE 'false' END AS guardar_c2_1000000,
    CASE WHEN current_setting('sismo.consulta')::INTEGER=3 AND current_setting('sismo.escala')::INTEGER=1000 THEN '
     true' ELSE 'false' END AS guardar_c3_1000,
    CASE WHEN current_setting('sismo.consulta')::INTEGER=3 AND current_setting('sismo.escala')::INTEGER=10000 THEN '
     true' ELSE 'false' END AS guardar_c3_10000,
    CASE WHEN current_setting('sismo.consulta')::INTEGER=3 AND current_setting('sismo.escala')::INTEGER=100000 THEN '
     true' ELSE 'false' END AS guardar_c3_100000,
    CASE WHEN current_setting('sismo.consulta')::INTEGER=3 AND current_setting('sismo.escala')::INTEGER=1000000 THEN
     'true' ELSE 'false' END AS guardar_c3_1000000,
    CASE WHEN current_setting('sismo.consulta')::INTEGER=4 AND current_setting('sismo.escala')::INTEGER=1000 THEN '
     true' ELSE 'false' END AS guardar_c4_1000,
    CASE WHEN current_setting('sismo.consulta')::INTEGER=4 AND current_setting('sismo.escala')::INTEGER=10000 THEN '
     true' ELSE 'false' END AS guardar_c4_10000,
    CASE WHEN current_setting('sismo.consulta')::INTEGER=4 AND current_setting('sismo.escala')::INTEGER=100000 THEN '
     true' ELSE 'false' END AS guardar_c4_100000,
    CASE WHEN current_setting('sismo.consulta')::INTEGER=4 AND current_setting('sismo.escala')::INTEGER=1000000 THEN
     'true' ELSE 'false' END AS guardar_c4_1000000,
    CASE WHEN current_setting('sismo.consulta')::INTEGER=5 AND current_setting('sismo.escala')::INTEGER=1000 THEN '
     true' ELSE 'false' END AS guardar_c5_1000,
```

```
    CASE WHEN current_setting('sismo.consulta')::INTEGER=5 AND current_setting('sismo.escala')::INTEGER=10000 THEN '
      true' ELSE 'false' END AS guardar_c5_10000,
    CASE WHEN current_setting('sismo.consulta')::INTEGER=5 AND current_setting('sismo.escala')::INTEGER=100000 THEN '
      true' ELSE 'false' END AS guardar_c5_100000,
    CASE WHEN current_setting('sismo.consulta')::INTEGER=5 AND current_setting('sismo.escala')::INTEGER=1000000 THEN
      'true' ELSE 'false' END AS guardar_c5_1000000
\gset

\if :guardar_c1_1000
\copy (SELECT * FROM resultado_mediciones ORDER BY CASE escenario WHEN 'sin_indice' THEN 1 ELSE 2 END, repeticion) TO
      'resultados/06_C1_1000.csv' WITH (FORMAT csv, HEADER true)
\endif
\if :guardar_c1_10000
\copy (SELECT * FROM resultado_mediciones ORDER BY CASE escenario WHEN 'sin_indice' THEN 1 ELSE 2 END, repeticion) TO
      'resultados/06_C1_10000.csv' WITH (FORMAT csv, HEADER true)
\endif
\if :guardar_c1_100000
\copy (SELECT * FROM resultado_mediciones ORDER BY CASE escenario WHEN 'sin_indice' THEN 1 ELSE 2 END, repeticion) TO
      'resultados/06_C1_100000.csv' WITH (FORMAT csv, HEADER true)
\endif
\if :guardar_c1_1000000
\copy (SELECT * FROM resultado_mediciones ORDER BY CASE escenario WHEN 'sin_indice' THEN 1 ELSE 2 END, repeticion) TO
      'resultados/06_C1_1000000.csv' WITH (FORMAT csv, HEADER true)
\endif
\if :guardar_c2_1000
\copy (SELECT * FROM resultado_mediciones ORDER BY CASE escenario WHEN 'sin_indice' THEN 1 ELSE 2 END, repeticion) TO
      'resultados/06_C2_1000.csv' WITH (FORMAT csv, HEADER true)
\endif
\if :guardar_c2_10000
\copy (SELECT * FROM resultado_mediciones ORDER BY CASE escenario WHEN 'sin_indice' THEN 1 ELSE 2 END, repeticion) TO
      'resultados/06_C2_10000.csv' WITH (FORMAT csv, HEADER true)
\endif
\if :guardar_c2_100000
\copy (SELECT * FROM resultado_mediciones ORDER BY CASE escenario WHEN 'sin_indice' THEN 1 ELSE 2 END, repeticion) TO
      'resultados/06_C2_100000.csv' WITH (FORMAT csv, HEADER true)
\endif
\if :guardar_c2_1000000
\copy (SELECT * FROM resultado_mediciones ORDER BY CASE escenario WHEN 'sin_indice' THEN 1 ELSE 2 END, repeticion) TO
      'resultados/06_C2_1000000.csv' WITH (FORMAT csv, HEADER true)
\endif
\if :guardar_c3_1000
\copy (SELECT * FROM resultado_mediciones ORDER BY CASE escenario WHEN 'sin_indice' THEN 1 ELSE 2 END, repeticion) TO
      'resultados/06_C3_1000.csv' WITH (FORMAT csv, HEADER true)
\endif
\if :guardar_c3_10000
\copy (SELECT * FROM resultado_mediciones ORDER BY CASE escenario WHEN 'sin_indice' THEN 1 ELSE 2 END, repeticion) TO
      'resultados/06_C3_10000.csv' WITH (FORMAT csv, HEADER true)
\endif
\if :guardar_c3_100000
\copy (SELECT * FROM resultado_mediciones ORDER BY CASE escenario WHEN 'sin_indice' THEN 1 ELSE 2 END, repeticion) TO
      'resultados/06_C3_100000.csv' WITH (FORMAT csv, HEADER true)
\endif
\if :guardar_c3_1000000
\copy (SELECT * FROM resultado_mediciones ORDER BY CASE escenario WHEN 'sin_indice' THEN 1 ELSE 2 END, repeticion) TO
      'resultados/06_C3_1000000.csv' WITH (FORMAT csv, HEADER true)
\endif
\if :guardar_c4_1000
\copy (SELECT * FROM resultado_mediciones ORDER BY CASE escenario WHEN 'sin_indice' THEN 1 ELSE 2 END, repeticion) TO
      'resultados/06_C4_1000.csv' WITH (FORMAT csv, HEADER true)
\endif
\if :guardar_c4_10000
\copy (SELECT * FROM resultado_mediciones ORDER BY CASE escenario WHEN 'sin_indice' THEN 1 ELSE 2 END, repeticion) TO
      'resultados/06_C4_10000.csv' WITH (FORMAT csv, HEADER true)
\endif
\if :guardar_c4_100000
\copy (SELECT * FROM resultado_mediciones ORDER BY CASE escenario WHEN 'sin_indice' THEN 1 ELSE 2 END, repeticion) TO
      'resultados/06_C4_100000.csv' WITH (FORMAT csv, HEADER true)
\endif
\if :guardar_c4_1000000
\copy (SELECT * FROM resultado_mediciones ORDER BY CASE escenario WHEN 'sin_indice' THEN 1 ELSE 2 END, repeticion) TO
      'resultados/06_C4_1000000.csv' WITH (FORMAT csv, HEADER true)
\endif
\if :guardar_c5_1000
\copy (SELECT * FROM resultado_mediciones ORDER BY CASE escenario WHEN 'sin_indice' THEN 1 ELSE 2 END, repeticion) TO
      'resultados/06_C5_1000.csv' WITH (FORMAT csv, HEADER true)
\endif
\if :guardar_c5_10000
```

```
\copy (SELECT * FROM resultado_mediciones ORDER BY CASE escenario WHEN 'sin_indice' THEN 1 ELSE 2 END, repeticion) TO
      'resultados/06_C5_10000.csv' WITH (FORMAT csv, HEADER true)
\endif
\if :guardar_c5_100000
\copy (SELECT * FROM resultado_mediciones ORDER BY CASE escenario WHEN 'sin_indice' THEN 1 ELSE 2 END, repeticion) TO
      'resultados/06_C5_100000.csv' WITH (FORMAT csv, HEADER true)
\endif
\if :guardar_c5_1000000
\copy (SELECT * FROM resultado_mediciones ORDER BY CASE escenario WHEN 'sin_indice' THEN 1 ELSE 2 END, repeticion) TO
      'resultados/06_C5_1000000.csv' WITH (FORMAT csv, HEADER true)
\endif

\echo === RESTAURACION ===
ROLLBACK;

DO $final$
DECLARE v_e INTEGER := current_setting('sismo.escala')::INTEGER;
BEGIN
    IF current_database() <> 'bd_sismologico_prueba_exp_' || v_e THEN
        RAISE EXCEPTION 'Base final incorrecta';
    END IF;
    IF EXISTS (
        SELECT 1 FROM pg_index i
        JOIN pg_class t ON t.oid = i.indrelid
        JOIN pg_namespace n ON n.oid = t.relnamespace
        LEFT JOIN pg_constraint c ON c.conindid = i.indexrelid
        WHERE n.nspname = 'monitoreo_sismologico' AND c.oid IS NULL
    ) THEN
        RAISE EXCEPTION 'Indices secundarios persistentes';
    END IF;
    RAISE NOTICE 'OK: indices y datos iniciales preservados';
END;
$final$;

\echo === ETAPA 6: COMPARACION FINALIZADA ===
```

Listing 6: Integrador 06: protocolo de mediciones por escala.

7.6.7. Demostración de la escala máxima

```
-- 07. Demostracion de las consultas C1-C5 con un millon de registros.
-- Control de condiciones y restablecimiento mediante ROLLBACK.
\set ON_ERROR_STOP on
\pset pager off
\connect bd_sismologico_exp_1000000
\ir modulos/demostracion/62_demostracion_video_1000000.sql
\connect bd_sismologico
```

Listing 7: Integrador 07: acceso a la demostración.

```
-- 62. Comparacion de planes C1-C5 para un millon de registros.
-- Consultas equivalentes a las empleadas en las mediciones experimentales.
-- Una ejecucion ilustrativa por condicion, sin reemplazar las series de medicion.
-- Los cambios transitorios de indices se revierten mediante ROLLBACK.

\set ON_ERROR_STOP on
\pset pager off
\timing off

DO $guard$
DECLARE v_total BIGINT;
BEGIN
  IF current_database() <> 'bd_sismologico_exp_1000000' THEN
    RAISE EXCEPTION 'Base incorrecta: conectar a bd_sismologico_exp_1000000';
  END IF;
  SELECT (SELECT count(*) FROM monitoreo_sismologico.estacion_sismica)
       + (SELECT count(*) FROM monitoreo_sismologico.sensor)
       + (SELECT count(*) FROM monitoreo_sismologico.evento_sismico)
       + (SELECT count(*) FROM monitoreo_sismologico.lectura_onda)
       + (SELECT count(*) FROM monitoreo_sismologico.deteccion)
       + (SELECT count(*) FROM monitoreo_sismologico.alerta) INTO v_total;
```

```
  IF v_total <> 1000000
     OR (SELECT count(*) FROM monitoreo_sismologico.estacion_sismica) <> 100
     OR (SELECT count(*) FROM monitoreo_sismologico.sensor) <> 300
     OR (SELECT count(*) FROM monitoreo_sismologico.evento_sismico) <> 120000
     OR (SELECT count(*) FROM monitoreo_sismologico.lectura_onda) <> 875000
     OR (SELECT count(*) FROM monitoreo_sismologico.deteccion) <> 3500
     OR (SELECT count(*) FROM monitoreo_sismologico.alerta) <> 1100 THEN
    RAISE EXCEPTION 'Distribucion distinta de la base de un millon';
  END IF;
  IF to_regclass('monitoreo_sismologico.idx_lectura_sensor_hora_c5') IS NULL
     OR to_regclass('monitoreo_sismologico.idx_det_sensor_hora_c5') IS NULL THEN
    RAISE EXCEPTION 'No se cumple la condicion inicial de los indices experimentales C5';
  END IF;
  IF EXISTS (
    SELECT 1 FROM pg_index i
    JOIN pg_class t ON t.oid=i.indrelid
    JOIN pg_namespace n ON n.oid=t.relnamespace
    JOIN pg_class ix ON ix.oid=i.indexrelid
    LEFT JOIN pg_constraint c ON c.conindid=i.indexrelid
    WHERE n.nspname='monitoreo_sismologico' AND c.oid IS NULL
      AND ix.relname NOT IN ('idx_lectura_sensor_hora_c5','idx_det_sensor_hora_c5')
  ) THEN
    RAISE EXCEPTION 'Hay indices secundarios inesperados; no se cambio nada';
  END IF;
  RAISE NOTICE 'VERIFICADO: 1000000 registros y solo dos indices C5 originales de las pruebas';
END;
$guard$;

\echo
\echo === A. RESUMEN DEL MILLON DE REGISTROS ===
SELECT 'estacion_sismica' AS tabla, count(*) FROM monitoreo_sismologico.estacion_sismica
UNION ALL SELECT 'sensor', count(*) FROM monitoreo_sismologico.sensor
UNION ALL SELECT 'evento_sismico', count(*) FROM monitoreo_sismologico.evento_sismico
UNION ALL SELECT 'lectura_onda', count(*) FROM monitoreo_sismologico.lectura_onda
UNION ALL SELECT 'deteccion', count(*) FROM monitoreo_sismologico.deteccion
UNION ALL SELECT 'alerta', count(*) FROM monitoreo_sismologico.alerta;

\echo
\echo === B. ESCENARIOS REVERSIBLES DENTRO DE TRANSACCION ===
BEGIN;
SET LOCAL search_path TO monitoreo_sismologico, public;
DROP INDEX monitoreo_sismologico.idx_lectura_sensor_hora_c5;
DROP INDEX monitoreo_sismologico.idx_det_sensor_hora_c5;

DO $check$
BEGIN
  IF EXISTS (
    SELECT 1 FROM pg_index i
    JOIN pg_class t ON t.oid=i.indrelid
    JOIN pg_namespace n ON n.oid=t.relnamespace
    LEFT JOIN pg_constraint c ON c.conindid=i.indexrelid
    WHERE n.nspname='monitoreo_sismologico' AND c.oid IS NULL
  ) THEN RAISE EXCEPTION 'Aun hay indices secundarios en la condicion de referencia';
  END IF;
END;
$check$;

CREATE TEMP TABLE evidencia_video_62 (
  consulta TEXT NOT NULL,
  escenario TEXT NOT NULL,
  execution_ms NUMERIC NOT NULL,
  shared_hit BIGINT,
  shared_read BIGINT,
  temp_read BIGINT,
  temp_written BIGINT,
  indice_usado TEXT NOT NULL,
  PRIMARY KEY(consulta,escenario)
) ON COMMIT DROP;

DO $demo$
DECLARE
    v_queries TEXT[] := ARRAY[
      $consulta_1$SELECT
    EXTRACT(YEAR FROM ev.fecha_hora_origen)::INTEGER AS anio_utc,
    COUNT(*) AS cantidad_eventos,
    ROUND(AVG(ev.magnitud), 2) AS magnitud_promedio,
    MAX(ev.magnitud) AS magnitud_maxima,
```

```
    ROUND(AVG(ev.profundidad_km), 2) AS profundidad_promedio_km
FROM evento_sismico ev
WHERE ev.id_evento BETWEEN 200000000 AND 299999999
  AND ev.fecha_hora_origen >= TIMESTAMP '2010-01-01 00:00:00'
  AND ev.fecha_hora_origen < TIMESTAMP '2026-01-01 00:00:00'
  AND ev.magnitud >= 6
GROUP BY EXTRACT(YEAR FROM ev.fecha_hora_origen)::INTEGER
HAVING COUNT(*) >= 1
ORDER BY anio_utc$consulta_1$,
      $consulta_2$SELECT
    e.id_estacion,
    e.codigo AS codigo_estacion,
    e.nombre AS nombre_estacion,
    COUNT(*) AS eventos_en_ventana,
    ROUND(AVG(ev.magnitud), 2) AS magnitud_promedio,
    MAX(ev.magnitud) AS magnitud_maxima
FROM estacion_sismica e
JOIN evento_sismico ev
  ON ev.latitud_epicentro BETWEEN e.latitud - 1.5 AND e.latitud + 1.5
 AND ev.longitud_epicentro BETWEEN e.longitud - 1.5 AND e.longitud + 1.5
WHERE ev.id_evento BETWEEN 200000000 AND 299999999
  AND ev.fecha_hora_origen >= TIMESTAMP '2000-01-01 00:00:00'
  AND ev.fecha_hora_origen < TIMESTAMP '2026-01-01 00:00:00'
  AND ev.magnitud >= 5
GROUP BY e.id_estacion, e.codigo, e.nombre
HAVING COUNT(*) >= 1
ORDER BY eventos_en_ventana DESC, e.id_estacion
LIMIT 10$consulta_2$,
      $consulta_3$SELECT
    e.id_estacion,
    e.codigo AS codigo_estacion,
    COUNT(DISTINCT s.id_sensor) AS sensores_con_lecturas,
    COUNT(*) AS total_lecturas,
    COUNT(*) FILTER (WHERE l.es_valida) AS lecturas_validas,
    COUNT(*) FILTER (WHERE NOT l.es_valida) AS lecturas_invalidas,
    ROUND(100.0 * COUNT(*) FILTER (WHERE NOT l.es_valida)
          / NULLIF(COUNT(*), 0), 2) AS porcentaje_invalidas
FROM estacion_sismica e
JOIN sensor s ON s.id_estacion = e.id_estacion
JOIN lectura_onda l ON l.id_sensor = s.id_sensor
WHERE l.fecha_hora >= TIMESTAMP '2026-08-01 00:00:00'
  AND l.fecha_hora < TIMESTAMP '2026-12-01 00:00:00'
GROUP BY e.id_estacion, e.codigo
HAVING COUNT(*) >= 10
ORDER BY porcentaje_invalidas DESC, total_lecturas DESC, e.id_estacion$consulta_3$,
      $consulta_4$WITH detecciones_resumen AS (
    SELECT id_evento, COUNT(*) AS total_detecciones,
           MIN(hora_deteccion) AS primera_deteccion
    FROM deteccion
    GROUP BY id_evento
), alertas_resumen AS (
    SELECT id_evento, COUNT(*) AS total_alertas,
           MIN(fecha_hora_emision) AS primera_alerta
    FROM alerta
    GROUP BY id_evento
)
SELECT
    ev.id_evento,
    ev.fecha_hora_origen,
    ev.magnitud,
    COALESCE(d.total_detecciones, 0) AS total_detecciones,
    COALESCE(a.total_alertas, 0) AS total_alertas,
    ROUND(EXTRACT(EPOCH FROM (d.primera_deteccion - ev.fecha_hora_origen))::NUMERIC, 2)
        AS segundos_primera_deteccion,
    ROUND(EXTRACT(EPOCH FROM (a.primera_alerta - ev.fecha_hora_origen))::NUMERIC, 2)
        AS segundos_primera_alerta
FROM evento_sismico ev
LEFT JOIN detecciones_resumen d ON d.id_evento = ev.id_evento
LEFT JOIN alertas_resumen a ON a.id_evento = ev.id_evento
WHERE (ev.id_evento BETWEEN 810000001 AND 810000008
    OR ev.id_evento BETWEEN 820300001 AND 820300030)
  AND (COALESCE(d.total_detecciones, 0) > 0
    OR COALESCE(a.total_alertas, 0) > 0)
ORDER BY ev.fecha_hora_origen DESC, ev.id_evento
LIMIT 15$consulta_4$,
      $consulta_5$SELECT
    s.id_sensor,
```

```
    s.numero_serie,
    e.codigo AS codigo_estacion,
    s.estado AS estado_sensor,
    COUNT(*) AS lecturas_periodo,
    COUNT(*) FILTER (WHERE NOT l.es_valida) AS lecturas_invalidas,
    MAX(l.fecha_hora) AS ultima_lectura
FROM sensor s
JOIN estacion_sismica e ON e.id_estacion = s.id_estacion
JOIN lectura_onda l ON l.id_sensor = s.id_sensor
WHERE l.fecha_hora >= TIMESTAMP '2026-08-01 00:00:00'
  AND l.fecha_hora < TIMESTAMP '2026-09-01 00:00:00'
  AND NOT EXISTS (
      SELECT 1
      FROM deteccion d
      WHERE d.id_sensor = s.id_sensor
        AND d.hora_deteccion >= TIMESTAMP '2026-08-01 00:00:00'
        AND d.hora_deteccion < TIMESTAMP '2026-09-01 00:00:00'
  )
GROUP BY s.id_sensor, s.numero_serie, e.codigo, s.estado
HAVING COUNT(*) >= 20
ORDER BY lecturas_invalidas DESC, lecturas_periodo DESC, s.id_sensor
LIMIT 15$consulta_5$
    ];
    v_idx TEXT[][] := ARRAY[
      ARRAY['idx_ev_igp_m6_fecha',''],
      ARRAY['idx_ev_igp_geo_c2',''],
      ARRAY['idx_lectura_sensor_fecha_c3',''],
      ARRAY['idx_det_evento_hora_c4','idx_alerta_evento_hora_c4'],
      ARRAY['idx_lectura_sensor_hora_c5','idx_det_sensor_hora_c5']
    ];
    v_idx_ddl TEXT[][] := ARRAY[
      ARRAY[$ddl_1_1$CREATE INDEX idx_ev_igp_m6_fecha ON monitoreo_sismologico.evento_sismico (fecha_hora_origen)
     INCLUDE (magnitud, profundidad_km) WHERE id_evento BETWEEN 200000000 AND 299999999 AND magnitud >= 6$ddl_1_1$,
     $ddl_1_2$$ddl_1_2$],
      ARRAY[$ddl_2_1$CREATE INDEX idx_ev_igp_geo_c2 ON monitoreo_sismologico.evento_sismico (latitud_epicentro,
     longitud_epicentro) INCLUDE (magnitud) WHERE id_evento BETWEEN 200000000 AND 299999999 AND fecha_hora_origen >=
      TIMESTAMP '2000-01-01 00:00:00' AND fecha_hora_origen < TIMESTAMP '2026-01-01 00:00:00' AND magnitud >= 5
     $ddl_2_1$, $ddl_2_2$$ddl_2_2$],
      ARRAY[$ddl_3_1$CREATE INDEX idx_lectura_sensor_fecha_c3 ON monitoreo_sismologico.lectura_onda (id_sensor,
     fecha_hora) INCLUDE (es_valida)$ddl_3_1$, $ddl_3_2$$ddl_3_2$],
      ARRAY[$ddl_4_1$CREATE INDEX idx_det_evento_hora_c4 ON monitoreo_sismologico.deteccion (id_evento,
     hora_deteccion)$ddl_4_1$, $ddl_4_2$CREATE INDEX idx_alerta_evento_hora_c4 ON monitoreo_sismologico.alerta (
     id_evento, fecha_hora_emision)$ddl_4_2$],
      ARRAY[$ddl_5_1$CREATE INDEX idx_lectura_sensor_hora_c5 ON monitoreo_sismologico.lectura_onda (id_sensor,
     fecha_hora) INCLUDE (es_valida)$ddl_5_1$, $ddl_5_2$CREATE INDEX idx_det_sensor_hora_c5 ON monitoreo_sismologico.
     deteccion (id_sensor, hora_deteccion)$ddl_5_2$]
    ];
    v_tables TEXT[][] := ARRAY[
      ARRAY['evento_sismico',''],
      ARRAY['evento_sismico',''],
      ARRAY['lectura_onda',''],
      ARRAY['deteccion','alerta'],
      ARRAY['lectura_onda','deteccion']
    ];
    i INTEGER; k INTEGER; phase INTEGER;
    v_plan JSONB; v_root JSONB; v_used TEXT;
BEGIN
    FOR i IN 1..5 LOOP
        FOR phase IN 1..2 LOOP
            IF phase = 2 THEN
                FOR k IN 1..2 LOOP
                    IF v_idx_ddl[i][k] <> '' THEN
                        EXECUTE v_idx_ddl[i][k];
                    END IF;
                    IF v_tables[i][k] <> '' THEN
                        EXECUTE format('ANALYZE monitoreo_sismologico.%I',v_tables[i][k]);
                    END IF;
                END LOOP;
            END IF;
            EXECUTE 'EXPLAIN (ANALYZE, BUFFERS, FORMAT JSON) ' || v_queries[i] INTO v_plan;
            v_root := v_plan->0->'Plan';
            v_used := '';
            FOR k IN 1..2 LOOP
                IF v_idx[i][k] <> '' AND POSITION(v_idx[i][k] IN v_root::TEXT) > 0 THEN
                    v_used := v_used || CASE WHEN v_used <> '' THEN ', ' ELSE '' END || v_idx[i][k];
                END IF;
            END LOOP;
```

```
            INSERT INTO evidencia_video_62 VALUES (
                'C'||i, CASE WHEN phase=1 THEN 'SIN' ELSE 'CON' END,
                (v_plan->0->>'Execution Time')::NUMERIC,
                COALESCE((v_root->>'Shared Hit Blocks')::BIGINT,0),
                COALESCE((v_root->>'Shared Read Blocks')::BIGINT,0),
                COALESCE((v_root->>'Temp Read Blocks')::BIGINT,0),
                COALESCE((v_root->>'Temp Written Blocks')::BIGINT,0),
                CASE WHEN v_used='' THEN 'ninguno' ELSE v_used END
            );
        END LOOP;
        FOR k IN 1..2 LOOP
            IF v_idx[i][k] <> '' THEN
                EXECUTE format('DROP INDEX monitoreo_sismologico.%I',v_idx[i][k]);
            END IF;
        END LOOP;
        RAISE NOTICE 'C% documentada en ambos escenarios; indice candidato retirado',i;
    END LOOP;
END;
$demo$;

\echo
\echo === C. PLANES DE DEMOSTRACION (UNA EJECUCION POR ESCENARIO) ===
SELECT consulta, escenario, execution_ms AS tiempo_ms,
       shared_hit, shared_read, temp_read, temp_written, indice_usado
FROM evidencia_video_62
ORDER BY consulta, CASE escenario WHEN 'SIN' THEN 1 ELSE 2 END;

\echo
\echo === D. ROLLBACK: RESTAURAR EL ESTADO ORIGINAL DE LOS INDICES ===
ROLLBACK;

\echo
\echo === E. VERIFICACION DE INDICES ORIGINALES TRAS ROLLBACK ===
SELECT indexname, tablename
FROM pg_indexes
WHERE schemaname='monitoreo_sismologico'
  AND indexname IN ('idx_lectura_sensor_hora_c5','idx_det_sensor_hora_c5')
ORDER BY indexname;

\echo
\echo === DEMOSTRACION COMPLETADA; NO SUSTITUYE LOS CSV DE 5 MEDICIONES ===
```

Listing 8: Módulo 62: demostración reversible de las consultas C1–C5.